

Capítulo	1
Título	Fundamentos del Lenguaje

Este capítulo cubre aspectos relacionados con los siguientes objetivos del Examen de Certificación en Java:

- Identificar la correcta construcción de archivos fuente, declaración de paquetes (**packages**), cláusulas de inclusión (**import**), declaración de clases (incluyendo clases anidadas), declaración e implementación de interfaces, declaración de métodos, declaración de variables e identificadores.
- Identificar todas las palabras reservadas del lenguaje e identificadores correctamente contruídos.
- Determinar el rango de todos los tipos primitivos y declarar valores literales para objetos de la clase *String* y para todos los tipos primitivos, utilizando todos los formatos, bases y representaciones permitidas.
- Escribir código que declare, construya e inicialice arreglos de cualquier tipo, utilizando cualquiera de las formas tanto para su declaración como para su inicialización.
- Determinar el efecto de utilizar una variable simple o elemento de un arreglo, de cualquier tipo, cuando no se ha inicializado explícitamente.
- Determinar la correspondencia existente entre los índices del arreglo de parámetros pasados a la función *main()* y los argumentos dados en la línea de comandos.
- Determinar el efecto de las asignaciones y otras modificaciones a objetos u otras variables primitivas, pasados como parámetros.
- Determinar el comportamiento garantizable del sistema de recolección de memoria (garbage collector) y escribir código que explícitamente marque los objetos para ser borrados de memoria.

Este curso no provee una introducción al lenguaje Java: está orientado a proveer la capacitación necesaria para pasar el examen de

certificación y asume por lo tanto que Ud. está familiarizado con los conceptos fundamentales del lenguaje. El propósito de este capítulo es entonces asegurar que Ud. está cien por ciento seguro de los conceptos fundamentales cubiertos en el examen de certificación.

Archivos de Código Fuente

Todos los archivos de código fuente deben terminar con la extensión **.java**. Todo archivo de código fuente debe contener como máximo la declaración de una clase pública^(*): si una clase pública aparece en el archivo, éste deberá tener el mismo nombre que la clase mas la extensión **.java**. Un archivo fuente puede contener un número ilimitado de clases no públicas.

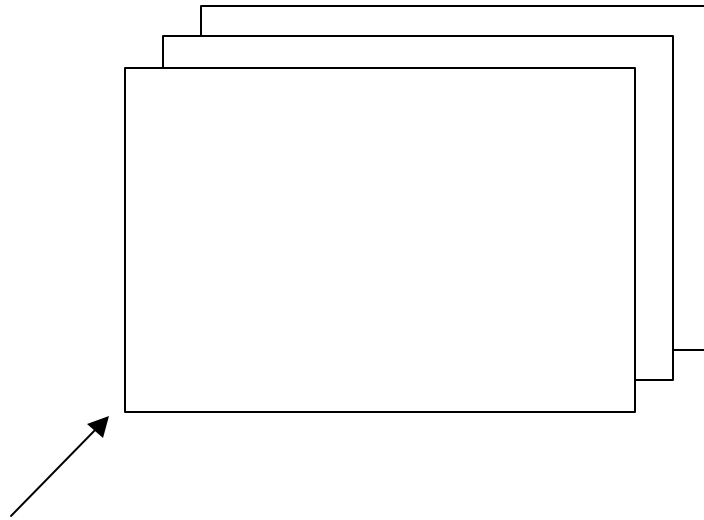


Figura 1. Archivo LoginApplet.java

Nota: este no es un requerimiento del lenguaje pero sí un requerimiento de implementación de muchos compiladores, incluyendo aquellos de Sun. Por lo tanto no es conveniente ignorar esta convención ya que de esta manera se estará limitando la portabilidad de los archivos fuente.

Existen tres elementos que pueden aparecer en un archivo fuente:

1. **package** → declaración de un paquete de clases
2. **import** → inclusión de la definición de otras clases
3. **class** → definición de una o más clases

Ninguno de los anteriores es obligatorio, pero si están presentes deben aparecer en el orden anterior.

La declaración de un paquete tiene el siguiente formato:

package nombre_del_paquete;

El nombre del paquete es una secuencia de nombres separados por puntos:

1.	package uv.applets.login;
----	----------------------------------

El nombre del paquete refleja la jerarquía de sub-directorios en el disco donde se encuentran las clases del mismo. Se debe tener mucho cuidado entonces que el nombre del paquete represente nombres válidos de directorios en todas las plataformas. No se deben utilizar, por lo tanto, caracteres tales como espacios, slash (/), backslash (\), u otros símbolos.

La inclusión de la definición de otras clases en un archivo fuente tiene un formato similar:

import nombre_del_paquete.nombre_de_la_clase

Por ejemplo:

1.	import nombre_del_paquete.*;
----	-------------------------------------

Para incluir todas las clases de un paquete se utiliza el siguiente formato:

Por ejemplo:

```
1.      import uv.applets.login.*;
```

De esta manera la definición de una clase puede ser algo así:

```
1.      // declaración del paquete
2.      package uv.applets.login;
3.
4.      // inclusiones
5.      import uv.servlets.login.*; // incluye todas las clases
6.
7.      // definición de la clase
8.      class LoginApplet
9.      {
10.         ...
11.     }
```

El ejemplo anterior muestra que se pueden agregar comentarios antes y/o después de cualquiera de estos elementos.

Palabras Reservadas e Identificadores

Java especifica 50 palabras reservadas listadas a continuación:

abstract	default	goto	null	synchronized
boolean	do	if	package	this
break	double	implements	private	throw
byte	else	import	protected	throws
case	extends	instanceof	public	transient
catch	false	int	return	true
char	final	interface	short	try
class	finally	long	static	void
const	float	native	super	volatile
continue	for	new	switch	while

Tabla 1. Palabras reservadas en Java.

Las palabras **goto** y **const** son reservadas: a pesar de que no tienen significado en Java no pueden ser utilizadas como identificadores.

Un identificador es una palabra utilizada para nombrar una variable, un método, una clase o una etiqueta. Las palabras reservadas no pueden ser utilizadas como identificadores. Un identificador debe comenzar con una letra, un signo de pesos (\$) or un underscore (_); los caracteres siguientes pueden ser letras, signos de pesos, underscores o dígitos.

var // legal

ISboolean	// legal: palabras reservadas embebidas
\$value	// legal
3_value	// ilegal: comienza con dígito
!isValid	// ilegal: debe empezar con letra, \$ o _

Los identificadores son sensibles a mayúsculas y minúsculas: value y Value identifican variables diferentes.

Tipos de Datos Primitivos

Los tipos de datos primitivos de Java son:

- **boolean**
- **char**
- **byte**
- **short**
- **int**
- **long**
- **float**
- **double**

El espacio de memoria ocupado por estos tipos está definido en la especificación del lenguaje y se listan a continuación:

TIPO	TAMAÑO (bits)	TIPO	TAMAÑO (bits)
boolean	1	char	16
byte	8	short	16
int	32	long	64
float	32	double	64

Tabla 2. Tipos primitivos y tamaño en bits.

Las variables de tipo **boolean** solo pueden tomar los valores **true** o **false**.

Los tipos de datos enteros con signo son:

- **byte**
- **short**
- **int**
- **long**

Las variables de este tipo son números 2-complemento. Los rangos de valores que pueden almacenar estas variables son los siguientes:

TIPO	TAMAÑO (bits)	MINIMO	MAXIMO
byte	8	-2^7	2^7-1
short	16	-2^{15}	$2^{15}-1$
int	32	-2^{31}	$2^{31}-1$
long	64	-2^{63}	$2^{63}-1$

Tabla 3. Tipos primitivos y rangos de valor.

Se debe notar que para cada tipo de los anteriores, los exponentes de dos para los valores mínimo y máximo es el tamaño en bits menos uno.

El tipo **char** es entero sin signo^C: el rango de valores es por lo tanto $0 \rightarrow 2^{16}-1$. Los caracteres en Java son codificados según es estándar UNICODE. Si los nueve bits más significativos son cero, el sistema de codificación coincide con el sistema ASCII de 7 bits.

^C Diferencias con lenguaje C/C++: en Java el tipo **byte** es con signo y el tipo **char** sin signo y de 16 bits. El tipo **long** es de 64 bits ($0 \rightarrow 2^{64}$ Gb).

Los tipos de punto flotante son:

- **float**
- **double**

Estos tipos se ajustan a la especificación IEEE-754. Muchas de las operaciones matemáticas pueden resultar en valores que no tienen representación en números: infinito, por ejemplo. Para describir estos valores no numéricos, las variables de tipo **float** y **double** utilizan patrones de bits definidos en las clases *Float* y *Double* respectivamente:

Float.NaN (**Not a Number**)

Float.NEGATIVE_INFINITY

Float.POSITIVE_INFINITY

Double.NaN

Double.NEGATIVE_INFINITY

Double.POSITIVE_INFINITY

El siguiente fragmento de código muestra el uso de estas constantes:

1.	<code>double v = -10.0/0.0;</code>
2.	<code>if (v == Double.NEGATIVE_INFINITY)</code>
3.	<code>{</code>
4.	<code> System.out.println("valor de v = " + v);</code>
5.	<code>}</code>

Literales

Un literal es un valor que puede ser asignado a una variable de tipo primitivo o instancia de la clase *String*, o pasado como argumento en la llamada a un método.

Literales **boolean**

Los únicos literales válidos para el tipo **boolean** son **true** y **false**:

```
boolean bIsValid = true;
```

```
boolean bIsInvalid = false;
```

Literales **char**

Un literal para el tipo **char** puede ser expresado colocando el valor deseado entre comillas sencillas:

```
char c = 'a';
```

Otra forma de expresar un literal de tipo caracter es como una secuencia UNICODE de cuatro dígitos hexadecimales, precedidos de `\u`, y colocados entre comillas sencillas:

```
char c = '\u1234';
```

Java soporta las siguientes secuencias de control para denotar caracteres especiales:

'\n'	Newline
'\r'	Return
'\t'	Tab
'\b'	Backspace
'\f'	Formfeed
'\''	Single Quote
'\"'	Double Quote
'\\'	Backslash

Literales enteros

Los literales enteros pueden ser expresados en forma decimal, octal o hexadecimal. La forma por omisión es decimal. Para indicar un literal octal se precede el valor de 0 (cero). Para indicar un literal hexadecimal se precede el valor de 0x o 0X; los dígitos pueden estar en mayúsculas o minúsculas. El valor cuarenta y dos puede ser entonces expresado de las siguientes formas:

- 42
- 052
- 0x2a
- 0x2A
- 0X2a
- 0X2A

Por omisión, un literal entero es un valor de 32 bits. Para indicar

un literal entero de 64 bits se debe agregar la letra L al final del valor. (Se puede utilizar una L minúscula pero entonces se confunde con un 1).

Literales de punto flotante

Un literal de punto flotante expresa un valor de punto flotante. Para ser interpretado como un literal de punto flotante, una expresión numérica debe contener uno de los siguientes:

Un punto decimal	1.23
La letra 'e' o 'E' que indica notación científica	1.23E+10
El sufijo 'f' o 'F' que indica un literal de 32 bits	1.23f
El sufijo 'd' o 'D' que indica un literal de 64 bits	1.23d

Literales *String*

Un literal de tipo *String* es una secuencia de caracteres entre comillas:

```
String str = "Los caracteres en Java se codifican en 16 bits";
```

Arreglos

Un arreglo en Java es una colección ordenada de variables primitivas, referencias a objetos u otros arreglos. Los arreglos en Java son homogéneos: excepto por lo que se puede hacer por medio del polimorfismo, todos los elementos de un arreglo son de un mismo tipo. Esto es, cuando se crea un arreglo se especifica el tipo de elementos que ha de contener y solo podrá contener instancias de esa clase o de una sub-clase de esa clase.

Para crear un arreglo se deben seguir los tres pasos siguientes:

1. declaración
2. construcción
3. inicialización

La declaración dice al compilador el nombre del arreglo y el tipo de de elementos que el arreglo ha de contener:

```
1.    int vi[];  
2.    double vd[];  
3.    String vs[];  
4.    float vf[][];  
5.    char[] sz;
```

Las líneas 1 y 2 declaran arreglos de tipos primitivos. La línea 3 declara un arreglo de referencias a objetos de la clase *String*. La línea 4 declara un arreglo de dos dimensiones, esto es, un arreglo de arreglos de tipo **float**. Los paréntesis cuadrados pueden ir antes o después del nombre del arreglo. Se debe notar que la declaración no especifica el tamaño del arreglo. El tamaño es especificado en tiempo

de ejecución cuando se pide memoria para el arreglo por medio de la palabra reservada **new**:

```
1.    int vi[];  
2.    vi = new int[ 80 ];
```

Como el tamaño de un arreglo no es utilizado sino en tiempo de ejecución, este puede ser especificado a través de una variable de tipo entero:

```
1.    int size = 80;  
2.    int vi[];  
3.    vi = new int[ size ];
```

La declaración y construcción de un arreglo pueden ser llevados a cabo en un solo paso como se muestra a continuación:

```
1.    int vi[] = new int[ 80 ];
```

Cuando un arreglo es construido, todos los elementos son automáticamente inicializados de la misma manera que las variables u objetos del tipo. Los elementos numéricos son inicializados en 0, los elementos no numéricos son inicializados en valores similares a 0, como se muestra en la siguiente tabla:

byte	0
short	0
int	0
long	0
float	0.0f
double	0.0d

char	'\u0000'
boolean	false
referencia a objeto	null

Tabla 4. Tipos primitivos y valores de inicialización por omisión.

Si se desea inicializar un arreglo de elementos con valores diferentes a los valores por omisión, se pueden combinar, declaración, construcción e inicialización en un solo paso:

1.	int vi[] = { 1, 2, 3, 4, 5 };
----	--------------------------------------

El tamaño del arreglo se infiere del número de elementos de inicialización. La inicialización también puede ser llevada a cabo, obviamente, asignando valores a cada elemento del arreglo:

1.	int vi[] = new int [80];
2.	for (int i = 0; i < vi.length; i++)
3.	{
4.	vi[i] = i; // los índices de un arreglo comienzan en 0
5.	}

Las clases son los componentes fundamentales de Java y el Examen de Certificación tiene como objetivo principal evaluar su profundo conocimiento.

El método `main()` es el punto de entrada para aplicaciones escritas en Java. Para crear una aplicación se escribe una definición de clase y en ésta se incluye un método `main()`. Para ejecutar una aplicación se escribe **java** en la línea de comandos seguida del nombre de la clase cuyo método `main()` se desea ejecutar:

```
1.    package uv.applets.login;  
2.    import uv.servlets.login.*;  
3.    class LoginApplet  
4.    {  
5.        ...  
  
6.        public static void main( String[] args )  
7.        {  
8.            ...  
9.        }  
10.   }
```

Ejemplo de ejecución:

```
c:\java LoginApplet ↵
```

El método main()

El método main() se hace público por convención. Sin embargo, debe ser estático (**static**) con el fin de que pueda ser ejecutado sin tener que construir un objeto de la clase correspondiente.

El arreglo 'args' contiene los argumentos dados en la línea de comandos:

```
c:\java LoginApplet 204.3.245.206 /dologin ↵
```

En el ejemplo anterior, 'args' contiene dos elementos:

▪ "204.3.245.206"	args[0]
▪ "/dologin"	args[1]

Debe notarse que ni el nombre de la clase ni el nombre del compilador (**java**) aparecen en el arreglo de argumentos^C.

El nombre 'args' es perfectamente arbitrario siempre que sea declarado como un arreglo unidimensional de objetos *String*.

Variables e Inicialización

Java soporta variables con dos tipos de vida:

- Variables miembro de la clase, son creadas cuando el objeto es creado y son accesibles por cualquier método de la clase
- Variables automáticas de un método, creadas al entrar al método, existen solamente durante la ejecución del método y solo pueden ser accedidas dentro del método

^C En C/C++ el primer parámetro (índice 0) contiene la ruta y nombre del ejecutable; de esta forma los argumentos que recibe la función main() comienzan en el índice 1.

A todas las variables miembro, que no son explícitamente inicializadas en su declaración, les es asignado un valor inicial de forma automática.

Esta inicialización depende del tipo de la variable miembro. Los valores de inicialización son los dados en la Tabla 4 (inicialización de los elementos de un arreglo).

Una variable miembro puede ser inicializada en la misma línea de código donde es declarada:

```
1.    package uv.applets.login;
2.    import uv.servlets.login.*;
3.    class LoginApplet
4.    {
5.        ...
6.        int          m_nIndex = -1;
7.        static int  m_nID = 0;
8.    }
```

Las variables no estáticas son inicializadas justo antes de que el constructor es ejecutado. Las variables estáticas son inicializadas cuando la clase es cargada ya que estas variables son únicas para todas las instancias de la clase.

En este caso, 'm_nIndex' es inicializada justo antes de que el constructor de LoginApplet es ejecutado; 'm_nID' es inicializada cuando la clase LoginApplet es cargada en memoria.

Las variables automáticas no son inicializadas por el sistema y deben ser inicializadas explícitamente antes de poder ser utilizadas:

```
1.    public int no_compila()
2.    {
```

```
3.         int i;  
4.         return i;  
5.     }
```

Mensaje del compilador:

```
LoginApplet.java:23: Variable i may not have been initialized.
```

Este error también aparece cuando la inicialización de una variable automática ocurre dentro de un par de corchetes de un nivel interior con respecto al lugar donde la variable es utilizada:

```
1.     public int no_compila( int x )  
2.     {  
3.         int i;  
4.         if ( x < 0 )  
5.         {  
6.             i = -1;  
7.         }  
8.         return i;  
9.     }
```

Mensaje del compilador:

```
LoginApplet.java:23: Variable i may not have been initialized.
```

La forma correcta es inicializar la variable 'i' en la misma línea donde es declarada:

```
1.      public int compila( int x )
2.      {
3.          int i = 0;
4.          if ( x < 0 )
5.          {
6.              i = -1;
7.          }
8.          return i;
9.      }
```

Paso de Argumentos

En Java todos los argumentos son pasados por valor. Esto es, cuando un argumento es pasado a un método, el método recibe una copia del argumento original. De esta forma los cambios en el valor del argumento recibido no afectan la variable original:

```
1.      public void inc( int x )
2.      {
3.          x += 1;
4.      }
5.      public static void main( String[] args )
6.      {
7.          int i = 0;
8.          inc( i );
9.          System.out.println( "i=", i );           // i=0
10.     }
```

Esto funciona de la misma forma si el argumento pasado al método es un objeto y no un tipo primitivo como en el ejemplo anterior. Sin embargo, el proceso es totalmente diferente. Para entenderlo es necesario revisar el concepto de referencia a objeto.

Los programas en Java no manipulan directamente los objetos que utilizan. Cuando un objeto es creado, el constructor retorna un valor que identifica al objeto en forma única. Este valor es conocido como una referencia al objeto.

```
1.      TextField tf = new TextField();
```

La línea anterior de código retorna una referencia al objeto construido, no el objeto ni una copia del objeto. Esta referencia es almacenada en la variable 'tf'. En muchas implementaciones de JVM (Java Virtual Machine), una referencia es simplemente un entero de 32 bits que almacena la dirección del objeto; sin embargo, las especificaciones de JVM dan una gran flexibilidad de como debe implementarse una referencia. De aquí que una referencia es mejor interpretada como un patrón de bits que identifica al objeto en forma única. Cada vez que un objeto es almacenado en una variable o pasado como argumento a un método, lo que se almacena o pasa es una referencia al objeto.

```
1.      public void no_cambia( TextField cotf )
2.      {
3.          cotf = new TextField( "abc" );
4.      }
5.      public static void main( String[] args )
6.      {
7.          TextField tf = new TextField( "texto" );
8.          no_cambia( tf );
9.          System.out.println( tf.getText() );      // 'texto'
10.     }
```

En el ejemplo anterior, un objeto `TextField` es construido en la línea 7 y la referencia almacenada en la variable `'tf'`. En la línea 8 la referencia es pasada al método `'no_cambia'` que recibe una copia de esta referencia. Allí en la línea 3 se construye un nuevo objeto y la referencia copia es sobrescrita en la asignación. Como la referencia original permanece inalterada, la línea 9 despliega el valor original: `'texto'`.

Como se ha visto, los métodos evocados no pueden modificar los valores originales de los argumentos recibidos, y que son almacenados en el método que ha hecho la llamada. Sin embargo, si el método modifica el objeto a través de la referencia recibida usando alguno de sus métodos, en ese caso los cambios si son visibles en el método que ha hecho la llamada:

```
1.      public void si_cambia( TextField cotf )
2.      {
3.          cotf.setText ( "abc" );
4.      }
5.      public static void main( String[] args )
6.      {
7.          TextField tf = new TextField( "texto" );
8.          si_cambia( tf );
9.          System.out.println( tf.getText() );           // 'abc'
10.     }
```

En el ejemplo anterior, la línea 3 modifica efectivamente el objeto `TextField` original mediante la evocación de uno de sus métodos públicos.

Los arreglos son objetos, por lo tanto, son manipulados de la misma forma que el resto de instancias: a través de referencias. Entonces, de igual forma que en el ejemplo anterior, el contenido de un arreglo puede ser modificado usando alguno de sus métodos de interfaz

pública.^C

```
1.      void f()                                // C++
2.      {
3.          WaitCursor wc;
4.          ...
5.          if ( i < 0 )
6.          {
7.              return;
8.          }
9.          ...
```

```
1.      public void m()                          // Java
2.      {
3.          setCursor( Cursor.getPredefinedCursor(
4.              Cursor.WAIT_CURSOR ) );
5.          ...
6.          if ( i < 0 )
7.          {
8.              setCursor( Cursor.getPredefinedCursor(
9.                  Cursor.DEFAULT_CURSOR ) );
10.             return;
11.         }
```

^C Como se ha visto, el manejo de referencias en Java es similar al manejo de apuntadores en lenguaje C/C++. Al manejar todo como referencias y no existir la posibilidad de construir objetos automáticos, Java no permite aprovechar la pareja constructor-destructor para producir cerramientos funcionales.

10. ...

Una forma de evitar la limitación de que todos los argumentos sean pasados por valor, y simular el paso de argumentos por referencia, es pasarlos en arreglos de tamaño 1:

```
1.      public void modifica( int[] vi )
2.      {
3.          vi[ 0 ] = 0;
4.      }
5.      public static void main( String[] args )
6.      {
7.          int[] i = { -1 };
8.          modifica( i );
9.          System.out.println( "i=" + i[ 0 ] );           // i=0
10.     }
```

En el ejemplo, el vector es pasado por valor, pero sus elementos son pasados por referencia.

La mayoría de los lenguajes modernos permiten asignar memoria en tiempo de ejecución (memoria dinámica). En Java esto se hace utilizando el operador **new**.

El asunto de este tipo de asignación está en cuándo debe ser esta memoria retornada al sistema operativo. Algunos lenguajes como C/C++ requieren que el programador libere en forma explícita la memoria que ya no se necesita. Este enfoque ha probado ser muy proclive a producir errores: o la memoria es liberada demasiado temprano, produciendo referencias corruptas, o la memoria no es liberada en absoluto, produciendo pérdidas de memoria (memory leaks).

En Java no es necesario liberar explícitamente la memoria asignada: Java provee un mecanismo automático de recolección de memoria. La máquina virtual de Java lleva un registro de la memoria pedida y es capaz de determinar cuando un bloque de memoria ya no está en uso. Esta tarea es realizada por un hilo de ejecución de baja prioridad llamado 'garbage collector'. Cuando este hilo encuentra un bloque de memoria que ya no es referenciado por ningún hilo activo, procede a retornarlo al banco de memoria disponible.

La recolección de memoria en desuso se puede llevar a cabo de diferentes formas, cada una presentando ventajas y desventajas, según el tipo de programa que se está ejecutando. Un sistema de tiempo real, por ejemplo, necesita que nada le impida responder inmediatamente a una interrupción; esto requiere un sistema de recolección de memoria que utilice pequeños tiempos de procesador y que pueda ser interrumpido fácilmente. Por otro lado, un programa que utilice la memoria en forma intensiva requiere un sistema de recolección que de tanto en tanto interrumpa el programa y libere toda la memoria en desuso.

En el momento, el sistema de recolección de memoria está embebido en la máquina virtual de Java y utiliza un algoritmo que trata de equilibrar el compromiso entre desempeño del programa y velocidad de liberación de memoria. En el futuro, se podrá escoger el algoritmo de recolección acorde las necesidades del programa en particular.

Lo anterior implica que el tiempo en el que será liberado un bloque de

memoria que ya no está en uso es indeterminado. Existen métodos tales como `System.gc()` o `Runtime.gc()`, pero estas llamadas no garantizan la liberación inmediata de la memoria ya que otro hilo de ejecución puede prevenir la ejecución del recolector de memoria. De hecho la documentación del método `gc()` dice:

“La evocación de este método *sugiere* que la máquina virtual de Java dedique cierto esfuerzo al reciclaje de objetos en desuso”.

A priori parecería que con un sistema de recolección de memoria es imposible generar pérdidas de memoria. Sin embargo, por la propia naturaleza del sistema de recolección, es posible dejar referencias vivas a objetos que ya no se necesitan en el programa y que no serán borrados de memoria. A continuación se presenta una implementación inapropiada del método `pop()` de una pila:

```
1.      public Object pop()
2.      {
3.          return m_vector[ m_index-- ];
4.      }
```

Aún en caso en que el método que ha llamado a `pop()` deje de utilizar la referencia retornada, el objeto no será liberado hasta tanto no se asigne un nuevo valor a `m_vector[ m_index ]`. Esto puede tomar mucho tiempo. El proceso se puede agilizar de la siguiente forma:

```
1.      public Object pop()
2.      {
3.          Object obj = m_vector[ m_index ];
4.          m_vector[ m_index-- ] = null;
5.          return obj;
6.      }
```

Resumen

Existen tres elementos que pueden aparecer en un archivo fuente y deben estar en el orden siguiente:

1. **package** → declaración de un paquete de clases
2. **import** → inclusión de la definición de otras clases
3. **class** → definición de una o más clases

En un archivo puede aparecer como máximo una clase pública y en ese caso el archivo debe tener el nombre de la clase mas la extensión **.java**.

Un identificador debe comenzar con una letra, un signo de pesos o un underscore. Los caracteres siguientes pueden ser letras, signos de pesos, underscores o dígitos.

Los tipos primitivos para almacenar números enteros son **byte**, **short**, **int** y **long**. Los cuatro utilizan notación 2-complemento. Los tipos primitivos para almacenar números de punto flotante son **float** y **double**. El tipo **char** no tiene signo y representa un caracter Unicode. El tipo **boolean** solo puede tomar los valores **false** y **true**.

Para crear un arreglo se deben seguir los tres pasos siguientes:

1. declaración
2. construcción
3. inicialización

Java inicializa automáticamente las variables miembro y los elementos de los arreglos, pero no las variables automáticas. El valor por omisión es cero para los tipos numéricos, **null** para las referencias a objetos, el

caracter nulo para una variable de tipo **char** y **false** para **boolean**. El operador **.length** retorna el número de elementos en un arreglo.

Toda clase que tenga un método `main()` puede ser ejecutada como una aplicación desde la línea de comandos. El prototipo del método `main()` es:

```
public static void main( String[] args )
```

El arreglo 'args' contiene los argumentos pasados en la línea de comandos después del nombre de la clase.

En Java todos los parámetros son pasados por valor, esto es, copia de los originales. Para los tipos primitivos esto significa que las modificaciones realizadas a los argumentos en el método que los recibe no son visibles en método que los ha pasado. Para argumentos que son referencias a objetos o referencias a arreglos, las modificaciones tampoco son visibles en método que las ha pasado; sin embargo, las modificaciones al objeto referenciado o a elementos del arreglo, sí son visibles en el método que ha hecho la llamada.

El sistema de recolección de memoria de Java se encarga de recuperar automáticamente la memoria que ya no está en uso. Sin embargo, no es posible predecir en que momento esta memoria será liberada.

Test Yourself

1.	Un tipo entero con signo representa una cantidad igual de valores negativos y de valores positivos. a. Verdadero b. Falso
2.	Seleccione los identificadores válidos de la siguiente lista: a. Nombre01LargoSinSignificadoAlguno b. \$char c. floats d. \$12 e. finalist
3.	¿ Cuáles de los siguientes prototipos son válidos para el método <code>main()</code> de una clase pública? <code>public static void main()</code> <code>public static void main(<i>String</i> args[])</code> <code>public void main(<i>String</i>[] args)</code> <code>public static void main(<i>String</i>[] args)</code> <code>public static int main(<i>String</i>[] args)</code>
4.	Si todos aparecen en un archivo, ¿ En que orden deben aparecer los elementos siguientes? a. import, package, class b. class, import, package c. package primero, import y class no importa en que orden d. package, import, class e. import primero, package y class no importa en que orden

5.	Considere la siguiente línea de código: <pre>long x[] = new long[64];</pre> Una vez ejecutada, ¿ cuál aseveración o aseveraciones son verdaderas? a. x[63] es igual a 0 b. x[63] tiene un valor indefinido c. x[64] es igual a 0 d. x[0] es null e. x.length es igual a 64
6.	Considere la siguiente aplicación: <pre>class A { public void inc(A a) { a.m_valor++; } int m_valor; } public class B { public static void main(String[] args) { A a = new A(); a.m_valor = 25; } }</pre>

	<pre> a.inc(a); System.out.println(a.m_valor); } } </pre> ¿Cuál es el valor desplegado? a. 0 b. 1 c. 25 d. 26
7.	Considere la siguiente aplicación: <pre> class A { public void dec(double d) { d -= 1.0; } } public class B { public static void main(String[] args) { double d = 10.0; A a = new A(); a.dec(d); System.out.println(d); } } </pre>

	<pre> } } </pre> ¿Cuál es el valor desplegado? 0.0 -1.0 10.0 9.0
8.	¿ De qué forma se puede forzar la recolección de un objeto en desuso? - No se puede - Llamando System.gc() - Llamando System.gc() y pasando una referencia al objeto - Llamando Runtime.gc() - Asignando null a todas las referencias al objeto
9.	¿Cuál es el rango de valores que se pueden asignar a una variable de tipo long? - Depende de la plataforma 0 a 2^{32} 0 a $2^{32}-1$ -2^{64} a $2^{64}-1$ 0 a $2^{64}-1$ -2^{63} a $2^{63}-1$
10.	¿Cuál es el rango de valores que se pueden asignar a una variable de tipo short? - Depende de la plataforma 0 a 2^8 -2^{15} a $2^{15}-1$

d. -2^{16} a $2^{16}-1$
e. 0 a $2^{16}-1$
f. 0 a $2^{15}-1$

Capítulo	2
Título	Operadores

Este capítulo cubre aspectos relacionados con los siguientes objetivos del Examen de Certificación en Java:

- Determinar el resultado de aplicar un operador, incluyendo el operador de asignación e **instanceof**, a operandos de cualquier tipo, clase, alcance o accesibilidad, o cualquier combinación de éstos
- Determinar el resultado de aplicar el método **boolean equals(Object)** a cualquier combinación de las clases *java.lang.String*, *java.lang.Boolean* y *java.lang.Object*.
- En una expresión que involucra los operadores **&**, **|**, **&&**, **||** y variables con valores conocidos, determinar el orden de evaluación de los operadores y el resultado de la expresión.

Java provee un conjunto completo de operadores, la mayoría tomados directamente de C/C++. Sin embargo, algunos difieren sustancialmente de sus correspondientes en estos lenguajes y es necesario conocer estas diferencias. Este capítulo describe todos los operadores, unos brevemente, otros que pueden ser confusos, en detalle. También se estudiará el comportamiento de expresiones en condiciones de rebosamiento (overflow). Los operadores de Java se listan en la siguiente tabla y están en orden de precedencia. Se han agrupado para obtener mayor claridad.

Unarios	++ -- + - ! ~ ()
Aritméticos	* / % + -
Corrimiento	<< >> >>>
Comparación	< <= > >= instanceof == !=
Bit a Bit	& ^
Cortocircuito	&&
Ternario	?:
Asignación	= "op="

Tabla 2.1 Operadores en Java

Orden de Evaluación

En Java, a diferencia de otros lenguajes, el orden aparente (?) de evaluación de los operandos de una expresión es fijo. Específicamente, todos los operadores son evaluados de izquierda a derecha, a pesar de que el orden de ejecución de las operaciones es algo diferente. Esto es notorio en el caso de las asignaciones. Considere el siguiente ejemplo:

```
1.      int[] a = { 4, 4 };  
2.      int b = 1;  
3.      a[ b ] = b = 0;
```

En este caso no es claro cuál de los elementos del arreglo es modificado. De otra manera, ¿cuál es el valor de b que se usa como índice del arreglo, 0 o 1? El orden de evaluación de izquierda a derecha implica que la expresión `a[ b ]` sea evaluada primero. Luego b, que una referencia a la variable b, es evaluada, y por último expresión constante 0 cero es evaluada, lo que obviamente no implica trabajo alguno. Una vez que los operandos han sido evaluados se llevan a cabo las operaciones. Esto se hace de acuerdo a las reglas de precedencia y asociatividad. Para las asignaciones, la asociatividad es derecha a izquierda, por lo tanto 0 es asignado a b y luego 0 es asignado al último elemento (`a[ 1 ]`) del arreglo.

Operadores Unarios

El primer grupo de operadores en la Tbla 2.1 es el de los operadores unarios. Los operadores unarios, a diferencia de la mayoría de operadores que requieren de dos operandos, como era de esperar se aplican a un solo operando. En Java existen 7 operadores unarios:

++ --	incremento y decremento
+ -	positivo y negativo
~	inversión de bits
!	complemento booleano
()	conversión de tipo (casting)*

Los operadores de incremento y decremento modifican el valor de una expresión sumando o restando 1 respectivamente. De esta manera si una variable *x* de tipo **int** contiene 1, entonces ++*x* resulta en 2 y --*x* resulta en 0. En ambos casos el valor resultante es almacenado en *x*.

En el ejemplo anterior los operadores son colocados antecediendo a la variable. Estos operadores también pueden ir después del nombre de la variable. Para entender como la posición del operador afecta el resultado de una operación, es necesario entender la diferencia que hay entre el valor almacenado en la variable y el resultado que arroja la expresión. En ambos casos, ++*x* ó *x*++, el valor almacenado en *x* es el mismo. Sin embargo, si esto hace parte de una expresión tal como

y = ++*x*;

o

y = *x*++;

* Estrictamente hablando, () no es un operador. Sin embargo, por simplicidad será discutido en esta sección.

el resultado es totalmente diferente. En el primer caso, el valor asignado a y es 0, y luego x es incrementado. En el segundo caso, x es incrementado y luego el valor asignado a y es 1.

Esto es, si alguno de estos operadores es utilizado a la izquierda de una expresión, el valor de la expresión es modificado *antes* de tomar parte en el cálculo. En este caso se llama pre-incremento o pre-decremento según el operador utilizado. En caso contrario, si el operador es utilizado a la derecha de una expresión, el valor resultante es calculado usando el valor *original* de la expresión.

valor inicial de x	expresión	valor final de y	valor final de x
1	y = x++	1	2
1	y = ++x	2	2
1	y = x--	1	0
1	y = --x	0	0

Tabla 2.2 Ejemplos de pre y post incremento/decremento

Los operadores unarios '+' y '-' son diferentes de los operadores binarios de suma y resta, '+' y '-'. El operador unario '+' no tiene otro efecto diferente al de enfatizar la naturaleza positiva del literal al que es aplicado. El operador unario '-' niega el valor de una expresión.

El operador de inversión de bits '~' se aplica a tipos enteros e invierte bit a bit el valor de una expresión (bit en 0 a 1 y bit en 1 a 0). Para los tipos primitivos, Java utiliza la representación que provee máquina virtual y que es independiente de la plataforma. Esto significa que el patrón de bits utilizado para representar un valor determinado, almacenado en una variable de un tipo determinado, es siempre el mismo. Esto hace que los operadores para la manipulación de bits sean aún más útiles, ya que no introducen ninguna dependencia respecto a la plataforma.

El operador de complemento booleano '!' invierte el valor de una expresión booleana. Por lo tanto **!false** resulta en **true** y **!true** resulta

en **false**. Este operador es generalmente utilizado en cláusulas **if()** de la siguiente forma:

```
1.      public Object m( Object x )
2.      {
3.          if ( x instanceof String )
4.          {
5.          }
6.          else
7.          {
8.              x = x.toString();
9.          }
10.         return x;
11.     }

12.     public Object m( Object x )
13.     {
14.         if ( !(x instanceof String) )
15.         {
16.             x = x.toString();
17.         }
18.         return x;
19.     }
```

El operador de conversión de tipo o 'casting' es utilizado en la conversión explícita del tipo de una expresión. Esto es posible solo para ciertos tipos de destino. Tanto el compilador como la máquina virtual realizan las verificaciones necesarias para asegurar el

cumplimiento de las reglas de conversión de tipos. Estas reglas serán descritas más adelante en el capítulo 4.

<pre>1. double x = 2.0; 2. int i = (int)(x*x);</pre>

Si en la línea 2 se omite la conversión (**int**), el compilador reporta un error de asignación: “Explicit cast needed to convert double to int.”. Esto porque el resultado de multiplicar dos números de punto flotante no puede ser representado con precisión en una variable de tipo entero.

La conversión de tipo puede ser aplicada también a referencias a objetos. Esto sucede usualmente cuando se utilizan objetos contenedores tales como una instancia de la clase *Vector*. Esta clase esta construída para almacenar referencias a objetos *Object*: por lo tanto, si se almacenan, por ejemplo, referencias a objetos *String* (lo cual es posible porque la clase *String* es descendiente de *Object*), cuando se utiliza el método `elementAt()` para recuperar un elemento, la referencia retornada es una referencia de tipo *Object* y si se quiere utilizar como referencia a *String* es necesario convertirla explícitamente.

<pre>1. <i>Vector</i> v = new <i>Vector</i>(); 2. v.addElement(“abc”); 3. <i>String</i> str = (<i>String</i>)v.elementAt(0);</pre>
--

A pesar de que el compilador acepta la conversión, en tiempo de ejecución la máquina virtual verifica que el objeto extraído es efectivamente un objeto de tipo *String*.

Operadores Aritméticos

Ahora que se han considerado los operadores unarios, que son los de mayor precedencia, se discutirán los operadores aritméticos. Estos operadores a su vez están divididos en dos grupos: el primero de mayor precedencia compuesto de '*', '/' y '%', y el segundo de menor precedencia compuesto de '+' y '-'.

Los operadores '*' y '/', de multiplicación y división respectivamente, operan sobre todos los tipos primitivos numéricos y sobre el tipo **char**. La división entera puede generar una excepción *ArithmeticException* en caso de división por cero.

Estas operaciones, multiplicación y división, presentan ciertas restricciones derivadas de la capacidad limitada de representación de números en un computador. Estas limitaciones se aplican a todos los tipos numéricos, desde **byte** hasta **double**, pero son más notorias en los tipos enteros.

Si se multiplican o dividen dos enteros, el resultado será calculado utilizando aritmética entera en representación **int** o **long**. Si los operandos contienen valores lo suficientemente grandes, el resultado será más grande que el máximo valor que puede ser representado en un tipo dado, arrojando un valor que no tiene significado alguno.

Por ejemplo, el rango de valores que puede ser almacenado en una variable de tipo **byte** es de -128 a +127. Si se multiplica 64 por 4, el resultado 256 (1 0000 0000) será almacenado en un **byte** como 0 ya que solo los ocho bits menos significativos pueden ser representados.

Por otra parte, cuando se utiliza división entera el resultado es forzado a un número entero, perdiéndose de esta forma la parte fraccionaria y por lo tanto precisión. Por ejemplo, al dividir $3/4=0.75$, el resultado almacenado es 0.

Por lo general, a pesar de que existe el riesgo de rebosamiento, en una expresión que involucra multiplicación y división, es preferible multiplicar primero y dividir después:

```
1.    int x = 123;
2.    int y = 456;
3.    int a = (y*x)/y;
4.    int b = y*(x/y);           // y*0
5.    System.out.println( "a=" + a );    // a=123
6.    System.out.println( "b=" + b );    // b=0
```

```
1.    int x = 1234567890;
2.    int y = 1234567890;
3.    int a = (y*x)/y;           // overflow
4.    int b = y*(x/y);
5.    System.out.println( "a=" + a );    // a=0
6.    System.out.println( "b=" + b );    // b=1234567890
```

El operador módulo '%', da como resultado el resto de la división entre dos números. Se aplica generalmente a números enteros pero puede ser aplicado también a números de punto flotante.

```
1.    int x = 12;
2.    int y = 5;
3.    int r = x % y;
4.    System.out.println( "r=" + r );    // r=2
```

A continuación se presentan varios ejemplos relacionados con operandos con valores negativos o de punto flotante.

```
1.      int x = -12;  
2.      int y = 5;  
3.      int r = x % y;  
4.      System.out.println( "r=" + r );      // r=-2
```

```
1.      int x = 12;  
2.      int y = -5;  
3.      int r = x % y;  
4.      System.out.println( "r=" + r );      // r=2 (!)
```

En el caso anterior se muestra que el signo del resultado está determinado por el signo del operando de la izquierda de la operación módulo, esto es, el signo del operando de la derecha es irrelevante.

A continuación se presenta un ejemplo de uso del operador módulo con números de punto flotante

```
1.      double x = -12.6;  
2.      double y = -5.1;  
3.      double r = x % y;  
4.      System.out.println( "r=" + r );      // r=-2.400000
```

El operador módulo lleva a cabo una división y por lo tanto puede arrojar una excepción de tipo *ArithmeticException* cuando se aplica entre números enteros y el denominador es cero.

Los operadores '+' y '-' realizan la suma y resta de dos números respectivamente. Hay que tener en cuenta, sin embargo, que puede haber cierta promoción de tipo según las reglas normales y a que aún así puede existir desbordamiento. En caso de desbordamiento o pérdida de precisión (overflow/underflow) en sumas o restas el

resultado carece de significado pero no es arrojada excepción alguna.

Adicionalmente, el operador '+' se puede utilizar para concatenar dos cadenas de caracteres representadas en objetos de tipo *String* o para producir un nuevo objeto de tipo *String* cuando uno de los operandos es de tipo *String* y el otro no lo es^c. En este caso, el operando que no es de tipo *String* es convertido a una cadena de caracteres utilizando el método *toString()*. Este método está definido en la clase **java.lang.Object** que es la superclase de todas las clases en Java y por lo tanto toda clase lo tiene.

```
1.      int x = 12;
2.      String s = "El valor es ";
3.      System.out.println( s + x );      // El valor es 12
```

En ciertos casos como el anterior, la conversión se lleva a cabo en forma indirecta: la variable de tipo **int** primero es encapsulada en un objeto de tipo *Integer* y luego convertida mediante el método estático *Integer.toString()*.

Para aplicar un formato determinado a la conversión se deben utilizar las utilidades contenidas en el paquete **java.text**.

Para la suma de dos operadores de tipos numéricos primitivos se aplican las siguientes reglas:

- ❑ El resultado es de tipo primitivo
- ❑ El resultado es por lo menos de tipo **int**
- ❑ El resultado es por lo menos del tipo de mayor tamaño (en bits) de los operandos
- ❑ El resultado es calculado promoviendo los operandos al tipo de la variable resultado y luego realizando la operación con el nuevo tipo

^c Java no permite la sobrecarga de operadores como en lenguaje C++.

Condiciones de Error Aritmético

Dado que las operaciones aritméticas son llevadas a cabo en una máquina que posee una capacidad limitada para representar valores numéricos, en ciertos casos se producen errores.

En caso de desbordamiento, pérdida de precisión u otra condición anómala se aplican las siguientes reglas:

- ❑ En caso de división entera por cero, incluyendo módulo, se genera una excepción *ArithmeticException*
- ❑ Ninguna otra operación genera una excepción: la operación produce un resultado a pesar de que éste pueda ser aritméticamente incorrecto
- ❑ Los valores fuera de rango tales como infinito, menos infinito y no-número (Not a Number/NaN), resultado de cálculos de punto flotante, son representados utilizando según la norma IEEE-754. Las constantes mnemónicas para estos valores se encuentran definidas en las clases **float** y **double**.
- ❑ Los cálculos con valores enteros, diferentes de la división por cero, que producen desbordamiento o un error similar, simplemente dejan el patrón de bits resultante de truncar el valor. Este puede inclusive ser de signo erróneo. Dado que la representación de números y las operaciones entre éstos son independientes de la plataforma en que se ejecutan, los resultados también son independientes de la plataforma.

Algunas operaciones de punto flotante pueden resultar en NaN, por ejemplo como resultado de calcular la raíz cuadrada de un número negativo. Dos valores (patrones de bits) de NaN son definidos en el paquete **java.lang**: *Float.NaN* y *Double.NaN*. Estas constantes no son consideradas ordinales en las comparaciones, esto es, para cualquier valor de x, incluyendo NaN, las siguientes comparaciones resultan todas en **false**:

1. $x < \text{Float.NaN}$
2. $x \leq \text{Float.NaN}$
3. $x == \text{Float.NaN}$
4. $x > \text{Float.NaN}$
5. $x \geq \text{Float.NaN}$

Como se puede deducir de la línea 3., las siguientes comparaciones resultan en **true**:

1. $\text{Float.NaN} != \text{Float.NaN}$
2. $\text{Double.NaN} != \text{Double.NaN}$

La manera apropiada de evaluar si el resultado de una operación es NaN es utilizar los métodos (estáticos) *Float.isNaN(**float** f)* y *Double.isNaN(**double** lf)*.

Operadores de Corrimiento

<<	corrimiento a la izquierda (con signo)
>>	corrimiento a la derecha (con signo)
>>>	corrimiento a la derecha (sin signo)

Java ofrece tres operadores de corrimiento de bits. Una ventaja de que la representación binaria de todos los tipos en Java esté predefinida, y sea independiente de la plataforma en uso, radica en que estos operadores se comportan de manera uniforme a través de las diferentes combinaciones máquina/ambiente operativo^c.

Con esto, Java provee un soporte multi-plataforma a muchas operaciones comunes en software de control o comunicaciones (e/s a través de puertos), computación gráfica y otros. Los operadores de corrimiento son utilizados también para realizar multiplicaciones y divisiones por números potencias de dos en forma rápida.

De hecho, la operación de corrimiento es una operación sencilla: se toma el patrón de bits en una variable numérica de tipo entero y se mueve a la izquierda o derecha. Sólo pueden ser aplicados a números de tipo entero y como se verá más adelante, sólo deberían ser aplicados a números de tipo **int** o **long**.

La siguiente figura muestra el mecanismo de corrimiento de bits:

^c No se puede decir lo mismo del código escrito en C/C++.

valor		binario					decimal
x		00000000	00000000	00000000	00000011		3
x << 1	0	00000000	00000000	00000000	00000110		6
x >> 1		00000000	00000000	00000000	00000001	0	1
x >>> 1		00000000	00000000	00000000	00000001	0	1

valor		binario					decimal
x		11111111	11111111	11111111	11111101		-3
x << 1	1	11111111	11111111	11111111	11111010		-6
x >> 1		11111111	11111111	11111111	11111110	1	-2
x >>> 1		01111111	11111111	11111111	11111101	0	2147483645

En todos los casos, los bits que salen del espacio reservado para la variable son descartados.

En el caso de corrimiento a la izquierda (<<) o a la derecha sin signo (>>>), los bits que entran son iguales a cero. En el caso de corrimiento a la derecha con signo (>>), los bits que entran son iguales al bit más significativo antes del corrimiento, preservando por lo tanto el signo original.

Capítulo 3: MODIFICADORES

Objetivos

Este capítulo cubre los siguientes aspectos del examen de certificación de Java:

1. Declarar clases, clases internas, los métodos, variables de instancia, variables estáticas y variables automáticas (método local), haciendo uso apropiado de todos los modificadores permitidos (tales como public, final, static, abstract, entre otros). Definir la importancia de cada uno de estos modificadores individualmente y en combinación, y definir el efecto de relaciones de paquete (package) sobre elementos declarados calificados por esos modificadores.

2. Identificar correctamente archivos fuente, declaraciones de paquete, estamentos import, declaraciones de clase (de todas las formas, incluyendo las clases internas), declaraciones de interfaz e implementaciones (para Java.lang.Runnable u otras interfaces descritas en el test), declaraciones de método (incluyendo el método main que se usa para comenzar la ejecución de una clase), declaraciones de variables, e identificadores.

Modificadores son palabras clave de Java que dan al compilador información acerca de la naturaleza del código, de los datos o de las clases. Los modificadores especifican, por ejemplo, que una característica particular es estática, final o transiente. (Una característica es una clase, un método, o una variable). Un grupo de modificadores llamados *modificadores de acceso*, determinan cuáles clases tienen permiso para usar una característica. Otros modificadores pueden ser usados en combinación para describir los atributos de una característica.

Este capítulo tratará los modificadores de Java que se aplican a clases de nivel superior (top-level). Las clases internas no serán discutidas, pero se cubrirán en el capítulo 6.

Los modificadores más comunes son los modificadores de acceso: public, protected y private. Los modificadores de acceso se verán en la siguiente sección. Los modificadores restantes no están claramente definidos dentro de categorías. Ellos son:

final
abstract
static
native
transient
synchronized
volatile

Cada uno de esos modificadores son discutidos en su propia sección.

Modificadores de Acceso

Los modificadores de acceso sirven para controlar cuáles clases pueden usar una característica. Una característica es:

la clase misma
sus variables de clase
sus métodos y constructores

Con raras excepciones, las únicas variables que pueden ser controladas por los modificadores son las variables de nivel de clase (class-level). Las variables declaradas dentro de los métodos de las clases pueden no tener modificadores de acceso; esto es comprensible porque una variable de método sólo puede ser usada dentro de ese método.

Los modificadores de acceso son:

public

protected

private

El único modificador de acceso permitido en una clase no interna es public; no existe algo como una clase de nivel superior que sea protected o private.

Una característica puede tener al menos un modificador de acceso. Si una característica no tiene un modificador de acceso, su acceso por defecto es "friendly". Friendly no es una palabra reservada de Java, sólo es el nombre usado cuando no se especifica ningún modificador de acceso.

Las siguientes declaraciones son todas legales (asumiendo que ellas aparecen en un contexto apropiado):

```
class Parser { ... }
public class EightDimensionalComplex { ... }
private int i;
Graphics offScreenGC;
protected double getChiSquared() { ... }
private class Horse { ... }
```

Las siguientes declaraciones son ilegales:

```
public protected int x; //Se permite a lo más 1 modificador
friendly Button getBtn() { ... } //"friendly" no es un modificador
```

public

El más generoso de los modificadores de acceso es public. Una clase, variable o método público puede ser usada por cualquier programa de Java sin restricción. Un Applet (esto es, una subclase modificada de la clase java.applet.Applet) es declarado como clase pública para que pueda ser instanciado por los navegadores. Una aplicación declara su método main() como público, así que el método main() puede ser invocado desde cualquier ambiente de ejecución de Java.

private

El menos generoso modificador de acceso es private. Las clases de nivel superior (esto es, no internas) no pueden ser declaradas como privadas. Una variable o método privado sólo puede ser usado por una instancia de la clase que declara ese método o variable. En el siguiente código se considera un ejemplo de acceso privado:

```

1. class Complex {
2. private double real, imaginary;
4. public Complex(double r, double i) { real = r; imaginary = i; }
6. public Complex add(Complex c) {
7. return new Complex(real + c.real, imaginary + c.imaginary);
8. }
9. }
10.
11.
12. class Client {
13. void useThem() {
14. Complex c1 = new Complex(1, 2);
15. Complex c2 = new Complex(3, 4);
16. Complex c3 = c1.add(c2);
17. double d = c3.real; // ilegal!
18. }
19. }

```

En la línea 16, se hace un llamado a `c1.add(c2)`. El objeto `c1` ejecutará el método usando el objeto `c2` como parámetro. En la línea 7, `c1` accede a sus propias variables privadas tanto como a las de `c2`. Ahí no hay nada incorrecto. La declaración de `real` e `imaginary` como privadas significa que ellas sólo pueden ser accesadas por instancias de la clase `Complex`, pero ellas pueden ser accesadas por cualquier instancia de `Complex`. Así `c1` puede acceder a sus propias variables `real` e `imaginary`, tan bien como a las `real` e `imaginary` de cualquier otra instancia de `Complex`. Los modificadores de acceso definen cuáles clases (no cuáles instancias) pueden acceder a las características.

La línea 17 es ilegal y causará un error de compilación. El mensaje de error dice: "Variable `real` in class `Complex` not accessible from class `Client`". La variable privada `real` puede ser accesada solamente por una instancia de `Complex`.

Los datos privados pueden ocultarse desde el mismo objeto al que pertenece el dato. Si la clase `Complex` tiene una subclase llamada `SubComplex`, entonces cada instancia de `SubComplex` heredarán sus propias variables `real` e `imaginary`. Sin embargo ninguna instancia de `SubComplex` podrá acceder a esas variables. De nuevo, las características privadas de `Complex` solamente pueden ser accesadas por instancias de `Complex`; una instancia de una subclase tiene acceso denegado. Así por ejemplo, el siguiente código no compilará:

```

1. class Complex {
2. private double real, imaginary;
3. }
4.
5.
6. class SubComplex extends Complex {
7. SubComplex (double r, double i) {
8. real = r; // problema!

```

```
9. }  
10 }
```

En el constructor para la clase SubComplex (en la línea 8), la variable real es accesada. Esta línea causa un error de compilación con un mensaje que es muy similar al mensaje del ejemplo previo: "Undefined variable: real.". La naturaleza privada de la variable real impide a una instancia de SubComplex acceder a una de sus propias variables!

friendly (amigable)

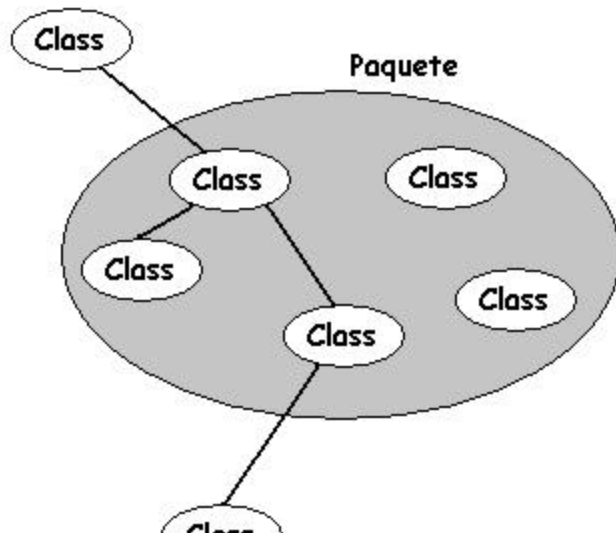
Friendly es el nombre del acceso por defecto a clases, variables y métodos, si no se especifica un modificador de acceso. Un dato de clase y los métodos pueden ser amigables al igual que la clase misma. Las características amigables de clase son accesibles a cualquier clase en el mismo paquete (package) de la clase en cuestión.

Friendly no es una palabra reservada de Java, es el nombre dado al nivel de acceso que resulta cuando no se especifica un modificador de acceso.

Podría pensarse que el acceso amigable es de interés sólo para quienes hacen paquetes. Esto es técnicamente cierto, pero actualmente cualquiera está haciendo paquetes sin darse cuenta. Java considera que todos los archivos que están en el directorio de trabajo constituyen un paquete.

Si deliberadamente desarrolla su propio paquete sin dar modificadores de acceso a sus clases, se requiere un trabajo extra: Debe poner una declaración de paquete en el código fuente y debe compilar con la opción -d. Cualquier característica de las clases del paquete que no se marcaron explícitamente con un modificador de acceso será accesible a todos los miembros del paquete, lo cual es probable que sea eso lo que desee.

Las clases que están por fuera del paquete no pueden tener acceso a las características amigables. Clases por fuera del paquete pueden tener subclases dentro del paquete (algo así se puede obtener cuando se crean applets, por ejemplo.) y esas subclases no podrán acceder a las características amigables. La siguiente figura ilustra el acceso amigable en ambos casos: dentro y fuera de un paquete.



protected

El término protegido (protected) es un poco engañoso, podría pensarse que es extremadamente restrictivo, quizás más que el acceso privado. De hecho las características protegidas son más accesibles que las características amigables.

Sólo las variables, los métodos y las clases internas pueden ser declaradas protegidas. Una característica protegida de una clase está disponible para todas las clases en el mismo paquete (como una característica amigable), y más aun, una característica protegida de una clase está disponible a todas las subclases de la clase que define la característica protegida. Este acceso está dado incluso para subclases que residen en un paquete diferente de la clase que define la característica protegida. El acceso protegido establece un nivel intermedio de protección entre el acceso público y el privado.

Ejemplo:

```
1. package sportinggoods;  
2. class Ski {  
3. void applyWax() { ... }  
4. }
```

El método applyWax() tiene acceso amigable. Ahora considere la siguiente subclase:

```
1. package sportinggoods;  
2. class DownhillSki extends Ski {  
3. void tuneup() {  
4. applyWax();  
5. // más código de tuneup  
6. }  
7. }
```

La subclase llama al método heredado applyWax(). No hay problema ya que ambas clases Ski y DownhillSki residen en el mismo paquete. Si cualquiera de las dos clases se ubicara en un diferente paquete, DownhillSki no tendría acceso al método heredado applyWax() y la compilación fallaría. El problema se soluciona haciendo protegido a applyWax():

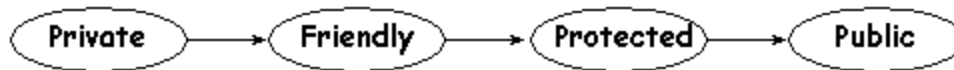
```
1. package unPaqueteDiferente; // la clase Ski está ahora en un paquete distinto  
2. class Ski {  
3. protected void applyWax() { ... }  
4. }
```

Privacidad de métodos y subclases

Java especifica que los métodos no pueden sobrescribirse para ser más privados. Por ejemplo, muchos applets proveen un método init() el cual sobrescribe la versión que

no hace nada heredado de la superclase `java.applet.Applet`. La versión heredada es pública, entonces la declaración en la versión de la subclase para que sea privada, protegido o amigable resultará en un error compilación. El mensaje de error dice: "Methods can't be overridden to be more private"

La siguiente figura muestra los tipos accesos legales para subclases. Un método con algún tipo de acceso particular puede ser sobrescrito por un método con un diferente tipo de acceso si (en la figura) hay un camino desde el tipo original hasta el nuevo tipo:



Las reglas para sobreescritura pueden ser resumidas como sigue:

Un método privado puede ser sobrescrito a un método privado, amigable, protegido o público.

Un método amigable puede ser sobrescrito a un método amigable, protegido o público.

Un método protegido puede ser sobrescrito a un método protegido o público.

Un método público sólo puede ser sobrescrito a un método público.

Resumen de los modos de acceso

public: Una característica pública puede ser accesada por cualquier clase.

protected: Una característica protegida puede ser accesada solamente por subclases de la clase que define la característica o por miembros del mismo paquete al que pertenece la clase que define la característica.

friendly: Una característica amigable puede ser accesada solamente por una clase del mismo paquete al que pertenece la clase que define la característica.

private: Una característica privada sólo puede ser accesada por la clase que define la característica.

Otros modificadores

El resto de este capítulo cubre otros modificadores de Java: `final`, `abstract`, `static`, `native`, `transient`, `synchronized` y `volatile`. (`transient` y `volatile` no son mencionados en los objetivos del examen de certificación, así que serán tratados brevemente.)

Java no tiene en cuenta el orden de aparición de los modificadores. Declarar una clase como `public final` no es diferente que declararla `final public`.

No todos los modificadores pueden ser aplicados a cada tipo de características, la tabla al final del capítulo muestra cuáles modificadores pueden aplicarse a cuáles características.

final

Aplica a clases, métodos y variables. El significado de `final` varía de acuerdo al

contexto, pero la idea esencial es la misma: las características finales no pueden ser cambiadas.

Una clase final no puede tener subclases. Por ejemplo:

```
Class.Sub.Math extends java.lang.Math { }
```

El error de compilación dice: "Can't subclass final classes." El código no compila porque una clase final no puede tener subclases y en este caso `java.lang.Math` es una clase final.

Una variable final no puede modificarse una vez se le asigne un valor. Una variable declarada como final no puede cambiar su valor a lo largo de la ejecución del programa. Puede ser considerada como una constante y equivale a la palabra `const` de C/++

Si una variable final hace una referencia a un objeto, es ésta referencia la que debe permanecer igual, no el objeto. Esto se muestra en el siguiente código:

```
1. class Walrus {  
2. int weight;  
3. Walrus(int w) { weight = w; }  
4. }  
5.  
6. class Tester {  
7. final Walrus w1 = new Walrus(1500);  
8. void test() {  
9. w1 = new Walrus(1400); // ilegal  
10. w1.weight = 1800; // legal  
11. }  
12. }
```

Aquí la variable final es `w1`, declarada en la línea 7. Ya que ella es final, `w1` no puede recibir un nuevo valor; la línea 9 es ilegal. Sin embargo, el dato dentro de `w1` no es final, y la línea 10 es perfectamente legal. En otras palabras,

No se puede cambiar una variable final de referencia a objeto.

Se puede cambiar el dato de un objeto que es referenciado por una variable final de referencia a objeto.

Un método final no puede ser sobrescrito. Por ejemplo el siguiente código no compilará:

```
1. class Mammal {  
2. final void getAround () { }  
3. }  
4.  
5. class Dolphin extends Mammal {  
6. void getAround() { }
```

7. }

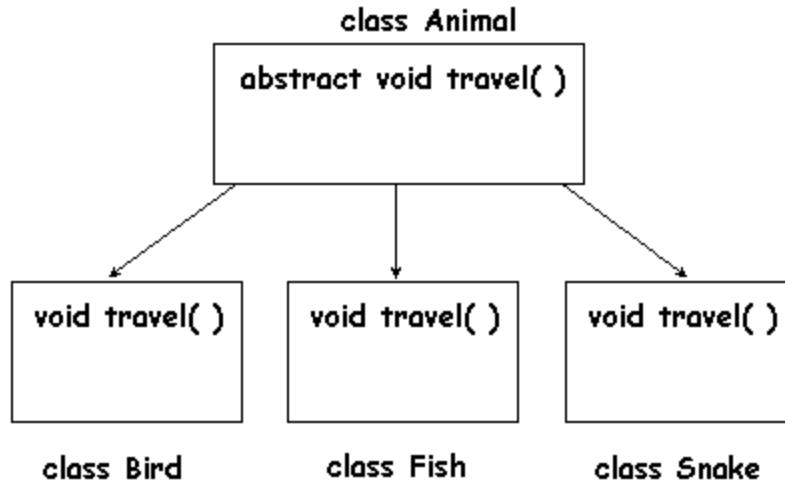
La clase Dolphin trata de sobrescribir el método heredado getAround(), pero getAround() es final, así que el único resultado es un error de compilación en la línea 6 que dice: "final methods can't be overridden."

abstract

El modificador abstract se aplica a clases y a métodos. Las clases abstractas no pueden ser instanciadas (Esto es no puede llamarse a su constructor).

Una clase abstracta provee un modo de delegar la implementación a las subclases.

Considere la jerarquía de clases mostrada en la siguiente figura:



El diseñador de la clase Animal ha decidido que cada subclase debería tener un método travel(). Cada subclase tiene su propio y único modo de viajar, así que no es posible proveer travel() en la superclase y tener que cada subclase herede la misma versión. En vez de eso, la superclase Animal declara travel() como abstracta. La declaración es:

```
abstract void travel();
```

Al final de la línea está el punto y coma donde se esperaba encontrar los corchetes conteniendo el cuerpo del método. El cuerpo del método - su implementación - es delegada a las subclases. La superclase solamente provee el nombre y la signatura del método. Cualquier subclase de Animal debe proveer una implementación de travel() o declararse ella misma como abstracta. En el último caso, la implementación de travel() es delegada de nuevo a una subclase de la subclase.

Si una clase tiene uno o más métodos abstractos, debe declararse abstracta. Mirar la declaración de la clase indica si está permitido instanciar la clase directamente o si hay

que contruir una subclase.

El compilador insiste que una clase debe ser declarada abstracta si cualquiera de las siguientes condiciones es cierta:

La clase tiene uno o más métodos abstractos

La clase hereda uno o más métodos abstractos para los cuales no provee implementación.

La clase declara que implementa una interfaz pero no provee implementación para cada método de esa interfaz

Esas condiciones son muy similares entre sí. En cada caso, en cierto sentido la clase está incompleta.

En cierto modo, el abstract es el contrario de final. Una clase final no puede tener subclases; una clase abstracta *debe* tener subclases.

static

Se aplica a variables, métodos y a ciertos códigos que no forman parte de un método. Las variables estáticas se utilizan para definir constantes comunes para todos los objetos de la clase o variables que sólo tienen sentido para toda la clase. Las variables estáticas son lo más parecido a las variables globales de C++ . Las variables estáticas se inicializan siempre antes que cualquier objeto de la clase.

Se puede pensar que las características státicas pertenecen a una clase, más que a una instancia específica de la clase.

Ejemplo:

```
1. class Ecstatic {  
2. static int x = 0;  
3. Ecstatic () { x++; }  
4. }
```

La variable x es estática, esto significa que solo hay una x, no importa cuantas instancias de la clase Ecstatic puedan existir en cualquier momento particular. La inicialización de una variable estática tiene lugar en el tiempo de cargue de la clase (class-load time). La variable estática es incrementada cada vez que el constructor es llamado, así es posible saber cuántas instancias se han creado.

Existen 2 maneras para referenciar una variable estática:

Por medio de una referencia a cualquier instancia de la clase
Por medio del nombre de la clase

El primer modo funciona, pero puede generar código confuso y es considerado un mal modo. El siguiente ejemplo muestra por qué:

1. Ecstatic e1 = new Ecstatic ();
2. Ecstatic e2 = new Ecstatic ();
3. e1.x = 100;
4. e2.x = 200;
5. reallyImportantVariable = e1.x;

Si no se sabía que x es estática, se puede pensar que reallyImportantVariable tendrá el valor de 100 en la línea 5. De hecho ella toma el valor 200, porque e1.x y e2.x hacen referencia a la misma variable (estática).

Un mejor modo para referenciar a una variable estática es por medio del nombre de la clase:

1. Ecstatic e1 = new Ecstatic ();
2. Ecstatic e2 = new Ecstatic ();
3. Ecstatic.x = 100;
4. Ecstatic.x = 200;
5. reallyImportantVariable = Ecstatic.x;

Ahora es claro que la línea 3 es inservible y el valor de reallyImportantVariable es 200 en la línea 5. Referir las características estáticas por medio del nombre de la clase es mejor que por medio de la instancia, dando como resultado un código más claro que describe lo que pasa en tiempo de ejecución.

Los métodos también pueden ser declarados estáticos. Los métodos estáticos pueden acceder sólo a variables estáticas y a otros métodos estáticos. Los métodos estáticos no pueden usar características no estáticas de su clase (aunque ellos pueden acceder a datos estáticos de la clase y llamar a otros métodos estáticos). Así los métodos estáticos no tienen que ver con instancias individuales de una clase. Ellos pueden ser invocados antes de que una instancia simple de la clase sea construida. Cada aplicación de Java es un ejemplo, porque estas contienen un método main () que es estático:

1. class SomeClass {
2. static int i = 48;
3. int j = 1;
- 4.
5. public static void main (string args []) {
6. i + = 100;
7. // j *=5; Por suerte esta línea está comentada!
8. }
9. }

Cuando la aplicación empieza, ninguna instancia de la clase SomeClass existe. En la línea 6, la variable i que es incrementada es estática, así que ella existe aunque no hay instancias. La línea7 podría dar como resultado un error de compilación si no estuviera comentada porque j no es estática.

Un ejemplo típico de métodos static son los métodos matemáticos de la clase

java.lang.Math (sin(), cos (ln exp ()), etc). Generalmente el argumento de estos métodos será de un tipo primitivo y se le pasará como argumento explícito. Los métodos estáticos son lo más parecido que java tiene a las funciones globales de C/++.

Los métodos tienen visibilidad directa de las variables miembro del objeto que es su argumento implícito, es decir pueden acceder a ellas sin cualificarlas con un nombre de objeto y el operador (.) También se puede acceder a las variables mediante la referencia this de modo discrecional (this.variable) o si alguna variable local o argumento las oculta. En código no estático se puede referir a una variable o método sin especificar cuál variable o método de objeto requiere. El compilador asume que lo que se requiere es this. Ejemplo:

```
1. class Xyzzy {  
2. int w;  
3.  
4. void bumpW() {  
5. w++;  
6. }  
7. }
```

En la línea 5, el programador no especificó cuál objeto w iba a ser incrementado. El compilador asume que la línea 5 es la abreviatura de: this.w++;

Los métodos estáticos no actúan sobre objetos concretos a través del operador punto (.). Pueden recibir objetos de su clase como argumentos explícitos, pero no tienen argumento implícito ni pueden utilizar la referencia this. Si se trata de acceder por medio de la referencia this, se emitirá un mensaje de error: "Undefined variable: this." El concepto de "instancia que está ejecutando el actual método" no tiene sentido porque Ninguna variable implícita referencia al objeto que está ejecutando el método. Como las variables estáticas, los métodos estáticos no están asociados con ninguna instancia individual de esa clase.

Si un método estático necesita acceder a una variable no estática o llamar a un método no estático, debe especificar cuál instancia de esa clase define la variable o ejecuta el método. Esta situación es familiar para quienes escriben una aplicación con un GUI:

```
1. import java.awt.*;  
2.  
3. public class MyFrame extends Frame {  
4. MyFrame() {  
5. setSize(300, 300);  
6. }  
7.  
8. public static void main(String args[]) {  
9. MyFrame theFrame = new MyFrame();  
10. theFrame.setVisible(true);  
11. }
```

12. }

En la línea 9 el método estático `main()` construye una instancia de la clase `MyFrame`. En la siguiente línea, esa instancia es llamada para ejecutar el método (no estático) `setVisible()`. Esta técnica brinda el truco de estático a no estático y es frecuentemente visto en aplicaciones.

Un método estático no puede sobrescribirse para ser no estático. El siguiente código no compilará:

```
1. class Cattle {
2. static void foo ( ) { }
3. }
4.
5. class Sheup extends Cattle {
6. void foo ( ) { }
7. }
```

El error de compilación dice: "Static methods can't be overridden". Si la línea 6 cambia a "static void foo()", entonces compilará satisfactoriamente.

Para resumir los métodos estáticos:

Un método estático sólo puede acceder a datos estáticos de su clase; no puede acceder a datos no estáticos.

Un método estático sólo puede llamar a métodos estáticos de su clase; no puede llamar a métodos no estáticos.

Un método estático no tiene **this**

Un método estático no puede ser sobreescrito para ser no estático

Inicializadores static

Son métodos que se llaman automáticamente al crear la clase, al utilizarla por primera vez. Se diferencia del constructor en que no es llamado para cada objeto, sino una sola vez para toda la clase. Los tipos primitivos pueden inicializarse directamente con asignaciones en la clase o en el constructor. Los inicializadores Static se crean dentro de la clase, como métodos sin nombre y sin valor de retorno, con tan solo la palabra Static y el código entre llaves { . . . } . En una clase pueden definirse varios inicializadores Static, que se llamarán en el orden en que han sido escritos. Los inicializadores Static se pueden utilizar para dar valor a las variables Static. Además se suelen utilizar para llamar a métodos nativos.

Es legal que una clase contenga un código estático que no existe dentro del código del método. Ejemplo:

```
1. public class StaticDemo {
2. static int i=5;
3.
4. static {
```

```

5. System.out.println ("Código estático: i = " + i++);
6. }
7.
8. public static void main (String[ ] arg) {
9. System.out.println("main: i = " + i++);
10. }
11. }

```

Algunas veces pareciera que algo se olvidó en la línea 4, tal vez se esperaba una declaración completa de un método, sin embargo la línea 4 es perfectamente válida, esto es conocido como el código de *inicializador estático*. el código dentro de los corchetes es ejecutado exactamente una vez, al tiempo que la clase es cargada. En el tiempo de carga de la clase todos las inicializaciones estáticas (tales como la línea 2) y todos los códigos libres (tales como las líneas 4-6) son ejecutadas en orden de aparición dentro de la definición de la clase.

El código inicializador debería ser usado con cautela, porque puede fácilmente volver confuso el código. El compilador soporta múltiples bloques inicializadores, pero no es una buena razón para tener más de uno.

native

Se aplica sólo a los métodos. El código del método queda completamente fuera de el Java Virtual Machine, en una librería dinámica (dll), normalmente escrito en C / C++ y compila para un solo tipo de máquina designada. Es la forma de utilizar conjuntamente funciones realizadas en otros lenguajes con códigos en Java.

La librería se carga llamando a: `system.loadLibrary ("nombre_librería");`

Para evitar demoras es recomendable hacer este llamado tan pronto como sea posible, muchos programadores usan la técnica que se muestra en el código siguiente:

```

1. class NativeExample {
2. native void doSomethingLocal (int i);
3.
4. static {
5. System.loadLibrary ("MiLibNativa");
6. }
7. }

```

Note la declaración native de la línea 2, la cual declara que el código que implementa `doSomethingLocal()` reside en una librería local. Las líneas 4-6 son el código inicializador estático, así que se ejecutará al tiempo que la clase `NativeExample` sea cargada; esto asegura que la librería estará disponible por el tiempo que se necesite.

Llamados a métodos nativos no tienen que saber que el método es nativo. Las llamadas son hechas exactamente del mismo modo como si no fuera nativo:

```

1. NativeExample natex;

```

2. `natex = new NativeExample();`
3. `natex.doSomethingLocal(5);`

Muchos métodos comunes son nativos incluyendo métodos de la clase `Math` y los métodos `clone()` y `notify()` de la clase `Object`.

transient

Sólo aplica a las variables. Una variable transitoria no se guarda como parte del estado de persistencia de su objeto. Esto es, indica que la variable no forma parte de la persistencia de un objeto (capacidad de los objetos de mantener su valor cuando termina la ejecución de un programa), por tanto no debe ser serializada (convertida en flujo de caracteres para poder ser almacenada en disco o en una base de datos) con el resto del objeto. Esto se hace pasando el objeto al método `writeObject()` de la clase `ObjectOutputStream`. Si el flujo es asociado a un `OutputStream` de un socket, entonces el estado del objeto es escrito en la red. En ambos casos el objeto puede ser reconstituído para leerlo a partir de un `ObjectInputStream`.

A veces el objeto puede contener información delicada:

1. `class WealthyCustomer extends Customer implements Serializable {`
2. `private float $wealth;`
3. `private String accessCode;`
4. `}`

Una vez el objeto es escrito a un destino fuera de JVM, ningún elaborado mecanismo de seguridad tiene efecto. Si una instancia de esta clase fuera escrita a un archivo o a internet, alguien pudiera acceder al código. La línea 3 deberá ser marcada con la palabra `transient`:

1. `class WealthyCustomer extends Customer implements Serializable {`
2. `private float $wealth;`
3. `private transient String accessCode;`
4. `}`

Ahora el valor de `accessCode` no será escrito durante la serialización.

Las variables `transient` no pueden ser estáticas o finales.

synchronized

Las secciones de código que acceden a un mismo recurso (un mismo objeto de una clase, un archivo de disco, etc.) desde threads distintos, se denominan códigos críticos. `Synchronized` controla el acceso al código crítico en programas con multihilado (`multithreaded`).

Estos métodos tienen la particularidad que sobre un objeto no pueden ejecutarse simultáneamente dos métodos que estén sincronizados.

Controlan el acceso al código crítico en el enhilado de los programas.

volatile

Sólo aplica a las variables. Indica que la variable puede ser utilizada por distintas threads sincronizadas y que el compilador no debe realizar optimizaciones con esta variable.

Las variables volátiles pueden modificarse asincrónicamente por el compilador, siendo de interés en ambientes multiprocesador.

Tabla: Combinaciones posibles con modificadores.

Modificador	Clase	Variable	Método / Constructor	Bloque libre
public	sí	sí	sí	no
protected	no	sí	sí	no
(friendly)	sí	sí	sí	no
private	no	sí	sí	no
final	sí	sí	sí	no
abstract	sí	no	sí	no
static	no	sí	sí	sí
native	no	no	sí	no
transient	no	sí	no	no
volatile	no	sí	no	no
synchronized	no	no	sí	no

PRUEBA

1.Cuál de las siguientes declaraciones son ilegales (Escoger una o más)

A. friendly String s;

- B. `transient int i = 41;`
- C. `public final static native int w ( );`
- D. `abstract final double hyperbolicCosine ( );`

2. cuál de los siguientes estamentos es cierto?

- A. Una clase abstracta no puede tener ningún método final
- B. Una clase final no puede tener ningún método abstracto.

3.Cuál es la mínima modificación que se requiere para que el código compile correctamente?

```
1. final class Aaa
2. {
3. int xxx;
4. void yyy ( ) { xxx = 1; }
5. }
6.
7.
8. class Bbb extends Aaa
9. {
10 final Aaa finalref = new Aaa ( );
11.
12 final void yyy ( )
13 {
14 System.out.println("In method yyy ( )" );
15 finalref.xxx = 12345;
16 }
17 }
```

- A. línea 1, remover el modificador final
- B. línea 10, remover el modificador final
- C. Remover la línea 15.
- C. líneas 1 y 10, remover el modificador final
- D. El código compilará tal como está. No se necesitan modificaciones

4.Cuál de los siguientes estamentos es cierto?

- A. métodos transientes no puede ser sobreescritos.
- B. métodos transientes deben ser sobreescritos.
- C. clases transientes no pueden ser serializadas.
- D. variables transientes deben ser estáticas.
- E. variables transientes no son serializadas.

5.Cuál estamento es cierto acerca de la siguiente aplicación?

```
1. class StaticStuff
2. {
3. static int x = 10;
```

```

4.
5. static { x += 5; }
6.
7. public static void main(String args [ ] )
8. {
9. System.out.println(" x = " + x );
10 }
11
12 static {x /= 5; }
13 }

```

- A. Línea 5 y 12 no compilará porque los nombres del método y el tipo de retorno se han perdido.
- B. Línea 12 no compilará porque sólo puede tenerse un inicializador estático
- C. El código compila y produce la salida x= 10.
- D. El código compila y produce la salida x= 15.
- E. El código compila y produce la salida x= 3.

6. Cuál estamento es cierto de acuerdo al siguiente código?

```

1. class HasStatic
2. {
3. private static int x = 100;
4.
5. public static void main (String args [ ] )
6. {
7. HasStatic hs1 = new HasStatic ( );
8. hs1.x++;
9. HasStatic hs2 = new HasStatic ( );
10 hs2.x++;
11 hs1 = new HasStatic ( );
12 hs1.x++;
13 HasStatic.x++;
14 System.out.println("x = " + X );
15 }
16 }

```

- A. Línea 8 no compilará porque es una referencia estática a una variable privada.
- B. Línea 13 no compilará porque es una referencia estática a una variable privada.
- C. El código compila y produce la salida x = 102.
- D. El código compila y produce la salida x = 103.
- E. El código compila y produce la salida x = 104.

7. Dado el siguiente código y no haciendo más cambios Cuál modificador de acceso (public, protected, o private) puede ser ubicado antes del método aMethod () en la línea 3? Si la línea se deja como está, cuál palabra reservada puede ser ubicada antes del método aMethod () en la línea 8?

```

1. class SuperDuper

```

```

2. {
3. void aMethod ( ) { }
4. }
5.
6l. class Sub extends SuperDuper
7. {
8. void aMethod ( ) { }
9. }

```

8. Cuál modificador o modificadores deberían ser usados para denotar que una variable no debería ser escrita como parte del estado persistente de la clase? (Escoja la respuesta más corta posible.)

- A. private
- B. protected
- C. private protected
- D. transient
- D. private transient

Las siguientes dos preguntas tienen en cuenta la siguiente definición de clase:

```

1. package abcde;
2.
3. public class Bird {
4. protected static int referenceCount = 0;
5. public Bird ( ) { referenceCount++; }
6. protected void fly ( ) { /* Flap wings, etc. */ }
7. static int getRefCount ( ) { return referenceCount; }
8. }

```

9. Cuál de los siguientes estamentos es cierto acerca de la clase Bird y de la clase Parrot?

```

1. package abcde;
2.
3. class Parrot extends abcde.Bird {
4. public void fly ( ) { /* Parrot specific flight code. */ }
5. public int getRefCount ( ) { return reference Count; }
6. }

```

- A. La compilación de Parrot.java falla en la línea 4 porque el método fly () es protegido en la superclase y las clases Bird y Parrot están en el mismo paquete.
- B. La compilación de Parrot.java falla en la línea 4 porque el método fly() es protegido en la superclase y público en la subclase y los métodos no pueden ser sobreescritos para ser más públicos.
- C. La compilación de Parrot.java falla en la línea 5 porque el método getRefCount() es estático en la superclase y los métodos estáticos no pueden ser sobreescritos a no estáticos.
- D. La compilación de Parrot.java es correcta, pero una excepción en tiempo de

ejecución es lanzada si el método fly() es llamado en una instancia de la clase Parrot.
E. La compilación de parrot.java es correcta, pero una excepción en tiempo de ejecución es lanzada si el método getRefCount() es llamado en una instancia de la clase Parrot.

9. Cuál estamento es cierto acerca de las clases Bird y Nightingale?

```
1. package singers;
2.
3. class Nightingale extends abcde.Bird {
4.     Nightingale ( ) { referenceCount++; }
5.
6.     public static void main(String args [ ] ) {
7.         System.out.print("BEFORE: " + referenceCount);
8.         Nightingale florence = new Nightingale ( );
9.         System.out.println(" AFTER: " + referenceCount);
10.        florence.fly ( );
11.    }
12. }
```

A. El programa compilará y la salida será: Before : 0 After: 2.
B. El programa compilará y la salida será: Before : 0 After: 1.
C. La compilación de Nightingale fallará en la línea 4 porque miembros estáticos no pueden ser sobreescritos.
D. La compilación de Nightingale fallará en la línea 10 porque el método fly () es protegido en la superclase.
E. La compilación de Nightingale es correcta, pero una excepción en tiempo de ejecución es lanzada en la línea 10 porque el método fly() es protegido en la superclase

Capítulo 4

4. Conversión y Casting

Este capítulo cubre el siguiente objetivo del examen de certificación en Java

- Determinar el efecto en los objetos y valores primitivos de pasar variables a métodos y realizar asignaciones u otras operaciones de asignación en ese método.

Cada variable de Java tiene un tipo. Los tipos de datos primitivos incluyen **int**, **long**, **double**, y así sucesivamente. Los tipos de datos de referencia a objetos pueden ser clases (tales como **Vector** o **Graphics**) o interfaces (como lo es **LayoutManager** o **Runnable**). Pueden ser también arreglos de primitivas, objetos o arreglos.

Este capítulo discute las formas en que un valor dato puede cambiar su tipo. Los valores pueden cambiar un tipo a otro explícita o implícitamente ; esto es, cualquier tipo al que cambie, ya sea por petición suya o iniciativa del sistema. Java brinda mucha importancia al tipo, y una programación exitosa en Java requiere que usted sea cuidadoso con los cambios de tipo.

Cambios de tipo Implícitos o explícitos

Usted puede explícitamente cambiar el tipo de un valor por casting. Para hacer casting de una expresión a un nuevo tipo, sólo ponga como prefijo de la expresión el nuevo tipo en paréntesis. Por ejemplo, la siguiente línea de código trae un elemento de un vector, realiza cast de ese elemento al tipo **Button**, y asigna el resultado a una variable llamada btn:

```
Button btn = (Button)myVector.elementAt( 5 );
```

Por supuesto, el sexto elemento del vector debe ser capaz de ser tratado como un botón. Hay reglas de tiempo de compilación y tiempo de corrida que deben ser observadas. Este capítulo lo familiarizará con tales reglas.

Hay situaciones en las cuales el sistema implícitamente cambia el tipo de una expresión sin que usted explícitamente elabore el cast. Por

ejemplo, suponga que usted tiene una variable llamada myCol que es una instancia de **Color**, y usted quiere almacenar myCol en un vector. Usted debería probablemente hacer lo siguiente:

```
myVector.addElement( myColor );
```

Hay más en éste código de lo que se ve a simple vista.

El método **addElement()** de la clase **Vector** es declarada con un parámetro de tipo **Object**, no del tipo **Color**. Como el argumento es pasado al método, el sistema entiende un cambio de tipo implícito. El cambio de tipo no explícito es conocido como conversión.

La conversión como el casting, es gobernado por un número de reglas. A diferencia de las reglas de casting, todas las reglas de conversión son forzadas en tiempo de compilación.

El número de reglas de casting y conversión es bastante largo, debido al gran número de casos que son considerados. (Por ejemplo, se puede hacer cast de char a double?, se puede convertir una interfaz a una clase final?).

Las buenas noticias son que la mayoría de las reglas van de acuerdo con el sentido común, y la mayoría de las combinaciones pueden ser generalizadas en reglas generalizadas. Hacia el final de éste capítulo, usted sabrá cuándo puede efectuar cast explícito, y cuándo el sistema implícitamente hará la conversión por usted.

Primitivas y Conversión

Las dos principales categorías de los tipos de datos Java son primitivas y objetos.

Los tipos de datos primitivos son int, float, Boolean, y así sucesivamente. (Hay 8 tipos de datos primitivos en total; vea el capítulo 1 para una explicación completa de las primitivas de Java).

Los tipos de dato **Object** (o más apropiadamente, tipos de dato de referencia a objetos) son todos los cientos de clases e interfaces del JDK, más la infinitud de clases e interfaces desarrolladas por los programadores de Java.

Tanto a los valores primitivos como referencias a objetos se les puede

efectuar procedimientos de casting y conversión, hay 4 casos a considerar:

- Conversión de primitivas.
- casting de primitivas.
- conversión de referencias a objetos.
- casting de referencias a objetos.

El tópico más sencillo es la conversión implícita de primitivas (que son: int, long, char,, y así sucesivamente). Todas las conversiones de tipos de datos primitivos toman lugar en tiempo de compilación; esto es porque toda la información necesaria para determinar si la conversión es legal o no en tiempo de compilar. (Este no es el caso de los datos Object, que verá usted más adelante en éste capítulo).

Hay 3 contextos de situaciones en que la conversión de una primitiva puede ocurrir:

- asignación.
- Llamada de métodos.
- Promoción aritmética.

Las siguientes secciones tartan con los contextos en turno.

Conversión de primitivas: Asignación

La conversión por asignación sucede cuando usted asigna un valor a una variable de un tipo diferente al valor original, por ejemplo:

1. **int** i;
2. **double** d;
3. i = 10;
4. d = i; // asigna un valor de tipo entero a una variable double.

Obviamente, *d* no puede retener un valor entero. Al momento que la cuarta línea de código es ejecutada, el entero 10 que es almacenado en la variable *i* se convierte al valor de doble precisión 10.000000000000 (los ceros restantes se omiten por brevedad).

El código anterior es perfectamente lícito. Algunas asignaciones, a la otra mano, son ilegales. Por ejemplo, el siguiente código no compilará:

1. **double** d;
2. **short** s;
3. d = 1.2345;
4. s = d; // asigna un valor double a una variable short

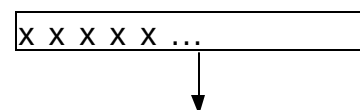
Este código no compilará. (El mensaje de error dice **"Incompatible type for =."**). El compilador reconoce que está tratando de introducir un valor de tipo **double** en una variable short, lo cual es como derremar un cuerto de café en un una taza de café de 8 onzas. Puede hacerse (ésto es, la asignación de valor puede hacerse; la cuestión del café es imposible), pero usted debe hacer uso de un cast explícito, que será explicado en la siguiente sección.

Las reglas generales para conversión por asignación pueden resumirse así:

- Un booleano no puede ser convertido a otro tipo de datos.
- Un no booleano puede ser convertido a otro no tipo booleano si la conversión es a un tipo más grande.
- Un no booleano no puede ser convertido en otro tipo no booleano, si la conversión debería ser a un tipo más pequeño.

Conversiones a tipos más grandes cambian un valor a un tipo que se acomoda a un mayor rango de valores a los que el valor original podía acomodarse. En la mayoría de los casos, el nuevo tipo tiene más bits que el original y puede ser visualizado como más amplio que el original, como se muestra en la figura 4.1

Figura 4.1
Conversión de agrandamiento



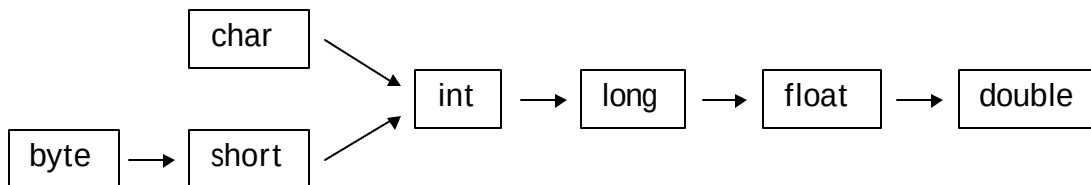
0	...	0	x	x	x	x	x	...
---	-----	---	---	---	---	---	---	-----

Las conversiones de agrandamiento no pierden información acerca de la magnitud de un valor. En el primer ejemplo de ésta sección, un valor entero fue asignado a una variable double. Esto fue lícito, porque los valores double son por así decirlo más amplios que los enteros, Las conversiones de ampliamiento en Java son:

- De un **byte** a un **short**, a un **int**, a un **long**, un **float** o **double**.
- De un **short** a un **int**, a un **long**, a un **float** o **double**.
- De un **char** a un **int**, a un **long**, a un **float**, o **double**.
- De un **int** a un **long**, a un **float**, o **double**.
- De un **long** a un **float** o **double**.
- De un **float** a un **double**.

La figura 4.2 ilustra todas las conversiones de ampliamiento. Las flechas pueden ser tomadas como “puede ser convertido a”. Para determinar si es legal convertir un tipo a otro, encuentre el primer tipo en la figura y vea si usted puede alcanzar el segundo tipo siguiendo las flechas.

Figura 4.2
Conversiones de ampliamiento



La figura muestra, por ejemplo, que es perfectamente legal para asignar un valor byte a una variable flota, porque usted puede trazar un camino de byte a flota siguiendo las flechas (byte a short a entero a long o float). Usted no puede al contrario, trazar un camino de long a short, así entonces no es legal asinar un valor long a una variable short.

La figura 4.2 es fácil de memorizar. La figura consiste en mayoría de los tipos numéricos en orden de tamaño. La única pieza extra de información es char, excepto que vaya en el único lugar en que puede ir: un char de 16 bits cabe en un entero de 32 bits. (nótese que usted no puede convertir un byte a char o char a short, aún cuando se ve razonable de hacer).

Cualquier conversión entre tipos primitivos que no es representada por un camino de las flechas de la figura 4.2 es un conversión de estrechamiento “narrowing”. Éstas conversiones pierden información acerca de la magnitud del valor que está siendo convertido, y no son permitidas como asignaciones . Es geométricamente imposible retratar las conversiones de estrechamiento en un gráfico como la figura 4.2. pero ellas pueden ser resumidas así:

- De un **byte** a un **char**.
- De un **short** a un **byte** o a **char**.
- De un **char** a un **byte** o a **short**.
- De un **int** a un **byte**, a **short**, o a **char**.
- De un **long** a un **byte**, a **short**, a **char**, o a **int**.
- De un **float** a un **byte**, a **short**, a **char**, a **int**, o a **long**.
- De un **double** a un **byte**, a **short**, a **char**, a **int**, a **long**, o a **float**.

Usted no necesita memorizar ésta lista. Simplemente representa todas las conversiones no mostradas en la figura 4.2, que más fácil de memorizar.

Conversión de primitivas: Llamado de métodos

Otra clase de conversión es el la conversión por llamado de métodos que sucede cuando usted pasa un valor de un tipo como un argumento de un método que espera un tipo diferente. Por ejemplo, el método **cos()** de la clase **Math**, espera un argumento sencillo de un tipo **double**. Considere el siguiente código:

1. **float** frads;
2. **double** d;
3. frads = 2.34567f;
4. d = **Math.cos**(frads); // pasa un flotante a un método que espera un double

El valor float en frads es automáticamente convertido a un valor double antes de ser manipulado por el método **cos()**. Tal como en las conversiones de asignación, hay reglas estrictas que gobiernan que son permitidas y que conversiones pueden ser rechazadas por el compilador. El código a continuación totalmente razonable genera un error de compilación (asumiendo que hay un vector llamado **myVector**).

```
double d = 12.0;  
Object ob = myVector.elementAt( d );
```

El error de compilador dice "Incompatible type for method. Explicit cast needed to convert **double** to **int**".

Ésto significa que el compilador no puede convertir el argumento de tipo *double* al tipo que es soportado por la versión del método *elementAt()*. El entiende que la única versión de *elementAt()* es la versión que toma como argumento a un entero. Entonces un valor puede ser únicamente pasado a *elementAt()* si ese valor es un *int* o puede ser convertido a *int*.

Afortunadamente, la regla que gobierna qué conversiones por llamado de métodos son permitidas, es la misma regla que gobierna las conversiones de asignación. Conversiones de ampliamento (como las mostradas en la figura 4.2) son permitidas; *las conversiones de estrechamiento son prohibidas*.

La última clase de conversión de primitivas a considerar es la promoción aritmética, que suceden con sentencias aritméticas. Mientras el compilador está tratando de percibir muchas diferentes clases de operando.

Considere el siguiente fragmento:

1. **short** s = 9;
2. **int** i = 10;
3. **float** f = 11.1f;
4. **double** d = 12.2;
5. if (++s*i >= f/d)
6. System.out.println(">>>>");
7. else
8. System.out.println("<<<<");

El código en la línea 5 multiplica un *short* incrementado por un *int*; entonces el divide un *flota* por un *double*; finalmente el compara los dos resultados. Atrás de las escenas, el sistema está haciendo una extensiva conversión de tipo para asegurar que los operandos pueden ser incrementados, multiplicados, divididos y comparados. Éstas conversiones son todas de ampliamento.

Entonces ellas son conocidas como conversiones de promoción aritmética, porque los valores son “promovidos” a tipos más amplios.

Las reglas que gobiernan la promoción aritmética difiere entre operadores unarios y binarios. Los operadores unarios operan en un solo operando. Los operadores binarios operan en 2 valores. La figura 4.3 muestra los operadores básicos unarios y binarios de Java.

Figura 4.3

Operadores unarios	+	-	++	--		~		
Operadores binarios	+	-	*	/	%	>>	>>>	<<
		&	^					

Para operadores unarios, dos reglas se aplican, dependiendo en el tipo de operando simple:

- Si el operando es un *byte*, un *short* o un *char*, es convertido a *int*.
- En otro caso, si el operando es de otro tipo, no es convertido.

Para operadores binarios, hay cuatro reglas, dependiendo de los tipos de los dos operandos:

- Si uno de los operandos es un *double*, el otro operando es convertido a *float*.
- Si uno de los operandos es *float*, el otro operando es convertido a *float*.
- Si uno de los operandos es un *long*, el otro operando es convertido a *long*.
- Si no, ambos operandos son convertidos a *int*.

Con éstas reglas en mente, es posible determinar que realmente sucede en el código de ejemplo dado al inicio de ésta sección:

El *short* es promovido a un *int* y luego incrementado. El resultado del paso 1 (un *int*) es multiplicado por el entero *i*. Desde que ambos operandos sean del mismo tipo, y éste tipo no sea más pequeño que un *int*, no se necesita conversión. El resultado de la multiplicación es un *int*.

Antes de dividir el *float* *f*, por el *double* *d*, *f* es ampliado a *double*. La división genera un resultado de doble precisión. El resultado del paso 2 (un *int*) está para comparar el resultado del paso 3 (un *double*). El entero es convertido a *double*, y los dos operados son comparados. El resultado de la comparación es siempre de tipo *boolean*.

Primitivas y Casting

Con lo visto en éste capítulo, se ha mostrado que Java es perfectamente capaz de realizar conversiones de ampliamento en primitivas. Éstas conversiones son implícitas y por detrás de escenas; usted no necesita escribir ningún código explícito para hacer que sucedan.

Casting significa dirección explícita a Java para hacer una conversión. Una operación de casting puede ampliar o disminuir su argumento. Para hacer cast, solo preceda un valor con el nombre del tipo deseado entre paréntesis. Por ejemplo, las siguientes líneas de código hacen un cast de *int* a *double*:

1. **int** i = 5;
2. **double** d = (**double**)i;

Por supuesto, el cast no es necesario. El siguiente código, en que el cast ha sido omitido, podría hacer una asinación en *i*, con el mismo resultado del ejemplo anterior:

1. **int** i = 5;
2. **double** d = i;

Los Casts son requeridos cuando usted desea realizar una conversión de estrechamiento, como la conversión no será nunca hecha implícitamente, usted tiene que programar un cast explícito que convenza al compilador de que usted realmente quiere esa conversión de reducción.

La reducción corre el riesgo de pérdida de información; el cast le dice al compilador que usted acepta el riesgo. Por ejemplo, el siguiente código genera un error de compilación:

1. **short** s = 259;
2. **byte** b = s; // error de compilación

```
3. System.out.println( s = " + s + ", b = " + b );
```

El mensaje de error de compilación para la segunda línea será (among other things), "Explicit cast needed to convert *short* to *byte*."),

Agregando un cast, es fácil:

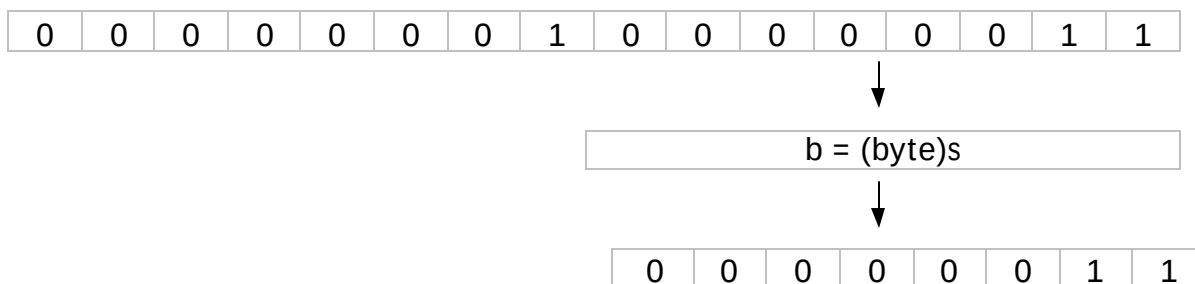
```
1. short s = 259;  
2. byte b = (byte)s;      // explicit cast  
3. System.out.println( b = " + b );
```

Cuando éste código es ejecutado, el número 259 (binario: 100000011) debe ser ajustado a un simple *byte*. Esto es logrado preservando los bytes de más bajo orden y descartando el resto. El código tendrá de salida éste mensaje:

```
b = 3
```

El bit 1 en la posición 8 ha sido descartado, dejando únicamente 3, como es mostrado en la figura 4.5. La conversión por reducción puede resultar en cambios radicales de valor; esto es porque el compilador requiere que usted haga el cast explícitamente. El cast le dice al compilador "sí, yo realmente deseo hacerlo".

Figura 4.4



Haciendo casting de un valor a un valor más amplio (fig. 4.2) es siempre permitido, pero nunca requerido; si usted omite el cast, una conversión implícita será efectuada a su nombre.

Sin embargo, el casting explícito puede hacer su código más leíble. Por ejemplo:

```
1. int i = 2;  
2. double radians;  
.  
.  
//cientos de líneas de código después.  
600. radians = (double)i;
```

El cast en la última línea no es requerido, pero el sirve como un buen recordatorio para los lectores (incluido usted mismo) que pueden haber olvidado el tipo de radianes. Hay dos simples reglas que gobiernan el casting de tipos primitivos:

- Usted puede pasar un no booleano a cualquier otro no booleano.
- Usted no puede pasar Boolean a otro tipo, ni otro tipo a Boolean.

Note que mientras el casting es ordinariamente en estrechamiento, es perfectamente lícito cuando se efectúa ampliación. El cast es innecesario, pero provee un poco de claridad.

Conversión de referencias a Objetos

Los valores de referencia a objetos, como los valores primitivos, participan en asignación, conversión, conversión por llamadas a métodos, y casting. (No hay promoción aritmética de referencias a objetos, desde que las referencias no sean operandos aritméticos). La conversión de referencias a objetos es más complicada que la conversión primitiva, porque hay más combinaciones posibles de viejos y nuevos tipos, y más combinaciones significan más reglas.

La conversión de referencias, como la conversión de primitivos, toma lugar al tiempo de compilación, porque el compilador tiene toda la información que se necesita para determinar si la conversión es legal. Después usted verá que éste no es el caso para el casting de objetos.

Las siguientes secciones examinan la asignación de referencias a objetos, llamadas de métodos y conversiones por casting

Conversión de referencias a objetos por asignación

La Conversión de referencias a objetos por asignación sucede cuando usted asigna un valor de referencia a objeto a una variable de tipo diferente. Hay 3 clases generales de tipos de referencia a objetos.

- Un tipo clase, como **Button** o **FileWriter**
- Un tipo interfaz, como **Cloneable** o **LayoutManager**
- Un tipo arreglo, como **int[][]** o **TextArea[]**

Generalmente hablando, la conversión por asignación de referencias, se ve así:

1. `OldType x = new OldType ();`
2. `NewType y = x;      // conversión por asignación de referencias`

Ésto es el formato general de una conversión por asignación de un viejo tipo a uno nuevo. Desafortunadamente, el viejo tipo puede ser una clase, una interfaz o un arreglo; el nuevo tipo puede también ser una clase, interfaz o arreglo. Entonces hay 9 posible combinaciones a considerar. La figura 4.6 muestra las reglas de los 9 casos:

Figura 4.6

	OldType es una clase	OldType es una interfaz	OldType es un arreglo
NewType es una clase	OldType debe ser subclase de NewType	NewType debe ser Object	NewType debe ser Object
NewType es una interfaz	OldType debe implementar la interfaz NewType	Oldtype debe ser sub-interfaz de NewType	NewType debe ser Cloneable
NewType is un arreglo	Error de compilación	Error compilación	OldType debe ser un arreglo de algunos tipos de referencia a objetos que puede ser convertido a cualquier NewType

Podría ser difícil memorizar las nueve reglas de la figura 4.5, afortunadamente, hay una regla general.

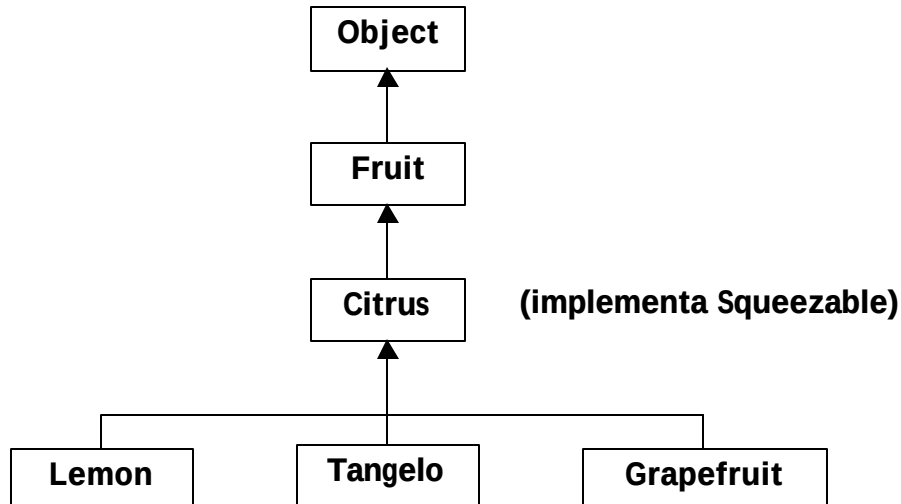
Recordar que con primitivas, las conversiones eran permitidas, dado que eran de ampliación “widening”. La noción de ampliación no se aplica realmente a referencias, pero hay un principio similar. En general, la conversión de referencias a objetos es permitida cuando la dirección de la conversión es hacia arriba en la jerarquía de herencia, que es, el viejo tipo debe heredar del nuevo tipo. Ésta regla general no cubre los 9 casos pero es útil.

Las reglas para la conversión de referencias a objetos son las siguientes:

- Un tipo interfaz puede ser únicamente convertido a un tipo interfaz o *objeto*. Si el nuevo tipo es una interfaz, ésta debe ser una superinterfaz del viejo tipo.
- Un tipo clase puede ser convertido a un tipo clase o un tipo interfaz. Si se convierte a una clase, el nuevo tipo será una superclase del viejo tipo. Si se convierte a un tipo interfaz, la vieja clase debe implementar tal interfaz.
- Un arreglo puede ser convertido a la clase **Object**, a la interfaz clonable o serializable, o a un arreglo. Únicamente un arreglo de referencias a objetos puede ser convertido a un arreglo, y el viejo tipo de elementos debe ser conversible al nuevo tipo de elementos.

Para ilustrar éstas reglas, considere la jerarquía de herencia mostrada en la figura 4.6 (asuma que hay una interfaz llamada squeezable)

Figura 4.6
La jerarquía frutal...



Como primer ejemplo, considere el siguiente código:

1. Tangelo tange = **new** Tangelo();
2. Citrus cit = tange;

Éste código funciona bien, *Tangelo* está siendo convertida a Citrus. El nuevo tipo es una superclase del viejo tipo, así que la conversión es permitida.

Convirtiendo en la otra dirección (hacia abajo en la jerarquía) no es permitida¹:

1. Citrus cit = **new** Citrus();
2. Tangelo tange = cit;

Éste código resultará en un error de compilador

Qué sucede si uno de los tipos es una interfaz?

¹ Recuerde: Esto no significa que usted no pueda hacerlo por casting.

1. Grapefruit g = **new** Grapefruit();
2. Squeezable squee = g; // no hay problema
3. Grapefruit g2 = squee; // error

La segunda línea cambia un tipo clase *Grapefruit* a un tipo interfaz. Esto es correcto, con tal que *Grapefruit* implemente *Squeezable*. Un vistazo a la figura 4.6 muestra que esto es de hecho el caso, porque *Grapefruit* hereda de *Citrus*, que implementa *Squeezable*.

La tercera línea es una error, *porque una interfaz nunca puede ser implícitamente convertida a cualquier tipo referencia más que el objeto*.

Finalmente, considere un ejemplo con arreglos:

1. Fruit fruits[];
2. Lemon lemons[];
3. Citrus citruses[] = **new** Citrus[10];
4. for (**int** i = 0; i < 10; i++) {
5. citruses[i] = **new** Citrus();
6. }
7. fruits = citruses; // no hay problema
8. lemons = citruses; // error

La línea 7 convierte un arreglo de *Citrus* a un arreglo de *Fruiti*. Esto está bien porque *Fruiti* es una superclase de *Citrus*. La línea 8 convierte en otra la dirección y falla, porque *lemon* no es superclase de *Citrus*.

CONVERSION DE OBJETOS POR EL MÉTODO CALL

Afortunadamente, las reglas de conversión por el método CALL de valores de referencias a objetos son las mismas reglas descritas para la conversión de objetos por asignación. La regla general es que la conversión a una superclase es permitida pero no a una subclase. Las reglas específicas, formales fueron dadas, en una lista del bulleted en la sección anterior y se muestra aquí de nuevo:

- Un tipo interfaz puede ser únicamente convertido a un tipo interfaz o *objeto*. Si el nuevo tipo es una interfaz, ésta debe ser una superinterfaz del viejo tipo.
- Un tipo clase puede ser convertido a un tipo clase o un tipo

interfaz. Si se convierte a una clase, el nuevo tipo será una superclase del viejo tipo. Si se convierte a un tipo interfaz, la vieja clase debe implementar tal interfaz.

- Un arreglo puede ser convertido a la clase **Object**, a la interfaz clonable o serializable, o a un arreglo. Únicamente un arreglo de referencias a objetos puede ser convertido a un arreglo, y el viejo tipo de elementos debe ser convertible al nuevo tipo de elementos.

Para ver cómo las reglas tienen sentido en el contexto de llamadas del método, considere la sumamente útil clase **Vector**. Puede guardar cualquier tipo de dato en un Vector (algo no-primitivo, eso es) llamando el método `addElement (obj del Objeto)`. Por ejemplo, el código siguiente almacena a Tangelo en un vector:

1. `Vector myVec = new VectorQ;`
2. `Tangelo tange = new TangeloQ;`
3. `myVec.addElement( tange );`

El argumento Tange automáticamente será convertido a tipo **Object**. La conversión automática quiere decir que quien escriba la clase **Vector** no tiene que escribir un método por separado para cada tipo de Objeto posible que se pueda almacenar en un vector. Por fortuna: la clase Tangelo fue desarrollada dos años después de la invención del **Vector**, así que posiblemente el diseñador de la clase Vector no podría escribir código del manejo específico de Tangelo. Un objeto de cualquier clase (incluso una arreglo de cualquier tipo) puede pasarse en el llamado sencillo al método `addElement (obj del Objeto)`.

Casting por Referencia a Objetos

El Casting por referencia a objetos es similar al Casting primitivo: por el uso de `Cast` puede pedir al compilador que le permita hacer una conversión hacia otro tipo que de otra forma no se permitiría.

Cualquier tipo de conversión que es permitida por asignación ó Método Call es permitida para el cast explícito. Por ejemplo el siguiente código es aceptado:

1. Lemon lem = **new** Lemon();
2. Citrus cit = (Citrus)lem;

El Cast es legal, pero no necesario; si se omite el compilador hará una conversión de la asignación implícita. El poder del Casting aparece cuando explícitamente se hace cast a un tipo que no es permitido por las reglas de conversión implícita.

Para entender como trabaja el Casting para Objetos, es importante entender la diferencia entre Objetos y variables de referencia a Objetos. Cada objeto (bien, casi cada objeto porque hay algunos casos oscuros) es construido con el operador **new**. El argumento para **new** determina durante todo el tiempo la clase correcta del objeto. Por ejemplo, si un objeto es construido llamando a **new Color** (222, 0, 255), entonces a lo largo de la vida de ese objeto su clase será Color.

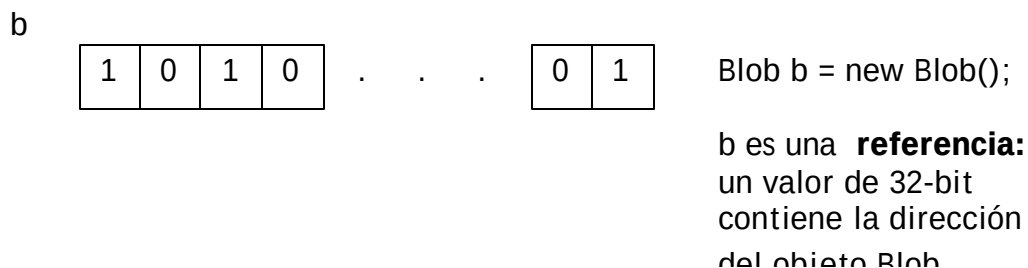
Recuerde que los programas de Java no se tratan directamente de objetos. Se tratan de referencias a los objetos. Por ejemplo, considere el código siguiente:

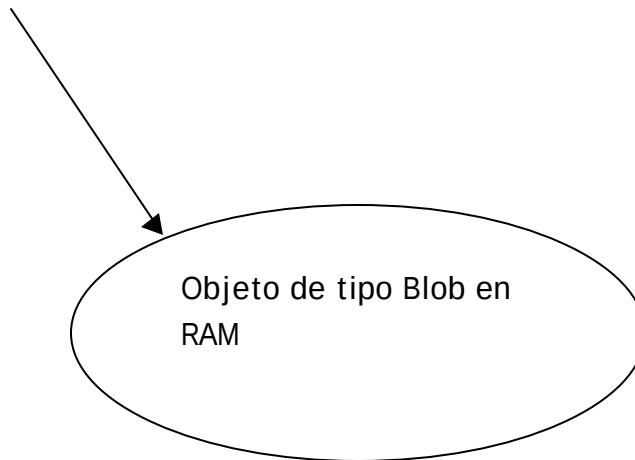
1. **Color** purple = **new Color**(222, 0, 255);

La variable purple no es un objeto, es la referencia a un objeto. El objeto reside en memoria en alguna parte de la Máquina Virtual de JAVA. La variable purple contiene algo similar a la dirección del objeto. Esta dirección es conocida como una referencia al objeto.

La diferencia entre una referencia y un objeto se ilustra en la figura 4.7

Figura 4.7
Referencias





Las referencias son almacenadas en variables, y las variables tienen tipos que son especificados por el programador en tiempo de compilación. Los tipos de variables de referencia a Objetos pueden ser clases (como **Graphics** o **FileWriter**), interfaces (como **Runnable** o **LayoutManager**), o arreglos (como **int [] []** o **Vector []**).

Mientras la clase de un objeto es inmutable, puede ser referenciada por variables de muchos tipos diferentes. Por ejemplo, considere una pila. Es construida llamando **new Stack()**, así que su clase realmente es **Stack**. Durante varios momentos del tiempo de vida de este objeto, puede ser referenciado por variables de tipo **Stack** (por supuesto), o del tipo **Vector** (porque **Stack** hereda de **Vector**), o del tipo **Objeto** (porque hereda todo del **Objeto**).

Incluso puede ser referenciada por variables de tipo **Serializable**, que es una interface, porqu el a Clase **Stack** implementa la interface de **Serializable**. Esta situación se muestra en Figura 4.8.

Serializable ser;



Object obj;



Objeto de tipo **Stack** in
RAM

Vector vec;



Stack stk;



El tipo de referencia a variable es obviado en tiempo de compilación. Sin embargo, la clase de referencia a objeto en forma semejante a la variable no puede conocerse hasta el runtime. Esta falta de conocimiento no es una limitación de tecnología de Java; es el resultado de un principio fundamental en ciencias de la computación. La distinción entre el conocimiento en tiempo de compilación y conocimiento en el runtime no es pertinente a nuestra discusión de conversiones; sin embargo, la diferencia llega a ser importante con el casting de referencia por valor. Las reglas para casting son un bit más ancho que aquéllos para la conversión. Algunas de estas reglas se preocupan por el tipo de referencia y pueden ser fortalecidas por el compilador en tiempo de compilación; otras reglas se preocupan por la clase de objeto y sólo pueden darse durante el tiempo de ejecución.

No hay ningún escape al hecho que hay varias reglas que gobiernan el

casting para objetos. Las buenas noticias son que la mayoría de las reglas cubre casos oscuros. Podría empezar viendo el cuadro de conversión por casting con toda sus complicadas relaciones, pero después de esto tendrá unas ideas simples que serán evidentes en la mayoría de las situaciones comunes.

Para el casting de referencia a objeto, no hat tres pero si cuatro posibilidades para el tipo viejo y el nuevo tipo. Cada tipo puede ser una clase no-final, una clase final, una interface, o un array. La primera regla fortalece lo que pasa en tiempo de compilación. Las reglas en tiempo de compilación se resumen en la Figura 4.9.

Figura 4.9
Reglas en tiempo de Compilación
para casting en referencia a objetos

1. NewType nt;
2. OldType t;
3. nt = (NewType)ot;

	OldType es una clase non-final	OldType es una clase final	OldType es una interface	Oldtype es un array
NewType Es una clase non-final	OldType debe extender NewType, o viceversa	OldType debe extender NewType	Siempre Bien	NewType debe ser Object
NewType es una clase final	NewType debe extender OldType	OldType y NewType deben ser la misma clase	NewType debe implementar la interfaz oldtype	Error de Compilación
NewType es una interface	Siempre bien	OldType debe implementar la interfaz NewType	Casi siempre bien	NewType debe ser Cloneable

NewType es un array	OldType debe ser Object	Error de compilación	Error de compilación	OldType debe ser un array del mismo tipo de referencia que puede ser convertido a cualquier tipo del arreglo NewType .
---------------------	--------------------------------	----------------------	----------------------	--

Asumiendo que el cast deseado sobrevive a la compilación, un segundo control ocurren en el runtime. El segundo control determina si el cast para la clase del objeto es compatible con el nuevo tipo. Aquí la compatibilidad significa que la clase puede convertirse según las reglas de la conversión presentadas en las dos secciones anteriores.

En forma sana, tenga presente que sólo algunas de las situaciones cubiertas por estas reglas normalmente se encuentra en la vida real. (Hay casos por ejemplo, cuando no es bueno tener interfaces para ambos tipos viejos y nuevos, pero estos casos son sumamente raros).

Unas reglas claves y algunos ejemplos deben ayudar a clarificar las cosas. primero, simplificando con las reglas en tiempo de compilación, tenga presente los hechos de bajo nivel sobre casting de Oldtype a NewType:

- Cuando OldType y NewType son clases, una clase debe ser un sub-clase de otra.
- Cuando OldType y NewType son arrays, ambos arrays deben contener la referencia al tipo (no primitivas), y debe hacer casting legal en elementos de Oldtype a elementos de NewType.
- Siempre puede hacer cast entre una interfaz y un objeto no-final.
- En cuanto a las reglas de runtime, recuerde que la conversión a NewType debe ser posible.

Las siguientes reglas claves son las más comunes:

- Si NewType es una clase, la clase de la expresión a convertirse

debe ser NewType o debe heredar de NewType.

- Si NewType es una interfaz, la clase de la expresión a convertirse debe implementar NewType.

Mire una vez más la jerarquía de Fruit/Citrus que vio antes en este capítulo. Primero, considere el código siguiente:

```
1. Grapefruit g1, g2;
2. Citrus c;
3. Tangelo t;
4. g1 = new Grapefruit();      // la clase es Grapefruit
5. c = g1;                     //Asignación legal de conversión, no
                               // necesita cast
6. g2 = (Grapefruit)c;         // cast válido
7. t = (Tangelo)c;             // cast no válido (throws an exception)
```

Este código tiene cuatro referencias pero sólo un objeto. La clase del objeto es Grapefruit, porque el constructor de Grapefruit que se llama en línea 4. La asignación `c = g` en la línea 5 es una conversión de la asignación absolutamente legal ("sobre" la jerarquía de herencia), así que ningún cast explícito se requiere. En las líneas 6 y 7, el Citrus hace cast a Grapefruit y a Tangelo. Para el llamado a casting entre los tipos de la clase, uno de las dos clases (no importa cuál) debe ser una subclase de la otra.

El primer cast es Citrus a la subclase Grapefruit; el segundo es de Citrus a la subclase Tangelo. Así ambos cast son legales en tiempo de compilación. El compilador no puede determinar la clase referenciada por el objeto `c` así que acepta cast y permite determinar el resultado en runtime.

Cuando el código se ejecuta, la Máquina Virtual Java intenta ejecutar línea 6: `G2 = (Grapefruit)c`; La clase de `c` se determina para ser Grapefruit, y no hay ninguna objeción a convertir Grapefruit a Grapefruit.

La línea 7 obliga (al runtime) cast de `c` para el tipo Tangelo. La clase de `c` todavía es Grapefruit, y un Grapefruit no puede hacer cast a un

Tangelo. Para que el cast sea legal, la clase `c` tendría que ser Tangelo o alguna subclase de Tangelo. Si éste no es el caso, una excepción del runtime (**`java.lang.ClassCastException`**) se lanza.

Ahora tome un ejemplo donde un objeto hace cast a un tipo interface. Considere el fragmento del código siguiente:

```
1. Grapefruit g1, g2;
2. Squeezable s;
3. g1 = new Grapefruit();
4. s = g1;           // convierte Grapefruit a Squeezable (bien)
5. g2 = s;           // convierte Squeezable a Grapefruit (error de compilación)
```

Este código no compilará. La línea 5 obliga convertir una interface (Squeezable) a una clase (Grapefruit). no importa que Grapefruit lleve a cabo Squeezable. Implícitamente convertir una interface a una clase nunca se permite; es uno de esos casos donde se tiene que usar un cast explícito para decirle al compilador que usted realmente sabe lo que está haciendo. Con el cast, línea 5 se vuelve

```
5. g1 = (Grapefruit)s;
```

Agregando el cast hace al compilador feliz. En runtime, la Máquina Virtual Java verifica si la clase de `s` (qué es Grapefruit) puede ser convertida a Grapefruit. Puede ciertamente, así que el cast se permite.

Para un ejemplo final, involucrando array, mire el código siguiente:

```
1. Grapefruit g[];
2. Squeezable s[];
3. Citrus c[];
4. g = new Grapefruit[ 500 ];
5. s = g;           // convierte el array Grapefruit en array Squeezable
6. c = (Citrus[])s; // hace cast de array Squeezable a array Citrus
```

La línea 6 hace cast de Squeezables (`s`) a un array de Citruses (`c`). Un cast de array es legal si el casting entre tipos del elemento array es legal (y si los elementos son referencias de tipo no primitivas). En este ejemplo, la pregunta es si un Squeezable (el tipo del elemento de array `s`) puede hacer cast a un Citrus (el tipo elemento hace cast a array). El ejemplo anterior mostró que éste es un cast legal.

Resumen del capítulo

Los valores primitivos y referencias del objeto son tipos muy diferentes de datos. Los dos pueden convertirse (implícitamente) o hacer cast (explícitamente). Los cambios del tipo Primitivos se producen por:

- Conversión por asignación
- Conversión por le método Call
- Conversión por promoción Aritmética
- Casting Explícito

Las Primitivas sólo puede convertirse si la conversión extiende los datos. Las Primitivas puede hacer recorte por casting, con tal que ni el viejo ni el nuevo tipo sea boolean.

La referencia a Objetos puede convertirse o hacer cast; las reglas que gobiernan estas actividades son extensas, porque hay muchas combinaciones de casos. En general, yendo "hacia arriba" en el árbol de herencia puede lograrse implícitamente a través de la conversión; yendo "hacia abajo" en el árbol se requiere casting explícito.

El cambio en la referencia a objetos puede suceder por:

- Conversión de Asignación
- Conversión por le Método Call
- Casting Explícito

Autoevaluación:

1. ¿Cual de las siguientes declaraciones es correcta? (Escoja más de una)
 - A. Solamente las primitivas son convertidas automáticamente; para cambiar el tipo de referencia a un objeto se debe hacer cast.
 - B. Solamente las referencias a objetos son convertidas automáticamente; para cambiar el tipo de una primitiva, se debe hacer cast.
 - C. La promoción Aritmetica de referencia a objetos requiere casting explícito.

D. Las primitivas y la referencia a objetos pueden ser convertidas y hechas por cast. El Casting de tipo numérico requiere control en el runtime.

2. ¿Cuáles líneas en el código siguiente no compilarán?

1. byte b = 5;
2. char c = 5;
3. short s = 55;
4. int i = 555;
5. float f = 555.5f;
6. b = s;
7. i = c;
8. if (f > b)
9. f = i;

3. El siguiente código compilará?

1. byte b1 = 2;
2. byte b2 = 3;
3. b = b1*b2;

4. En el siguiente código, que tipo de variables resultan (escoja la respuesta más correcta).

1. byte b = 11;
2. short s = 13;
3. result = b * ++s;

- A. **byte, short, int, long, float, double**
- B. **boolean, byte, short, char, int, long, float, double**
- C. **byte, short, char, int, long, float, double**
- D. **byte, short, char**
- E. **int, long, float, double**

5. Considere la siguiente clase:

1. **class** Cruncher {
2. **void** crunch(int i) { System.out.println("int version"); } }
3. **void** crunch(String s) { System.out.println("String version"); }

```

4.    public static void main( String[] args ) {
5.        Cruncher crun = new Cruncher();
6.        char ch = 'p';
7.        crun.crunch( ch );
8.    }
9. }

```

Cual de las siguientes afirmaciones es correcta (Escoja una)

- A. La Línea 3 no compilará, porque el método en void no puede ser sobreescrito.
- B. La línea 8 no compilará, porque no hay version de crunch() que tome un argumento char.
- C. El código compilará pero habrá una excepción 8.
- D. El código compilará y produce la siguiente salida :
int version
- E. El código compilará y produce la siguiente salida :
String versión

6. ¿Cuál de las siguientes afirmaciones es correcta? (Escoja una)

- A. La referencia a objeto puede ser convertida por asignación pero no por llamada al Método.
- B. La referencia a Objeto puede ser convertida por llamada al método pero no por asignación.
- C. La referencia a objeto puede ser convertida por llamada al método y por asignación, pero las reglas que manejan esta conversión son muy diferentes.
- D. La referencia a objeto puede ser convertida por llamada al método y por asignación, pero las reglas que manejan esta conversión son idénticas.
- E. La referencia a Objetos nunca puede ser convertida.

7. Considere el siguiente Código:

```

1. Object ob = new Object();
2. String[] strArr = new String[ 50 ];
3. Float floater = new Float( 3.14f );
4.
5. ob = strArr;

```

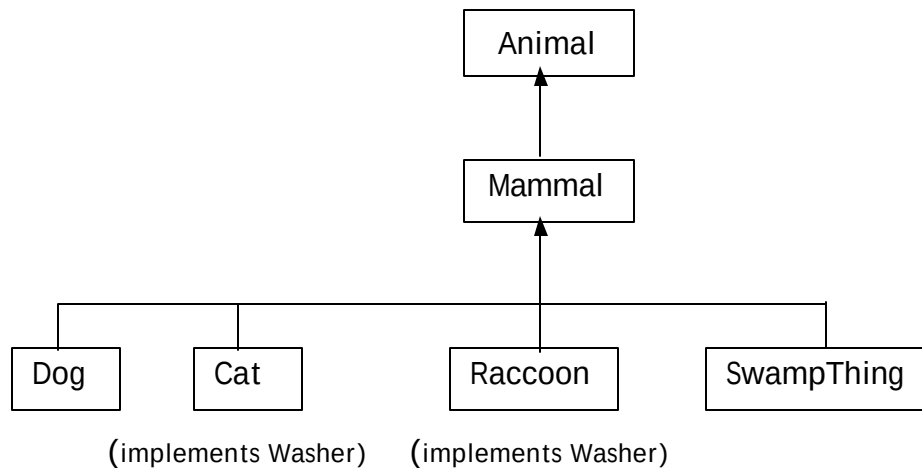
```

6. ob = strArr[ 5 ];
7. floater = ob;
8. ob = floater;

```

¿Cuál línea no compila?

8. ¿Consideré el siguiente código:



```

1. Dog rover, fido;
2. Animal anim;
3.
4. rover = new Dog();
5. anim = rover;
6. fido = (Dog)anim;

```

¿Cuál de las siguientes afirmaciones es correcta? (Escoja una)

- A. La línea 5 no compila.
- B. La línea 6 no compila.
- C. El código compila pero se produce un error de Excepción en la línea 6.
- D. El código compila y corre.
- E. El código compila y corre, pero el cast en la línea seis no es requerido y puede ser eliminado.

9. Considere el siguiente código:

1. Cat sunflower;
2. Washer wawa;
3. SwampThing pogo;
- 4.
5. sunflower = **new** Cat();
6. wawa = sunflower;
7. pogo = (SwampThing)wawa;

¿Cuál de las siguientes afirmaciones es correcta? (Escoja una)

- A. La línea 6 no compila; un cast explícito es requerido para convertir un Cat en un Washer
- B. La línea 7 no compila, porque no se puede hacer cast de una interfaz hacia una clase.
- C. El código compila y corre, pero el cast en la línea 7 no es requerido y puede ser eliminado.
- D. El código compila pero se produce una excepción en la línea 7, porque la conversión en runtime de una interfaz hacia una clase no es permitida.
- E. El código compila pero se produce una excepción en la línea 7, porque en runtime la clase wawa no puede ser convertida a tipo SwampThing.

10. Considere el siguiente código:

1. Raccoon rocky;
2. SwampThing pogo;
3. Washer w;
- 4.
5. rocky = **new** RaccoonQ;
6. w = rocky;
7. pogo = w;

¿Cuál de las siguientes afirmaciones es correcta? (Escoja una)

- A. La línea 6 no compila, un cast explícito es requerido para convertir Raccoon en Washer.
- B. La línea 7 no compila; un cast explícito es requerido para convertir Washer en SwampThing.
- C. El código compila y corre.

- D. El código compila pero se produce una excepción en la línea 7, porque la conversión en runtime de una interfaz hacia una clase no es permitida.
- F. El código compila pero se produce una excepción en la línea 7, porque en runtime la clase w no puede ser convertida a tipo SwampThing.

CAPITULO 5: CONTROL DE FLUJO Y EXCEPCIONES

Este capítulo cubre aspectos de los siguientes objetivos del Examen de Certificación en Java:

- Escribir código utilizando las sentencias if y switch e identificar tipos de argumentos legales para estas sentencias.
- Escribir código utilizando todas las formas de loops, incluyendo usos etiquetados y no etiquetados de break y continue, y enunciar los valores tomados por las variables durante y después de la ejecución del loop.
- Escribir código que haga uso propio de excepciones y cláusulas de manejo de excepciones (try, catch, finally), declarar métodos y sobre escribir métodos que lancen excepciones .

El control de flujo es una facilidad fundamental de casi cualquier lenguaje de programación. La secuencia, iteración, y selección son los principales elementos de control de flujo, y Java las provee de formas muy familiares para los programadores de C y C++. Adicionalmente, Java provee manejo de excepciones.

El control de secuencia es provisto simplemente con la especificación de un bloque de código, la ejecución comienza en la cima y continúa hacia la base del bloque. La iteración se caracteriza por tres clases de loops: for(), while(), y do(). La selección ocurre cuando es utilizada la sentencia if()/else o switch(). Java omite un elemento común de control de flujo, este es el caso de la sentencia goto. Cuando Java estaba siendo diseñado, el grupo responsable hizo algún análisis de un amplio cuerpo de código existente y determinó que había dos situaciones en donde el uso de goto era apropiado en el nuevo código. Estas ocasiones eran el abandono de un loop anidado y el manejo de condiciones de excepción o error. Así que los diseñadores omitieron goto y en su lugar crearon una alternativa para el manejo de estas condiciones particulares. Las sentencias break y continue que controlan la ejecución de un loop fueron extendidas para manejar los ciclos anidados, y el manejo de excepciones formalizado fue introducido, utilizando ideas similares para estas de C++.

Este capítulo discute las facilidades de control de flujo de Java. Miraremos con detenimiento los mecanismos de excepción, puesto que esta es un área que ocasiona alguna confusión. Pero, primero discutiremos los mecanismos de loops.

Los constructores de loops

Java provee tres construcciones de loops. Tomadas de C y C++, estos son los constructores: while(), do() y for(). Cada uno provee la facilidad para repetir la ejecución de un bloque de código hasta que alguna condición ocurra. Discutiremos el loop while() primero, el cual es quizás el más simple.

El loop while()

La forma general del loop while es:

1. while(boolean_condition)
2. repeated_statement

En estos constructores el elemento boolean_condition puede ser cualquier expresión que retorne un resultado booleano. Observe que esto es diferente de C y C++, donde una variedad de tipos puede ser usada: en Java usted puede utilizar solamente expresiones booleanas. Típicamente usted utilizará una comparación de alguna clase, tal como $x > 5$.

La repeated_statement será ejecutada una y otra vez hasta que la boolean_condition se haga falsa. Si la condición nunca llega a ser falsa el loop se repetirá por siempre. En

la práctica, esto realmente significa que el loop se repetirá hasta que el programa sea detenido o la máquina sea apagada.

Usted frecuentemente necesitará un loop para ejecutar no solo una única sentencia sino una secuencia de sentencias. De hecho un bloque, separado por corchetes, es tratado como una única sentencia, así que comúnmente verá un ciclo while() de esta forma:

1. while(boolean_condition){
2. do_something...
3. do_some_more...
4. }

Note que el par de corchetes ({ y }) hace que las sentencias entre ellos parezca ser como una única sentencia desde el punto de vista de los loops.

Nota: la posición exacta del corchete de abrir ({) que marca un bloque de código es tema de argumentación religiosa. Algunos programadores la colocan al final de un línea, como en los ejemplos de este libro. Otros la colocan en una nueva línea sola. Colocarla de otra manera implica colocarla en la secuencia correcta, no importa cuantos espacios, tabulaciones, y caracteres de nueva línea se coloquen antes o después del corchete de abrir. En otras palabras, este posicionamiento no es relevante para corrección sintáctica. Usted debería ser consciente, sin embargo, que el estilo utilizado para presentar las preguntas del exámen, como también las utilizadas para el código en el exámen de nivel de desarrollador, es el estilo mostrado aquí, donde el corchete de apertura es localizado al final de la línea.

Consejo: Un segundo punto del estilo se relaciona con la redundancia del uso de corchetes. Si solamente una sentencia está subordinada a la condición while() u a otra construcción, entonces usted puede omitir los corchetes. Sin embargo, es buena idea utilizar estos corchetes de todas maneras, porque al utilizarlos evita la introducción de errores si subsecuentemente agrega sentencias para ser subordinadas al ciclo.

Observe que si boolean_condition es falsa cuando se va a entrar al ciclo, entonces el cuerpo del loop no se ejecutará. Esto se relaciona con la principal característica del ciclo do, el cual discutiremos en seguida..

El loop do

La forma general del loop do es

1. do
2. repeat_statement
3. while(boolean_condition);

Este es similar al loop while() ya discutido y, como el anterior, es común tener un cuerpo conformado por varias sentencias. Bajo estas condiciones, usted puede utilizar un bloque:

1. do {
2. do_something
3. do_more
4. } while (boolean_condition);

Nuevamente, la repetición de este ciclo es terminada cuando boolean_condition se convierte en falsa. La diferencia significativa es que este ciclo siempre ejecuta el cuerpo al menos una vez, puesto que la prueba es hecha al final del cuerpo.

En general, el loop do es probablemente menos frecuente utilizado que el loop while(), pero el tercer formato es quizás el más común. La tercera forma es el loop for(), el cual se discute a continuación..

El loop for()

Un requerimiento común en programación es ejecutar un ciclo a medida que una variable es incrementada en un rango de valores entre dos límites. Esto es generalmente facilitado por un loop que utiliza la palabra reservada for. El loop while() puede conseguir este efecto, pero es más comúnmente conseguido utilizando el loop for(). Sin embargo, como con C y C++, la utilización del loop for() es más general que simplemente suministrar iteraciones a través de una secuencia de valores.

La forma general del loop for() es

1. for (Init_statement; boolean_condition; iter_expresion)
2. loop_body

De nuevo, un bloque como este puede ser utilizado

1. for (Init_statement; boolean_condition; iter_expresion) {
2. do_something
3. do_more
4. }

Las claves para este loop se encuentran contenidas en las tres partes dentro de los paréntesis siguiendo las palabras clave:

- La init_statement es ejecutada inmediatamente antes de que el loop se inicie. Es frecuentemente utilizada para colocar las condiciones iniciales. Usted notará que puede contener declaraciones de variables.
- La boolean_condition es tratada exactamente lo mismo que en el loop while(). El cuerpo del loop se ejecutará repetidamente hasta que la condición deje de ser verdadera. Como con el ciclo while(), es posible que el cuerpo de un loop for() nunca sea ejecutado. Esto ocurre si la condición ya es falsa cuando se inicia el ciclo.
- La iter_expresion (contracción de "expresión de iteración") es ejecutada inmediatamente después del cuerpo del loop, justo antes de que la prueba sea ejecutada nuevamente. Con frecuencia, esto es utilizado para incrementar un contador de loop.

Si usted ya ha declarado una variable de tipo int x, usted puede codificar una secuencia de conteo como esta:

1. for (x = 0; x < 10; x++){
2. System.out.println("value is " + x);
3. }

Esto produce 10 líneas de salida comenzando con

Value es 0

Y terminando con

Value is 9

De hecho, porque los loop for() necesitan comúnmente una variable de conteo, está permitido declarar variables en la parte de init_statement. La visibilidad de esta variable está restringida a la sentencia o bloque que esté incluida dentro del for(). Esto protege las variables de conteo del ciclo de interferencias con otros y prevenir valores de residuo de contadores de ciclo de re_usos accidentales. Lo anterior da como resultado un código como el siguiente:

1. for (int x = 0; x < 10; x++){
2. System.out.println("value is " +x);

```
3. }
```

Será útil mirar el equivalente de este código implementado utilizando el loop while():

```
1. {  
2. int x = 0;  
3. while (x < 10) {  
4. System.out.println("value is " + x);  
5. x++;  
6. }  
7. }
```

Esta versión refuerza un par de puntos. Primero, la visibilidad de la variable x, declarada en la parte init_statement del loop for(), es restringida al loop y sus partes de control (estas son, Init_statement, boolean_condition e iter_expresion). Segundo, iter_expresion es ejecutada después del resto del cuerpo del loop, efectivamente antes de que el control se devuelva a probar la condición..

El loop for() y el separador coma

El loop for() permite el uso del separador coma en una forma especial. Las partes Init_statement, e iter_expresion descritas previamente pueden actualmente contener una secuencia de expresiones en lugar de solo una. Si usted quiere una secuencia, usted debería separar estas expresiones, no con un punto y coma (el cual sería confundido con el separador entre las tres partes de la estructura de control del loop for()) sino con una coma. Este comportamiento es copiado de C y C++ donde la coma es un operador, pero en Java la coma sirve solamente como un caso especial de separador para las condiciones donde el punto y coma sería inadecuado. Este ejemplo lo demuestra:

```
1. int j, k;  
2. for (j = 3, k = 6; j + k < 20; j++, k += 2) {  
3. System.out.println("j is " + j + " k is " + k);  
4. }
```

Note que mientras usted puede usar la coma para separar varias expresiones, usted no puede mezclar expresiones con declaraciones de variables. Así esto sería ilegal:

```
1. int i;  
2. for (i = 7, int j = 0; i < 10; j++) { } //ilegal!
```

Hemos discutido las tres construcciones de loops es sus formas básicas. En la siguiente sección veremos controles de flujo más avanzados en los loops, específicamente el uso de las sentencias break y continue..

Las sentencias break y continue en los loops

Algunas veces usted necesita abandonar la ejecución del cuerpo de un loop o quizás un número de loops anidados. El equipo de desarrollo de Java reconoció esta situación como un uso legítimo para la sentencia goto. Java provee dos sentencias, break y continue, las cuales pueden ser usadas en lugar de goto para conseguir este efecto.

Uso de continue

Suponga que usted tiene un loop que está procesando un arreglo de ítems cada uno de los cuales contiene dos referencias a String. El primer String es siempre no nulo, pero el segundo podría no estar presente. Para procesar esto, usted decidirá lo que quiere, en pseudocódigo, algo a lo largo de estas líneas:

Para cada elemento del arreglo

Procesar el primer String

Si el segundo String existe

Procese el segundo String

Fin de si

Fin de para

Usted reconocerá que esto puede ser codificado fácilmente utilizando bloque if para controlar el procesamiento del segundo String. Sin embargo, usted puede también usar la sentencia continue como esta:

```
1. for (int i = 0; i < array.length; i++) {  
2. // proceso del primer String  
3. if (array[i].secondString == null) {  
4. continue;  
5. }  
6. // proceso del segundo String  
7. }
```

En este caso, el ejemplo es suficientemente simple así que usted probablemente no verá ninguna ventaja de usar la condición if() para controlar la ejecución de la segunda parte. Si el procesamiento del segundo String fue largo o quizás pesado en si mismo, usted encontrará que el uso de continue fue ligeramente más simple visualmente.

El poder real de continue es que es capaz de salir de múltiples niveles del loop.

Suponga nuestro ejemplo, en lugar de ser dos objetos String, tiene dos arreglos de valores char. Ahora necesitaremos anidar nuestros loops. Considere este ejemplo:

```
1. mainLoop: for (int i = 0; i < array.length; i++) {  
2. // proceso del primer array  
3. for (int j = 0; j < array[i].secondArray.length; j++) {  
4. if (array[i].secondArray[j] == '\u0000') {  
5. continue mainLoop;  
6. }  
7. }  
8. }
```

Advierta particularmente la etiqueta mainLoop que ha sido empleada en el for() de la línea 1. El hecho de que esto sea una etiqueta es indicado por los dos puntos. Usted puede aplicar etiquetas de esta forma al comienzo de las sentencias de loop: do, while() o for().

Aquí, cuando el procesamiento del segundo array llega a un valor cero, abandona todo el proceso no solamente para el segundo arreglo, sino para el actual objeto en el arreglo principal. Esto es equivalente a saltar la sentencia i++ en la primera sentencia for().

Usted pensaría aún que esto no es realmente una ventaja sobre el uso de sentencias if(), pero imagine que más adelante el procesamiento fue hecho entre las líneas 6 y 7 y que para encontrar el carácter cero en el arreglo fue necesario procesar más adelante, también. Para lograr esto sin utilizar continue, usted tendría que fijar una bandera en loop interior y usarla para abandonar el loop de procesamiento de más afuera. Esto puede ser hecho, pero es más confuso..

Uso de break

La sentencia break, cuando se aplica a un loop, es algo similar a la sentencia continue. Sin embargo, en lugar de cumplir prematuramente la actual iteración de un loop, break hace que todo el loop sea abandonado. Considere este ejemplo:

```
1. for (int j = 0; j < array.length; j++) {  
2. if (array[j] == null) {  
3. break; // escape del loop interior
```

```
4. }  
5. // proceso de array[j]  
6. }
```

En este caso, en vez de simplemente saltar algunos procesos para array[j] y proceder directamente a procesar array[j+1], esta versión abandona completamente el loop interior tan pronto como un elemento null sea encontrado.

Usted puede también utilizar etiquetas en las sentencias break, y como antes, debe localizar una etiqueta correspondiente en uno de los bloques encerrados. Las sentencias break y continue proveen una forma conveniente para hacer partes de un loop condicional, especialmente cuando se usa en sus formatos etiquetados.

Consejo De hecho, las etiquetas pueden ser aplicadas a cualquier sentencia, pero son solamente útiles, y el objetivo del examen requiere que usted entienda su uso, en el contexto de las construcciones de loop break y continue.

En la siguiente sección se discuten las construcciones if()/else y switch(), las cuales proveen la forma normal de implementar código condicional..

Las sentencias de selección

Java provee una opción de dos constructores de selección. Estos son los mecanismos if()/else y switch(). Usted puede fácilmente escribir código simple condicional o escoger dos rutas de ejecución basadas en el valor de una expresión booleana utilizando if()/else. Si usted necesita selecciones más complejas entre múltiples rutas de ejecución, y si un argumento apropiado está disponible para controlar la selección, entonces usted puede utilizar switch(); en otros casos usted puede usar anidamientos o secuencias de if()/else.

La estructura if() / else

La estructura if()/else toma un argumento booleano como base para la elección. Frecuentemente usted usará una expresión de comparación para proveer este argumento, por ejemplo:

```
1. if(x > 5)  
2. System.out.println("x is more than 5 ");
```

Este ejemplo ejecuta la línea 2, puesto que la prueba (x > 5) en la línea 1 retorna true. Frecuentemente usted requerirá más de una línea de código para ser condicional sobre el resultado de la prueba, y puede lograr esto usando un bloque, como en los loop discutidos previamente.

Adicionalmente, usted puede usar una parte else para la parte del código que es ejecutado bajo la condición de que la prueba retorne false. Por ejemplo:

```
1. if (x > 5){  
2. System.out.println("x is more than 5 ");  
3. }  
4. else {  
5. System.out.println("x is not more than 5 ");  
6. }
```

Más allá de esto, usted puede utilizar if()/else en forma anidada, refinando las condiciones para más especificidad, o estrechar, las pruebas en cada punto.

La estructura if()/else hace una prueba entre sólo dos posibles rutas de ejecución, aunque usted puede crear anidamientos o secuencias para seleccionar entre un gran rango de posibilidades. En la siguiente sección se discute la estructura switch(), la cual permite a un valor individual seleccionar entre múltiples posibles rutas de ejecución..

La estructura switch()

Si usted necesita hacer una selección entre múltiples alternativas de rutas de ejecución, y la selección puede estar basada en valores enteros, usted puede usar la estructura switch(). Considere este ejemplo:

```
1. switch(x) {  
2. case 1:  
3. System.out.println("Got a 1");  
4. break;  
5. case 2:  
6. case 3:  
7. System.out.println("Got a 2 or 3");  
8. break;  
9. default:  
10. System.out.println("Got something other than 1, 2, or 3");  
11. break;  
12. }
```

Note que, aunque usted no puede determinar el hecho por inspección de este código, la variable x debe ser cualquier byte, short, char, o int. No deber ser long, cualquier tipo de punto flotante, boolean, o una referencia a objeto.

La comparación de valores sigue las etiquetas case con el valor de la expresión suministrado como argumento par que el switch() determine la ruta de ejecución. Los argumentos para las etiquetas case deben ser constantes, o en últimas una expresión constante que debe ser evaluada completamente en tiempo de compilación. Usted no puede usar una variable o expresión que involucre variables.

Cada etiqueta case toma un solo argumento, pero cuando la ejecución salta a una de estas etiquetas, continúa descendiendo hasta que alcance una sentencia break. Esto ocurre aún si se pasa otra etiqueta case o la etiqueta default. Así en el ejemplo anterior, si x tiene el valor 2, la ejecución va a través de las líneas 1, 5, 6, 7, y 8 y continúa hasta la línea 12. Este requisito para break indica la terminación de la parte case y es importante. Muy frecuentemente, usted no quiere omitir el break, si no desea que la ejecución fracase. Sin embargo, para lograr el efecto mostrado en el ejemplo, donde más de un valor particular de x ocasiona la ejecución del mismo bloque de código, usted utiliza múltiples etiquetas case con un solo break.

La sentencia default es comparable con la parte else de una estructura if()/else. La ejecución salta a sentencia default si ninguno de los valores case explícitos corresponde al argumento provisto por switch(). Aunque la sentencia default es mostrada al final del bloque switch() en el ejemplo (y esto es convencional y razonablemente lógico colocarlo así), no hay reglas que requieran su colocación.

Ahora que hemos examinado las estructuras que proveen iteración y selección bajo programas de control normal, miraremos el control de flujo bajo condiciones de excepción, esto es, condiciones donde algunos problemas en tiempo de ejecución pueden surgir..

Excepciones

Algunas veces cuando un programa se está ejecutando, algo ocurre que no es realmente normal desde el punto de vista de matas de manejo. Por ejemplo, que un usuario entre un nombre de archivo inválido, o que un archivo contenga datos corruptos, un enlace de red podría fallar, o podría haber en error en el programa que ocasiona un intento por hacer un acceso ilegal a memoria, tal como la referencia a un elemento más allá del final de un arreglo.

Circunstancias de este tipo son llamadas condiciones de excepción en Java. Si usted no toma medidas para tratar una excepción, la ejecución salta al final del método actual. La excepción entonces aparece en el llamador del método, una excepción salta al final del método llamado. Esto continúa hasta que la ejecución alcanza la cima del hilo afectado, en este punto el hilo muere.

El proceso de una excepción "aparece" de cualquier causa inmediata de un problema, o porque un método llamado es abandonado y pasa la ejecución a su llamador, esto es llamado lanzar una excepción en Java. Usted escuchará otros términos usados, particularmente una excepción siendo alcanzada.

Las excepciones son actualmente objetos, y un subárbol de la jerarquía de clases es dedicada a describirlas. Todas las excepciones son subclases de una clase llamada `java.lang.Throwable`.

Control de flujo y condiciones de excepción

Uso de `try { } catch() { }`

Para interceptar, y por consiguiente controlar, una excepción, usted utiliza una estructura `try/catch/finally`. Usted sitúa las líneas de código que son parte del proceso normal de secuencia en un bloque `try`. Entonces pone el código a tratar con una excepción que se alcanzará durante la ejecución del bloque `try`, luego varios bloques `catch` son autorizados para manejarla. El código que debe ser ejecutado no importa lo que pase puede ser situado en un bloque `finally`. Tomemos un momento para considerar un ejemplo:

```
1. int x = (int)(Math.random()*5);
2. int y = (int)(Math.random()*10);
3. int [] z = new int[5];
4. try {
5. System.out.println("y/x gives " + (y/x));
6. System.out.println("y is " + y + "z[y] is " + z[y]);
7. }
8. catch (ArithmeticException e) {
9. System.out.println("Arithmetic problem " + e);
10.}
11.catch (ArrayIndexOutOfBoundsException e) {
12.System.out.println("Subscript problem " + e);
13.}
```

En este ejemplo, hay una posibilidad de una excepción en la línea 5 y en la línea 6. La línea 5 tiene la causa potencial de una división por 0, lo cual en aritmética entera resulta en el lanzamiento de una `ArithmeticException`. La línea 6 algunas veces lanzará una `ArrayIndexOutOfBoundsException`.

Si el valor de `x` llega a ser 0, entonces la línea 5 generará la construcción de una instancia de la clase `ArithmeticException` que luego es lanzada. La ejecución continúa en la línea 8, donde la variable `e` toma la referencia de la excepción recientemente creada. En la línea 9, el mensaje impreso incluye una descripción del problema, la cual viene directamente de la excepción en si. Un flujo similar ocurre si la línea 5 se ejecuta sin problema pero el valor de `y` es 5 o mayor, ocasionando un subíndice fuera de rango en la línea 6. En ese caso la ejecución salta directamente a la línea 11.

En cualquiera de estos casos, donde una excepción es lanzada en un bloque `try` y es cogida por un bloque `catch` correspondiente, la excepción se considera que ha sido manejada si: La ejecución continúa después del último `catch` como si nada hubiera pasado. Si no hay un bloque `catch` que nombre cualquier clase de excepción que ha

sido lanzada, entonces la excepción es considerada como no manejable. En estas condiciones, la ejecución generalmente sale del método directamente, como si no hubiera sido usado el try.

La tabla 5.1 resume el flujo de excepción que ocurre en los escenarios de manejo de excepciones ya discutidos. Usted no debería confiar en esta tabla para la preparación del examen, porque solamente describe la historia hasta aquí. Encontrará una referencia de estudio más completa en el resumen de final de este capítulo.

Excepción	try { }	Correspondiente catch() { }	comportamiento
No			Flujo normal
Sí	No		Termina el método
Sí	Sí	No	Termina el método
Sí	Sí	Sí	Termina el bloque try { }. Ejecuta el cuerpo del bloque catch correspondiente. Continúa el flujo normal después de los bloques catch

Tabla 5.1 descripción del flujo en condiciones de excepción sencillas..

Uso de finally

El código generalizado de manejo de excepciones tiene una parte más que la que usted vio en el último ejemplo. Este es el bloque finally. Si usted pone un bloque finally después de un try y sus bloques catch asociados, entonces una vez que la ejecución entra al bloque try el código en ese bloque finally será definitivamente ejecutado sin importar las circunstancias, bueno, más o menos definitivamente. Si una excepción se origina con un bloque catch correspondiente, entonces el bloque finally es ejecutado después del bloque catch. Si no se origina excepción, el bloque finally es ejecutado después del bloque try. Si una excepción aparece y no hay un bloque catch apropiado, entonces el bloque finally es ejecutado después del bloque try.

Las circunstancias que pueden impedir la ejecución del código en un bloque finally son:

- La muerte del hilo.
- El uso de System.exit().
- Apagar la CPU.
- Una excepción que se origine en el mismo bloque finally.

Note que una excepción en el bloque finally se comporta exactamente como cualquier otra excepción; puede ser manejada a través de try/catch. Si no se encuentra un catch, entonces el control salta fuera del método desde el punto en el cual la excepción es originada, quizás saliendo del bloque finally incompletamente ejecutado..

Capturando múltiples excepciones

Cuando usted define un bloque catch, eses bloque capturará excepciones de la clase especificada, incluyendo cualquier excepción que sea subclase de una específica. En este forma, usted puede manejar categorías de excepciones en un único bloque catch. Si usted especifica una clase de excepción en otro bloque catch particular y una clase padre de esa excepción en otro bloque catch, usted puede manejar la excepción más específica, aquellos de la subclase, separadamente de otros de la misma clase padre general. Bajo estas condiciones estas reglas aplican:

- Un bloque catch más específico debe preceder uno más general en el código. Una falla para encontrar este requisito de orden ocasiona un error de compilación.

- Solamente un bloque catch, que es el primero aplicable, será ejecutado.

Ahora miremos la estructura general para try, múltiples bloques catch, y finally.

```

1. try {
2. // sentencias ...
3. // algunas son seguras, algunas lanzarán una excepción.
4. }
5. catch (SpecificException e) {
6. // haga algo, quizás intente recuperar
7. }
8. catch (OtherException e) {
9. // manejo para la otra excepción
10.}
11. catch (GeneralException e) {
12. // manejo para GeneralException
13.}
14. finally {
15. // código que será ejecutado bajo
16. // condiciones exitosas o no exitosas
17.}
18. // más líneas de código del método ...

```

En este ejemplo, GeneralException es una clase padre de SpecificException. Varios escenarios pueden surgir bajo estas condiciones:

- No ocurren excepciones.
- Una SpecificException ocurre.
- Una GeneralException ocurre.
- Una excepción completamente diferente ocurre, la cual llamaremos UnknownException.

Si no ocurre ninguna excepción, la ejecución completa el bloque try, líneas 1, 2, 3, y 4 y después procede al bloque finally, líneas 14, 15, 16 y 17. El resto del método, línea 18 en adelante, son ejecutadas.

Si una SpecificException ocurre, la ejecución abandona el bloque try en el punto donde le excepción se origina y salta al bloque catch SpecificException. Típicamente, esto resultará en las líneas 1, 2 después 5, 6, y 7 siendo ejecutadas. Después del bloque catch, el bloque finally y el resto del método son ejecutados, líneas 14 - 17 y línea 18 en adelante.

Si una GeneralException que es una SpecificException ocurre, entonces la ejecución procede fuera del bloque try, en el bloque catch GeneralException líneas 11, 12 y 13. Después del bloque catch, la ejecución procede al bloque finally y el resto del método, como en el último ejemplo.

Si una UnknownException ocurre, la ejecución procede fuera del bloque try directamente al bloque finally. Después de que el bloque finally es completado, el resto del método es abandonado. Esto es una excepción no capturada; aparecerá en el llamador como si nunca hubiera existido un bloque try en primer lugar.

Ahora que hemos discutido qué ocurre cuando una excepción es lanzada, procederemos a ver cómo las excepciones son lanzadas y las reglas relativas a los métodos que lanzarán excepciones..

Lanzamiento de excepciones

En la última sección discutimos como las excepciones modifican el flujo de ejecución en un programa Java. Ahora continuaremos examinando como son emitidas las

excepciones en primer lugar, y como puede usted escribir métodos que usen excepciones para reportar dificultades.

La sentencia throw

Lanzar una excepción, en su forma más básica, es simple. Usted necesita hacer dos cosas. Primero, cree un instancia de un objeto que es una subclase de `java.lang.Throwable`. En seguida use la palabra reservada `throw` para lanzar actualmente la excepción. Estas dos son normalmente combinadas en un simple sentencia como esta:

```
throw new IOException(" File not Found ");
```

Hay una importante razón por la cual la sentencia `throw` y la construcción de la excepción son normalmente combinados. La excepción construye información acerca del punto en el cual fue creada, y esa información es mostrada en la pila de señales cuando la excepción es reportada. Es conveniente si la línea reportada al origen de la excepción es la misma línea de la sentencia `throw`, así que es buena idea combinar las dos partes, y `throw new xxx()` llega a ser la norma..

La sentencia throws

Usted ha visto como es fácil generar y lanzar una excepción; sin embargo, la representación completa es más compleja. Primero, como una regla general, Java requiere que cualquier método que lanzará una excepción debe ser declarado de hecho. En una forma, esto en una manera de forzar la documentación, pero verá que hay un poco más que solo eso.

Si usted escribe un método que lanzará una excepción (y esto incluye excepciones no manejadas que son generadas por otros métodos llamados por su método), entonces usted debe declarar la posibilidad de usar un sentencia `throws`. Por ejemplo, el método (incompleto) mostrado aquí puede lanzar una `MalformedURLException` o una `EOFException`.

```
1. Public void doSomeIO(String targetUrl)
2. Throws MalformedURLException, EOFException {
3. // el constructor URL puede lanzar una MalformedURLException
4. URL url = new URL(targetUrl);
5. // abra el url y léalo ...
6. // Ponga la bandera 'completado' cuando IO completado satisfactoriamente
7. // ...
8. // Si se obtiene aquí completed == false, tenemos un
9. // inesperado fin de archivo.
10.if (!completed) {
11.throw new EOFException("Invalid file contents");
12.}
13.}
```

la línea 11 demuestra el uso de la sentencia `throw` (es usual para una sentencia `throw` ser condicional en alguna forma); de otra manera el método no tiene forma de completarse exitosamente. La línea 2 muestra el uso de la sentencia `throws`. En este caso, hay dos excepciones distintas listadas que el método lanzará bajo diferentes condiciones de fallo. Las excepciones son dadas como una lista separada por comas. La sección "Capturando Múltiples Excepciones", explicada recientemente en este capítulo indicaba que la jerarquía de clases de excepciones es significativa en los bloques `catch`. La jerarquía es también significativa en la sentencia `throws`. En este ejemplo, la línea 2 puede ser acortada a `throws IOException`. Esto es porque ambos `MalformedURLException` y `EOFException` son subclases de `IOException`..

Excepciones de chequeo

Hasta el momento hemos discutido el lanzamiento de excepciones y declarar métodos que lanzarán excepciones. Hemos dicho que cualquier método que lance una excepción debe usar la sentencia `throws` para declararla. La verdad completa es tan claramente sutil como eso.

La jerarquía de clases que existe bajo la clase `java.lang.Throwable` es dividida en tres partes. Una parte contiene los errores, la cual es `java.lang.Error` y todas las subclases. Otra parte es llamada excepciones en tiempo de ejecución, y es `java.lang.RuntimeException` y todas las subclases de esta. La tercera parte contiene las excepciones de chequeo, las cuales son todas las subclases de `java.lang.Exception` (excepto `java.lang.RuntimeException` y sus subclases). La figura 5.1 muestra esto por medio de un diagrama.

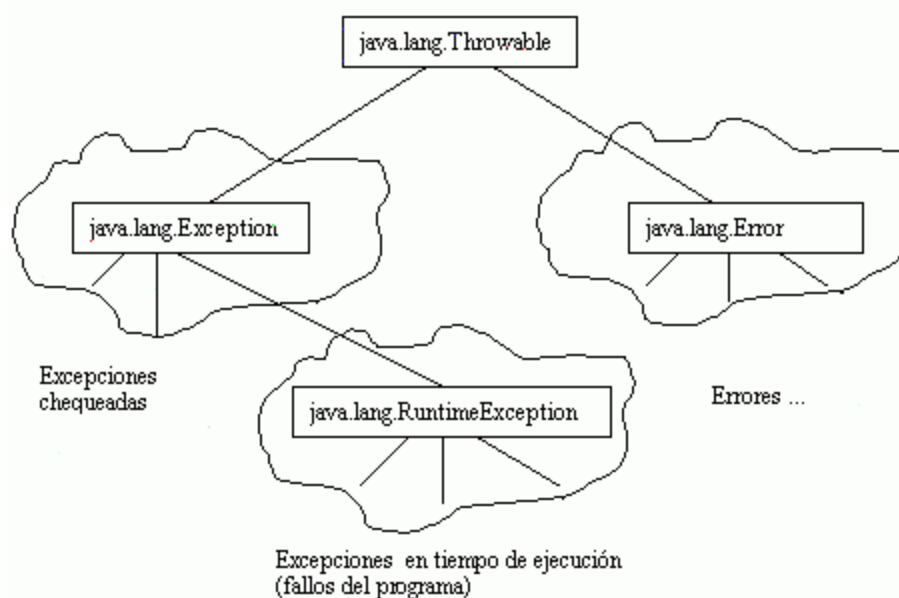


Figura 5.1 Diagrama de jerarquía de clases

Usted se preguntará porque la jerarquía es dividida y qué significan estos nombres. Las excepciones chequeadas describen problemas que pueden surgir en un programa correcto, típicamente dificultades con el ambiente como errores de usuario o problemas de I/O. Por ejemplo, intentar abrir un socket de una máquina que no está respondiendo puede fallar si la máquina remota no existe o no es suministrado el servicio solicitado. Ninguno de estos problemas indica un error de programación; es más probable que sea un problema con el nombre de la máquina (que el usuario lo digite mal) o con la máquina remota (quizás está mal configurada). Como estas condiciones pueden surgir a cualquier momento, en un programa comercial usted debe escribir el código para manejar y recobrarlas. De hecho el compilador de Java chequea que usted haya declarado correctamente lo que se va a hacer cuando ellas aparezcan, y esta es la razón por la cual se llaman excepciones de chequeo.

Las excepciones en tiempo de compilación describen los fallos de un programa. Usted podría usar una excepción de tiempo de ejecución como un deliberado control de flujo, pero podría ser una extraña forma de diseñar código y más bien un estilo pobre. Las

excepciones en tiempo de ejecución generalmente surgen de cosas como acceso a arreglos fuera de límite, y normalmente esto sería evitado por un programa correctamente codificado. Como las excepciones de tiempo de ejecución nunca aparecerán en un programa correcto, usted no requiere manejarlas. Después de todo, podrían desordenar el programa si usted ha escrito código que los estados de diseño nunca ejecutarán.

NOTA: un acercamiento al diseño de programas e implantación que es altamente efectiva en producir código robusto y confiable es conocido como programación por contrato. Brevemente, esta metodología requiere claramente definir responsabilidades para los métodos y para los llamadores de estos métodos. Por ejemplo, un método de raíz cuadrada requiere que sea llamado solamente con argumentos no negativos. Si se llama con un argumento negativo, el método reacciona lanzando una excepción, puesto que el convenio entre él y su llamador ha sido roto. Esta metodología simplifica el código, ya que los métodos solamente intentan manejar llamadas correctamente formuladas. Esto también brinda prevención de errores al abrirlas tan rápidamente como sea posible, por esta razón ellos se aseguran de que sea fijado. Usted debería usar excepciones en tiempo de ejecución para implementar esta estrategia, ya que es claramente inapropiado para el llamador tener que chequear los errores de programación; el programador debería fijarlos.

Los errores generalmente describen problemas que son suficientemente inusuales, y muy difíciles de recuperar, así que no se requiere que usted los maneje. Ellos reflejarán un error de programación, pero más comúnmente reflejan problemas del ambiente, como rebosamientos de memoria. Como con las excepciones en tiempo de ejecución, Java no requiere que usted declare cómo serán manejados estos..

Verificación de Excepciones de chequeo

Hemos declarado que de las tres categorías de excepciones, la verificación de excepciones de chequeo hacen ciertas demandas de los programadores: Usted está obligado a afirmar cómo serán manejadas las excepciones. De hecho usted tiene dos opciones. Puede poner un bloque try alrededor del código que lanzará la excepción y proporcionar un bloque catch correspondiente que se aplicará a la excepción en cuestión.

Este manejo de excepciones efectivamente funciona. Alternativamente, usted decidirá que si una excepción ocurre, su método no procederá y sería abandonado. En este caso, usted no necesita suministrar una estructura try/catch , pero deberá asegurarse sin embargo que la declaración del método incluye una parte throws que informa a los llamadores potenciales que la excepción ha surgido. Note que la insistencia en que el método sea declarado en esta forma implica la responsabilidad de que el manejo de la excepción es pasado explícitamente al llamador del método, el cual debe hacer la misma elección – si declarar o manejar la excepción. El siguiente ejemplo ilustra estas opciones:

1. public class DeclareOrHandle {
2. // este método hace que no se intente recobrar la
3. //excepción, mas bien se declara que será lanzada
4. // y usada sin un bloque try
5. public void declare(String s) throws IOException {
6. URL u = new URL(s); // lanzará una IOException
7. // haga cosas el objeto URL u ...
8. }

```

9. // este método maneja la excepción que surgirá
10.// cuando se llame al método declare().
11.// Porque este, no declara una excepción
12.// y no hace uso de la declaración throws
13.public void handle(String s) {
14.    boolean success = false;
15.    while(!success) {
16.        try {
17.            declare(s); // lanzará una IOException
18.            success = true; // ejecuta esto si declare() es exitoso
19.        }
20.        catch(IOException e) {
21.            // advierte al usuario que el String s es por
22.            // ahora inusable pregunta por uno nuevo
23.        }
24.    } //final del ciclo while(),Sale cuando success es verdadero
25.    }
26.}

```

Note que el método `declare()` no intenta manejar la excepción que surgirá durante la construcción del objeto URL. Sin embargo, el método `declare()` declara que lanzará la excepción. Por el contrario, el método `handle()` usa una estructura `try/catch` para asegurar que el control permanezca dentro del método mismo hasta que sea posible sortear el problema.

Hemos discutido el manejo de excepciones y las estructuras que le permiten a usted lanzar excepciones propias. Antes de que finalicemos las excepciones, debemos considerar una regla relacionada con la sobrescritura de métodos y excepciones. La siguiente sección discute esta regla..

Excepciones y sobrescritura

Cuando usted extiende una clase y sobrescribe un método, Java insiste en que el nuevo método no sea declarado como lanzador de excepciones de chequeo de otras clases que los han declarado en el método original. Considere este ejemplo (asuma que ellos son declarados en archivos separados; los números de línea son solamente como referencia):

```

1. public class BaseClass {
2.     public void method() throws IOException {
3.     }
4. }
5. public class LegalOne extends BaseClass {
6.     public void method() throws IOException {
7.     }
8. }
9. public class LegalTwo extends BaseClass {
10.    public void method() {
11.    }
12.}
13. public class LegalThree extends BaseClass {
14.    public void method()
15.    throws EOFException, MalformedURLException {
16.}

```

```

17.}
18.public IllegalOne extends BaseClass {
19.    public void method()
20.    throws IOException, IllegalArgumentException {
21.}
22.}
23.public IllegalTwo extends BaseClass {
24.    public void method()
25.    throws Exception {
26.}
27.}

```

Note que el método original `method()` en `BaseClass` es declarado para lanzar una `IOException`. Esto le permite, y cualquier método sobrescrito definido en una subclase, lanzar una `IOException` o cualquier objeto que sea una subclase de `IOException`. Sobrescribir métodos puede, sin embargo, no lanzar cualquier excepción de chequeo que no sea subclase de `IOException`.

Dadas estas reglas, usted notará que la línea 7 en `LegalOne` es correcta, puesto que `method()` es declarado exactamente de la misma forma que el original que es sobrescrito. De la misma forma, en la línea 18 en `LegalThree` es correcta, puesto que ambas `EOFException` y `MalformedURLException` son subclases de `IOException` – así que esto se adhiere a la regla de que nada puede ser lanzado que no sea una subclase de excepciones ya declarada. La línea 12 en `LegalTwo` es correcta, puesto que no lanza excepciones y por consiguiente no puede lanzar cualquier excepción que no sea subclase de `IOException`.

Los métodos en la línea 23 y 30 no son permitidos, ya que ambos lanzan excepciones de chequeo que no son subclases de `IOException`. En `IllegalOne`, `IllegalArgumentException` es una subclase de `Exception`; en `IllegalTwo`, `Exception` es en si misma una superclase de `IOException`. Ambas `IllegalArgumentException` y `Exception` son excepciones de chequeo, así que en los métodos que intenten lanzarlas será ilegal la sobreescritura de métodos como `method()` en `BaseClass`.

El punto de estas reglas es el relativo al uso de las variables de clases bases como referencias a objetos de tipos de subclases. El capítulo 4 explica que usted puede declarar una variable de una clase `X` y después usar las variables para referirse a cualquier objeto que es de la clase `X` o de una subclase de `X`.

Imagine que en el ejemplo antes descrito, usted ha declarado una variable `myBaseObject` de la clase `BaseClass`; la puede utilizar para referirse a cualquier objeto de las clases `LegalOne`, `LegalTwo` y `LegalThree`. (Usted no puede utilizarlas para referirse a objetos de las clases `IllegalOne` o `IllegalTwo` ya que estos objetos no fueron creados en primer lugar: Pues su código no compiló). El compilador impone chequeos sobre cómo llama usted a `myBaseObject.method()`. Estos chequeos aseguran que para cada llamada, usted tiene aseguradas dos cosas, la llamada en un bloque `try` y suministra un correspondiente bloque `catch` o ha declarado que la llamada al método en si misma lanzará una `IOException`. Ahora suponga que en tiempo de ejecución la variable `myBaseObject` fue utilizada para referirse a un objeto de la clase `IllegalOne`. Bajo estas condiciones, el compilador creería que solamente las excepciones que deben ser tratadas por él son de la clase `IOException`. Esto ocurre porque él cree que `myBaseObject` se refiere a un objeto de la clase `BaseClass`. El compilador por consiguiente no insiste que usted suministre una estructura `try/catch` que capture la `IllegalArgumentException`, ni tampoco que declare la llamada a un método que lance la

excepción. Esto significa que si la clase `IllegalOne` fuera permitida, entonces los métodos sobrescritos serían capaces de desviar el chequeo forzado por las excepciones chequeadas.

NOTA: Ya que un método sobrescrito no puede lanzar más excepciones que las que fueron declaradas en el método original, es importante considerar la probable necesidad de que las subclases definan una clase. Por ejemplo, la clase `InputStream` no puede, por sí misma, actualmente lanzar una excepción, ya que interactúa con los dispositivos reales que podrían fallar. Sin embargo, es usada como la clase base para toda la jerarquía de clases que interactúan con los dispositivos físicos: `FileInputStream` y así sucesivamente. Es importante que el método `read()` de estas subclases sea capaz de lanzar excepciones, así que el correspondiente método `read()` en la clase `InputStream` debe ser declarada para lanzar `IOException`.

Hemos visto todos los aspectos del manejo de excepciones que usted necesita para preparar el examen de Certificación y hacer uso efectivo de excepciones en sus programas. La siguiente sección resume todos los puntos clave del control de flujo y excepciones..

RESUMEN DEL CAPÍTULO

Estructuras de loop

- Tres estructuras de loop son suministradas: `while()`, `do`, y `for()`.
- Cada sentencia de loop es controlada por una expresión que debe ser de tipo boolean.
- En ambos, `while()` y `for()`, la evaluación ocurre en la "cima" del loop, así que el cuerpo no será ejecutado del todo.
- En `do`, la prueba ocurre al final del loop así que el cuerpo del loop se ejecutará al menos una vez.
- El loop `for()` toma tres elementos en sus paréntesis. El primero es ejecutado solamente una vez, antes de que el loop comience. Típicamente usted la utilizará para inicializar el loop o para declarar una variable contador de loop. La segunda es la prueba de control de loop. La tercera es ejecutada al final del cuerpo del loop, justo antes de efectuar la prueba.
- En el primer elemento entre paréntesis de una estructura `for()` puede ser declarada una variable. En este caso, el alcance de esta variable es restringido a la parte de control dentro de los paréntesis, y la siguiente sentencia. La siguiente sentencia es frecuentemente un bloque de código, en cuyo caso la variable tiene visibilidad sólo en este bloque.
- La sentencia `continue` ocasiona que la iteración actual del loop sea abandonada. El flujo se reinicia en la base del loop. Para `while()` y `do`, esto significa que la evaluación es ejecutada después. Para el loop `for()`, la tercera sentencia dentro de los paréntesis es ejecutada, continuando con la evaluación.
- La sentencia `break` abandona el loop totalmente; la evaluación no se ejecuta de ninguna forma.
- Ambas `break` y `continue` pueden tomar una etiqueta y ocasionar el salto de múltiples niveles de loops anidados. La etiqueta correspondiente debe ser colocada en el encabezado del loop y es indicado utilizando un identificador seguido de dos puntos (:).

Sentencias de Selección

- La sentencia `if()` toma un argumento boolean.

- La parte else es opcional después del if().
- La sentencia switch() toma un argumento que es de asignación compatible con int (esto es, byte, short, char, o int).
- El argumento de case debe ser una expresión constante que puede ser calculada en tiempo de compilación.
- La etiqueta case toma solamente un único argumento. Para crear una lista de valores que dirijan el mismo tiempo, el uso de múltiples sentencias case y permiten la ejecución de "fracasos".
- La etiqueta default puede ser usada en una estructura switch() par que sea ejecutada si ninguno de los case explícitos se cumple..

Flujo y Manejo de Excepciones

- Una excepción ocasiona un salto al final de un bloque encerrado try aún si la excepción ocurre dentro de un método llamado dentro del bloque try, en cuyo caso el método llamado es abandonado.
- Si cualquiera de los bloques catch asociados con el bloque try ya terminado especifica una clase excepción que es la misma, o un una clase padre , de la excepción que fue lanzada, la ejecución procede al primero de dichos bloques catch. La excepción es ahora se considera manejada. Si no se encuentra un bloque catch apropiado, la excepción es considerada no manejada.
- No obstante de si ocurre o no una excepción, o si no fue manejada, la ejecución prosigue hacia el bloque finally asociado con el bloque try, si dicho bloque finally existe.
- Si no hubo excepción, o si la excepción fue manejada, la ejecución continúa después del bloque finally.
- Si la excepción no fue manejada, el proceso se repite, buscando el siguiente bloque try adjunto. Si la búsqueda del bloque try alcanza la cima de la jerarquía del método llamado (esto es, el punto en el cual el hilo fue creado), entonces el hilo es eliminado y un mensaje y ruta de pila es arrojado a System.err..

Lanzamiento de Excepciones

- Para lanzar una excepción, se utiliza la estructura throw new XXXException();.
- Cualquier objeto que es de la clase java.lang.Exception, o cualquier subclase de ésta excepto la subclase java.lang.RuntimeException, es una excepción de chequeo.
- Si cualquier método que contiene líneas que lanzarán una excepción de chequeo, usted debe manejar la excepción utilizando una estructura try/catch o declarar que el método lanza la excepción usando la estructura throws en la declaración del método.
- Un método sobrescrito puede no lanzar una excepción de chequeo a menos que el método sobrescrito también lance la excepción o una superclase de esa excepción..

TEST YOURSELF

1. Considere el siguiente código:

```

1. for (int i = 0; i < 2; i++) {
2. for (int j = 0; j < 3; j++) {
3. if (i == j) {
4. continue;
5. }
6. System.out.println("i = " + i + "j = " + j);
7. }

```

8. }

¿ Cuáles líneas deberían ser parte de la salida ?

- A. i = 0 j = 0
- B. i = 0 j = 1
- C. i = 0 j = 2
- D. i = 1 j = 0
- E. i = 1 j = 1
- F. i = 1 j = 2

2. Considere el siguiente código :

```
1. outer : for ( int i = 0 ; i < 2 ; i++ ) {  
2.   for ( int j = 0; j < 3 ; j++ ) {  
3.     if (i == j) {  
4.       continue outer;  
5.     }  
6.     System.out.println("i = " + i + "j = " + j );  
7.   }  
8. }
```

¿ Cuáles líneas deberían ser parte de la salida?

- A. i = 0 j = 0
- B. i = 0 j = 1
- C. i = 0 j = 2
- D. i = 1 j = 0
- E. i = 1 j = 1
- F. i = 1 j = 2

3. Cuáles de las siguientes son estructuras LOOP legales (escoja una o más)

- A. while(int i < 7) { i++; System.out.println("i is " + i); }
- B. int i = 3; while(i) { System.out.println("i is " + i); }
- C. int j = 0; for(int k = 0; j + k != 10; j++, k++) { System.out.println("j is " + j + "k is " + k); }
- D. int j = 0;

do{

System.out.println("j is " + j++);

if(j == 3) { continue loop; }

} while (j < 10);

4. Cuál debería ser la salida de este fragmento de código :

```
1. int x = 0, y = 4, z = 5;  
2. if (x > 2) {  
3.   if ( y < 5 ){  
4.     System.out.println("Message one");  
5.   }  
6.   else {  
7.     System.out.println("Message two");  
8.   }  
9. }  
10. else if (z > 5) {  
11.   System.out.println("Message three");  
12. }
```

```
13.else {
14.System.out.println("Message four");
15.}
```

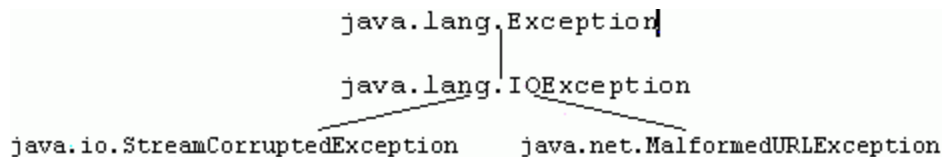
- A. Message one
- B. Message two
- C. Message three
- D. Message four

5. ¿Cuál sentencia es verdadera acerca del siguiente fragmento de código?

```
1. int j = 2;
2. switch(j) {
3. case 2:
4. System.out.println("value is two");
5. case 2 + 1:
6. System.out.println("value is three");
7. break;
8. default:
9. System.out.println("value is " + j);
10.break;
11.}
```

- A. El código es ilegal por la expresión de la línea 5.
- B. El tipo aceptable para la variable j, como argumento de la estructura switch(), deben ser cualquiera de tipo byte, short, int o long.
- C. La salida debería ser solamente el texto value is two.
- D. La salida debería ser el texto value is two seguido del texto value is three.
- E. La salida debería ser el texto value is two seguida del texto value is three, seguida del texto value is 2.

6. Considere la siguiente jerarquía de clases y el fragmento de código:



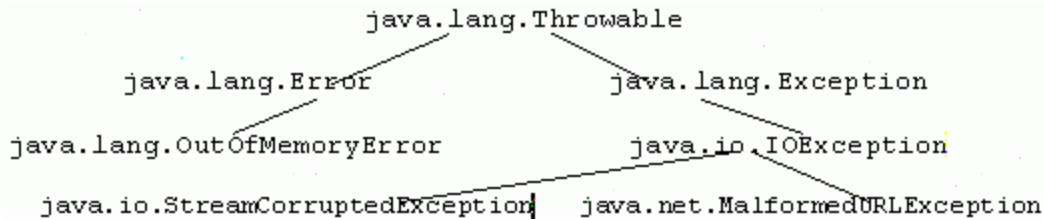
```
1. try {
2. URL u = new URL(s); // asuma que s es previamente definido
3. Object o = in.readObject(); //in es un ObjectInputStream
4. System.out.println("Success");
5. }
6. catch (MalformedURLException e) {
7. System.out.println("Bad URL");
8. }
9. catch (StreamCorruptedException e) {
10.System.out.println("Bad file contents");
11.}
12.catch (Exception e) {
13.System.out.println("General exception");
14.}
15.finally {
```

```
16.System.out.println("doing finally part");
17.}
```

```
18.System.out.println("Carrying on");
```

¿ Cuáles líneas son la salida si el método de la línea 2 lanza una MalformedURLException?

- A. Success
- B. Bad URL
- C. Bad file contents
- D. General Exception
- E. doing finally part
- F. Carrying on



Considere la siguiente jerarquía de clases y el fragmento de código:

```
1. try {
2. URL u = new URL(s); // asuma que s es previamente definido
3. Object o = in.readObject(); //in es un ObjectInputStream
4. System.out.println("Success");
5. }
6. catch (MalformedURLException e) {
7. System.out.println("Bad URL");
8. }
9. catch (StreamCorruptedException e) {
10.System.out.println("Bad file contents");
11.}
12.catch (Exception e) {
13.System.out.println("General exception");
14.}
15.finally {
16.System.out.println("doing finally part");
17.}
18.System.out.println("Carrying on");
```

¿ Cuáles líneas son la salida si el método de la línea 2 y 3 se completan exitosamente sin lanzar excepciones?

- A. Success
- B. Bad URL
- C. Bad file contents
- D. General Exception
- E. doing finally part
- F. Carrying on

8. Considere la siguiente jerarquía de clases y el fragmento de código:

```
1. try {
```

```

2. URL u = new URL(s); // asuma que s es previamente definido
3. Object o = in.readObject(); //in es un ObjectInputStream
4. System.out.println("Success");
5. }
6. catch (MalformedURLException e) {
7. System.out.println("Bad URL");
8. }
9. catch (StreamCorruptedException e) {
10. System.out.println("Bad file contents");
11. }
12. catch (Exception e) {
13. System.out.println("General exception");
14. }
15. finally {
16. System.out.println("doing finally part");
17. }
18. System.out.println("Carrying on");

```

¿ Cuáles líneas son la salida si el método de la línea 3 lanza un OutOfMemoryError?

- A. Success
- B. Bad URL
- C. Bad file contents
- D. General Exception
- E. doing finally part
- F. Carrying on

9. ¿ Cuáles de los siguientes fragmentos muestran la forma más apropiada de lanzar una excepción? Asuma que las variables no declaradas han sido apropiadamente declaradas en algún lugar, son visibles y tiene valores significativos.

A. `Exception e = new IOException("File not found");`

```

    if (!f.exists()) { // f es un objeto File
        throw e;
    }

```

B. `if (!f.exists()) { // f es un objeto File`
 `throw new IOException("File" + f.getName() + "not found");`
`}`

C. `if (!f.exists()) {`
 `throw IOException;`
`}`

D. `if (!f.exists()) {`
 `throw e;`
`}`

E. `if (!f.exists()) {`
 `throw new IOExcepcion();`
`}`

10. Dado que el método `dodgy()` lanzará una `java.io.IOException`, `java.lang.RuntimeException`, o `java.net.MalformedURLException` (el cual es una subclase de `java.io.IOException`), ¿ cuáles de las siguientes clases y conjuntos de clases son legales? (escoja una o más).

- A. `public class aClass {
 public void aMethod() {
 dodgy();
 }
}`
- B. `public class aClass {
 public void aMethod() throws java.io.IOException {
 dodgy();
 }
}`
- C. `public class aClass {
 public void aMethod() throws java.lang.RuntimeException {
 dodgy();
 }
}`
- D. `public class aClass {
 public void aMethod() {
 try {
 dodgy();
 }
 catch (IOException e) {
 e.printStackTrace();
 }
 }
}`
- E. `public class aClass {
 public void aMethod() throws java.net.MformedURLException {
 try { dodgy();
 }
 catch (IOException e) { /* ignórela */ }
 }
}`
- F. `public class anotherClass extends aClass {
 public void aMethod() throws java.io.IOException {
 super.aMethod();
 }
}`

CAPITULO 6 : OBJETOS Y CLASES

Este capítulo cubre aspectos relacionados con los siguientes objetivos del Examen de Certificación en Java:

Declara los beneficios de encapsulación en EL DISEÑO ORIENTADO A OBJETOS, y escribe la codificación que se lleva a cabo clase herméticamente encapsuladas y envía las relaciones "es un" y "tiene un."

Declara los tipos del retorno legales para cualquiera método que dan las declaraciones de todos métodos relacionados en éste o la clase del padre.

Escribe código que involucra sobrecarga o cargó excesivamente métodos paternos, Constructores; y describe el efecto de invocar estos métodos.

Declara clases, clases internas, métodos, variables del ejemplo, variables estáticas, y automáticas (método local), hace uso de todos los modificadores permitidos (tales como público, final, estático, abstracto etc.). Afirma la importancia de cada uno de estos modificadores; ambos, individualmente y en combinación, y determina el efecto de relaciones del paquete declarado por los artículos calificados por estos modificadores.

Escribir la codificación para construir ejemplos de cualquier clase concreta, normal e incluso clases cima-niveladas, clases internas, clase estática y clases anónimas internas.

Identificar correctamente las construcción de los archivos fuentes, la declaraciones de los paquete, declaraciones de las clases (de todas las formas, incluso clases internas), declaraciones de la interfase e implementaciones (por java.lang Runnable U otra interfase descrita en la prueba), declaraciones del método incluso el método principal que se usa para comenzar la ejecución de una clase), variables declaradas e identificadores.

Este capítulo discute el características orientadas a objeto de Java. Un buen código en Java requiere una comprensión legítima del orientación a objetos (OO) paradigma, y requiere un bueno dominio del idioma estos se implementas a objetos y clases. Muchos beneficios de orientación a objeto han sido el asunto de considerado debate público, pero la mayoría de programadores no se han dado cuenta de estos beneficios. En la mayoría de casos, la promesa no se ha cumplido; simplemente los programadores no han escrito objetos. En cambio, mucho programadores de C++ han escrito una forma híbrida de C con una mezcla de lenguaje procedural y codificación orientada a objeto-. Desgraciadamente, tal proximidad ha dado un levantamiento, no a algunos de los beneficios de OO, pero en cambio a todo las desventajas de ambos estilos.

Beneficios de implementación orientada a objetos

Iniciando con el examen 1.2 , se requiere entender los beneficios de diseño-orientado a objetos. Estos beneficios aumentan con dos características particulares del paradigma orientado a objetos. El primero de estos, y quizás el más importante, es la noción de un tipo de dato abstracto. El segundo, quizás conoce mejor, las extensibilidad de la herencia.

Tipos del Datos abstracto

Un tipo del datos abstracto es un nombre muy agradable para un buen encapsulamiento agregado de datos y su conducta. Considere los datos primitivos cualquiera tipo de programación que alguna vez ha usado. No sabe cómo se guardan estos artículos del datos y por la mayor parte no les importa. Qué asuntos son los funcionamientos que puede ejecutar en estos items de del datos y las condiciones limites que puede esperar trabajar propiamente. Estos tipos primitivos son lo abstracto de hecho datos, aunque el usuario no lo definió.

Su primera meta en definir una buena clase que debe estar claramente definida para los funcionamientos que se pueden aplicar a los ejemplos de esa clase. Lo siguientes es considerar cómo representar el estado de las instancias, se tiene presente se debe hacer sólo con variables de acceso privado. Se debe acceder toda conducta con sólo una vía de métodos.

Por insistir que las variables dentro de un objeto no son accesible fuera del objeto, asegura que las naturaleza de esas variables no son pertinentes fuera del objeto. Este asegura que se puede cambiar el naturaleza del almacenamiento, por mantenimiento se propone, la mejora de la ejecución, o por cualquiera otra razón, libremente.

A veces, quizás como una consecuencia de la manera que se han guardado el estado en una clase, hay condiciones que se debe aplicar a su métodos. Una condición que límite en el rango de argumentos por un método puede operar a propiamente. Por ejemplo una función raíz del cuadrado no puede operar convencionalmente con un número negativo, y además de agregar al funcionamiento que si ambos de su argumentos son más que mitad del rango entonces retornan el máximo del tipo del retorno.

Cuando encuentra una condición limite que estructuran los resultados de su opción de almacenamiento, se debe hacer una opción. Si considera que las condiciones del limita son razonables, entonces se debe hacer dos cosas. Primero, documento el limite se condiciona. El próximo, prueba el limite se condiciona a la entrada del método y, si el limite condiciona que se ha excedido, lanzando una excepción del runtime.

Como , se puede decidir que la condición del limite no sea aceptable, en que caso que debe rediseñar el almacenamiento usó en la clase.

Ahora, considera éste: si había dejado un acceso a cualquiera de las variables representara el objeto que se declara, entonces se redefinara la manera que se guarda causaría inmediatamente cualquiera otra codificación que usa estas variables del estado del objeto tiene que volver a escribir. De cualquier modo , por usar variables del miembro privado que hemos insistido que se hace métodos acabados toda interacción con este objeto y nunca por acceso directo inconstante (para que eliminemos este problema). En consecuencia, estamos hábiles a rediseñar nuestro almacenamiento interior libremente y, provisto de las firmas de todo los métodos quedan el mismo, ninguna otra codificación necesita cambiar.

A veces se dará cuenta de que los métodos que ha provisto por una clase son inadecuados. Éste usualmente no es un problema, o. Agregaría métodos nuevos provistos funcionalidad adicional. Ninguna codificación del cliente romperá los métodos creados que ya se utilizaron quedan disponibles, con las mismas firmas y las mismas semánticas. sólo las emisiones se puede levantarse durante mantenimiento es si se da cuenta de que un método que ha definido tiene un defecto fundamentalmente en el prototipo. Éste no debe pasar a menudo, implica que el funcionamiento que definió por la clase era conceptualmente defectuosa.

Encapsulation y eficiencia observada

Muchos programadores tienen (hondo-sembró) preocupaciones acerca de la ejecución que no se pueden forzar todo el acceso a sus objetos se hace a través de métodos y que se resisten creando clases con variables del miembro completamente privadas. Hay varias razones porqué éste es una proximidad imprudente. Primero, totalmente la encapsulación de clases son más probables si se usa correctamente, sobre todo si el límite se condiciona se marca apropiadamente con excepciones por eso, la codificación usada está más probable a estar correcto. Segundo, el cambio de mantenimiento cambiada es menos probable que rompa el programa, se aceptan los efectos del cambio a la clase afectada. Estas razones se caen en el título ancho de "Haría su cliente prefiere un programa lento ése trabaja y se entrega a tiempo (y se puede hacer más rápido más tarde) o un programa que se entrega tarde, trabaja incorrectamente, pero corre rápidamente?"

Hay más razones porqué totalmente clases del encapsuladas son la manera correcta comenzar un diseño. La optimización de la máquina virtual tal como HotSpot puede optimizar los métodos del acceso de la variable simples por "inlining." Esta proximidad deja el programa todo la robustez, fiabilidad, y se puede mantener que resulta de la encapsulación, mientras da la ejecución del runtime asoció con él dirige acceso inconstante. Además, si decide que la ejecución lenta de una programa sea atribuible al uso de variables privadas y acceso a métodos, entonces al cambiar la variable es estar más accesible no requiere cualquier cambio a la codificación, o dentro de o fuera de la clase. En cambio, si tiene codificación que deja de correr propiamente como un resultado de hacer dirigir acceso inconstante, el hallará eso reduce la accesibilidad de la variable pertinente requerirá garantía subsidiaria considerable cambia en muchos otro pedazos de codificación (toda codificación que hace acceso tal directo).

Reusar

Hemos discutido cómo la encapsulación puede hacer que la codificación sea más fiable y robusta. Ahora consideraremos la ventaja segunda más significativa de la programación orientada a objetos. Éste código de reusa.

Escritura buena, de una clase encapsulada usualmente requieren más trabajos en las fases iniciales que se habría requerido producir la misma funcionalidad con una programación tradicional. Mientras esto es verdadero, normalmente hallará un uso riguroso en la técnicas orientada a objetos que reducirán realmente el tiempo global. Éste pasa por dos razones.

La primera razón es que las clases robustas que produce actualmente requieren menos tiempo integrar en el final programa y menos tiempo reparar daños. La razón del segundo es ese con plan cuidadoso que puede reusar las clases igual en unas circunstancias que están diferentes del intento original de la clase.

El rehusó es realmente posible en dos maneras diferentes , usa o composición (el "tiene un" relación) o herencia (el "es un" relación). Composición es probablemente más segura y más fácil de controlar, que la herencia, porque se percibe como "OO puro," parece estar más interesante y suplicante a la mayoría de programadores.

El examen de la certificación de java no requiere discutir en detalle la técnicas de programación orientada a objetos y la debilidad de la composición de la herencia. De cualquier modo siempre, debe apreciar una significativa secuencia de pasos: si una clase es bien encapsulada, será más fácil el rehusó con buen éxito. Lo mejor si se usa una clase que es rehusada, lo mejor se probara que tendrá menos problemas.

Ahora que lo hemos discutido porqué querría escribir objeto-orientó codificación, miremos cómo se alcanza objetivo.

Implementar Relaciones orientada a objetos

No se piensa discutir la programación orientada a objetos, más bien se considera la implementaciones de las clases por que se ha dado una descripción básica.

Hay dos cláusulas que normalmente se usan cuando describe una clase clara en inglés. Éstos son "es un" y "tiene un."

Como una simplificación del funcionamiento, se usan describir el superclases y variables miembro, respectivamente. Por ejemplo considere esta descripción:

"Un hogar es una casa que tiene a una familia y un animal doméstico."

Esta descripción daría lugar a clase java con esta forma de entorno:

```
public class Home extends House {
    Family inhabitants;
    Pet thePet;
}
```

Aviso la correspondencia directa entre cláusula "es un" y la cláusula extendida. En este ejemplo, hay también una correspondencia directa entre los artículos listados después de "tiene un" y las variables del miembro. Tal correspondencia es representativa en ejemplos simples y en una situación de la prueba; de cualquier modo, se debe trabajar con esos en ejemplos reales hay otras maneras que pueden proveer una clase con atributos.

Probablemente la más importante de estas alternativas es la proximidad tomada por Java Beans, que están suministrados por un asesor y cambio de métodos que operan entre los miembros de los datos privados.

Sobrecarga y Sobrescritura

Cuando se construye unas clases y agregan métodos a ellos, hay unas circunstancias cuando se querrá reusar el mismo nombre por un método. Hay dos maneras que pueden hacer este con Java. Reusing el mismo método apellida con argumentos diferentes y quizás retorna un tipo diferente es conocido como sobrecarga. Usar el mismo método que identifica argumentos idénticos y tipo de retornos se conoce como sobrescritura.

Un nombre del método puede ser rehusado en cualquier parte, como ciertas condiciones de longitud se encuentran:

En una clase no relacionada, ninguna de las condiciones especiales se aplican y no se consideran que dos métodos se relacionen de cualquiera manera.

En la clase que define el método original, o una subclase de esa clase, el nombre del método puede ser rehusado si el argumento lista difiere en cuanto al tipo de por lo menos un argumento. Ésta carga excesiva. Es importante dar cuenta de que un cambio en el tipo de retorno como no es solo suficiente constituir una sobrecarga y está es ilegal.

En una subclase estricta de la clase que se define en el método original, el nombre del método puede ser rehusado con argumentos del mismo tipo y orden y con tipo el tipo de retorno idéntico. Ésta sobre escritura. En este caso, se adicionan

restricciones que se aplican a la accesibilidad y excepciones que se aplican por el método.

En general, una clase se considera una subclase de si misma. Esto es, si las clases de A,B y C son definidas como C extiende a B y B se extiende de A entonces las subclases de A son A,B y C . El término estricto de subclases es usado para describir las subclases excluyendo la clase misma. Así que las subclases únicas de A son solamente B y C

Ahora miremos estas ideas en detalle. Primero, consideraremos la sobrecarga de métodos.

Sobrecarga de Métodos

En Java un método es definido excepcionalmente por la combinación de su clase totalmente definidas y de los métodos , y la secuencias exactas de su tipos del argumentos.

La sobrecarga es rehusada renombrando el método en una clase o una subclase por un método diferente.

No se le relaciona con la orientación a objetos, aunque hay una correlación completamente coincidente que muestra que los lenguajes orientado a objetos son más probables soportar la sobrecarga. Se debe notar que la sobrecarga esencialmente es un truco con nombres, el título de esta sección sería "Sobrecargar nombres de métodos" en lugar de "Sobrecarga de Métodos."

Lo siguientes son todos métodos diferentes:

```
public void aMethod( String s ) { }  
public void aMethod() { }  
public void aMethod( int i, String s ) { }  
public void aMethod( String s, int i ) { }
```

Estos métodos todos tienen tipos del retorno idénticos , pero sus listas de argumentos toman cualquier tipo de argumento o de lo contrario en su orden. Sólo los tipos de argumentos son considerados no renombrados consecuentemente al método.

```
public void aMethod( int j, String name ) { }
```

no se distingue el método definido sobre la línea 3.

¿ Qué es Sobrecargar For?

¿ Porqué carga excesivamente es útil? Hay tiempos cuando creará varios métodos que ejecutan íntimamente con funciones relacionadas bajo condiciones diferentes. Por ejemplo imagine métodos que calculan el área de un triángulo. Tal método puede tomar los tres vértices de las coordenadas cartesianas y otro puede tomar las coordenadas polares. Un tercer método puede tomar las longitudes de los tres lados, mientras un cuarto puede tomar los tres ángulos y la longitud de un lado. Éstos haría desempeñar la misma función esencial, y así propicio usar el mismo nombre para los métodos.

En algunos lenguajes no se permiten la sobrecarga, tendría que pensar en usar cuatro nombres de métodos diferentes, `areaOfTriangleByCoordinate` tal como (Punto p, Punto q, Punto r), `areaOfTriangleByPolarCoordinates` (PolarPoint p, PolarPoint q, PolarPoint r), así también en adelante.

La sobrecarga verdaderamente no es nada nuevo. Casi todo lenguajes tiene un tipo de sistema para usar la sobrecarga de cierto modo, aunque no se ha dejado que el programador use libremente la sobrecarga. Considere a los operadores de la aritméticos `+`, `-`, `*`, y `/`. En la mayoría de los lenguajes se pueden usar éstos operadores con enteros o puntos flotantes.

La actual implementación, dice, la multiplicación de un entero y operándole de punto flotante generalmente envuelve codificación completamente diferente, y todavía el compilador permite usar el mismo símbolo. Porque los tipos del operadores son diferentes, el copilador puede decidir que versión de funcionamiento se debe usar. Se conoce como sobrecargar un operador y es una ley de la sobrecarga de métodos.

Así que es bastante útil, imaginar renombrar un método por que mejora la legibilidad del programa, poder usar un método relacionando varios de ellos requiere una implementación diferente. De cualquier modo que, debe restringir su uso de sobrecarga de métodos a situaciones donde los métodos verdaderamente ejecutan la misma función básica con juegos de datos diferentes. Los Métodos que ejecutan trabajos diferentes deben tener nombres diferentes.

Un último punto a considerar es el retorno de la sobrecarga de métodos. ¿ los lenguajes que tratan los métodos con sobrecarga como métodos totalmente diferentes, y como a tal pueden tener tipos del retorno diferentes (verá brevemente ésa sobre escritura de métodos no tiene esta libertad). de cualquier modo que, si

dos métodos ejecutan el mismo trabajo con datos diferente , no debe producen el mismo tipo del resultado? Generalmente esto está verdadero, y se debe esperar sobrecarga de métodos si se define con los mismos tipos de resultado. Hay una condición en particular, de cualquier modo que, está claramente sensato que se puede definir diferentes tipos de retorno en la sobrecarga e métodos.

Invocar Sobrecarga de Métodos

Cuando usted escribe múltiples métodos se ejecuta básicamente la misma función con diferentes argumentos , a menudo usted encuentra la llamamda de esos métodos en diferentes versiones. Considere un el método `printRightJustified()` esté toma dos versiones la cadena o un valor entero. La versión que toma un valor entero podía ser codificado para convertir de un entero a una cadena y entonces llama la versión que opera con objetos tipo cadena.

Puede hacer esto fácilmente. Recuerde que el compilador decide que método debe llamar simplemente mirando la liste de argumentos y que métodos de echo se encuentran relacionados. Todo que tiene que hacer es escribir la llamada del método exacta como es normal-el copilador hará el resto. Considere este ejemplo:

```
public class RightJustify {
    private static final String padding =
        "                " +
        "                " // 80 spaces
    public static void print( String s, int w ) {
        System.out.print( padding.substring( 0, w-s.length() ) );
        System.out.print( s );
    }
    public static void print( int i, int w ) {
        print( ""+i, w );
    }
}
```

Método sobrescrito

Acaba de ver que la sobrecarga está relacionado esencialmente con un truco con nombres, efectivamente se trata a la lista de argumentos como parte del método identificado.

Para sobrescribir está algo más elegante. Cuando se extiende una clase producir una nueva , que hereda y tiene acceso a todo los métodos no-privados de la clase original. A veces, de cualquier modo que, puede necesitar modificar la conducta de uno de estos métodos para satisfacer la clase nueva. En este caso realmente se quiere una redefinición del método, y éste es el propósito esencial de la sobrescritura.

Hay varias diferencias importantes entre sobrecarga y la sobrescritura:

la sobrecarga de métodos complementa el uno al otro; un método sobrescrito (grande) reemplaza el método que se sobrescribe.

La sobrecarga de métodos puede existir, en cualquiera número de veces , en la misma clase. Se puede sobrescribir un método en una clase padre al máximo una vez en cualquier una de las subclase.

La sobrecarga de argumentos puede tener una lista diferente de argumentos; Sobrescribir métodos debe tener lista de argumentos de idéntico tipo y orden (por otra parte se tratan simplemente como sobrecarga de métodos).

El tipo de retorno de la sobrecarga de métodos se puede escoger libremente.

¿Qué es sobrescritura For?

Sobrecargar permite la implementación múltiple de la misma funcionalidad de usada por el mismo nombre. Sobrescribir , en cambio, modifica la implementación de un parte en particular de la subclase.

Considere una clase que describe un rectángulo. Imaginativamente lo llamaremos Rectángulo. Esta clase tiene un método llamó setSize(), que se usa los valores de alto y ancho.

En la clase del Rectángulo se utilizan el método setSize() que utiliza variables privadas de anchura y altura por más tarde se usaran. Ahora imaginemos que creamos una clase DisplayedRectangle que es una subclase de la clase Rectángulo original. Ahora, cuando el método setSize ()sea llamado, necesitamos recordarle su comportamiento. Especificando las variables de anchura y altura, pero también se debe volver a dibujar la representación visual. Esto se cumple con la sobrescritura.

Si define un método que tiene exactamente el mismo nombre y exactamente el mismo tipo de argumentos como un método en una clase del padre entonces usted ha sobrescrito un método.

Considere este ejemplo:

```

class Rectangle {
    int x, y, w, h;

    public void setSize( int w, int h ) {
        this.w = w; this.h = h;
    }
}
class DisplayedRectangle extends Rectangle {
    public void setSize( int w, int h ) {
        this.w = w;  this.h = h;
        redisplay(); // implementation
    }
    public void redisplay() {
        // implementation not shown
    }
}

public class TestRectangle {
    public static void main( String args[] ) {
        Rectangle[] recs = new Rectangle[ 4 ];
        recs[ 0 ] = new Rectangle();
        recs[ 1 ] = new DisplayedRectangle();
        recs[ 2 ] = new DisplayedRectangle();
        recs[ 3 ] = new Rectangle();
        for ( int r = 0; r < 4; r++ ) {
            int w = ((int)( Math.random() * 400 ));
            int h = ((int)( Math.random() * 200 ));
            recs[ r ].setSize( w, h );
        }
    }
}

```

Claramente este ejemplo esta incompleto, puesto que ninguna codificación existe para causar un despliegue del objeto DisplayedRectangle, pero es suficiente para discutirlo.

En la línea 20 que se mira un arreglo recs que es creado como un arreglo del objeto rectángulo, sin embargo a las líneas 21-24 del arreglo se usa no solamente para guardar las dos instancias del rectángulo pero además las dos instancias del DisplayedRectangle. Subsiguientemente, cuando el método setSize () se llamaba, será importante que la codificación que se ejecuta debe ser asociado con el objeto actual de referencia por el elemento de l arreglo, preferiblemente siempre debe constituirse por la clase Rectángulo.

Esto es lo qué Java hace realmente, y éste es el punto esencial de sobrescribir métodos.

En orden para cualquiera método en particular puede sobrecibir este

correctamente, hay varios requisitos que se deben cumplir. Se ha mencionado algunos de éstos de han mencionado anteriormente con la sobrecarga, pero se lista aquí todos por integridad:

El nombre del método y el tipo y orden de argumentos debe ser idéntico al método en la clase del padre.

El tipo del retorno debe ser idéntico.

La accesibilidad no debe ser mas restringida que la del método original.

El método no debe lanzar excepciones de las clases que no son posible realizarlas por el método original.

Ligación retrasada o Invocación del Método Virtual

Normalmente, cuando el compilador para el lenguajes de programación no orientada a objetos desciende a través de una invocación a un método (o función o procedimiento), se determina exactamente qué código objetivo se llama para representar el lenguaje de máquina y así representarlo con dicha llamada.

En la programación orientado a objetos, éste no es posible desde la invocación de una clase del objeto para realizar la respectiva llamada, no en el tipo de la variable. En cambio, el código se generara en runtime. Ésta retardó de decisión se refiere a a la ligación retrasada (la ligación es un término que se trabaja en un linker cuando varios bits de la maquina hacen que juntos ejecuten un archivo del programa ejecutable) o la invocación del método virtual.

La Máquina Virtual de Java se ha diseñado para soportar o apoyar un sistema de programación orientada a objetos, el nivel de instrucciones se hacen a través de llamadas a métodos. El compilador solo necesita preparar la lista de argumentos y producto y la invocación del método; el trabajo de identificación y llamada es ejecutado adecuadamente por la Máquina Virtual.

Si la Máquina Virtual puede decidir que método debe ser invocado para una llamada en particular.

Invocar Métodos sobrescritos

Cuando discutimos métodos sobrecargados como se miro que se invocar una versión de un método de otro. Está también útil poder invocar un métodos sobrescritos. Considere esto cuando escribe un método sobrescrito, ese método reemplaza totalmente al método original, pero a veces usted sólo quiere agregar un pequeño comportamiento extra y conservar toda la conducta original. Se puede alcanzar éste objetivo , aunque se requiere un truco pequeño que se observa en el siguiente ejemplo.

Observe este ejemplo:

```
1.    class Rectangle {
2.        private int x, y, w, h;
3.        public String toString() {
4.            return "x = " + x + ", y = " + y + ", w = " + w + ", h = " + h;
5.        }
6.    }
7.
8.    class DecoratedRectangle extends Rectangle {
9.        private int borderWidth;
10.       public String toString() {
11.           return super.toString() + ", borderWidth = " + borderWidth;
12.       }
13.    }
```

en la línea 11 el método sobrescrito de la clase padre DecoratedRectangle utiliza el método() toString que ejecuta gran parte del trabajo. Nota la diferentes variables x, y, w, y y h en la clase del Rectángulo son privadas, seria imposible trabajar para un método sobrescrito habría estado imposible trabajar directamente con el DecoratedRectangle.

Una llamada de la forma `() super.xxx` siempre invoca la conducta que se habría usado en la sobrecarga del método que o se ha definido. No importa si se define el método padre en la superclase, o en la clase del antepasado más arriba la jerarquía, súper invoca la versión de este método que está "próximo arriba el árbol."

Resumir, éstos son los puntos importantes acerca de métodos sobrescritos:

Un método que tiene un nombre, número, tipo y orden de argumentos idéntico como un método en una clase del padre es un método sobrescrito.

Cada método de la clase padre al máximo puede ser sobrescrito una vez en cualquiera de sus subclases. (Ése está, no puede tener dos métodos idénticos en la misma clase.)

Métodos sobrescritos debe devolverle exactamente el mismo tipo como el que se sobrescribieron.

Un método del atropella no debe estar menos accesible que el método `n atropella`.

Un método sobrescrito no debe lanzar excepciones que no se declaran por el método que se sobrescribió.

Un método sobrescrito se reemplaza completamente otro método sobrescrito a menos que método que se sobrescribió se invoca deliberadamente dentro de la subclase.

Un método sobrescrito se puede invocar de dentro de la subclase si se usa el constructor `() super.xxx`, donde `[xxx]` es el nombre del método.

Hay mucho para trabajar en los métodos sobrescritos se puede recolectar mas información en el capítulo de tema de constructores.

Constructores y Subclasificación

Generalmente la herencia que el código y los datos se han definidas en una clase padre disponible para ser usada en una subclase. Éste está sujeto a la accesibilidad, por ejemplo, los items privados de la clase del padre no son directamente accesible en los métodos de la subclases, aunque existan en ella.

De hecho, no se pueden heredar constructores de la manera del normal pero se deben definir en la clase donde se encuentran. Cuando escribe código un constructor de una instancia de una clase particular, usted puede definir el constructor MyClass y este constructor puede tomar argumentos de los diferentes tipos (o unos de superclasses) de las variables y 'argumentos de la listas. En el caso que un constructor, no se encuentre definido en la clase no es suficiente que se encuentre definido en esta clase ; más bien, un constructor generalmente es disponible por una clase sólo si es definido explícitamente en la clase.

La excepción a lo anterior es el constructor estándar. El constructor estándar no toma argumentos y es creado por el compilador si no se definen ningunos otros constructores por esa clase. Aviso el constructor estándar no es heredado sino que se crea por usted por el compilador si, y sólo si, no se provisto por otros constructores en la fuente de la clase particular.

A menudo usted definirá un constructor que toma los argumentos y querrá usar esos argumentos y controlar la construcción de la parte objeto del padre. Puede pasarle el control a un constructor de la clase padre usando el

keyword super (). controlar el constructor que se este usando ,simplemente provee los argumentos apropiados.

Considere este ejemplo:

```
class Base {
    public Base( String s ) {
        // initialize this object using s
    }
    public Base( int i ) {
        // initialize this object using i
    }
}

class Derived extends Base {
    public Derived( String s ) {
        super( s );          // pass control to Base constructor at line 2
    }
    public Derived( int i ) {
        super( i );          // pass control to Base constructor at line 5
    }
}
```

el código de las líneas 12 y 15 muestra el uso de super() que controla el constructor de la clase padre de un objeto. Las definiciones de los constructores

están e la líneas 11 y 14 selecciona una manera apropiada construir la clase padre e invocar supper() con una listas de argumentos de uno de los constructores por la clase del padre. Es importante saber que el constructor de la superclase debe se llamaba antes de cualquiera hacer cualquier referencias en el objeto. Se debe garantizar esta regla que alguna vez se accede en un estado de inicialización. Generalmente la regla significa ese si supper() este aparecer en el constructor, entonces deberá ser la primera estamento.

Aunque el ejemplo muestra que la invocación del constructores con la lista de argumentos se ubica en le constructor original, éste no es un requisito. Que se escribe de la manera no es la mas aceptable, por ejemplo, si línea 15 había leído:

```
15. super( "Value is " + i );
```

Éste habría causado que el control se le pasa a la linea 2 que toma un argumento de string, al constructor que tiene lugar en la línea 5.

Cargar excesivamente Constructores

Aunque se acaba de ver que los constructores no heredan en la misma manera como se heredan los métodos, los mecanismos de sobrecarga se aplican normalmente. De hecho, el ejemplo que se discute en el uso súper () controlar la invocación de constructores paternales que se indicaron en la sobrecarga de constructores.

Además se cómo podría invocar un método de otro que se nombra con sobrecarga con una lista apropiada de parámetros. En otras situaciones sería útil invocar un constructor de otro. Imagínelo tiene un constructor que toma cinco argumentos y hace proceso considerable y inicializando el objeto. Quiere jugar armando otro constructor que toma sólo dos argumentos y le queda tres que no tienen valores.

Estaría bueno evitar la codificación del primer constructor y en cambio simplemente instala los valores de la presunción por omisión y el control paso al primer constructor. Esto es posible pero requiere un pequeño de sintaxis para lograrlo.

Usualmente invocaría un método por usar su nombre seguido por una lista de argumentos en paréntesis, e invocaría un constructor por usar el nuevo keyword, seguido por el nombre de la clase, seguido de nuevo por un lista de argumentos en paréntesis. Este puede llevarlo tratar de usar la construcción de el nuevo ClassName(args) invocando otro constructor de su clase propia. Desgraciadamente, aunque ésta está sintaxis legal, resulta ser completamente separado del objeto que se creó. La proximidad de Java toma es la de proveer otro significado para el keyword.

Observe este ejemplo:

```

public class AnyClass {
    public AnyClass( int a, String b, float c, Date d ) {
        // complex processing to initialize based on arguments
    }
    public AnyClass( int a ) {
        this( a, "default", 0.0F, new Date() );
    }
}

```

El constructor en la línea 5 toma y usa solo un argumento solo, a lo largo del programa los tres valores los toma por omisión, llama el constructor en la línea 2. La llamada se hace usando This() y su construcción en la línea 6. Como con supper(), this() se debe posicionar como el primer estamento del constructor.

Hemos indicado que cualquiera uso de o Súper() o this() en un constructor se debe poner a la primera línea. Claramente, no puede poner ambos en la primera línea. De hecho, éste no es un problema. Si le escribe una llamada a un constructor que no tiene una llamada a super(.) ni una llamada a this(.), entonces el compilador automáticamente realiza inserciones con una llamada al constructor de la clase del padre sin los argumentos. Si se hace uso de una llamada explícita a otro constructor (.), entonces no se llama el constructor de la superclase hasta que las otras lo permitan. Java insiste que el objeto sea inicializado por encima de la jerarquía de la clase descendente; porqué la llamada a supper(.) o this (.) debe ocurrir a la salida de un constructor.

Resumamos los puntos importantes acerca de constructores antes de continuar con las clases internas:

No se heredan en la misma manera los constructores que los métodos. sólo puede crear un objeto si un constructor con una lista argumentos en contraparte que uno puede definirlo en la clase misma.

Si no define un constructor en una clase, entonces el copilador presume por omisión y no toma los argumentos. Si define igual un constructor solo, no se provee esta presunción por omisión.

Es común proveer múltiples sobrecarga de constructores: estos constructores tienen listas del argumentos diferentes. Un constructor puede llamar otro usando la sintaxis this (argumentos...).

Un constructor de tarda en le funcionamiento hasta que la clase padre ha sido inicializada. Este normalmente es causado con una llamada implícita a supper () agregada por el compilador. Puede proveer su propia llamada a supper (argumentos.) controlando realizando su inicialización en el padre. Si hace esto, se debe hacer primero la declaración del constructor.

Un constructor puede usar sobrecarga versiones de constructores para apoyar su trabajo. Se invocan usando la sintaxis `this` (argumentos.) y si suministró, esta llamada debe ser la primera declaración del constructor. En tales condiciones se ejecuta la inicialización de la clase del padre en la que se sobrecarga el constructor.

Clases internas

El material que hemos mirado hasta ahora hace parte de la primeras versiones de Java. Las Clases internas son un rasgo que se agregó con el descarga de JDK 1.1. clases Internas, que a veces se llaman clases anidadas, puede darles claridad adicional a sus programas y ser mucho mas concisos.

Fundamentalmente, una clase interna es el mismo como cualquiera otra clase, pero se declara dentro de (esto es, entre la abertura y cierre) algunas otras clases. La complejidad de clases internas se observa con el particular acceso de variables y encerrar las mismas. Antes consideramos estas material, miremos la sintaxis de una clase básica interna, que es verdaderamente bastante simple. Considere este ejemplo:

```
public class OuterOne {
    private int x;
    public class InnerOne {
        private int y;
        public void innerMethod() {
            System.out.println( "y is " + y );
        }
    }
    public void outerMethod() {
        System.out.println( "x is " + x );
    }
    // other methods...
}
```

En este ejemplo no hay ningún beneficio obvio en por haber declarado la clase `InnerOne` que se colocó como una clase interna; hasta ahora estamos mirando la sintaxis.

Cuando se declara como una clase interna, la clase que se nombra internamente llega a ser parte total del nombre la clase interna. En este caso los nombres de dos clases son `OuterOne` y `OuterOne.InnerOne`. Este formato es recordativo de cómo se llamó a la clase `InnerOne` que se declaró como un paquete que se llamo de la clase `OuterOne`. Este punto de vista no es completamente erróneo.

Aunque por el propósito de renombrar hay unos beneficio de organización en poder definir una clase dentro de otra clase, éste no es el fin del cuento. Objetos que son

ejemplos de la clase interna generalmente retienen la habilidad acceder a los miembros de la clase exterior. Se discutirá en la próxima sección.

Encierra Referencia This y la Construcción de Clases Internas

Cuando se crea un ejemplo de una clase interna, se debe tener normalmente un ejemplo del preexisting de la acción de la clase exterior como del contexto. Este ejemplo de la clase exterior será accesible por el objeto interno. Considere este ejemplo, que se extiende de anterior:

```
public class OuterOne {
    private int x;
    public class InnerOne {
        private int y;
        public void innerMethod() {
            System.out.println( "enclosing x is " + x );
            System.out.println( "y is " + y );
        }
    }
    public void outerMethod() {
        System.out.println( "x is " + x );
    }
    public void makeInner() {
        InnerOne anInner = new InnerOne();
        anInner.innerMethod();
    }
    // other methods...
}
```

se observa dos cambios en el código en las versiones anteriores. Primero, en la línea 6, el innerMethod () ahora la salida no será el valor de y, que se define la clase InnerOne, pero también el valor de x, que se define en OuterOne. El cambio del segundo de las líneas 13-16, en la codificación tenemos que crea un ejemplo de la clase InnerOne e invocar el innerMethod().

La accesibilidad de los miembros de la clase son muy importantes y cruciales. Esto es posible porque la clase interna realmente tiene una referencia oculta al ejemplo de la clase exterior que era el contexto presente cuando se creó el objeto de la clase interna. En efecto se asegura que la clase interna y la clase exterior a la que pertenece, en lugar del ser la instancias interna sólo otro miembro de la instancias exterior.

A veces puede querer crear una instancias de la clase interna de un método estático, o en algunos otras situación donde no hay ningún objeto disponible. La situación se inicia con el método del main() o si necesita crear la clase interna de un método de unos objeto de una clase que no se relaciona. Puede alcanzar este

por usar al operador nuevo como si era un método del miembro de la clase exterior. Por supuesto todavía debe tener un ejemplo de la clase exterior. La codificación siguiente, que es la del método main () en aislamiento, se puede aumentar la codificación hasta que se produzca un ejemplo completo:

```
1. public static void main(String args[]) {  
2. OuterOne.InnerOne i = new OuterOneQ.new InnerOneQ;  
3. i .innerMethodO;  
4. }
```

Del punto de vista del ejemplo de la clase interna, este uso de dos declaraciones nuevas en la misma línea es una manera de hacer mas compacto el ejemplo:

```
public static void main( String[] args ) {  
    OuterOne o = new OuterOne();  
    OuterOne.InnerOne i = o.new InnerOne();  
    innerMethodQ;  
}
```

Si intenta usar el nuevas opresiones para construir instancias de una clase interna sin una referencia del prefijo a una instancias de una clase exterior, entonces el prefijo estará implícito. se asume. Esta conducta es idéntico a ese que halla con miembro ordinario accede e invocaciones del método. Como con miembro accede e invocación del método. Dentro de un método estático no hay ninguna referencia, que está porqué usted debe tomar condiciones especiales.

Clases Internas estática

Los mecanismos de la clase internos de Java dejan que una clase interna se marque como estática. Cuando se aplicada a una variable, los medios de de la variables estática que se asocia con la clase, en lugar de con cualquiera instancias en particular de la clase. Cuando es aplicado a una clase interna, el significado es similar. Específicamente, una clase estática interna no tiene ninguna referencia a un instancias que la encierra. A causa de este métodos de una clase estática interna no puede acceder variables de la instancias ejemplo de la clase que se encierra; esos métodos pueden, de cualquier modo, que el acceso a las variables estáticas de la clase que se encierra. Su comportamiento es similar a las reglas que aplican a métodos estáticos en clases ordinarias. Cuando esperarías, puede crear una instancias de una clase interna estática sin la necesidad por una instancias este presente en la clase que se encierra.

El resultado de la clase estática interna es una clase cima-nivelada con un nombre que se renombro en el esquema. De hecho, puede usar las clases internas estáticas como una extensión a empaquetamiento.

No sólo puede declara una clase dentro de otra clase, pero puede declarar también una clase dentro de un método de otra clase. Discutiremos este próximo contenido.

Clases Definidas en los Métodos Interiores

Hasta ahora se ha visto clases definidas dentro de otra clases, pero Java también deja que se defina una clase dentro de un método. Éste es superficialmente y es similar a lo que se ha visto ya, pero en este caso hay dos aspectos particulares que se debe considerar. Primero, un objeto que se creó de una clase interna dentro de un método puede tener algo de acceso a las variables del método que lo encierras. Segundo, esto es posible crear una clase anónima, literalmente una clase sin especificar su nombre, y éste puede ser muy elocuente cuando funcionamiento con eventos del tipo oyentes.

La regla que gobierna acceso a las variables de un método que lo encierra es simple. Cualquiera variable, o una variable local o un parámetro formal, se puede acceder por métodos dentro de una clase interna, con tal de que sea la marca final de la variable.

Una variable final es efectivamente es una constante, así que esto sea quizás realmente una restricción estricta, pero el punto es simplemente: un objeto que se creó dentro de un método es probable utilizarlo en la invocación del método. Desde variables locales y se destruyen los argumentos del método convencionalmente cuando su método no se utiliza, estas variables estarían inválidas por acceso a los métodos de la clase internos después de las salidas del método que las encierran.

Las otras proximidades posibles a este problema les escribirían a dos copias del mismo datos cada tiempo hizo cambió o pone método variables locales hacia las instancias de la pila. O de estas proximidades degradaría significativamente la ejecución.

Miremos un ejemplo:

```
public class MOuter {
    public static void main( String[] args ) {
        MOuter that = new MOuterQ();
        that.go( (int)(Math.random()*100), (int)(Math.random()*100));
    }

    public void go( int x, final int y ) {
        int a = x + y;
    }
}
```

```

        final int b = x - y;
    class MInner {
        public void method() {
            // System.out.println( x is " + x ) // Illegal!
            System.out.println( "y is " + y );
            // System.out.println( "a is " + a ); // Illegal!
            System.out.println( "b is " + b );
        }
    }

    MInner that = new MInner();
    that.method();
}

```

En este ejemplo la clase MInner que se define en las líneas 11-17. Dentro de él método () se tiene acceso a las variables del miembro de la clase que la encierra (como con los ejemplos previos) pero también a las variables conclusivas de método () anterior. Las líneas 12 y 14 son ilegales, porque intentan referir a variables no finales en un método (): si se incluyeron éstos en la fuente, causarían errores en el compilador.

Clases anónimas

Unas clases que se define dentro de un método no requieren un nombre. Una clase se definió de esta manera sin un nombre será llamada una clase anónima. Claramente no puede usar nuevo en la manera usual crear un ejemplo de una clase si no sabe su nombre. De hecho, se definen clases anónimas en el lugar de que se construyan:

```

1.    public void aMethod() {
2.        theButton.addActionListener(
3.            new ActionListener() {
4.                public void actionPerformed( ActionEvent e ) {
5.                    System.out.println( "The action has occurred" );
6.                }
7.            } );
8.    }

```

En este fragmento theButton a línea 2 es un objeto del Botón. Se define en las líneas 3-7 avisa al oyente la acción que realiza el botón. La declaración entera forma un argumento en el método addActionListener () llama a la línea 2; el paréntesis de cierre que completa esta llamada del método está línea delante de 7.

En la línea 3, se sigue con una nueva llamada seguida por la definición de la clase; la clase no tiene nombre pero se refiere simplemente usando el nombre de una interfase. El efecto de esta sintaxis es la declarar en una clase y no requiere pensar por una clase superior. Más allá, la clase lleva a cabo una interfase específica sin usar la implementación del keyword.

Una clase anónima es la manera mas conveniente evitar que tener que usar nombres triviales para las clases, pero se debe usar la facilidad con aflicción.

Éstos son los puntos que necesita entender acerca de clases anónimas internas para tener éxito en el Examen de la Certificación:

La clase se deben instanciar y declarar en el mismo lugar.

La declaración y instancia toma una forma

```
Xxxx nuevo () { . Cuerpo }
```

donde Xxxx es el nombre de la interfase.

Una clase anónima no puede tener un constructor. Desde que no se especifica un nombre para la clase, no puede usar ese nombre para especificar un constructor.

Características adicionales de la Clases Anónimas Internas

Los objetivos del Examen de la Certificación sólo discute clases anónimas internas que implementes a cabo interfases específicas. Hay algunos puntos del adicional que se anuncian en las clases anónimas internas que se extiende de las clases del padre. Puede hallar estos puntos útil, aunque no se utilizan en el examen.

Una clase anónima puede ser una subclase de otra clase explícita, o puede llevar a cabo un interfase explícita. Una clase anónima no puede una subclase explícita o una implementar una interfase.

Si una clase anónima se extiende una clase que ya existe, en lugar de llevar a cabo la interfase, entonces se ponen argumentos en el constructor de la superclase, como éste:

```
nuevo Botón ( "Press Me") { // define una modificación de Botón }
```

Nota esas clases anónimas llevan a implementación de interfaces, la clase

padre es `java.lang.Object`. El constructor es `java.lang.Object` no toma argumentos, así es imposible usar cualquier argumento por estas clases.

Resumen del capítulo

Hemos cubierto mucho material en este capítulo, pero todo de él es muy importante. Repasemos los puntos más importantes.

Diseño e implementación orientado a objetos

Justamente las clases encapsuladas son hechas para ser más robustas y el código sea reutilizable. La codificación envuelve la implementación de la codificación, que más allá mejora fiabilidad y lo robustece.

Justamente las clases encapsuladas esconden sus variables de estado de la interferencia externa, típicamente marca todas las variables del miembro como privadas y provee los métodos apropiados por interacción con las instancias.

Se puede reusar la Codificación simplemente reusando las clases, por composición (la relación "tiene un") o por herencia (el "es un" relación).

El "es un" relación es llevada a cabo por herencia, usa en Java el keyword que se extiende.

El "tiene un" se lleva a cabo implementados por los miembros de una variable.

Sobrecarga y sobrescritura

§ Un método puede tener el mismo nombre como otro método en la misma clase, tienen la misma forma en la sobrecarga y en la sobrescritura.

§ Una sobrecarga válida difiere en el número o tipo de sus argumentos. Diferencias en nombres del argumento no son significantes. Se permite un tipo del retorno sea diferente, pero no es suficiente para distinguir el mismo método sobrecargado.

§ Métodos que se sobrecargan con nombres de métodos diferentes y pueden trabajar con una sola clase.

§ la sobrecarga y la carga excesiva de métodos se accede simplemente por instancia correcta del método que se llama.

§ Una sobrescritura tiene argumentos idéntico en orden y tipo del retorno , y no está menos accesible que el método original. El método de sobrescritura no debe lanzar excepciones que no se declararon por el método original.

§ los métodos sobrescritos reemplaza el método original a menos que se derive una clase que hace referencia específica a ese método original usada por el constructor() super.xxx.

§ no se puede definir en la misma clase el método que se sobrescribe; más bien, se debe definir en una subclase.

§ El super.xxx () es un mecanismo que le da acceso a un método sobrescrito dentro de la subclase que define el método sobrescrito.

Constructores y Subclases

§ no se heredan Constructores en las subclases; se debe definir cada forma de constructor que requiere en ellas .

§ Una clase que no tiene constructores definió en la fuente se da exactamente un constructor. Éste es el constructor estándar; no toma argumentos y es accesible y público.

§ Un constructor puede llamar en otro constructor de su clase ayudando con su trabajo. El constructor this (). Si usa el mecanismo this (), debe ocurrir a la salida del constructor.

§ Un constructor puede llamar el constructor de la clase explícita padre por usar el mecanismo súper (). Si usa el súper (), debe ocurrir a la salida del constructor.

Clases internas

Se puede declarar Una clase en cualquiera campo. Clases definieron en otro clases, incluso ésos definieron en métodos, son llamadas clases internas.

Una clase interna en cualquier campo puede tener cualquier accesibilidad, incluso privada. De cualquier modo, una clase interna se declara local a un bloque (por ejemplo, en un método), no debe tener nada acceso al modificador. Tal clase es efectivamente privada al bloque.

Las clases Internas se definieron locales a un bloque que no es estático.

Clases que se definieron en los métodos puede ser anónimas, en este caso se deben instanciar en el mismo punto que se definen.

Clases Internas, excepto las estática, tiene una referencia implícita a las instancias a la que la encierra. Se debe proveer la instancias que la encierra a la nueva llamada que se construye len la clase interna. En muchos casos se construyen métodos de las instancias interiores de la clase que las encierran.

Clases Internas, a menos que estática, tiene acceso a las variables del ejemplo de la clase del encierra. Adicionalmente, clases internas definieron en mira del método les ha leído acceso a variables conclusivas del método del encierra.

Clases Anónimas internas llevarían a cabo interfases o se extienden otro clases.

Clases Anónimas internas no pueden tener ningún tipo de constructores explícitos.

Preguntas

1. Considera esta clase:

```
public class Test1 {  
    public float aMethod( float a, float b ) {  
    }  
  
}
```

¿Cuál de los métodos siguientes estaría legal si agregó (individualmente) a línea 4?

- A. `public int aMethod( int a, int b) { }`
- B. `public float aMethod( float a, float b) { }`
- C. `public float aMethod( float a, float b, int c ) throws Exception { }`
- D. `public float aMethod( float c, float d) { }`
- E. `private float aMethod( int a, int b, int c ) { }`

2. Considera estas clases, definió en archivos de fuente separadas:

```
1. public class Test1 {  
2.     public float aMethod( float a, float b) throws IOException {  
3.     }
```

```
1. public class Test2 extends Test1 {  
2.  
3. }
```

¿Cuál de los métodos siguientes estaría legal (individualmente) a línea 2 en clase Test2?

- A. `float aMethod( float a, float b ) { }`
- B. `public int aMethod( int a, int b ) throws Exception { }`
- C. `public float aMethod( float a, float b ) throws Exception { }`
- D. `public float aMethod( float p, float q ) { }`

3. se ha dado un plan documenta por un sistema del registro veterinario para implementar en Java. Que dice:

"Un animal doméstico tiene a un dueño, un registro de fecha, y una debida fecha de vacunación. Un gato es un animal doméstico ése tiene de una bandera indica si ha sido castrado, y una descripción textual de su clasificación."

¿ Dado que se ha definido la clase pet ya, cuál de los campos siguientes estaría apropiado por inclusión en la clase del Gato como miembros?

- A. thePet pet;
- B. Date registrado;
- C. Date vaccinationDue;
- D. [theCat] gat;
- E. neutered boolean;
- F. Encorda markings;

4. se ha dado un plan documenta por un sistema del registro veterinario para implementar en Java. Se afirma:

"Un animal doméstico tiene a un dueño, un registro fecha, y una fecha vacunación-debida. Un gato es un animal doméstico ése tiene una bandera indica si ha sido castrado, y una descripción textual de su clasificación."

¿ Dado que se ha definido la clase pet y se espera la clase del Gato se usa libremente por todas partes de la aplicación, cómo haría hace la declaración de la abertura de la clase del Gato, arriba a pero no incluso la primera abrazadera de la abertura? Use sólo estas palabras y espacios: boolean, Gato, clase, Fecha, se extiende, Objeto, Dueño, animal doméstico, privado, protegido, público, Cordón.

Public class cat extends pet

5. Considera las clases siguientes, declarado en archivos separados:

```
public class Base {
```

```

        public void method( int i ) {
            System.out.println( "Value is " + i );
        }
    }

    public class Sub extends Base {
        public void method( int j ) {
            System.out.println( "This value is " + j );
        }
        public void method( String s ) {
            System.out.println( "I was passed " + s );
        }
        public static void main( String[] args ) {
            Base b1 = new Base();
            Base b2 = new Sub();
            b1.method( 5 );
            b2.method( 6 );
        }
    }
}

```

¿Qué rendimiento resulta cuando se corre el método principal del Submarino de la clase?

A. Valor es 5

Valor es 6

B. Este valor es 5

Este valor es 6

C. Valor es 5

Este valor es 6

D. Este valor es 5

Valor es 6

E. que se pasé 5

Se pasé 6

6. Considera la definición de la clase siguiente:

1. clase pública Prueba se extiende Base{

```

2.      public Test( int j ) {
3.      }
4.      public Test( int j, int k ) {
5.          super( j, k );
6.      }
7.  }

```

¿Cuál de los siguientes formas es legitima para llamar instancias de la test ?

- A. Test t = new Test();
- B. Test t = new Test(1);
- C. Test t = new Test(1, 2);
- D. Test t = new Test(1, 2, 3);
- E. Test t = (new Base()).new Test(1);

7. Considere las siguientes definiciones de las clases:

```

1.  public class Test extends Base {
2.      public Test( int j ) {
3.      }
4.      public Test( int j, int k ) {
5.          super( j, k );
6.      }
7.  }

```

¿Cuál de las formas siguientes de constructor debe existir explícitamente en la definición de la clase inferior?

- A. Base() { }

- B. Base(int j) { }
- C. Base(int j, int k) { }
- D. Base(int j, int k, int l) { }

¿ 8. Que de las declaraciones siguientes es verdadero? (Escoge uno o más.)

- A. Una clase interna puede ser privada.
- B. Una clase interna puede ser estática.
- C. Una clase interna definió en un método siempre debe ser anónimo.
- D. Una clase interna definió en un método puede acceder todo el método con la variables locales.
- E. Construcción de una clase interna requeriría un ejemplo de la clase exterior.

9. Considera la definición siguiente:

- 1. círculo exterior de un blanco de la clase público{
- 2. public int a = 1;
- 3. private int b = 2;
- 4. public void method(final int c) {
- 5. int d = 3;
- 6. class Inner {
- 7. private void iMethod(int e) {
- 8.
- 9. }
- 10. }
- 11.)
- 12. }

¿ Cuáles variables podrían ser referenciada correctamente en la línea 8?

- A. a
- B. b
- C. c
- D. d
- E. e

10. ¿Cuál de las declaraciones siguientes es verdadero? (Escoge uno o más.)

A. Da una clase Interno no-estática que se declaró dentro de un círculo exterior que incide en la clase pública, y un constructor apropiado forma se define, una instancia Interna se construye como éste:

New outer () .new Interno ()

B. Si se define llevar a cabo el interfase una clase anónima interna dentro del círculo exterior de un blanco de la clase ActionListener, se construye como:

New outer () .new ActionListener ()

C. Dan una clase Interno es un non-estática que declara dentro de ella un círculo exterior de un blanco de la clase público y constructor apropiado para definirlo, una instancias Interna se construye en un método estático:

nuevo Interno ()

D. Un ejemplo de la clase anónimo que lleva a cabo el interfase que se construye MyInterface y volvió de un método como:

1. return new MyInterface(int x) {
2. int x;
3. public MyInterface(int x) {

4. `this.x = x;`

5. `}`

6. `};`

7. Hilos

Objetivos

Este capítulo cubre los siguientes aspectos, de los objetivos del examen de certificación en Java:

- Escribir código para definir, instanciar e iniciar nuevos hilos usando **java.lang.Thread** y **java.lang Runnable**.
- Reconocer condiciones que podrían impedir la ejecución de un hilo.
- Escribir código usando **synchronized**, **wait**, **notify**, y **notifyAll** para proteger contra problemas de acceso concurrente y para comunicación entre hilos. Definir la interacción entre hilos y entre hilos y objetos cuando se ejecuta **synchronized**, **wait**, **notify**, o **notifyAll**.

Los Hilos son una forma que tiene Java de hacer ver una sola Máquina Virtual de Java como muchas máquinas, todos corren a la vez. Este efecto, usualmente, es una ilusión: hay solamente una MVJ y muy a menudo solo una CPU; pero la CPU alterna entre las MVJ de varios proyectos para dar la impresión de muchas CPUs.

Java suministra un buen número de herramientas para crear y manejar hilos. Los hilos son una herramienta muy valiosa para simplificar el diseño de programas, permitiendo que el trabajo no relacionado o relacionado muy poco sea programado separadamente, mientras se ejecutan concurrentemente. Hay sistemas de hilos que trabajan detrás de las aplicaciones, escuchando entradas de usuarios y administrando la recolección de basura. La mejor forma para contribuir con estas facilidades es entender que son realmente los hilos.

Los objetivos de la Certificación requieren que usted se familiarice con el soporte de los hilos de Java, incluyendo los mecanismos para crear, controlar, y comunicarse entre ellos.

Fundamentos sobre hilos

El soporte de hilos de Java reside en tres lugares:

- La clase **java.lang.Thread**
- La clase **java.lang.Object**
- El lenguaje Java y la máquina virtual

Muchos soportes (pero no definitivamente todos) residen en la clase **Thread**. En Java, cada hilo corresponde a una instancia de la clase **Thread**. Estos objetos pueden estar en varios estados: en cualquier momento, al menos uno de estos se está ejecutando por CPU, mientras otros podrían estar

esperando por recursos, o esperando por un cambio para ejecutarse, o dormirse, o morir.

En orden para demostrar una comprensión de hilos, usted necesita ser capaz de responder una pocas preguntas:

- Cuándo un hilo se ejecuta, que código ejecuta este?
- En que estados puede estar un hilo?
- Como cambia un hilo su estado? Las próximas secciones verá cada una de estas preguntas.

Como se ejecuta un hilo

Para hacer que un hilo se ejecute, usted debe llamar su método **start()**. Este registra el hilo con una pieza del código del sistema llamada el programador de hilos (*thread scheduler*). El programador podría ser parte de la MVJ o ser parte del sistema operativo. El programador determina cual hilo esta actualmente corriendo en cada CPU disponible en cualquier momento dado. Note que llamar el método **start()** no hace que el hilo corra inmediatamente; este solo hace que esté elegible para correr. El hilo podría aun competir por un tiempo de CPU con todos los otros hilos.

Si todo está bien, en algún momento el programador de hilos permitirá que su hilo se ejecute. Durante su tiempo de vida, un hilo gasta algún tiempo ejecutándose y otro en cualquiera de los diversos estados de no-ejecución. En esta sección, usted puede ignorar (de momento) las preguntas de como el hilo cambie entre estados. La pregunta de hecho es: cuándo el hilo consigue ejecutarse, que hace esta ejecución?

La respuesta más simple es que este ejecuta un método llamado **run()**. Pero el método **run()** de cual objeto? Usted tiene dos elecciones:

- El hilo puede ejecutar su propio método **run()**.
- El hilo puede ejecutar el método **run()** de algunos otros objetos.

Si usted quiere que el hilo ejecute su propio método **run()**, usted necesita para la subclase la clase **Thread** y darle a su subclase un **run()**. Por ejemplo

```
1. public class CounterThread extends Thread {
2.     public void run() {
3.         for ( int i = 1; i <= 10; i++ ) {
4.             System.out.println( "Counting: " + i );
5.         }
6.     }
7. }
```

Este método **run()** solamente imprime los números de 1 a 10. Para hacer esto en un hilo, usted primero construye una instancia del CounterThread y

entonces invoca su método **start()**:

1. CounterThread ct = new CounterThread();
2. ct.start(); // **start()**, no **run()**

Que no llama directamente **run()** ; que podría solo contar hasta 10 en el actual thread. En su lugar, usted llama **start()**, en el cual la clase CounterThread hereda de su clase padre , **Thread**. El método **start()** registra el hilo (que es, ct) con el programador de hilos; eventualmente el hilo se ejecutará, y a la vez su método **run()** será llamado.

Si usted quiere que su hilo ejecute el método **run()** de cualquier otro objeto , se necesitará construir una instancia de la clase **Thread** . La única diferencia es que cuando se llama el constructor del **Thread** , se tiene que especificar cual objeto es el dueño del método **run()** que usted quiere. Para hacer esto, se invoca una forma alterna del constructor del **Thread** :

```
public Thread( Runnable target );
```

La interface Runnable describe un único método:

```
public void run();
```

Así se puede pasar cualquier objeto que quiera dentro del constructor, suministrado este implementa la interface **Runnable** (dado que este realmente tiene un método **run()** para el programador de hilos que invoca).

Habiendo construido una instancia del **Thread**, se procede como antes : Usted invoca el método **start()** . Como antes, este registre el hilo con el programador, y eventualmente el método **run()** del objetivo será llamado.

Por ejemplo, la siguiente **class** tiene un método **run()** que cuenta en forma descendente de 10 a 1:

```
1. public class DownCounter implements Runnable {
2.     public void run() {
3.         for ( int i = 10; i >= 1; i-- ) {
4.             System.out.println( "Counting Down: " + i );
5.         }
6.     }
7. }
```

Esta **class** no extiende de **Thread**. Sin embargo, esta tiene un método **run()** , y este declara que implementa la interface **Runnable** . Así cualquier instancia de la clase DownCounter es candidata para ser pasada en el constructor alterno para el **Thread**:

1. DownCounter dc = new DownCounter();
2. Thread t = new Thread(dc);

3. `t.start();`

Esta sección ha presentado dos estrategias para construir hilos. Superficialmente, la única diferencia entre estas dos estrategias es la ubicación del método **run()**. La segunda estrategia es quizás un poco mas complicada que en el caso de los ejemplos sencillos que hemos considerado. Sin embargo, hay muy buenas razones por la que usted debería hacer un esfuerzo extra.

El método **run()**, como cualquier otro método miembro, tiene permiso de acceder los datos privados, y llamar los métodos privados, de la clase de la cual es miembro. Colocando **run()** en una subclase del **Thread** principal significa que el método no puede obtener las características que este necesita (o no puede obtener estas características de una manera razonable).

Otra razón que le podría persuadir para implementar sus hilos usando **Runnable** mejor que hacer una subclase **Thread** es la única implementación de la regla de herencia. Si usted escribe una subclase de **Thread**, entonces esta no puede ser una subclase de cualquier otra.

Mientras que si usa **Runnable**, usted puede hacer subclases de cualquier otra clase padre que elija. De esta manera implementar **Runnable** es mucho mas aplicable generalmente, esto podría tener mas sentido si lo tomara como habito, mejor que complicar su código con hilos creados en diferentes formas en diferentes lugares. Finalmente, desde un punto de vista orientado a objetos, la "thread-iness" de una clase es usualmente periférica para la naturaleza esencial de cualquiera que usted este creando, y por lo tanto hacer una subclase **Thread**, la cual dice en efecto "Esta clase 'es un hilo' es probablemente una elección de diseño inapropiada. Usando **Runnable**, el cual dice "esta clase esta asociada con un hilo," tiene mas sentido.

Para resumir, hay dos enfoques para especificar cual método **run()** será ejecutado por un hilo:

- Subclase **Thread**. Defina su método **run()** en la subclase.
- Escribe una clase que implementa **Runnable**. Defina su método **run()** en la clase. Pasa una instancia de la clase dentro de su llamada al constructor **Thread**.

Cuando finaliza una Ejecución

Cuando el método **run()** retorna, el hilo ha finalizado su tarea y es considerado muerto. No hay forma fuera de este estado. Una vez un hilo esta muerto, este no podría inicializarse de nuevo; si usted quiere que la tarea del hilo sea ejecutada de nuevo, usted tiene que construir e inicializar una nueva instancia del hilo. El hilo muerto está listo para existir; este es un objeto como cualquier otro objeto, y usted puede aun acceder sus datos y

llamar su método. Lo que no puede hacer es correrlo de nuevo. En otras palabras,

- Usted no puede reiniciar un hilo muerto.
- Usted puede llamar los métodos de un hilo muerto.

Los métodos de **Thread** incluyen un método llamado **stop()**, cual forzosamente termina un hilo, colocando este en el estado muerto. Este método es desaprobado desde JDK 1.2, porque este puede causar corrupción de datos o un punto si salida si usted mata un hilo que esta en una sección critica del código. El método **stop()** por lo tanto no se cubre en gran parte del examen de Certificación. En lugar de usar **stop()**, Si un hilo necesitara terminar su actividad desde otro hilo, debería mandarle a este un **interrupt()** desde el método anulador .

Aunque se puede reiniciar un hilo muerto, si usted usa runnables, usted puede ceder la instancia antigua **Runnable** al nuevo hilo. Sin embargo, este es generalmente un diseño pobre para constantemente crear, usar, y descartar hilos, porque construir un **Thread** es una operación relativamente pesada, involucrando fuentes significativas del kernel . Es mejor crear un conjunto de hilos reutilizables que puedan ser asignados a tareas cada que sea necesario . Esto se discute mejor en el capítulo 17 (en la sección examen del desarrollador).

Estados de Hilos

Cuando se llama el **start()** de un hilo, el hilo no corre inmediatamente. Este va al estado "listo-para-correr" y permanece ahí hasta que el programador cambia este al estado "Ejecutándose". Luego el método **run()** es llamado. En el curso de ejecución **run()**, el hilo podría temporalmente ganar la CPU y entrar a algún otro estado por un while. Es importante estar enterado de los posibles estados en los que un hilo podría estar y de los disparadores que pueden causar que el hilo cambie.

Los estados de los hilos son:

Ejecutándose	El estado a el que todos los hilos aspiran
Varios estados de espera	Waiting, Sleeping, Suspended, Blocked
Listo	No espera nada excepto la CPU
Muerto	Todo hecho

La Figura 7.1 muestra los estados . Note que la figura no muestra como el estado muerto.

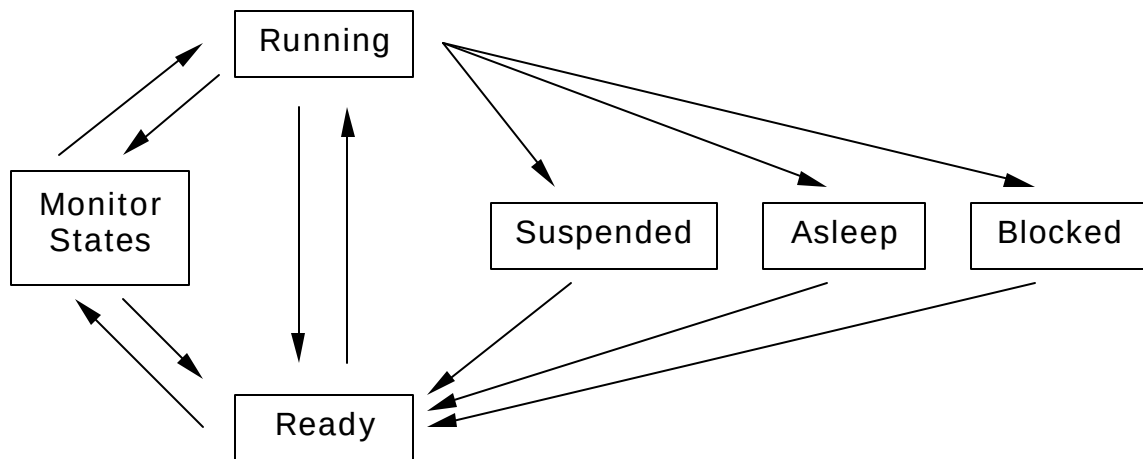
En la parte superior de la figura 7.1 esta el estado Ejecutándose . En la parte inferior esta el estado Listo. Y entre ellos están los demás estados. Un hilo en uno de estos estados intermedios espera hasta que algo pase; cuando algo eventualmente pasa, el hilo cambia al estado Ready, y eventualmente el

programador de hilos permitirá que este corra de nuevo. Note que los métodos asociados con el estado suspendido son ahora ignorados; usted no tendrá que probar con este estado o sus métodos asociados en el examen. Por esta razón, nosotros no los discutiremos en detalle en este libro.

Las flechas entre los cuadros en la Figura 7.1 representa las transiciones de los estados .Note que solamente el programador de hilos puede cambiar a hilo a listo dentro de la CPU.

FIGURA 7.1

Estados de los hilos



Después en este capítulo, usted examinara en detalle los diversos estados de espera. Por ahora, lo importante para observar en la Figura 7.1 es el flujo general :

Un hilo ejecutándose entra en un estado intermedio por alguna razón; después el hilo espera a que algo pase, y el hilo entra en el estado Ready ; después, El programador concede la CPU al hilo. La excepción para este flujo general involucra código **synchronized** y la secuencia **wait/notify** ; la parte correspondiente de la Figura 7. 1 describe como a un cuadro no definido marcado como "Monitor States." Este monitor de estados se discute después en la sección "**Monitors, wait()**, y **notify()**".

Prioridades de los hilos

Cada hilo tiene una prioridad. La prioridad es un entero de 1 a 10. Los hilos con mas alta prioridad tienen preferencia sobre los hilos con mas baja prioridad es considerado por el programador de hilos cuando este decide este podría ejecutarse. El programador generalmente elige el hilo mas alto. Si hay mas de un hilo esperando, El programador elige uno de ellos. No hay garantía que el hilo elegido sea el que mas tiempo ha estado esperando.

La prioridad por defecto es 5, pero todos los nuevos hilos creados tienen que establecer su prioridad en la creación del hilo. Para definir la prioridad de un hilo, llame el método **setPriority()**, pasándole la prioridad deseada. El método **getPriority()** retorna la prioridad de un hilo.

El fragmento de código abajo incrementa la prioridad de un hilo, suministrando la prioridad menor que 10. En lugar de colocar con código duro el valor 10, el fragmento usa la constante `MAX_PRIORITY`. La clase **Thread** también define constantes para `MIN_PRIORITY` (la cual es 1), y `NORM_PRIORITY` (la cual es 5).

1. **int** oldPriority = theThread.getPriority();
2. **int** newPriority = Math.min(oldPriority+1, **Thread**.MAX_PRIORITY);
3. theThread.setPriority(newPriority);

Los detalles específicos de cómo las prioridades de los hilos afectan la programación son dependientes de la plataforma. La especificación de Java declara que hilos podrían tener prioridades, pero esta no fija precisamente que debería hacer el programador con ellas. Esta ambigüedad es un problema: Los algoritmos que confían en la manipulación de las prioridades de los hilos podrían quizás no correr consistentemente sobre todas las plataformas.

Control de Hilos

El control de hilos, es el arte de mover hilos de estado a estado. Usted controla los hilos disparando estados de transición. Esta sección examina varias formas del estado corriendo (Running). Estas formas son

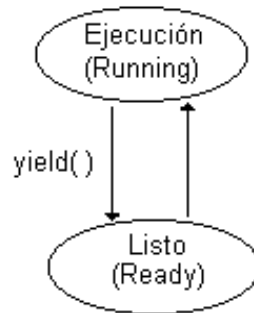
- Yielding
- Suspendido y luego resumido
- Dormido y luego waking up
- Bloqueado y luego continuando
- Esperando y luego ser anunciado

Yielding

Un hilo puede ofrecerse para moverse fuera de la CPU virtual por yielding. Una llamada al método **yield()** causa que la corriente ejecución del hilo pase al estado listo (Ready) si el programador está pendiente para correr cualquier otro hilo en lugar del hilo yielding. La transición del estado es mostrada en la Figura 7.2.

Un hilo que ha sucumbido va al estado listo (Ready). Hay dos posibles escenarios. Si cualquier otro hilo está en estado Ready, entonces el hilo que solo sucumbe podría esperar un tiempo mientras este se ejecuta de nuevo.

FIGURA 7.2



Si embargo, si ningún otro hilo está esperando, entonces el hilo que solo sucumbe se quedará para continuar ejecutándose inmediatamente. Note que la mayor parte de los programadores no paran el hilo que sucumbe de correr en favor de un hilo de baja prioridad.

El método **yield()** es un método estático de la clase **Thread**. Este siempre origina que el hilo ejecutándose corrientemente sucumba.

Al ceder se le permite a un hilo consumir un tiempo para permitir que otro hilo se ejecute. Por ejemplo, considere un applet que calcula una imagen de píxeles 300 x 300 usando un algoritmo ray-tracing. El applet podría tener un botón "Calcular" y un botón de "Interrumpir". El evento generado por el botón "Calcular" crearía e iniciaría un hilo separado, el cual llamaría un método `traceRaysQ`. Una primera parte para este código podría verse como sigue:

```
1. private void traceRays() {  
2.   for (int j=0; j<300; j++) {  
3.     for (int i=0; i<300;i++) {  
4.       computeOnePixel(i, j);  
5.     }  
6.   }  
7. }
```

Hay 90,000 valores color de píxeles que calcular. Si toma 0.1 segundo calcular el valor del color de un píxel, entonces tomaría dos horas y media calcular la imagen completa.

Suponga que después de media hora el usuario ve la imagen parcialmente y se da cuenta que algo está mal. (Quizás el punto de vista o el factor de zoom son incorrectos.) El usuario hará click en el botón "Interrumpir" de esta forma no tiene sentido seguir calculando la imagen inservible.

Desafortunadamente, el hilo que maneja la entrada GUI podría quizás no conseguir una oportunidad de ejecutarse hasta que el hilo que está ejecutando `traceRays()` suelte la CPU. De esta manera el botón "Interrumpir"

podría no tener efecto durante las otras dos horas.

Si las prioridades son implementadas en el programador, entonces bajando la prioridad del hilo ray-tracing tendrá el efecto deseado, asegurando que el hilo GUI correrá cuando este tenga algo útil que hacer. Sin embargo, esto no es confiable entre plataformas (aunque es una buena acción de cualquier forma, dado que este trabajará en muchas plataformas). La aproximación confiable es tener el hilo ray-tracing periódicamente cediendo. Si no hay entradas pendientes cuando el hilo yield es ejecutado, entonces el hilo ray-tracing no dejará la CPU. Si, de otra parte, hay una entrada para ser procesada, el hilo input-listening tendrá la oportunidad de ejecutarse.

El hilo ray-tracing puede obtener su prioridad como se muestra a continuación:

```
rayTraceThread. setPriority(Thread. NORM_PRIORITY-1);
```

El método traceRaysQ listado anteriormente puede ceder después que cada valor de pixel es calculado, después de la línea 4. La versión revisada se ve como sigue:

```
1. private void traceRays() {  
2.   for (int j=0; j<300; {  
3.     for (int i=0; i<300; {  
4.       computeOnePixel(i, j);  
5.       Thread.yield();  
6.     }  
7.   }  
8. }
```

Suspendido (Suspending)

Suspender un hilo es un mecanismo que le permite a cualquier hilo arbitrario hacer que otro hilo pase a no-ejecución (un-runnable) por un periodo de tiempo indefinido. El hilo suspendido vuelve estar en ejecución cuando algún otro hilo reanuda este. Esto podría considerarse como un mecanismo útil, pero hace que sea muy fácil causar un punto muerto en un programa usándolo, ya que un hilo no tiene control sobre si cuando este es suspendido (el control se obtiene de un hilo externo) y este podría estar en una sección crítica. El efecto exacto de suspender y resumir es mejor implementarlo usando **wait** y **notify** y quizás un pequeño cambio para el diseño. Ya que suspend() y resume() no son tenidos en cuenta y no se evalúan en el examen de certificación, no trataremos estos mas adelante.

Dormido (Sleeping)

Un hilo dormido pasa el tiempo sin hacer nada y sin usar la CPU. Una llamada al método sleep() requiere que el hilo se este ejecutando

corrientemente para cesar su ejecución por (aproximadamente) una cantidad específica de tiempo. Hay dos formas de llamar este método , dependiendo si se quiere especificar el periodo para dormir con una precisión en milisegundos o nanosegundos:

- **public static void** sleep(long milliseconds) throws InterruptedException
- **public static void** sleep(long milliseconds, **int** nanoseconds) throws InterruptedException

Note que sleep(), como yield(), es estático. ambos métodos operan sobre el hilo que se esta ejecutando corrientemente.

El diagrama de estados para dormido(sleeping) es mostrado en la Figura 7.3. Note que cuando el hilo ha finalizado el estado sleeping, este no continua la ejecución. Como podría esperarse, este entra al estado listo y solamente se ejecutará cuando el programador de hilos se lo permita. Por esta razón, podría esperarse que una llamada a sleep() bloqueará un hilo al menos por el tiempo requerido, pero este podría bloquearse por mucho mas tiempo. Esto sugiere piense con cuidado lo que haría en dado momento su diseño o antes de esperar cualquier resultado de la precisión de la versión nanosegundos del método sleep().

La clase **Thread** tiene un método llamado interrupt(). Un hilo dormido(sleeping) que recibe una llamada interrupt() cambia inmediatamente al estado listo(Ready); cuando este logra correr, ejecutara su manejador InterruptedException.

FIGURE 7.3
El estado Dormido (Sleeping)



Bloqueado(Blocking)

Muchos métodos que ejecutan entrada o salida deben esperar por alguna ocurrencia en el mundo exterior antes que ellos puedan proceder; este comportamiento es conocido como bloqueado(blocking). Un buen ejemplo es la lectura de un socket:

```

1. try {
2. Socket sock = new SocketC"magnesium", 5505);
3. InputStream istr = sock.getInputStreamQ;
4. int b = istr.readQ;
5. }
6. catch (IOException ex) {
7. // Manejador para la excepción
8. }

```

Si no está familiarizado con sockets en Java y funcionalidad de flujo, no se preocupe: Esta es cubierta en el capítulo 13. El problema no es complicada .

Esta muestra como en la Línea 4 lee un byte de una entrada de flujo que es conectada al puerto 5505 sobre una maquina llamada "magnesium."

Actualmente, la línea 4 trata de leer un byte. Si un byte está disponible (Si magnesium tiene previamente escrito un byte), entonces la línea 4 puede retornar inmediatamente y la ejecución puede continuar. Si magnesium no tiene escrito nada, sin embargo, la llamada a read() tiene que esperar. Si magnesium esta ocupada haciendo otras cosas y le toma una hora y media para escribir un byte de nuevo, entonces el llamado read() tendrá que esperar por una hora y media.

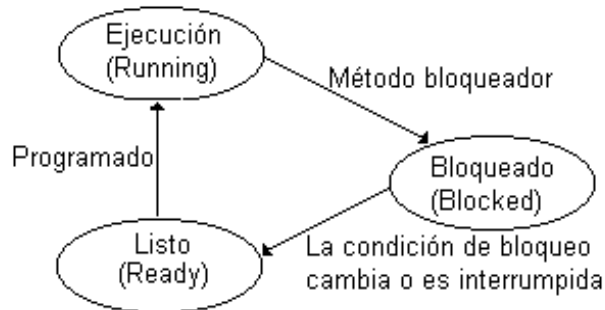
Claramente, esto podría ser un serio problema si el hilo en ejecución llama a read() en la línea 4 permaneciendo en el estado ejecución (Running) por toda la media hora . Note que esto podría pasar. En general, si un método necesita esperar una cantidad de tiempo indeterminada hasta que algún evento de I/O tome lugar, entonces un hilo en ejecución podría graciosamente pasar fuera del estado de ejecución. Todos los métodos de I/O de Java tienen este comportamiento. Un hilo que tiene este gracioso paso de esta manera pasa al estado bloqueado. La Figura 7.4 muestra estas transiciones.

Estado Bloqueado (Blocked)

En general, si usted ve un método con un nombre que sugiere que este podría no hacer nada hasta que algo este listo, por ejemplo esperar por una entrada, o esperar tForImages(), se esperaría que la llamada al hilo quedara bloqueada, llegando a estar fuera de ejecución y perdiendo la CPU, cuando el método es llamado. No necesita saber acerca de las APIs para suponer esto; este es un principio general de las APIs, ambas son el centro , en un medio Java.

FIGURE 7.4

El estado Bloqueado



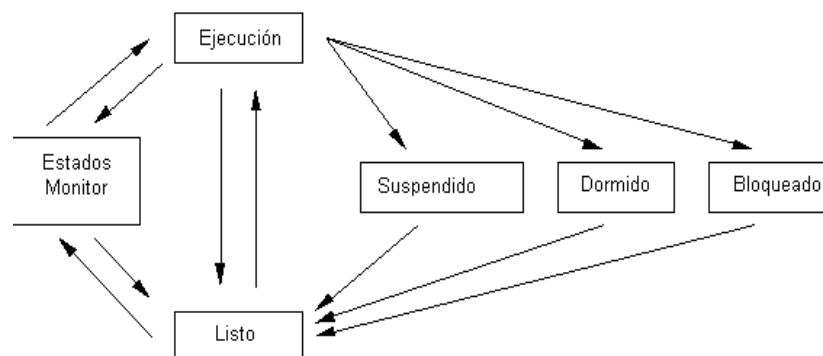
Un hilo puede también llegar a quedar bloqueado si este falla para adquirir el seguro para un monitor o si este emite un llamado al método **wait()**. Los seguros y monitores son explicados en detalle más adelante en este capítulo, en la sección "Monitores, **wait()**, y **notify()**." Internamente, muchos bloqueos para I/O, como llamadas a **read()** son discutidas, se implementan usando llamadas a **wait()** y **notify()**.

Estado Monitor

La Figura 7.5 muestra todos los estados de transición de los hilos. Los estados intermedios del lado derecho de la figura (Suspendido, Dormido, y Bloqueado) han sido discutidos en las secciones previas. Los estados de Monitor son dibujados solos en el lado izquierdo de la figura para hacer énfasis en que ellos son muy diferentes de los otros estados intermedios.

FIGURE 7.5

Estados de los hilos (reprise)



EL método **wait()** coloca un hilo en ejecución en el estado de espera (**Waiting**), y los métodos **notify()** y **notifyAll()** colocan los hilos que están en espera al estado listo (Ready). Sin embargo, Estos método son muy diferentes de **suspend()**, **resume()**, y **yield()**. Por lo siguiente, Ellos esrtan implementados en la clase **Object**, no en la clase **Thread**. Por otro lado, ellos pueden solamente ser llamados en el código **synchronized**. el estado de espera (**Waiting**), y lo asuntos asociados, son discutidos en la sección final de este capítulo. Pero primero, hay un tópico mas concerniente al control de hilos que tratar.

Implementaciones de la Programación

Históricamente, dos aproximaciones han surgido para implementar los programadores de hilos:

- Programación Preferencial
- Divisiones de tiempo o programación round-robin

Hasta ahora, las facilidades descritas en este capítulo han sido preferenciales. En programación preferencial, hay solo dos formas para que un hilo salga del estado de ejecución (Running) sin llamar explícitamente un método programador de hilos al como **waitQ** or **suspendQ**:

- Este puede cesar de estar listo para ejecutarse (llamando un método de bloqueo I/O, por ejemplo).
- Este puede conseguir ser cambiado fuera de la CPU por un hilo de mas alta prioridad que este listo para ejecutarse.

Con divisiones de tiempo, un hilo puede solamente ejecutarse por una cantidad de tiempo limitada. Este luego es cambiado al estado listo, donde este competiría con todos los otros hilos en este mismo estado. Las divisiones de tiempo aseguran de nuevo la posibilidad de que un único hilo de alta prioridad consiga el estado de ejecución y nunca estén fuera, previniendo a todos los otros hilos de hacer estas tareas. desafortunadamente, las divisiones de tiempo crean un sistema no-determinístico; en cualquier momento no se puede saber con certeza cual hilo se está ejecutando o durante cuanto tiempo este continuará la ejecución. Es natural preguntar que implementación usa Java. La respuesta es que esto depende de la plataforma; la especificación Java da implementaciones un poco libres. Las maquinas Solaris son preferenciales. Las Macintosh son por intervalos de tiempo. Las plataformas Windows fueron originalmente preferenciales, pero cambiaron a intervalos de tiempo con la versión 1.0.2 del JDK.

Monitores, **wait()**, y **notify()**

Un monitor es un objeto que puede bloquear y revivir hilos. El concepto es simple, pero toma un poco de trabajo entender para que son buenos los

monitores y como usarlos eficientemente.

La razón de tener monitores es que algunas veces un hilo no puede ejecutar su tarea hasta que un objeto alcance un determinado estado. Por ejemplo, considere una clase que maneja peticiones para escribir por una salida estandar:

```
1. class Mailbox {  
2.     public boolean request;  
3.     public String message;  
4. }
```

La intención de esta clase es que un cliente pueda colocar algún mensaje de algún valor, luego colocar la solicitud a verdadero:

```
1. myMailbox.message = "Hello everybody."  
2. myMailbox.request = true;
```

Aquí podría haber un hilo que chequee la solicitud; al encontrar este en verdadero, el hilo podría escribir un mensaje para el sistema, fuera y luego colocar la solicitud a falso. (Colocando la solicitud a falso indica que el objeto mailbox esta listo para manejar otra solicitud) es atractivo para implementar el hilo como esto:

```
1. public class Consumer extends Thread {  
2.     private Mailbox myMailbox;  
3.  
4.     public Consumer(Mailbox box) {  
5.         this.myMailbox = box;  
6.     }  
7.  
8.     public void run() {  
9.         while (true) {  
10.            if (myMailbox.request) {  
11.                System.out.println(myMailbox.message);  
12.                myMailbox.request = false;  
13.            }  
14.  
15.            try {  
16.                sleep(50);  
17.            }  
18.            catch (InterruptedException e) { }  
19.        }  
20.    }
```

El hilo consumidor entra en un ciclo infinito, chequeando las peticiones cada 50 milisegundos. Si hay una solicitud (línea 10), el consumidor escribe el mensaje a la salida estandar (línea 11) y luego coloca las solicitudes a falso

para mostrar que esta listo para mas solicitudes.

La clase Consumer puede verse bien aparentemente, pero esta tiene dos problemas serios:

- La clase Consumer accesa datos internos de la clase Mailbox , introduciendo la posibilidad de corrupción. En un sistema time-sliced, el hilo consumidor podría solo posiblemente ser interrumpido entre las líneas 10 y 11. El hilo interruptor podría solo posiblemente ser un cliente que coloca mensajes para sus propios mensajes (ignorando la convención de chequear solicitudes para ver si el manejador esta disponible). El hilo consumidor podría enviar un mensaje equivocado.
- La elección de 50 milisegundos para el retardo puede no ser ideal. Algunas veces 50 milisegundos será demasiado tiempo, y los clientes recibirán un servicio lento; algunas veces 50 milisegundos será muy frecuente, y los ciclos serán desaprovechados. Un hilo que quiere enviar un mensaje tiene un dilema similar si este encuentra las solicitudes colocadas por solicitud de bandera: El hilo estaría off por un rato , pero durante cuanto tiempo? Idealmente, Estos problemas podrían ser solucionados haciendo algunas modificaciones a la clase Mailbox:
- La clase mailbox podría ser capaz de proteger sus datos de clientes irresponsables.
- Si el mailbox no esta disponible—lo que significa, si la bandera de solicitud ya esta colocada—entonces un cliente consumidor no tendría que adivinar cuanto tiempo tener que esperar antes de chequear la bandera de nuevo . El manejador llamaría el cliente cuando el tiempo este bien.

Los monitores de Java soportan direcciones estos asuntos lo suministran el siguiente código:

- Una clave para cada objeto
- La palabra clave **synchronized** para accesar un objeto llave
- Los métodos **wait()**, **notify()**, y **notifyAll()**, los cuales le permiten al objeto controlar los hilos cliente

La sección siguiente describe las llaves, código **synchronized** ,y los métodos **waitQ**, **notifyQ**, y **notifyAll()** ,y muestra como estos pueden ser usados para hacer el código del hilo mas robusto.

Los Objetos llave y Sincronización

Cada objeto tiene una llave. En cualquier momento, la llave es controlada por, al menos , un único hilo. La llave controla el acceso a los objetos de

código sincronizado. Un hilo que quiere ejecutar un código sincronizado de un objeto debe primero tratar de adquirir la llave del objeto. Si la llave está disponible—lo que significa, si este no está ya controlado por otro hilo — entonces todo esta bien. Si la llave esta bajo el control de otro hilo, entonces el hilo trata de ir a buscar en el estado llave y solo estará listo cuando la llave este disponible. Cuando un hilo que posee una llave pasa fuera del código sincronizado, el hilo automáticamente abandona la llave . Todo esto chequeo de llaves y cambios de estados es hecho detrás de la escena; La única programación explícita que usted necesita es declarar el código sincronizado(**synchronized**).

La figura 7.6 muestra el estado de búsqueda de llaves. Esta figura es el primer estado en nuestra expansión de los estados monitores, como se describe en la Figura 7.5.

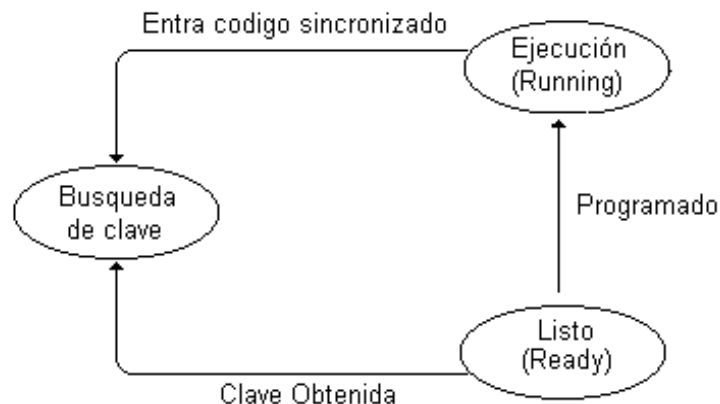
Hay dos formas para marcar un código como sincronizado:

- Sincronizar un método entero colocando el modificador `synchronized` en la declaración del método. Para ejecutar el método, un hilo debe adquirir la llave del objeto que posee el método.
- Sincronizar un grupo de métodos encerrando las líneas deseadas del código con paréntesis (`{ }`) e insertando la expresión **synchronized**(someOb)fCt) antes de abrir el paréntesis. Esta técnica le permite sincronizar el bloque sobre la llave de cualquier objeto, no necesariamente el objeto que posee el código.

La primera técnica esta muy lejos de ser común; la sincronización sobre otros objetos a los que poseen el código **synchronized** puede ser explícita. peligrosa

FIGURA 7.6

Los estados de búsqueda de llaves



El examen de certificación requiere que usted sepa como aplicar la segunda técnica, pero el examen no hace que usted piense en escenarios complicados de sincronización sobre objetos externos. La segunda técnica es discutida al final de este capítulo.

La sincronización hace fácil poner en orden algunos de los problemas con el mailbox

```
1.class Mailbox {
2.  private boolean    request;
3.  private String     message;
4.
5.  public synchronized void storeMessage(String message) {
6.    request = true;
7.    this. message = message;
8.  }
9.
10. public synchronized String retrieveMessage() {
11.  request = false;
12.  return message;
13. }
14. }
```

Ahora la bandera de solicitud y el mensaje string son privados , así ellos pueden solamente ser modificados por medio de los métodos **públicos** de la clase. Dado que storeMessage() y retrieveMessage() son **synchronized**, no hay peligro de que un hilo produzca mensajes que dañe la bandera y eche a perder cosas como hilos consumidores de mensajes o viceversa.

La clase Mailbox es ahora segura para sus clientes, pero los clientes aun tienen problemas. Un cliente productor de mensajes debería solamente llamar storeMessage() cuando la solicitud de bandera este en falso; un cliente consumidor de mensajes debería solamente llamar retrieveMessage() cuando la solicitud de bandera este en verdadero. En la clase Consumer de la sección previa, los hilos consumidores principales verifican la solicitud de bandera cada 50 segundos. (Presumiblemente un hilo productor de mensajes debería hacer algo similar.) ahora la solicitud de bandera es privada, así usted debería encontrar otra forma.

Es posible traer a colación un numero de formas ágiles para los hilos que están continuamente verificando el correo, pero todas las aproximaciones es volver atrás. El correo llega a estar disponible o no disponible basado en los cambios de su propio estado.

El correo debería estar encargado del progreso de los clientes. Los métodos de Java **wait()** y **notify()** suministran los controles necesarios, como se vera en la próxima sección.

wait() y notify()

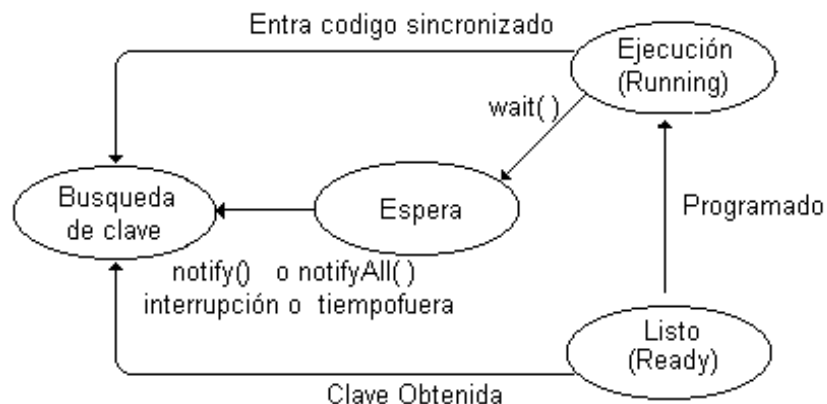
Los métodos **wait()** y **notify()** suministran una forma para que un objeto compartido coloque en pausa un hilo cuando este no disponible para aquel hilo, y le permita al hilo continuar cuando sea apropiado. Los hilos en si mismos no tiene nunca que chequear el estado de los objetos compartidos.

Un objeto que controla sus hilos clientes de esta manera es conocido como un monitor. En la estricta terminología de Java, un monitor es cualquier objeto que tiene algún código **sincronizado**. Para ser realmente útil, muchos monitores hacen uso de los métodos **wait()** y **notify()**. De esta manera la clase Mailbox es un monitor.

La Figura 7.7 muestra los estados de transición de **wait ()** y **notifyO**.

Ambos **wait()** y **notify()** tiene que ser llamados en un código **sincronizado**. Un hilo que llama a **wait()** suelta la CPU virtual; al mismo tiempo este suelta la llave. Este entra a un grupo de hilos en espera, el cual es administrado por el objeto cuyo método **wait()** fue llamado. Cada objeto tiene tal grupo. El código siguiente muestra como el método `retrieveMessageO` de la clase Mailbox podría ser modificado para comenzar a tener ventaja de la llamada a **wait()**.

FIGURA 7.7
Estados Monitor



```
1. public synchronized String retrieveMessage() {
2.     while (request == false) {
3.     try{
4.         wait();
5.     catch (InterruptedException e) { }
6. }
```

```
7. request = false;  
8. return message;  
9. }
```

Ahora considere que pasa cuando un hilo consumidor de mensajes llama este método. La llamada podría verse como esto:

```
myMailbox. retrieveMessage();
```

Cuando un hilo consumidor de mensajes llama este método, el hilo debería primero obtener la clave para myMailbox. Adquiriendo la clave podría pasar inmediatamente o este podría incurrir en una espera si cualquier otro hilo esta ejecutando cualquier código **sincronizado** de myMailbox. De una u otra forma, eventualmente el hilo consumidor tiene la clave y comienza a ejecutarse en la línea 2. El código primero verifica la bandera de solicitud. Si la bandera no se ha colocado, entonces myMailbox no tiene mensajes que recuperar para el hilo. En este caso el método **wait()** es llamado en la línea 4 (este puede lanzar un InterruptedException, de esta manera el código try/catch es requerido, y el while verificará la condición). Cuando la línea 4 se ejecuta, el hilo consumidor suspende la ejecución; Esto también libera la clave para myMailbox y entra al grupo de hilos en espera administrados por myMailbox.

El hilo consumidor ha estado exitosamente evitando de dañar el monitor myMailbox. Desafortunadamente, Este esta introducido en el grupo de monitores de hilos en espera. Cuando el monitor cambia a un estado donde este puede suministrar el consumidor con algo que hacer, entonces algo tendrá que haber hecho para llevar el consumidor fuera del estado de espera. Esto se hace llamando a notify() cuando la bandera de solicitud del monitor se coloca a verdadero, lo cual solamente pasa en el método storeMessage(). El método storeMessage() se ve como sigue:

```
1. public synchronized void storeMessage(String message) {  
2.   this.message = message;  
3.   request = true;  
4.   notifyO;  
5. }
```

En la línea 4, el código llama a **notify()** solo después de cambiar el estado del monitor. Lo que **notify()** hace es seleccionar uno de los hilos del grupo de monitores en espera y cambiar este al estado de búsqueda de clave. Eventualmentel el hilo obtendrá la clave de mailbox y proceder con la ejecución.

Ahora imagine un escenario completo. A hilo consumidor llama retrieveMessage() en un mailbox que no tiene mensajes. Este obtiene la clave y comienza a ejecutar el método. Este observa que la bandera de solicitud esta en falso, así este llama **wait()** y lo inserta al grupo de espera del correo. (En

este ejemplo sencillo, no hay otros hilos en el grupo). Dado que el consumidor ha llamado a **wait()**, este abandona la clave.

Después , un hilo productor de mensajes llama `storeMessage()` en el mismo correo. Este adquiere la clave, almacena su mensaje en una variable del monitor, y coloca la bandera de solicitud a verdadero. El produce entonces **notify()**. En este momento hay solo un hilo en el grupo de espera del monitor.

Así el consumidor consigue salir del estado de espera y va al estado de búsqueda de clave. Ahora el productor retorna del mensaje almacenado; Así el productor ha salido del código **sincronizado**, este abandona la clave del monitor. Después el consumidor pacientemente adquiere la clave y consigue ejecutarse; una vez esto pasa, este chequea la bandera de solicitud y (finalmente!) observa que hay un mensaje disponible. El consumidor retorna el mensaje; después de retornar este automáticamente suelta la clave.

Para resumir: Un consumidor trata de consumir nada, pero no hubo nada que consumir, entonces espera. El productor genera algo. De tal forma que hay algo que consumir por el consumidor, de esta forma el consumidor será notificado; una vez el monitor estuvo listo el monitor, el consumidor consumió el mensaje.

Como se muestra en la Figura 7.7 , un hilo que esta en espera tiene varias formas de salir de este estado donde no requiere ser notificado. Una versión de la llamada a **wait()** toma un argumento que especifica un tiempo fuera en milisegundos; si este expira, el hilo cambia al estado de búsqueda de clave, aun este no haya sido notificado.

No importa que versión de **wait()** es invocada, si el hilo que espera recibe un llamado de una interrupción este cambia inmediatamente al estado de búsqueda de clave.

Este ejemplo protege el consumidor contra la posibilidad de que el monitor pueda estar vacío; la protección fue implementada con una llamada a **wait()** en `retrieveMessage()` y una llamada a **notify()** en `storeMessage()`. Una precaución similar debe tenerse en cuenta en caso que un hilo productor quiera producir dentro de un monitor que ya contenga un mensaje. Para ser mas robusto, `storeMessage()` necesita llamara **wait()**, y `retrieveMessage()` necesita llamar al **notify()**. La clase Mailbox completa se vería como esto :

```
1.class Mailbox {
2.private boolean request;
3.private String message;
4.
5.public synchronized void storeMessage(String message) {
6.while( request == true) { // No room for another message
7.try {
8.wait();
```

```

9.     } catch (InterruptedException e) { }
10.  }
11.  request = true;
12.  this. message = message;
13.  notifyO;
14.  }
15.
16.  public synchronized String retrieveMessage() {
17.  while( request == false) { // No hay mensajes para recuperar
18.  try {
19.  waitO;
20.  } catch (InterruptedException e) { }
21.  }
22.  request = false;
23.  notify();
24.  return message;
25.  }
26.  }

```

Sincronizando el código y llamando cuidadosamente a `wait()` y **`notify()`** los monitores tales como la clase Mailbox pueden asegurar la propia interacción de hilos clientes y proteger los datos compartidos de daño.

Aquí están los principales puntos para recordar acerca de `wait()`:

- La llamada al hilo abandona la CPU.
- La llamada al hilo abandona la clave.
- La llamada al hilo va dentro del grupo de monitores en espera.

Aquí están los principales puntos para recordar acerca de **`notify()`**:

- Un hilo consigue cambiar y estar fuera del grupo de monitores en espera y pasa al estado listo (Ready) .
- El hilo que fue notificado debe readquirir la clave del monitor antes que este pueda ser procesado.

Mas alla del Modelo Puro

El ejemplo del correo de las secciones previas ha sido muy simple son ejemplos de situaciones que involucran un productor y un consumidor. En situaciones reales no son siempre tan simples. Usted debe tener un monitor que tiene métodos que no producen puramente o consumen puramente. Todo lo que usted puede decir en general acerca de tales métodos es que ellos no pueden proceder a menos que el monitor este en un cierto estado y

ellos mismos pueden cambiar el estado del monitor de diversas formas que podrían ser de vital interés para los otros métodos

El método **notify()** no es preciso: Usted no puede especificar cual hilo va a ser notificado. En un escenario confuso tal como el descrito arriba, un hilo puede alterar el estado del monitor en una forma que es inútil para el hilo en particular que será anunciado. En tal caso, Los métodos del monitor tomarían dos precauciones:

- Siempre verifique el estado del monitor en un while antes de una declaración if.
- Después de cambiar el estado del monitor, llame **notifyAll()** antes que **notify()**.

La primera precaución significa que usted no debería hacer lo siguiente:

```
1. public synchronized void mixedUpMethod() {  
2. if (i<16 || f>4.3f || message.equals("UH-OH")) {  
3. try { wait(); } catch (InterruptedException e) { }  
4. }  
5.  
6. // Proceder de una forma que cambia el estado , y luego..  
7. notifyO;  
8. }
```

El peligro es que algunas veces un hilo podría ejecutar la prueba en la línea 2, entonces note que este es (por ejemplo) 15, y tiene que esperar .después otro , hilo puede cambiar el estado del monitor colocando i a -23444, y entonces llamar a **notify()**. Si el hilo original es el único que consigue ser anunciado, este continuará donde se detuvo, aun cuando el monitor no este en un estado donde este listo para **mixedUpMethod()**.

La solución es cambiar **mixedUpMethod()** como sigue:

```
1. public synchronized void mixedUpMethod() {  
2. while (i<16 || f>4.3f || message.equalsC'UH-OH11)) {  
3. try { wait(); } catch (InterruptedException e) { }  
4. }  
5.  
6. // Proceed in a way that changes state, and then...  
7. notifyAll();  
8. }
```

Los métodos **sincronizados** del otro monitor deberían ser modificados en 1 de una manera similar.

Ahora cuando un hilo en espera consigue ser anunciado , este no asume que el estado del monitor es aceptable. Este verifica de nuevo, en un ciclo while chequea en la línea 2. Si el estado aun no es conducente, El hilo espera de nuevo.

En la línea 7, Se ha hecho sus propias modificaciones al estado del monitor, El código llama a **notifyAll()**; esta llamada es como **notify()**, pero este cambia cada hilo en el grupo de monitores en espera al estado listo. Presumiblemente cada hilo llama a `wait()` dentro de un ciclo como en las líneas 2-4, así cada hilo conseguirá verificar el estado del monitor y esperar o proceder.

Usando un ciclo while para verificar el estado del monitor es una buena idea aun cuando usted este codificando un modelo puro de un productor o un consumidor, después de todo, usted nunca puede estar seguro que alguien no querrá tratar de adicionar un productor extra o un consumidor.

Formas extrañas para Sincronizar

Hay dos formas de sincronizar código que todavía no han sido explicadas. Ellas son poco comunes y generalmente no deberían ser usadas sin una razón obligatoria. Estas dos formas son

- Sincronizando la clave de un objeto diferente
- Sincronizando la clave de una clase

Esto fue brevemente mencionado en una sección anterior ("La clave del objeto y Sincronización") que usted puede sincronizar la clave de un objeto. Suponga, por ejemplo, que usted tiene la siguiente clase, la cual tiene un buen grado de creatividad.

```
1. class StrangeSync {
2.   Rectangle rect = new Rectangledl, 13, 1100, 1300);
3.   void doitQ {
4.     int x = 504;
5.     int y = x / 3;
6.     rect.width -= x;
7.     rect.height -= y;
8.   }
9. }
```

Si usted adiciona la clave sincronizada en la línea 3, entonces un hilo que quiere ejecutar el método `doit()` de alguna instancia de `StrangeSync` debe primero adquirir la clave para la instancia. Que puede ser exactamente lo que usted quiere. Sin embargo, quizás usted solamente quiera sincronizar las líneas 6 y 7, y quizás usted quiera que un hilo intentar ejecutar estas líneas para sincronizar la clave de `rect`, antes que la clave del objeto en ejecución. La forma de hacer esto se muestra a continuación:

```

1.class StrangeSync {
2. Rectangle rect = new Rectangle(11, 13, 1100, 1300);
3. void doitQ {
4.     int x = 504;
5.     int y = x / 3;
6.     synchronized(rect) {
7.         rect.width -= x;
8.         rect.height -= y;
9.     }
10. }
11. }

```

El código de arriba sincroniza la clave de, algún objeto arbitrario (especificado en el paréntesis después de la palabra clave **sincronizada** en la línea 6), antes de sincronizar la clave del objeto actual. También, el código de arriba sincroniza solo dos líneas, en cambio del método entero.

Es difícil encontrar una buena razón para sincronizar un objeto arbitrario. Sin embargo, sincronizar solo un subconjunto de un método puede ser útil; algunas veces usted quiere obtener la clave tan pronto como sea posible, dado que los otros hilos pueden conseguir sus turnos tan pronto como sea posible. El compilador de Java insiste que cuando usted sincroniza una porción de un método (no todo el método), usted tiene que especificar un objeto en paréntesis después de la clave **sincronizada**. Si usted coloca esta en paréntesis, Entonces logra el objetivo: Usted ha **sincronizado** una porción de un método, con la clave usando la clave del objeto que es dueño del método.

De esta manera sus opciones son

- Para sincronizar un método entero, usando la clave del objeto que es dueño del método. Para hacer esto, coloque la palabra clave **sincronizada** en la declaración del método.
- Para sincronizar parte de un método, usando la clave de un objeto arbitrario. Para hacer esto, coloque corchetes alrededor del código que será **sincronizado**, precedido por **sincronizado**(theArbitraryObject).

Para sincronizar parte de un método, use la clave del objeto que es dueño del método. Para hacer esto, coloque corchetes alrededor del código que será **sincronizado**, precedido por **synchronized**(this). Las clases, así como los objetos, tienen claves. Una clave de una clase es usada para sincronizar los métodos estáticos de la clase. Los objetivos de la certificación no hacen gran énfasis sobre las claves de las clases.

Resumen del capítulo

La programación de hilos en Java pueden ser con preferencia o con divisiones de tiempo, dependiendo del diseño o del JVM. No importa cual diseño sea usado, un hilo llega a ser elegible para ser ejecutado ("Ready") cuando su método `start()` es invocado. Cuando un hilo comienza la ejecución, el programador llama el método **`run()`** del hilo objetivo (si hay un objetivo) o el método **`run()`** del hilo en si mismo (si no hay objetivo).

En el curso de la ejecución, un hilo puede llegar a ser no elegible por un numero de razones : Un hilo puede suspender, o dormir, o bloquear, o esperar. A su debido tiempo (una esperanza!), las condiciones cambiaran dado que el hilo una vez mas llega a ser elegible para la ejecución; entonces el hilo entra al estado listo y eventualmente puede ejecutarse. ,

Cuando un hilo retorna de su método(), este entra al estado muerto y no puede ser reinicializado.

Usted puede encontrar la siguiente lista como un resumen útil de los hilos de Java.

Implementaciones del programador:

- Preferencia
- Divisiones de tiempo en la construcción de un hilo:
- **`new Thread()`**: sin objetivo;el método **`run()`** del propio hilo se ejecuta
- **`new Thread (Runnable target)`**: el método **`run()`** del hilo se ejecuta

Estados de no ejecución:

- Suspendido(Suspended): causado por `suspend()`, espera a `resume()`
- Dormido (Sleeping): causado by `sleep()`, espera por pausa
- Bloqueado (Blocked): causado por varias llamadas a I/O o por una falla para conseguir una clave de un monitor, espera por una I/O o por la clave del monito
- Espera (**Waiting**): causado por **`wait()`**, **`waits`** por **`notify()`** o **`notifyAll()`**
- Muerto: causado por `stop()` o retornando de **`run()`**

Test Yourself

1. Cual de las siguientes declaraciones abajo respecto al siguiente código son verdaderas?

```
1. class Greebo extends java.util.Vector implements Runnable {  
2.     public void run( String message ) {  
3.         System.out.println( "in run() method: " + message );  
4.     }  
5. }  
6.  
7. class GreeboTest {  
8.     public static void main( String[] args ) {  
9.         Greebo g = new Greebo();  
10.        Thread t = new Thread( g );  
11.        t.start();  
12.    }  
13. }
```

- A. Habrá un error del compilador, porque la clase Greebo no implementa la interface **Runnable** .
- B. Habrá un error del compilador en la línea 10, porque no se pueden pasar parámetros al constructor de un hilo.
- C. El código compilará correctamente pero lanzará una excepción en al línea 10.
- D. El código compilará correctamente pero lanzará una excepción en línea 11.
- E. El código compilará correctamente y se ejecutará sin lanzar ninguna excepción.

2. Cual declaración abajo es siempre verdadera acerca de la siguiente aplicación?

```
1. class HiPri extends Thread {  
2.     HiPri() {  
3.         setPriority( 10 );  
4.     }  
5.  
6.     public void run() {  
7.         System.out.println( " Another thread starting up." );  
8.         while ( true ) { }  
9.     }  
10.  
11.    public static void main( String[] args ) {  
12.        HiPri hp1 = new HiPri();
```

```

13.          HiPri  hp2 = new HiPri();
14.          HiPri  hp3 = new HiPri();
15.          hp1.start();
16.          hp2.start();
17.          hp3.start();
18.      }
19. }

```

- A. Cuando la aplicación corre, el hilo hp1 se ejecutará; los hilos hp2 y hp3 nunca conseguirán la CPU.
- B. Cuando la aplicación corre , todos los hilos (hp1, hp2, y hp3) conseguirán ejecutarse, tomando intervalos de tiempo en la CPU.
- C. A o B serán verdadera, dependiendo de la plataforma que corra.

3.Un hilo quiere que un segundo hilo no sea elegible para la ejecución. Para Hacer esto, el primer hilo puede llamar el método yield() sobre el segundo hilo.

- A. Verdadero
- B. Falso

4. El método **run()** de un hilo incluye las siguientes líneas de código:

```

1. try {
2.     sleep( 100 );
3. } catch ( InterruptedException e ) { }

```

Asumiendo que el hilo no es interrumpido, cual de las siguientes declaraciones es correcta?

- A. El código no compilará, porque las excepciones no pueden ser atrapadas en el método **run()** de un hilo.
- B. En la línea 2,el hilo parará de correr. La ejecución comenzará de nuevo al menos en 100 milisegundos.
- C. En la línea 2, el hilo parará de correr . Este comenzará de nuevo a correr en exactamente 100 milisegundos.
- D. En la línea 2, el hilo parará de correr . Este comenzará de nuevo a correr en algún momento después que hayan transcurrido 100 milisegundos .

5. Un monitor llamado mon tiene 10 hilos en su grupo de espera; todos estos hilos en espera tienen la misma prioridad. Uno de los hilos es thr1. Como se puede anunciar thr1 dado que este solo cambia del estado en espera al estado listo?

- A. Ejecute **notify(thr1);** Desde adentro del código **synchronized** del monitor.
- B. Ejecute mon.**notify(thr1);** desde el código **synchronized** de cualquier objeto.

- C. Ejecute `thr1.notify()`; desde el código **synchronized** de cualquier objeto.
- D. Ejecute `thr1.notify()`; desde cualquier código **synchronized** o no, de cualquier objeto.
- E. Usted no puede especificar cual hilo será anunciado.

6. Cual de las declaraciones abajo esta verdaderamente relacionada con la siguiente aplicación?

```
1. class TestThread2 extends Thread |
2.     public void run() {
3.         System.out.println( "Starting" );
4.         yield();
5.         resume();
6.         System.out.println( "Done" );
7.     }
8.
9.     public static void main( String[] args ) {
10.        TestThread2 tt = new TestThread2();
11.        tt.start();
12.    }
13. }
```

- A. La compilación fallará en la línea 4, porque `yied()` debe ser llamado en un código **sincronizado**.
- B. La compilación fallará en la línea 5, porque `resume()` debe ser llamado en código **sincronizado**.
- C. La compilación será exitosa. Durante la ejecución, nada se imprimirá.
- D. La compilación será exitosa . Durante la ejecución, solamente una línea de la salida (Starting) se imprimirá.
- E. La compilación será exitosa . Durante la ejecución, ambas líneas de la salida (Starting and Done) serán impresas.

7. Si usted trata de compilar y ejecutar la aplicación listada abajo, imprimirá el mensaje `In xxx`?

```
1. class TestThread3 extends Thread {
2.     public void run() {
3.         System.out.println( "Running" );
4.         System.out.println( "Done" );
5.     }
6.
7.     private void xxx() {
8.         System.out.println( "In xxx" );
9.     }
10.
11.     public static void main( String[] args ) {
12.        TestThread3 ttt = new TestThread3();
13.        ttt.xxx();
14.        ttt.start();
15.    }
16. }
```

15. }
16. }

8. Un monitor en Java debe heredar de **Thread** o implementar **Runnable** .
A. Verdadero
B. Falso

8. java.lang / java.util Packages

Objectives

Este capítulo cubre aspectos relacionados con los siguientes objetivos del examen de certificación:

- Determinar el resultado de aplicar el método booleano **equals(Object)** a los objetos de cualquier combinación de las clases **java.lang.String**, **java.lang.Boolean**, y **java.lang.Object**.
- Escribir código que usa los siguientes métodos de la clase **java.lang.Math**: **abs**, **ceil**, **floor**, **max**, **min**, **random**, **round**, **sin**, **cos**, **tan**, **sqrt**.
- Determine el resultado de aplicar a cualquier operador, incluso operadores de la asignación e instanceof, a los operando de cualquier tipo, clase, alcance, o accesibilidad, o cualquier combinación de éstos.
- Describir la importancia de la inmutabilidad de los objetos string.
- Hacer una apropiada selección de la colección de , para satisfacer los requisitos de conducta especificados.

El paquete de java.lang contiene clases que son el eje central del funcionamiento del lenguaje y ambiente Java. Muy poco puede hacerse sin la clase String, por ejemplo, y la clase OBJECT es completamente indispensable. *El compilador de Java importa todas las clases automáticamente en el paquete en cada archivo fuente.*

Este capítulo examina algunas de las clases más importantes del paquete de java.lang:

- **Object**
- **Math**
- The wrapper classes
- **String**
- **StringBuffer**

Además, este capítulo también cubre las clases de la colección del paquete **java.util**.

Los paquetes son grupos relacionados de clases e interfaces y proporcionan un mecanismo conveniente para manejar un gran juego de clases e interfaces y evitar los conflictos de nombres. Además de los paquetes de

Java puede crear tus propios paquetes y poner en ellos definiciones de clases y de interfaces utilizando la sentencia **package**.

Supongamos que se está implementando un grupo de clases que representan una colección de objetos gráficos como círculos, rectángulos, líneas y puntos. Además de estas clases tendrás que escribir un interface Draggable para que las clases que lo implementen puede moverse con el ratón. Si quieres que estas clases estén disponibles para otros programadores, puedes empaquetarlas en un paquete, digamos, **graphics** y entregar el paquete a los programadores (junto con alguna documentación de referencia como qué hacen las clases y los interfaces y qué interfaces de programación son públicos).

De esta forma, otros programadores pueden determinar fácilmente para qué es tu grupo de clases, cómo utilizarlos, y cómo relacionarlos unos con otros y con otras clases y paquetes. Los nombres de clases no tienen conflictos con los nombres de las clases de otros paquetes porque las clases y los interfaces dentro de un paquete son referenciados en términos de su paquete (técnicamente un paquete crea un nuevo espacio de nombres).

Se declara un paquete utilizando la sentencia package:

La clase Object

La clase del Objeto es la última antepasado de todas las clases de Java. Si una clase no contiene la palabra clave EXTENDS en su declaración, el compilador construye una clase que se extiende directamente de **Object**.

Todos los métodos de Objeto son heredados por cada clase. Tres de estos métodos (**wait()**, **notify()**, and **notifyAll()**) soportan el mando de thread de apoyo; ellos se discuten en detalle en Capítulo 7. Otros dos métodos, **equals()** and **toString()**, proporcionan tan solo una pequeña funcionalidad.. La intención es que programadores que desarrollan las clases reusables pueden realizar override(atropellar) a **equals()** and **toString()** para proporcionar la funcionalidad y utilidad de una clase específica.

equals() se utiliza para comparar si dos objetos son iguales. Este método devuelve **true** si los objetos son iguales, o **false** si no lo son. Observe que la igualdad no significa que los objetos sean el mismo objeto.

La estructura de **equals()** es:

```
public boolean equals( Object object )
```

Se supone que el método proporciona " la comparación profunda", en contraste con la" comparación poco profunda" proporcionada por el == operador.

Para ver la diferencia entre los dos tipos de comparación, considere la clase java.util.Date , la cual representa un tiempo en un momento dado. Suponga

usted tiene dos referencias del tipo `Fecha(date)`: `d1` y `d2`. Una manera de comparar estos dos está con lo siguiente la línea de código:

```
if ( d1 == d2 )
```

La comparación será verdad si la referencia en el `d1` es igual a la referencia en el `d2`: es decir, si ambas variables contienen los modelos idénticos. Claro, éste es sólo el caso cuando ambas variables se refieren al mismo objeto.

A veces usted quiere un tipo diferente de comparación. A veces usted no conoce si el `d1` y `d2` se refieren al mismo objeto de la `Fecha(date)`. A veces usted sabe que ellos son los objetos diferentes; en eso usted tiene que tener mucho cuidado sobre es si los dos objetos representan el mismo momento de tiempo. En este caso usted no quiere la comparación referencia-nivelada poco profunda de `==`. La manera de hacerlo está con el método **`equals()`** es:

```
if ( d1.equals( d2 ) )
```

La versión **`equals()`** suministrada por la clase **`Object`** no es muy útil; de hecho, apenas hace una comparación `==`. Todas las clases deben `atropellar(override)` al método `equals()` para que realice una comparación útil. Ése es eso que la mayoría de las clase estandar de java hacen: ellos comparan las variables del caso pertinentes de dos objetos.

El propósito del método **`toString ()`** es proporcionar un string que representa el estado de un objeto. Esto es especialmente útil para depurar.

El método **`toString ()`** es similar a **`equals()`** en que la versión que porvee la clase **`Object`** no es especialmente útil. (Apenas imprime fuera el nombre de la clase del objeto, siguiida por un código) Muchas clases de JDK `atropellan` el `toString ()` para proporcionar información más útil. La facilidad de encadenamiento de string de Java hace uso de este método, como usted verá después en este capítulo, en la sección. "**`String`** Concatenation"

The Clase Math

La clase **`Math`** de Java contiene una colección de métodos y dos constantes que apoyan el cómputo matemático. La clase es final, para que usted no puede extenderla. El constructor es privado, para que usted no puede crear una instancia. Afortunadamente, los métodos y constantes son estáticas, para que ellos pueden accederse a través del nombre de la clase sin tener que construir un objeto **`Math`**. (Vea Capítulo 3 para una explicación de los modificadores de Java, incluso el modificador final, static y privado).

Las dos constantes de la clase **`Math`** son

- **Math.PI**
- **Math.E**

Ellos se declaran para ser públicos, estático, final, y double.

Los métodos de la clase **Math** suministran un rango ancho de funcionalidad matemática, incluso la trigonometría, logaritmos y exponenciación , y redondeo. Los métodos número-rozonando intensivos son generalmente nativos, aprovecharse la de cualquier hardware de aceleración de matemática que podría estar presente en la máquina subyacente.

El Examen de la Certificación le exige que sepa sobre lo siguiente los métodos de la clase **Math**:

- **int** abs(**int** i): retorna el valor absoluto de i
- **long** abs(**long** l): retorna el valor absoluto de l
- **float** abs(**float** f): retorna el valor absoluto de f
- **double** abs(**double** d): retorna el valor absoluto de d
- **double** ceil(**double** d): retorna como **double** el entero mas mas pequeño menor que d
- **double** floor(**double** d): retorna como **double** el entero mas grande que es menor que d
- **int** max(**int** i1, **int** i2): retorna mayor entre i1 y i2
- **long** max(**long** l1, **long** l2): retorna mayor entre l1 y l2
- **float** max(**float** f1, **float** f2): retorna mayor entre f1 y f2
- **double** max(**double** d1, **double** d2): retorna mayor entre d1 y d2
- **int** min(**int** i1, **int** i2): retorna menor entre i1 y i2
- **long** min(**long** l1, **long** l2): retorna menor entre l1 y l2
- **float** min(**float** f1, **float** f2): retorna menor entre f1 y f2
- **double** min(**double** d1, **double** d2): retorna menor entre d1 y d2
- **double** random(): retorna un numero random entre 0.0 y 1.0
- **int** round(**float** f): retorna la parte entera de f
- **double** sin(**double** d): retorna el seno de f
- **double** cos(**double** d): retorna el coseno de d
- **double** tan(**double** d): retorna la tangente de d
- **double** sqrt(**double** d): retorna la raíz cuadrada de de d

La clase Wrapper

Cada dato de tipo primitivo en java tiene una clase de la envoltura correspondiente. Una clase de la envoltura simplemente es una clase que encapsula un solo, inmutable valor. Por ejemplo, las envolturas de clase de Entero a un valor del int, y una clase FLOAT envuelve un valor FLOAT. Los nombres de clase de envoltura no coinciden con los nombres del tipo de

datos primitivos correspondientes. La tabla 8.1 listas los primitivas y envolturas.

TABLE 8.1 Primitives and Wrapers	
Primitive Data Type	Wrapper Class
boolean	Boolean
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double

Todas las clases de la envoltura pueden construirse pasando el valor a ser envuelto en el constructor apropiado. El fragmento del código debajo muestra cómo construir un caso de cada tipo de la envoltura:

```
1. boolean    primitiveBoolean = true;
2. Boolean    wrappedBoolean = new Boolean( primitiveBoolean );
3.
4. byte        primitiveByte = 41;
5. Byte        wrappedByte = new Byte( primitiveByte );
6.
7. char        primitiveChar = 'M';
8. Character   wrappedChar = new Character( primitiveChar );
9.
10. short      primitiveShort = 31313;
11. Short      wrappedShort = new Short( primitiveShort );
12.
13. int        primitiveInt = 12345678;
14. Integer    wrappedInt = new Integer( primitiveInt );
15.
16. long       primitiveLong = 12345678987654321L;
17. Long       wrappedLong = new Long( primitiveLong );
18.
19. float      primitiveFloat = 1.11f;
20. Float      wrappedFloat = new Float( primitiveFloat );
21.
22. double     primitiveDouble = 1.11111111;
23. Double     wrappedDouble = new Double( primitiveDouble );
```

Hay otra manera de construir cualquiera de estas clases, con la excepción de **Carácter**. Usted puede pasar en el constructor un string que representa el valor a ser envuelto.

La mayoría de estos constructores tira una excepción **NumberFormatException**, porque hay siempre la posibilidad que el string no representará un valor válido. Sólo un dato **Booleano** no tire esta excepción: el constructor acepta cualquier string de entrada, y envolturas un verdadero valor si el string (ignorando el caso) es "verdad."

El fragmento del código siguiente muestra cómo construir las envolturas de los string:

```
1. Boolean wrappedBoolean = new Boolean( "True" );
2. try {
3.     Byte wrappedByte = new Byte( "41" );
4.     Short wrappedShort = new Short( "31313" );
5.     Integer wrappedInt = new Integer( "12345678" );
```

```

6.      Long wrappedLong = new Long( "12345678987654321" );
7.      Float wrappedFloat = new Float( "1.1f" );
8.      Double wrappedDouble = new Double( "1.11111111" );
9.  }
10. catch ( NumberFormatException e ) {
11.      System.out.println( "Bad Number Format" );
12. }

```

Pueden verificarse los valores envueltos dentro de dos envolturas del mismo tipo por la igualdad usando el método **equals()** que se discutió en la sección anterior. Por ejemplo, el siguiente fragmento de código chequea dos instancias de los cheques dos casos de **Double** :

```

1. Double d1 = new Double( 1.01055 );
2. Double d2 = new Double( "1.11348" );
3. if ( d1.equals( d2 ) ) {
4.     // do something
5. }

```

Después de que un valor ha sido ajustado, puede ser necesario extraerlo en el futuro. Para un caso de Booleano, usted puede llamar el **booleanValue()**. Para una instancia de Carácter, usted puede llamar el **charValue()**.

Las otras seis clases se extienden desde la superclase abstracta **Number**, que proporciona los métodos para recuperar el valor ajustado como un byte, un short, un int, un long, un float, o un double. En otras palabras, el valor de cualquier número ajustado puede recuperarse como cualquier tipo numérico.

Los métodos de la recuperación son:

- **public byte** byteValue()
- **public short** shortValue()
- **public int** intValue()
- **public long** longValue()
- **public float** floatValue()
- **public double** doubleValue()

Las clases de la envoltura son útiles siempre que fuera conveniente tratar un pedazo de datos primitivos como si fuera un objeto. Un ejemplo bueno es la clase del Vector que es una colección dinámicamente creciente de objetos de tipo arbitrario.

El método por agregar un objeto a un vector es

```
public void addElement( Object obj )
```

Usando este método, usted puede agregar cualquier objeto de cualquier tipo a un Vector; usted puede agregar una serie incluso (usted vio por qué en Capítulo 4). Usted no puede, sin embargo, agregue un int, un long, o cualquier otro primitivo a un Vector. No hay ningún método especial por hacer para que, y addElement (el obj del Objeto) no trabajará porque no hay ninguna conversión automática de un primitivo a un objeto.

Así, el código que se presenta abajo de no compilará:

1. **Vector** vec = **new Vector**();
2. **boolean** boo = false;
3. vec.**addElement**(boo); // illegal

La solución es envolver el primitivo booleano, como se muestra abajo:

1. **Vector** vec = **new Vector**();
2. **boolean** boo = false;
3. **Boolean** wrapper = **new Boolean**(boo);
4. vec.**addElement**(wrapper); // legal

Las clases de la envoltura son útiles de otra manera: ellos proporcionan una variedad de métodos de utilidad, la mayoría de los cuales son estáticos. Por ejemplo, el Character.isDigit (el char ch) es un método que devuelve un booleano que dice si el carácter representa un dígito de base-10.

Todas las clases de la envoltura exceptuando el Carácter tiene un método estático llamado el valueOf (String s) que analiza un string y construye y retorna una instancia de la envoltura del mismo tipo como la clase cuyo método fue llamado. Por ejemplo, así Long.valueOf (" 23L") construye y retorna una instancia de la clase LONG que envuelve el valor 23.

Para resumir los hechos mayores sobre las clases de la envoltura primitivas:

Cada tipo primitivo tiene un tipo de clase de envoltura correspondiente. Todos los tipos de la envoltura pueden construirse desde las primitivas; todos exceptúan el Carácter también puede construirse de los strings.

Pueden probarse los valores envueltos por la igualdad con el método **equals()**.

Pueden extraerse los valores envueltos con varios métodos XXXvalue (). Todas las seis envoltura numérica se apoyan en los métodos seis XXXvalue numéricos .

Las clases de la envoltura proporcionan los varios métodos de utilidad, incluso el método estático valueOf que analiza un string de la entrada.

Strings

Java usa la clase **String** y **StringBuffer** para encapsular strings de caracteres. Java usa el 16-bit caracteres de Unicode para apoyar un rango más ancho de alfabeto internacional que sería posible con los tradicional 8-bit caracteres. Ambos string y stringBuffer contienen sucesiones de 16-bit de caracteres de Unicode. Las próximas secciones examinan estas dos clases, así como el encadenamiento de string de Java.

La clase String

La clase **String** representa un string inmutable. Una vez se ha creado una instancia, el string que lo contiene no puede modificarse. Hay numerosos formas de constructores, permitiéndole construir una instancia de un arreglo de bytes o caracteres, como un subconjunto de arreglo de bytes o caracteres, otro string o un string buffer.

Muchos de estos constructores le dan la opción de especificar un carácter poniendo en código, especificado como string; sin embargo, el Examen de la Certificación no le exige que sepa los detalles de codificaciones del carácter.

Probablemente el constructor del string simplemente toma otro string simplemente como su entrada. Esto es útil cuando usted quiere especificar un valor literal por el nuevo string:

1. **String** si = **new String**("immutable");

Una abreviación aun más fácil podría ser:

1. **String** si = "immutable";

Es importante ser consciente de lo que pasa cuando usted usa un String literal (" immutable" en ambos ejemplos). Cada string literal se representa internamente como una instancia de String. Las clases de Java pueden tener una bolsa(conjunto) de strings. Cuando un literal es compilado, el recopilador agrega un string cordón apropiado a la bolsa

En la práctica, si el mismo literal ya apareció como un literal en otra parte en la clase, entonces ya se representa en la bolsa. El recopilador no crea una nueva copia; en cambio, usa la existente en la bolsa. Esto ahorra en la memoria y no puede hacer el daño. Desde que los strings son inmutables, no hay ninguna manera que un pedazo de código puede dañar otro pedazo de código modificando un string compartido.

Antes en este capítulo, usted vio cómo el método equals() puede usarse para proporcionar un chequeo de igualdad profundo de dos objetos. Con los

strings, del método equals() hace lo que usted chequee el contenido de la colección de caracteres. El código siguiente muestra cómo esto se hace:

```
1. String s1 = "Compare me";
2. String s2 = "Compare me";
3. if ( s1.equals( s2 ) ) {
4.     // whatever
5. }
```

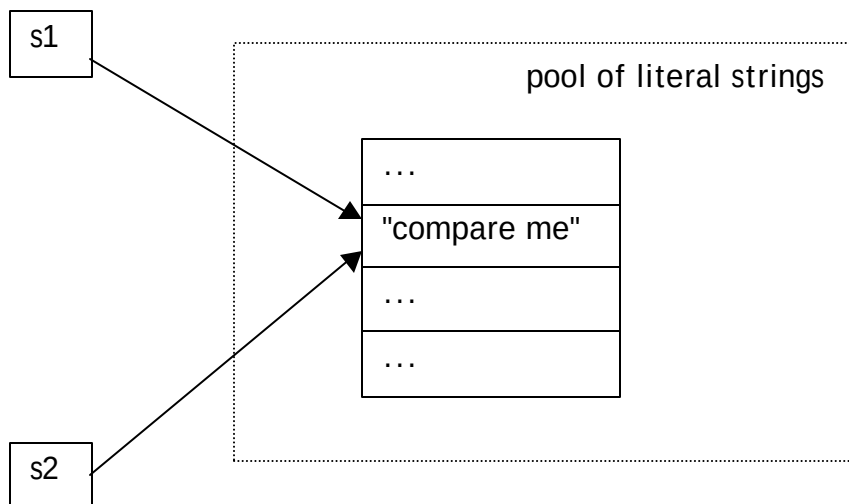
No sorprende, línea 3 tiene éxito. Dado que usted conoce acerca de cómo los literales strings trabajan, usted puede ver que si línea 3 se modifica para usar la comparación, como se muestra abajo, la prueba todavía tiene éxito:

```
1. String s1 = "compare me";
2. String s2 = "compare me";
3. if ( s1 == s2 ) {
4.     // whatever
5. }
```

El == la prueba es verdad porque el s2 se refiere al **String** en la bolsa que ha sido creado en línea 1. Figure 8.1 muestra esto gráficamente:

Figure 8.1

Literales identicos(Identical literals.)



String

Usted también puede construir un **String** llamando al constructor explícitamente, como se muestra abajo; sin embargo, esto causa la asignación memoria extra paor lo cual no es ninguna ventaja obvia.

```
1. String s2 = new String( "constructed" );
```

Cuando esta línea es compilada, el literal **String** " construido" se pone en la bolsa. En tiempo de ejecución, la declaración **new String ()** se ejecuta y un nueva instancia del de **STRING** se construye, duplicando en la bolsa.literal. Finalmente, una referencia al nuevo **String** se asigna al s2. La Figura 8.2 muestras la cadena de eventos.

FIGURA 8.2

Llamado explícito al constructor **String** constructor

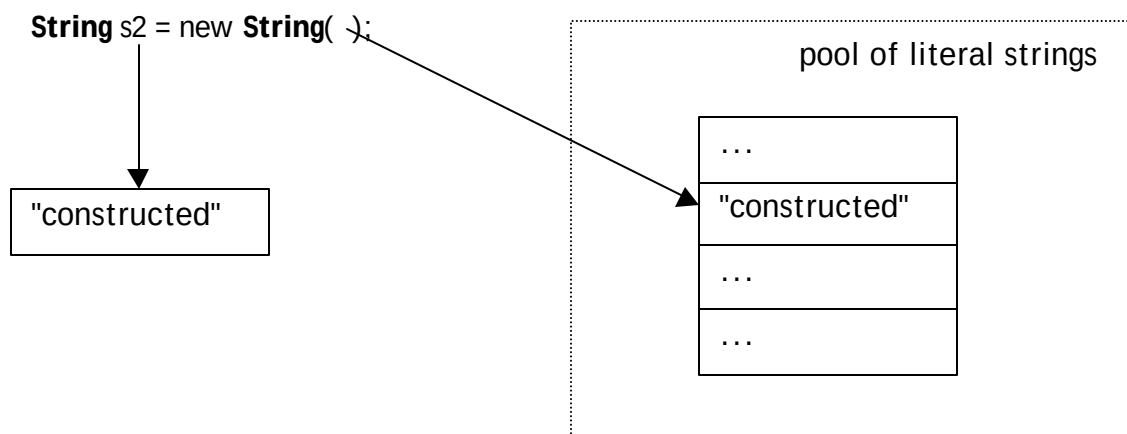


Figure 8.2 muestras el llamado explícito a **new String** como resultado de la existencia de dos objetos , uno en la bolsa literal y el otro en el espacio del programa.

Usted simplemente ha visto que si usted crea una instancia a **new String** en tiempo de ejecución, no estará en la bolsa, pero realmente será un nuevo y distinto objeto. Usted puede poner en orden para su **new String** a ser puesto en la bolsa para posible reuso, o para reusar un **String** idéntico existente de la bolsa, usando el metotdo `intern ( )` de la clase del **String**

En programas que usan muchos strings que podrían ser similares, esto puede reducir los requisitos de memoria. Más pretenciosamente en programas que hacen muchas comparaciones de igualdad entre **String**, asegurando que todas los **Strings** están en la bolsa, usted puede usar la comparación `==` en lugar del método `equals( )`.

Hay varios métodos convenientes en la clase **String**. Varios de estos métodos realizan una transformación en un **String**. Por ejemplo, **toUpperCase ()** retorna un nuevo **String** que es una copia de todos los caracteres en el **String**, convertido a mayúsculas. Es importante recordar que el **String** original no se modifica. Eso sería imposible, desde que **String** son inmutables. Lo que realmente pasa es que un nuevo **String** es **construido y retornado**. Generalmente, este nuevo **String** no estará en la bolsa explícitamente llame a **intern()** para ponerlo allí.

Los métodos listados debajo son simplemente algunos de los métodos más útiles de la clase **String**. Hay más métodos de los listados aquí, y algunos de aquéllos son cargados excesivamente para formas que toman las entradas diferentes. Esta lista incluye todos los métodos que le exigen que conozca para la el examen de Certificación, los más útiles son::

- **char** charAt(**int** index): recibe un argumento entero que utiliza como índice y devuelve el carácter en esa posición., el índice inicial es 0.
- **String** concat(**String** addThis): retorna un nuevo string consistente el el viejo string seguido po el string addThis.
- **int** compareTo(**String** otherString): Esto realiza una comparación léxica; devuelve un int que está menos de 0 si el string actual es menor del otherString, igual a 0 si las strings son idénticos, y mayores que 0 si el string actual es mayor que el otherString.
- **boolean** endsWith(**String** suffix): Esto retorna verdadero si el string actual termina con suffix; de otra forma retorna falso
- **boolean** equals(**Object** obj): devuelve true si los objetos son iguales y false en caso contrario.
- **boolean** equalsIgnoreCase(**String** s): Igual que el método **equals()**, pero si el argumento es un **String** la comparación ignora mayusculas.
- **int** indexOf (**int** ch): Esto devuelve el índice dentro del string actual de la primera ocurrencia de ch.
- **int** lastIndexOf(**int** ch): Esto devuelve el índice dentro del string actual de la última ocurrencia de ch.
- **int** length(): Retorna el numero de caracteres del string actual
- **String** replace(**char** oldChar, **char** newChar): Retorna un nuevo string, generado al remplazar cada ocurrencia de f oldChar por

newChar.

- **boolean** startsWith(**String** prefix): Retorna verdadero si el actual string empieza por prefix; de otra forma retorna falso
- **String** substring(**int** startIndex): retorna un substring, que comienza por startIndex del actual string and y extiende al final el actual string.
- **String** toLowerCase(): Esto devuelve un nuevo string que es una copia de todos los caracteres en el string, convertido minúsculas.
- **String** toString(): Esto devuelve el objeto ejecutando.
- **String** toUpperCase(): Esto retorna un nuevo string que es la copia de todos los caracteres del string convertidos a mayúsculas.
- **String** trim(): Esto devuelve el string que es el resultado de quitar los caracteres en blanco desde el comienzo. (muy usado al trabajar con las bases de datos).

El código siguiente muestra cómo usar dos de estos métodos para "modificar" un string. El string original es " 5 + 4 = 20."

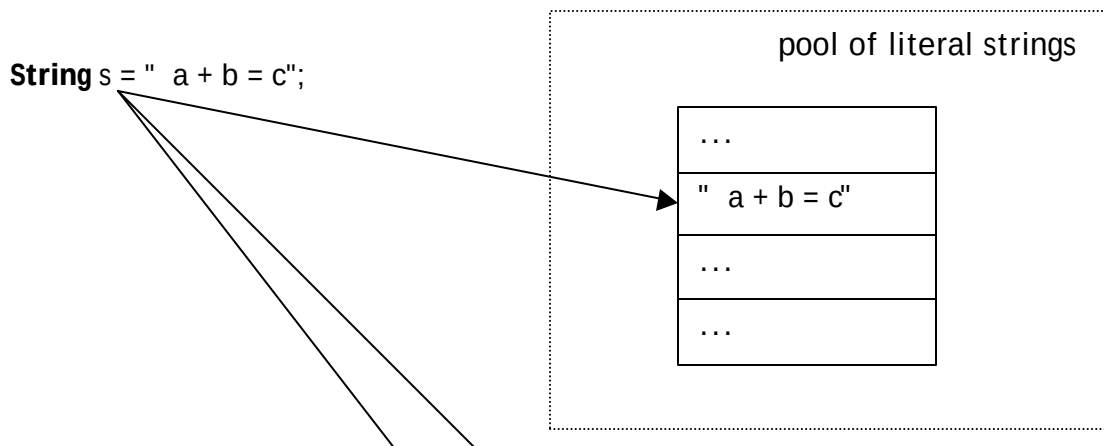
```
1. String s = " 5 + 4 = 20";  
2. s = s.trim(); // "5 + 4 = 20"  
3. s = s.replace( '+', 'x' ); // "5 x 4 = 20"
```

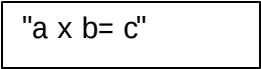
Después de línea 3, s se refiere a un string cuya apariencia se muestra en el comentario de la línea 3. Claro, la modificación no ha tenido lugar dentro del string original. Ambos el **trim()** y el **replace()** llamados sobre la línea 3 construye y retorna un nuevo string; la dirección de cada nuevo string se asigna a su vez a la referencia de la variable s

Figura 8.3 muestra gráficamente esta secuencia.

FIGURE 8.3

Arreglando y reemplazando (Trimming and replacing)





"a x b= c"

Figure 8.3 muestra que el string original sólo parece ser modificado. Realmente se reemplaza, porque los strings son inmutables. Si se requiere mucha modificación, entonces esto se pone muy ineficaz. La próxima sección discute una clase que las ayudas alivian estos problemas porque representa un string mutable: la clase de StringBuffer.

La clase StringBuffer

Una instancia de Java la clase StringBuffer representa un string que puede modificarse dinámicamente.

El constructor normalmente usado toma una instancia del **String** como la entrada. Usted también puede construir un buffer del string vacío (probablemente con la intención de agregar los caracteres después a él). Un **String** buffer vacío puede tener su capacidad inicial especificada en momento de la construcción. Los tres constructores son

- `StringBuffer ()`: Esto construye un **String** buffer vacío.
- `StringBuffer (int capacity)`: Esto construye un **String buffer** vacíos con la capacidad inicial especificada.
- `StringBuffer (String initialString)`: Esto construye un buffer de s que inicialmente contiene el **String** especificado.
- Un **String buffer** tiene que una capacidad que es el **String** de máximo-longitud él puede representar sin necesitar asignar más memoria. Un **String** buffer puede crecer más allá de esto como sea necesario, pero normalmente usted no tiene que preocuparse por la capacidad.

- La lista debajo presenta algunos de los métodos que modifican el contenido de un **String buffer**. Todos ellos devuelven el propio **String buffer** original:
- **StringBuffer append (String str):** Esto añade el str al **String buffer** actual. Forma alternativa de agregar primitivas y arreglos de caracteres; esto son convertidos a string antes de añadir.
- **StringBuffer append (Object obj):** Esto llama el toString () en el obj y añade el resultado al string buffer actual.
- **StringBuffer insert (int offset, String str):** Esto inserta el str en el **String buffer** actual a la posición offset. Hay numerosas alternativas.
- **StringBuffer reverse():** Esto invierte los caracteres del **String** buffer actual.
- **Stringbuffer setCharAt (int offset, char newChar):** Esto reemplaza el carácter a posición compensada con el newChar.

StringBuffer setLength (int newLength): Esto pone la longitud **StringBuffer** al newLength. Si el newLength está menos de la longitud actual, el string se trunca. Si el newLength es mayor que la longitud actual, el **String** se rellena con los caracteres nulos.

El siguiente código muestra el efecto de usar algunos de estos métodos

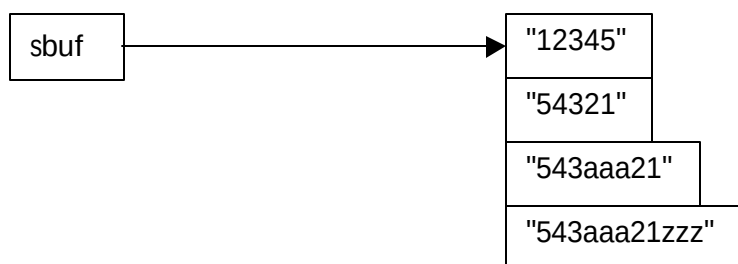
```

1. StringBuffer sbuf = new StringBuffer( "12345" );
2. sbuf.reverse();                      // '54321'
3. sbuf.insert( 3, "aaa" );             // "543aaa21"
4. sbuf.append( "zzz" );                 // "543aaa21zzz"

```

Los métodos llamados anteriormente realmente modifican el string buffer ellos operan adelante (diferente el ejemplo de clase string de la sección anterior). Figure que 8.4 muestra graficammente lo que este código hace:

FIGURE 8.4
Modificando un StringBuffer



Un último método string buffer que mencionamos es el método `toString()`. Usted vio antes en este capítulo que cada clase tiene uno de estos métodos. No sorprende, la versión del string buffer apenas devuelve el string encapsulado, como un caso de la clase string. Usted verá en la próxima sección que este método juega a un papel crucial en el encadenamiento de strings.

Una diferencia con la clase string que nosotros tenemos que mencionar es que la clase de StringBuffer no tienen el método **`equals()`** para comparar dos string encapsulados. Si usted usa el método **`equals()`** de la clase string para comparar un string con un StringBuffer, la llamada del método siempre volverá falso, aun cuando las dos strings encapsulados eran idénticos.

Encadenamiento de String de manera fácil

El método `concat()` de la clase **String** y el método `append()` de la clase StringBuffer une dos strings juntos. Una manera más fácil de encadenar los strings es usar Java's overloaded `+` operador.

El encadenamiento del **String** con el operador `+` y los operadores aritméticos son las situaciones que Java provee cuando se incorpora el operador cargando excesivamente. Sin embargo, no se olvida que usted, que el programador, no puede definir las cargas excesivas del operador adicionales.

El encadenamiento de String es útil en muchas situaciones, por ejemplo, en cuando se imprimen las declaraciones. Así que, para imprimir el valor de un radio **double**, todos que usted tiene que hacer es esto:

```
System.out.println( "radius = " + radius );
```

Esta técnica también trabaja para los datos de tipo objeto. Imprimir el valor de una **Dimension** llamado `dim`, todo lo que usted tiene que hacer es:

```
System.out.println( "dim=" + dim );
```

Es importante entender cómo trabaja esta técnica. En tiempo de compilación, si cualquier operando de un operador `+` (es decir, si lo en que aparece cualquier lado de un signo `+`) es un objeto **String**, entonces el compilador reconoce que está en un contexto del string. En un contexto del string, el

signo + se interpreta como el llamado para concatenar strings, en lugar de la suma aritmética.

Un contexto del string simplemente es una carrera arbitraria de sumas donde uno de los operandos es un string. Por ejemplo, si la variable `aaa` es un string, entonces la siguiente línea de código es un contexto del string, sin tener en cuenta los tipos de los otros operandos,:

```
aaa + bbb + ccc
```

El compilador de Java trata el código si hiciera lo siguiente:

```
new StringBuffer().append( aaa ).append( bbb ).append( ccc ).  
toString();
```

Si cualquiera de las variables (`aaa`, `bbb`, o `ccc`) es un primitivo, el método `append()` computa una representación del string apropiada. Para una variable objeto, el método `append()` usa el string devuelto desde el llamado del `toString()` en el objeto.

La conversión empieza con un string buffer vacío, entonces añade cada elemento a su vez al string buffer, y finalmente llama el `toString()` para convertir el string buffer a un string.

El código siguiente implementa una clase con su propio método `toString()`.

```
1. class Abe {  
2.     private int a;  
3.     private int b;  
4.     private int c;  
5.  
6.     Abc( int a, int b, int c ) {  
7.         this.a = a;  
8.         this.b = b;  
9.         this.c = c;  
10.    }  
11.  
12.    public String toString() {  
13.        return "a = " + a + ", b = " + b + ", c = " + c;  
14.    }  
15. }
```

Ahora el método **`toString()`** (línea 12-14) puede usarse por cualquier código que quiere aprovecharse la de encadenamiento de string. Por ejemplo,

```
1. Abe theAbc = new Abc( 11, 13, 48 );  
2. System.out.println( "Abe: " + theAbc );
```

La salida es:

Abe: a = 11, b = 13, c = 48

En resumen, la sucesión de eventos para un contexto de `sstring` son:

1. Se construye un string buffer vacío.
2. Cada argumento se encadena a su vez con el string buffer, usando el método `append()`.
3. El string buffer se convierte a un string con un llamado a `toString()`.

That is all you need to know about string manipulation for the Certification Exam, and probably all you need to know to write effective and efficient code, too. Next, we're going to look at collections.

Esto es todo lo que usted necesita saber sobre la manipulación de string para el Examen de Certificación, y probablemente todos que usted necesita saber para escribir el código eficaz y eficiente.

The Collections API (Las colecciones API)

Yo cualquiera, si no la mayoría, de los programadores necesitan guardar huella de grupos de elementos de los datos relacionados. Uno de los mecanismos más básicos por hacer esto son los arreglos. Java tiene también los arreglos y también algunas clases adicionales, como el **Vector** y las clases **Hashtable**, para permitirle manipular tales grupos o items.

Desde JDK 1.2, sin embargo, hay un rasgo del API significativo para apoyar la dirección de la colección mucho más generalizada. Los objetivos de Examen de Certificación también requieren que usted tenga un buen conocimiento de los conceptos de API.

Las colecciones API están a referenciadas con un armazón. Es decir, las clases se han diseñado con una abstracción común de recipiente de datos en la mente, mientras asegurando la semántica uniforme dondequiera que posible.

Al mismo tiempo, cada tipo de colección implementada es libre perfeccionar su propias operaciones. La fábrica clase de la **java.util.Collections** complementa el apoyo para, el cual se discuten abajo, con una variedad de auxiliador estático y métodos de la fábrica. Estos métodos apoyan los funcionamientos como sincronizar el recipiente, mientras estableciendo la inmutabilidad, ejecutando las búsquedas binaria, y así sucesivamente. Ya no se exigen a programadores que construyan sus propios datos básicos con estas clases en el lugar, estructura desde el principio.

Los Tipos de la colección (Collection Types)

Hay varias colecciones diferentes. Por ejemplo, ellos varían en los mecanismos del almacenamiento usados, de la manera que ellos pueden acceder a los datos, y en las reglas sobre qué datos podría guardarse. La colección API proporciona una variedad de interfaz y alguna aplicación concreta de clases, cubriendo estas variaciones.

Hay una interfaz general, **java.util.Collection** que define el armazón básico para todas las colecciones. Esta interfaz estipula los métodos que le permiten agregar los items, remover items, determina si los items son de una colección, y contar el número de items en la colección. Una colección algunas veces es conocida como una bolsa o una multiset. Una colección simple no pone los constraints en el tipo de elementos, el orden de elementos, o repetición de elementos dentro de la colección.

Algunas colecciones son ordenadas, hay una noción clara de un item y del siguiente. Una colección de este tipo normalmente es conocida como una lista o una sucesión. En algunas listas, el orden es el orden en que se

agregan los items a la colección; en otros, los elementos se asume que ellos tienen un orden natural, y ese orden se entiende por la lista. En el Java Colecciones API, el java de la interfaz. **util.List define un armazón básico para las colecciones de esta clase.**

Si una colección impone la condición específica que no puede contener el mismo valor más de una vez, entonces está conocido como un conjunto. La interfaz **java.util.Set** define el armazón básico para este tipo de colección. En algunos conjuntos, el valor nulo es una entrada legítima, pero si se permite, nulo sólo puede ocurrir una vez en un conjunto.

El tipo final de conducta especializada directamente apoyada por las Colecciones API es conocido como un mapa. Un mapa usa un conjunto de valores importantes para buscar, o índices, de datos guardados. Por ejemplo, si usted guarda un objeto que representa a una persona, entonces como llave usted podría usar cualquier nombre de persona o algún otro único identificador como un número del seguro social. Los mapas son particularmente apropiados para llevar a cabo las bases de datos pequeñas en línea, sobre todo si normalmente si los datos que han sido guardados son usualmente accedidos por un {único identificador}.

Es un requisito para un mapa que la llave sea única, y por esta razón si usted estuviera guardando los datos sobre una persona en un mapa, el nombre no sería una llave muy buena porque es muy posible que para dos personas se tenga el mismo nombre.

Tomemos un momento para recapitular estos puntos:

- Una colección no tiene ningún orden especial y no rechaza los duplicados.
- Una lista es ordenada y no rechaza los duplicados.
- Un conjunto no tiene ningún orden especial pero rechaza los duplicados.
- Un mapa soporta la búsqueda sobre un campo llave, valor que debe ser único.

Claro que es posible para las combinaciones de estas conductas ser significante. Por ejemplo, un mapa también podría ser ordenado. Sin embargo, el examen de certificación sólo le exige que entienda estos cuatro tipos fundamentales de colección.

Hay muchas maneras en que el almacenamiento asociado con cualquier colección puede ser implementada, pero las Colecciones API implementa de los mas usados. Éstos están usando un arreglo, usados en una lista vinculada, usados en un árbol. Cada uno de estas técnicas tiene beneficios y constraints. Consideremos estos beneficios y constraints a su vez para cada técnica del almacenamiento.

El almacenamiento de un array tiende a ser rápido el acceso, pero es relativamente ineficaz cuando el número de elementos en la colección crece o si los elementos necesitan ser insertados o anulados en el medio de una lista. Estas limitaciones ocurren porque la propia serie es una sucesión fija. Agregando o quitando los elementos en el medio requiere que todos los elementos de ese punto deben subirse adelante o abajo una posición. Agregando más datos una vez el arreglo está lleno exige asignar una nueva serie entera, y los volúmenes enteros copiaron encima de a la nueva serie. Otra limitación de una serie(arreglo) es que no proporciona ningún mecanismo de la búsqueda especial. A pesar de estas debilidades, una serie puede ser todavía una opción apropiada para datos que se piden, no cambia a menudo, y no necesita ser investigadoa menudo.

Una lista vinculada permite agregar los elementos, o removerlos desde, la colección a cualquier localización del contenedor, y permite que el tamaño de la colección crezca arbitrariamente sin las multas asociadas con el arreglo copiado. Esta mejora ocurre porque cada elemento es un objeto individual que se refiere al próximo (y a veces anterior, en una lista doble-vinculada) el elemento en la lista.

Sin embargo, es significativamente más lento que acceder por el índice de un arreglo, y todavía no proporciona ningún mecanismo de la búsqueda especial. Porque las listas vinculadas pueden insertar los nuevos elementos con localizaciones arbitrarias, sin embargo, ellas pueden aplicar el orden fácilmente, haciéndolo simple (si no siempre eficaz) la materia para investigar un subconjunto, o rango, de datos.

Un árbol, como una lista vinculada, permite la suma fácil y borrado de elementos y el crecimiento arbitrario de la colección. Al contrario de una lista, los árboles son un medio para el ordenamiento. De hecho, construir un árbol requiere de algún mecanismo de comparación de los datos a ser guardados (aunque esto puede crearse artificialmente en algunos casos). Un árbol normalmente proporcionará una eficiente búsqueda que un arreglo o una lista vinculada, pero este beneficio puede disimularse si que se estan guardando están distribuidos irregularmente.

Hashing requiere que alguna única identificador llave pueda asociarse con cada item de los datos que a su vez proporciona búsqueda eficiente.

Para trabajar propiamente en los varios tipos de la colección, los items de los datos pueden necesitar exhibir cierta conducta específica. Si usted desea buscar un item, por ejemplo, que la clase de ese items debe implementar correctamente el método equals().

Investigando en las colecciones ordenadas también pueden requerir que los datos llevan a cabo la interfaz **java.lang.Comparable** que define el

compareTo(), un método para determinar el orden inherente de dos items del mismo tipo. La mayoría de las aplicaciones de **Map** también requerirán una aplicación correcta del método **hashCode()**. Es aconsejable tener presente estos tres métodos siempre que usted defina una nueva-clase, aun cuando usted no se anticipa casos del acopio de esta clase en las colecciones.

Collection Implementations in the API (La implementación de la colección en el API)

Una variedad de clases de aplicación concretas se proporciona en las colecciones API para llevar la implementación de interfaces **Collection**, **List**, **Set**, y **Map**, usando los diferentes tipos de almacenamiento.

Estos se listan aquí:

HashMap/Hashtable

Estas dos clases son muy similares, usando el almacenamiento hash-based para implementar un mapa. El Hashtable ha estado en el API de Java desde las versiones más tempranas, y el HashMap se agregó a JDK 1.2. La diferencia principal entre los dos es que ese Hashtable no permite guardar el valor nulo, aunque hace algunos esfuerzos para apoyar el uso multi-enhebrado.

HashSet

Éste es un conjunto, que no permite duplicar y usar hashing para el almacenamiento.

LinkedList

Esta es una implementación, basado en un almacenamiento de lista-vinculada.

TreeMap

Esta clase proporciona un mapa ordenado. Los elementos deben ser los ordenables, o por de una interfaz **Comparable** o proporcionando una clase de Comparator para realizar las comparaciones.

TreeSet

Esta clase proporciona un conjunto ordenado, usando un árbol para el almacenamiento. Como con el TreeMap, los elementos deben tener un orden asociado a cada item.

TreeMap

Los elementos deben tener un orden asociado con ellos.

Vector

Esta clase que ha estado en el API de Java desde la primera versión, implementa una lista usando un arreglo interno para el almacenamiento. El arreglo es dinámicamente relocalizado, necesariamente como el número de artículos en el vector crece.

El resumen de Colecciones

Los puntos esenciales en esta sección han sido:

- Las colecciones no imponen ningún orden, ni restricciones, en la duplicación satisfecha.
- Las listas mantienen un orden (posiblemente inherente en los datos, los posiblemente externamente impuesto).
- Los conjuntos rechazan las entradas duplicadas.
- Los mapas usan llaves únicas para facilitar localización de sus volúmenes.

Para el almacenamiento:

- Usando arreglos se puede insertar, borrar, y crecimiento del arreglo con más dificultad.
- Usando listas vinculada, se soporta insercción, borrado, y crecimiento la tienda, pero se utiliza un índice dejando el el acceso más lentamente.
- Usando un árbol soporta inserción, borrado, y crecimiento, el acceso puesto en un índice es lento, pero la búsqueda es más rápida.
- Usando hashing se puede tener inserción, borrado, y crecimiento. El acceso puesto en un índice es lento, pero la búsqueda es particularmente rápida. Sin embargo, hashing requiere el uso de llaves únicas para guardar los elementos del datos.

Resumen del capítulo

El paquete **java.lang** contiene clases que son indispensable en el funcionamiento de Java, para que se importen todas las clases del paquete automáticamente en todos los archivos fuente. Algunas de las clases más importantes en el paquete son

- **Object**
- **Math**
- The wrapper classes

- **String**
- **StringBuffer**

En un contexto de string, se añaden los operandos a su vez a un string buffer, que los convierte en un string; se convierten los operandos primitivos a los strings, y los objetos son convertidos invocando el método `toString()`.

El paquete de **java.util** contiene muchas utilidades, pero para el examen de la certificación, el API de las colecciones es de interés. Las colecciones proporcionan las maneras para guardar y recuperar los datos en un programa. Los tipos diferentes de colección mantienen las reglas diferentes el almacenamiento, y las aplicaciones de la colección diferentes perfeccionan el acceso diferente y ponen al día las conductas.

El método `toString()` devuelve una cadena de texto que representa al objeto. Se puede utilizar **toString** para mostrar un objeto

CAPITULO 9 CERTIFICACION EN JAVA

OBJETIVOS

Este capítulo cubre el objetivo siguiente:

Escribir código que use componentes, container, y el administrado de clasifica del java.awt empaquete para presentar una GUI con la apariencia especificada y conducta del tamaño, y distinga las responsabilidades de Administrador Layout de aquéllos de recipientes.

El acercamiento de Administrador de Layout de Java a las Interfaces del Usuario Gráficas es una novedad. Muchos sistemas de GUI animan a que los programadores de GUI piensen por lo que se refiere a la especificación precisa del tamaño y situación de componentes de la interface. El Windowing Toolkit Abstracto (AWT) proporciona un manajo de Administradores de Layout cada uno de los cuales los instrumentos posee su propia política del Layout. En Java, usted crea un GUI escogiendo a uno o más Administrador de Layout y permitiéndolos que cuiden de los detalles.

Cuando usted empieza trabajando con un Administrador de Layout, usted tenía dos impresiones probablemente:

- Usted ya no se aburre por la carga de especificar la posición exacta y dimensiones de cada componente.
- Usted ya no tenía el poder para especificar la posición exacta y dimensiones de cada componente.

Algunas personas disfrutan trabajando con Administrador de Layout, y otros los no tanto.

Ellos están aquí para quedarse, para que el trabajo se haga mas facil de dominar este rasgo del lenguaje.

Para Adquirir esta competencia requiere tres cosas:

- Una comprensión de por qué Java usa un Administrador de Layout
- Una comprensión de las políticas del Layout de los Administrador de Layout más básicos
- Alguna práctica

La próxima sección explica por qué Java usa a Administrador de Layout. Entonces, después de alguna teoría intermedia sobre cómo Administrador de

Layout trabajan, los últimos tres secciones de este capítulo describen a los tres Administrador de Layout más simples de Java: El Layout de Flow, Layout de la Grid, y el Layout Border. (Cuarto Administrador del Layout, el Layout de Bolsa de Grid, es más complejo que estos tres por un orden de magnitud). el Layout de Bolsa de Grid se menciona brevemente en este capítulo, entonces discutido en detalle en el contexto del examen del diseñador.

Por qué Java Usos Administradores de Layout

Hay dos razones por qué el AWT de Java usa un Administrador de Layout. La primera razón es un poco teórica, y usted puede o no puede encontrarla convincente. La segunda razón es completamente práctica.

La teórica queda en la precisar la posición del Layout (es decir, especificación en los pixel del tamaño de cada componente y posición) es una tarea redundante y a menudo realizada ; por consiguiente, según los principios de programación orientada a objetos, la funcionalidad del Layout debe encapsularse en uno o más clases para automatizar la tarea. Ciertamente los Administrador de Layout eliminan mucho tedio aburrimiento de desarrollo. Muchos programadores detestan la idea de Administrador de Layout al principio, pero vienen a apreciarlos cada vez más ya que los quehaceres tediosos se eliminan.

La razón práctica por tener los Administradores de Layout de la independencia del plataforma de Java. Los componentes de Java piden prestado su conducta del sistema de la ventana del hardware subyacente en que la Máquina Virtual de Java este corriendo. Así en un Macintosh, un botón de AWT se parece cualquier otro botón de Mac; en una plataforma del Motif, un botón de Java se parece cualquier otro botón del Motif, y así sucesivamente. El problema aquí es los botones y otros componentes tienen los tamaños diferentes cuando son colocados en las plataformas diferentes.

Por ejemplo, considere el botón que se construye por la línea siguiente de código:

```
Button b = new Button( "ok" );
```

En un Windows 95, este botón será 32 pixel ancho por 21 pixel alto. En una plataforma del Motif, el botón será todavía 32 pixel ancho, pero será 22 pixel alto, aunque usa el mismo conjunto de caracteres. La diferencia parece pequeña hasta que usted considere el efecto si coloca una columna de muchos botones. Otros componentes también pueden variar en el tamaño de la plataforma a la plataforma. Si Java hubiera encarado clasificando según tamaño pixel preciso y posicionado, habría muchos Java GUIs que parecían exquisito en su plataforma de origen, y terribles en otros organizadores.

No hay ninguna garantía que los conjuntos de caracteres con los nombres idénticos serán de verdad 100 por ciento idénticos de una plataforma a otra plataforma; podría haber diferencias diminutas. Por consiguiente, Java incluso no puede garantizar que dos cordones dibujados con el mismo texto y el conjunto de caracteres desplegará al mismo tamaño por las diferentes plataformas.

Similarmente, no hay ninguna manera de lograr la consistencia del tamaño entre componentes que tienen que tratar con el conjunto de caracteres diferentes y con las diferencias de la decoración.

Java trata este problema delegando el trabajo de Layout al Administrador de Layout. El resto de este capítulo investiga qué Administrador de Layout y se explora los tres Administradores comunes.

La Teoría de Administradores de Layout

Hay que cinco clases de Administradores del Layout que se clasifican en el toolkit de AWT. Usted podría esperar que habría un superclass abstractos comunes, llamada algo como `LayoutManager` de la cual estos cinco Administrador de Layout heredarían la funcionalidad común.

Hay un `java.awt.LayoutManager` de hecho, pero es una interface, no una clase, porque los Administrador de Layout son entre si tan diferentes que entre ellos no tienen nada en común excepto un manojito de nombres del método. (Hay también una interface de `java.awt.LayoutManager2` que los `GridBag`, `Frontera`, y Administradores de Layout de Tarjeta llevan a cabo. El examen de la certificación no cubre a los `GridBag` y Administradores de Layout de Tarjeta, aunque estos dos se discuten en el curso del diseñador.

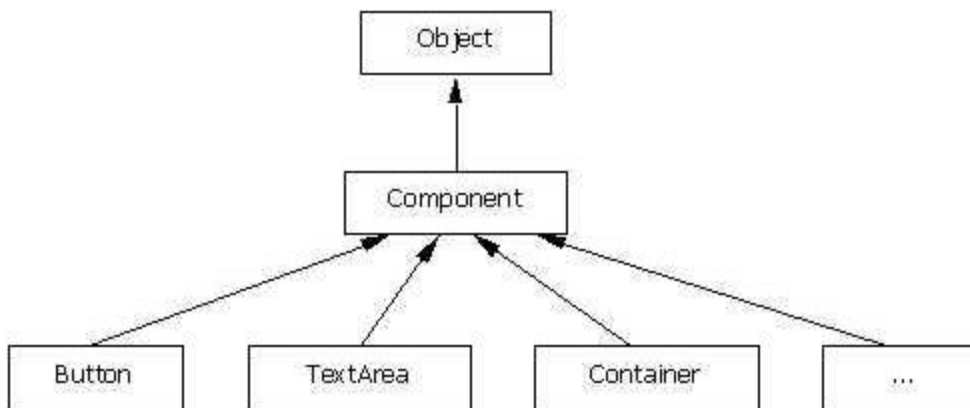
Administrador de Layout trabajan en la sociedad con los recipientes. Para entender un Administrador de Layout, es importante entender lo que es un recipiente y lo que pasa cuando un componente se inserta en un recipiente. Las próximas dos secciones exploran estos temas; la información no se dirige directamente por el examen de la certificación, pero alguna teoría pertinente lo hará muy más fácil a estas alturas de entender el material que se requiere para el examen.

Los recipientes y Componentes

Los recipientes son componentes de Java que pueden contener otros componentes. Hay una clase de `java.awt.Container` que, como `java.awt.Button` y `java.awt.Choice`, hereda de la superclase de `java.awt.Component`. Esta relación de herencia se muestra en Figura 9.1.

FIGURE 9.1

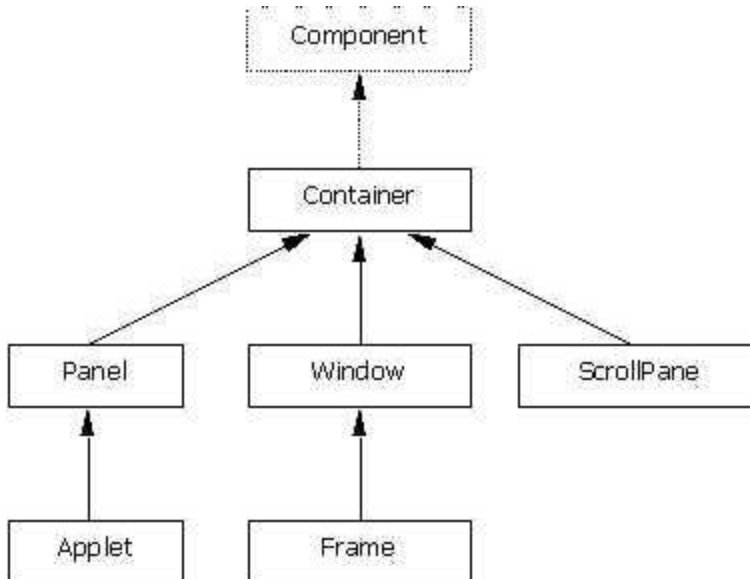
La herencia de java.awt.Container



La clase del Recipiente era abstracta, pero ahora no es, en la mayoría de los casos normalmente se usa los subclasses concretas como lo son Applet, Frame, y Panel, como son mostrados en la Figura 9.2. (la Nota que Applet es un subclass de Panel).

FIGURE 9.2

El subclasses común de java.awt.Container



Java GUIs residen en el applets o en los Frames. Para los applets, usted debe colocar apenas sus componentes en su applet; para las aplicaciones simples,

usted coloca apenas sus componentes en sus Frame. (En ambos casos, usted podría preguntarse cómo los componentes terminan dónde ellos lo hacen; Administrador de Layout están acechando en el fondo, cuidando los detalles).

Para GUIs más complicado, es conveniente dividir el applet o idear en las regiones más pequeñas. Por ejemplo, estas regiones pueden hacer constituir un toolbar o una matriz de botones de la radio. En Java, se llevan a cabo normalmente las implementaciones de GUI la mayoría con el contenedor del Panel.

Los Panels, sólo como los applets y Frames, puede contener otros componentes: los botones, las cajas del cheque, las barras, las áreas del texto, que el texto presenta, y claro otros Panels. GUIs complicados a veces tienen jerarquías de la contención muy complicadas de Panels dentro de los Panels dentro de los Panels dentro de los Panels, y así sucesivamente, abajo a través de muchas capas de contención.

En Java, el término jerarquía es ambiguo. Al discutir las clases, y la herencia se refiere a la jerarquía de herencia del superclass al subclass. Al discutir GUIs, la jerarquía se refiere a la jerarquía de la contención de applets o Frames que contienen Panels que contienen Panels que contienen los Panels.

El GUI en Figura 9.3 es un Frame de tamaño moderado por especificar un color. Usted puede ver de una ojeada que el Panel contiene las etiquetas, las barras del pergamino, que el texto presenta, y botones. Usted probablemente ha supuesto que el Frame también contiene algunos Panels, aunque ellos no pueden verse. De hecho, el Frame contiene cinco Panels. Cada uno de los seis recipientes (los cinco Panels, más el propio Frame) tiene su propio Administrador del Layout: hay cuatro casos de Administradores de Layout de Grid, un Administrador de Layout de Flow, y un Administrador del Layout Border.

FIGURE 9.3

Un GUI con varios niveles de contención

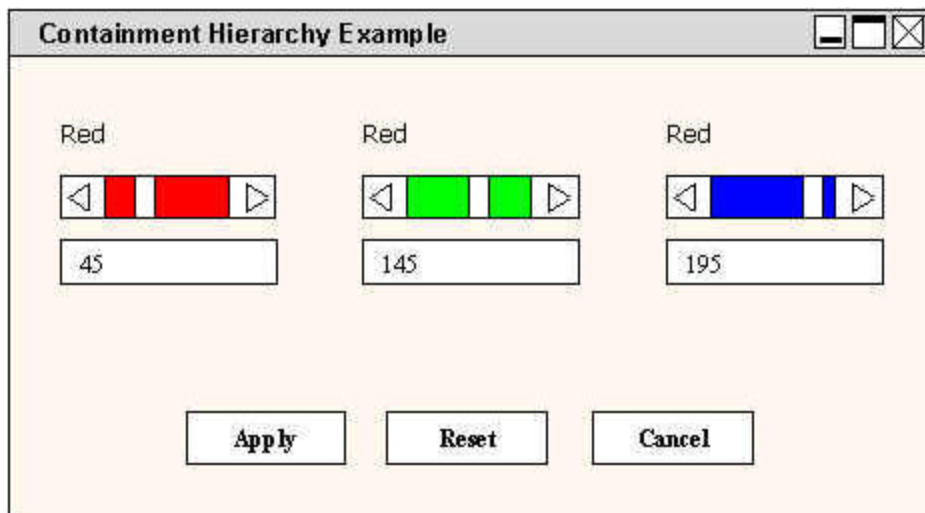
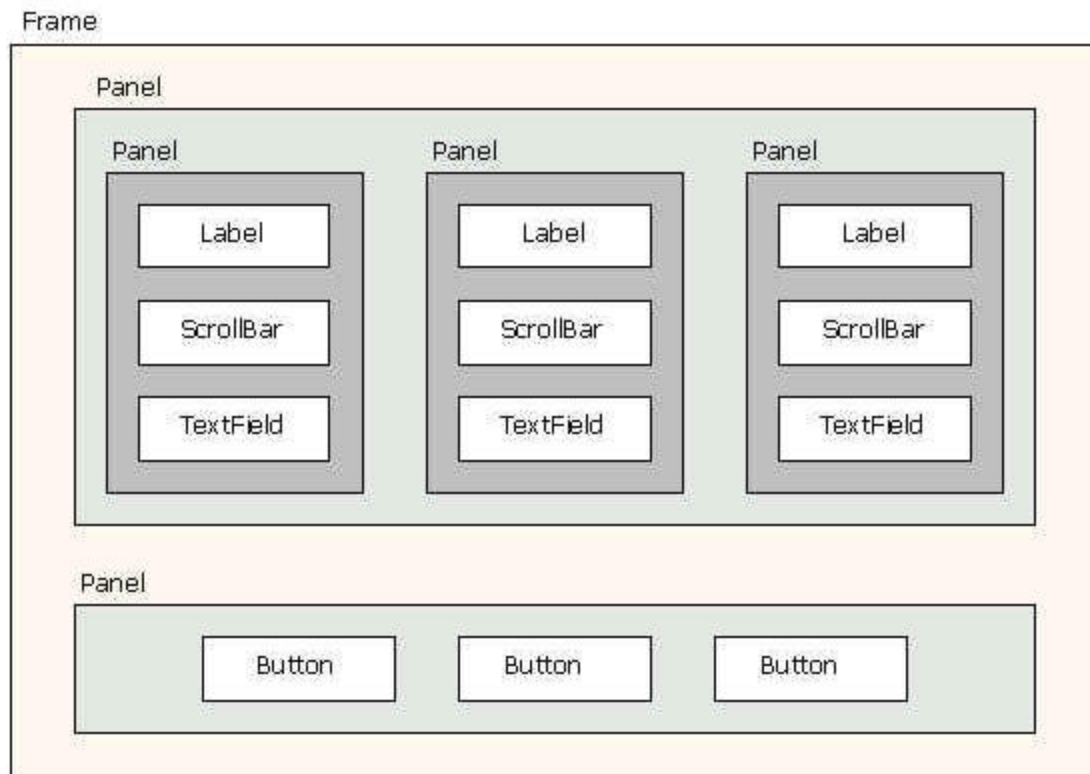


Figure 9.4 esquemáticamente muestra la jerarquía de la contención del Frame. Un programador de JavaGUI debe dominar el arte de transformar un GUI propuesto en una jerarquía de la contención laborable y eficaz. Ésta es una habilidad que viene con la experiencia, una vez los principios se entienden. El Java Certificación Examen del Programador no le exige que desarrolle cualquier contención complicada, pero le exige que entienda los principios.



El código que lleva a cabo el escogedor colorido se lista debajo:

```
1. import java.awt.*;
2. public class Hier extends Frame {
3. Hier() {
4. super( "Containment Hierarchy Example" );
5. setBounds( 20, 20, 300, 180 );
6. setLayout( new BorderLayout( 0, 25 ) );
7. // Build upper panel with 3 horizontal "strips"
8. String strings[] = { "Red", "Green", "Blue" };
9. Panel bigUpperPanel = new Panel();
10.    bigUpperPanel.setLayout( new GridLayout( 1, 3, 20, 0 ) );
11.    for ( int i = 0; i < 3; i++ ) {
12.        // Add strips: each strip is a panel within bigUpperPanel
13.        Panel levelPanel = new Panel();
14.        levelPanel.setLayout( new GridLayout( 3, 1, 0, 10 ) );
15.        levelPanel.add( new Label( strings[ i ] ) );
16.        levelPanel.add( new Scrollbar( Scrollbar.HORIZONTAL, i, 10, 0,
            255 ) );
17.        levelPanel.add( new TextField( "0" ) );
18.        bigUpperPanel.add( levelPanel );
19.    }
20.    add( bigUpperPanel, BorderLayout.CENTER );
21.    // Build lower panel containing 3 buttons
22.    Panel lowerPanel = new Panel();
23.    lowerPanel.add( new Button( "Apply" ) );
24.    lowerPanel.add( new Button( "Reset" ) );
```

```
25.     lowerPanel.add( new Button( "Cancel" ) );
26.     add( lowerPanel, BorderLayout.SOUTH );
27.     }
28.     }
```

Como usted puede ver de la lista de código, no hay ningún código que especifique exactamente en cualquier parte donde deben ir las etiquetas, barras del pergamino, campos del texto, botones, o Panels deben ir. Hay varios llamadas en cambio, (línea 7, 12, y 16) a constructores de Administrador de Layout. En esas mismas líneas, los nuevos Administrador de Layout son fijos como los Administradores para los recipientes correspondientes. El más bajo Panel construyó en la línea 25 usa su Administrador del Layout predefinido, por que no es necesario darle un nuevo.

Un componente dentro de un recipiente recibe ciertas propiedades del recipiente. Por ejemplo, si un componente no se asigna un conjunto de caracteres explícitamente, usa el mismo conjunto de caracteres que sus usos del recipiente. Los mismos sostenimientos del principio se toman para el color de frente y color del fondo. Administrador de Layout, sin embargo, son diferentes. El Administrador de Layout de valor por defecto de un Panel siempre es el Flow. El Administrador de Layout de valor por defecto de un applet también es siempre el Flow. El Administrador de Layout de valor por defecto de un Frame siempre es Border.

Después de que cada Panel se construye y asignó un Administrador de Layout apropiado, el Panel se puebla con los componentes ha contener. Por ejemplo, el más bajo Panel, construido en línea 25, se puebla con los botones en líneas 26, 27, y 28. Finalmente, el Panel ahora-poblado se agrega al recipiente que es sostenerlo (línea 29).

El add() la llamada del método en línea 29 no especifica qué objeto es el de ejecución de la llamada. Es decir, la forma de la llamada es agregar (el params), y no el algunos objetos. añadidos (el params). En Java, cada llamada de método de no-estático se ejecuta por algún objeto; si usted no especifica uno, Java asume que usted pensó ejecutar el método por esto. Así que línea 29 se ejecuta por el caso de Hier que es el recipiente extremo en la jerarquía. Línea 23 que agregan el Panel superior grande es similar: ningún objeto ejecutando se especifica en el add () la llamada, para que el Panel se agrega a esto.

En líneas 16-19, y también en líneas 26-28, un recipiente se especifica para ejecutar el add () la llamada. En esas líneas, se agregan los componentes para interponerse los recipientes.

Cada Panel en el código de la muestra se construye en cuatro pasos:

1. Construya el Panel.
2. De un Administrador del Layout al Panel.
3. Puele el Panel.
4. add el Panel a su propio recipiente.

Cuando un recipiente se construye (paso 1), se da un Administrador del Layout predefinido. Para los Panels, el valor por defecto es un Administrador de Layout de Flow, y camina pueden saltarse 2 si éste es el Administrador deseado. En paso 3, poblando el Panel involucra hacer trampas estructuras los componentes y agregándolos al Panel; si cualquiera de estos componentes se es un Panel, pasos 1-4 deben ser recursivos.

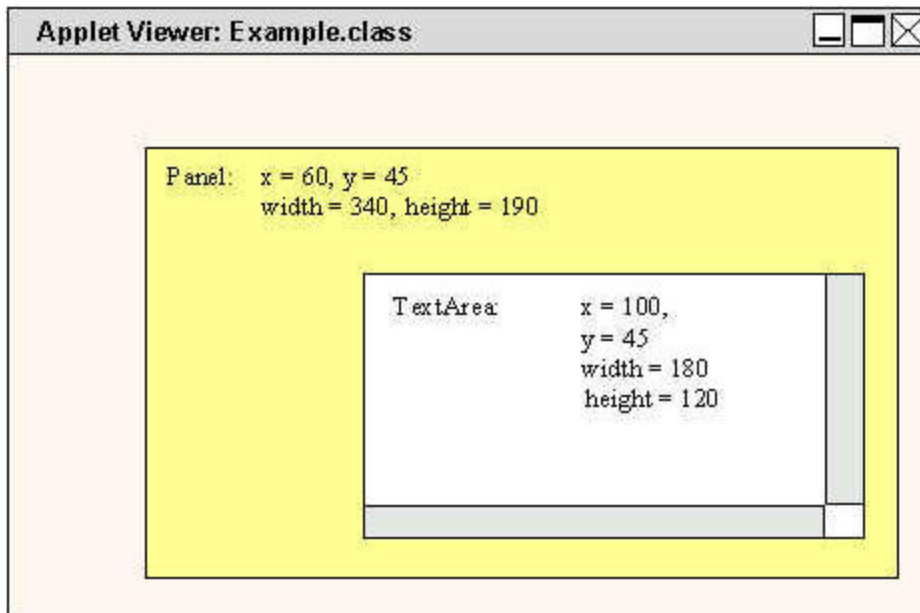
Un recipiente confiere con su Administrador del Layout determinar donde se pondrán los componentes y (opcionalmente) cómo ellos serán los tamaños. Si el hacer trampas-recipientes como consecuencia consigue el tamaño, el Administrador del Layout pone los componentes del recipiente de nuevo (probablemente con los resultados diferentes, desde que tiene una área del diferente para trabajar). Esta "conferencia" entre el recipiente y el Administrador del Layout es el asunto de la próxima sección.

El Tamaño del componente y Posición

Los componentes saben donde ellos son y cómo grande ellos son. Es decir, la clase de `java.awt.Component` tiene las variables del caso llamado `x`, `y`, anchura, y altura. Los `x` y variables de `y` especifican la posición de la esquina superior-izquierda del componente (como moderado de la esquina superior-izquierda del recipiente que contiene el componente), y la anchura y altura están en los pixel. Figure 9.5 ilustra el `x`, `y`, anchura, y altura de un campo del texto dentro de un Panel dentro de un applet.

FIGURE 9.5

La posición y tamaño



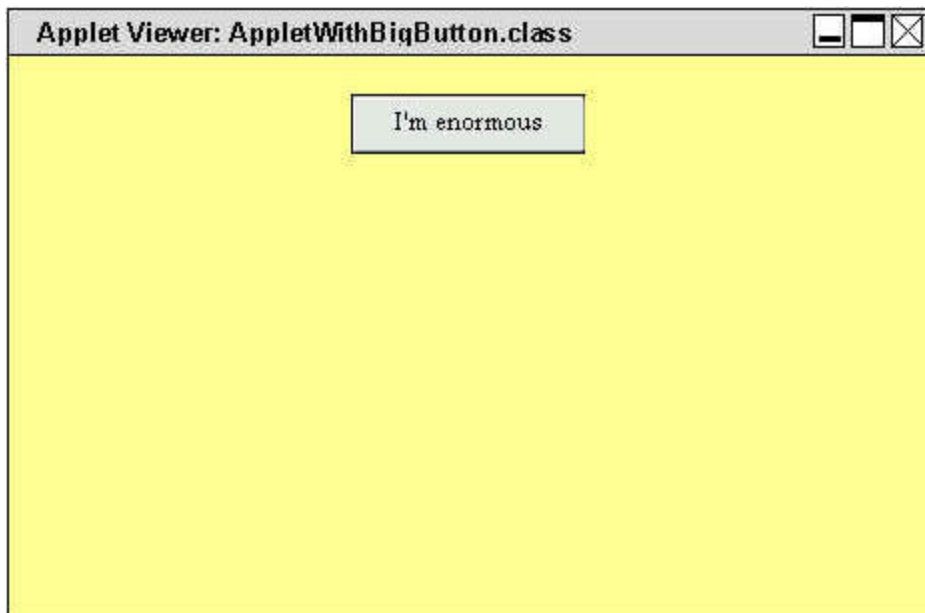
La posición de un componente y tamaño pueden ser cambiados llamando el `setBounds` del componente () el método. (En los descargos del JDK antes de las 1.1, el método se llamó `reforme` (); esto se ha desaprobado a favor del `setBounds` (). parece razonable esperar que el código siguiente que llama el `setBounds` () en un botón, crearía un applet con un botón bastante grande:

1. **`import java.awt.Button;`**
2. **`import java.applet.Applet;`**
3. **`public class AppletWithBigButton extends Applet {`**
4. **`public void init() {`**
5. **`Button b = new Button( "I'm enormous!" );`**
6. **`b.setBounds( 3, 3, 333, 333 ); // should make button really big`**
7. **`add( b );`**
8. **`}`**
9. **`}`**

Si usted ha probado algo así, usted sabe que el resultado está defraudante. Un tiro de la pantalla aparece en Figura 9.6.

FIGURE 9.6

Un botón defraudante



Parece que esa línea 7 debe obligar al botón ser 333 pixel ancho por 333 pixel alto. De hecho, el botón es simplemente el tamaño o el que sería si línea 7 fue omitida o comentada fuera.

Línea 7 no tiene el efecto porque después de que ejecuta, el botón se agrega al applet (línea 8). En el futuro (después de una sucesión bastante complicada de eventos), el applet llama en su Administrador del Layout dar fuerza a su política del Layout en el botón. El Administrador del Layout decide donde y que tan grande el botón debe ser: en este caso, el Administrador del Layout quiere el botón para simplemente ser grande bastante para acomodar su etiqueta. Cuando este tamaño ha sido calculado, el Administrador del Layout llama el `setBounds()` en el botón, golpeando el trabajo que se hicieron en línea 7.

En general, es útil llamar el `setBounds()` en un componente, porque Administrador de Layout siempre corren la voz; es decir, su llamada al `setBounds()` pasa después del suyo. Hay maneras de derrotar esta funcionalidad, pero ellos cuidan ser complicados, difícil mantener, y no es el espíritu de Java.

El toolkit de AWT de Java quiere que se permita a los Administrador de Layout hacer el trabajo del Layout. Java lo supone a usar a su Administrador de Layout, y el Examen de la Certificación espera que usted sepa las políticas del Layout de los Administradores más básicos. Estas políticas se cubren en las próximas secciones.

Las Políticas del Layout

Cada componente de Java tiene un tamaño preferido. El tamaño preferido expresa cómo grande al componente le gustaría ser, excepto el conflicto con un

Administrador del Layout. El tamaño preferido generalmente es el tamaño más pequeño necesario dar el componente de una manera visualmente significativa.

Por ejemplo, el tamaño preferido de un botón es el tamaño de su texto de la etiqueta, más un poco la frontera de espacio vacío alrededor del texto, más las decoraciones sombreadas que marcan el o el límite el botón. Así el tamaño preferido de un botón es "simplemente grande bastante."

El tamaño preferido es la persona a cargo de la plataforma, desde que las decoraciones de límite de componente varían de sistema a sistema.

Cuando un Administrador del Layout pone los componentes del niño de su recipiente, tiene que equilibrar dos consideraciones: la política del Layout y el tamaño preferido de cada componente. Primero la prioridad va a dar fuerza a la política del Layout. Si honrando el tamaño preferido de un componente significaran violando la política del Layout, entonces el Administrador del Layout toma el tamaño preferido del componente.

Entendiendo a un Administrador del Layout significa comprensión dónde pondrá un componente, y también cómo tratará el tamaño preferido de un componente. Las próximas secciones discuten algunos Administradores Layout más simple : FlowLayout, GridLayout, y BorderLayout. Éstos son los tres Administradores que usted debe conocer por el Examen de la Certificación.

El Administrador de Layout de Flow

El Administrador de FlowLayout coloca los componentes en las filas horizontales. Es el tipo del Administrador predefinido para los Panels y applets, para que normalmente es el primer Administrador del Layout que programadores encuentran. Es una experiencia común para los nuevos diseñadores de Java agregar unos componentes a un applet y maravilla cómo ellos llegaron a ser colocados tan pulcramente. El código siguiente es un ejemplo bueno:

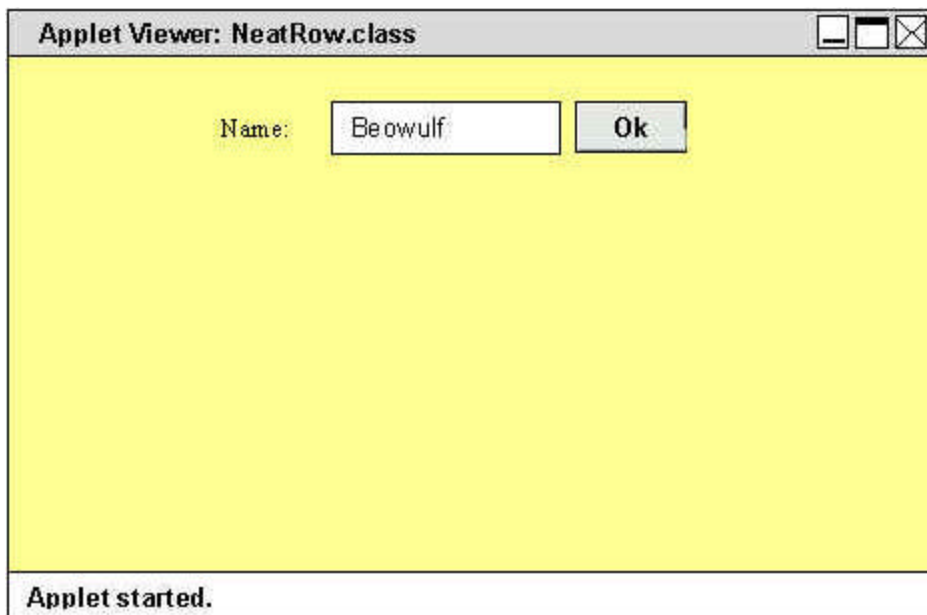
```
1. import java.awt.*;  
2. import java.applet.Applet;  
3. public class NeatRow extends Applet {  
4. public void init() {  
5. Label label = new Label( "Name:" );  
6. add( label );  
7. TextField textfield = new TextField( "Beowulf" );
```

```
8. add( textfield );
9. Button button = new Button( "Ok" );
10.    add( button );
11.    }
12.    }
```

El applet resultante se muestra en Figura 9.7.

FIGURE 9.7

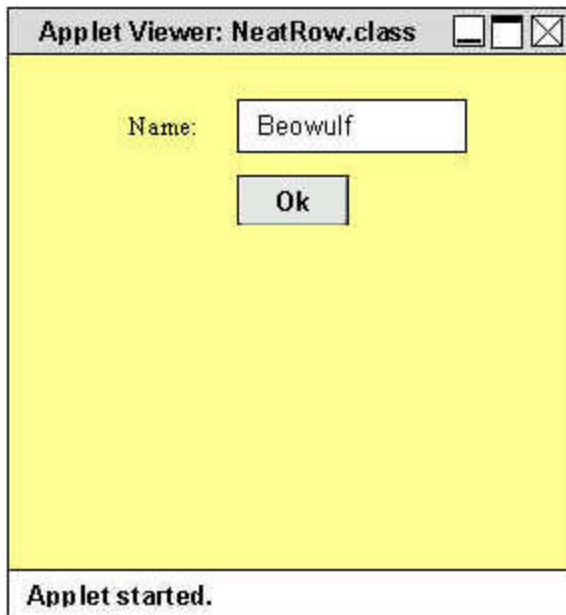
Applet simple que usa a Administrador de Layout de Flow



Si los mismos tres componentes aparecen en un applet del tipo angosto, como mostrado en Figura 9.8, no hay bastante espacio para todos los tres encajar en una sola fila. El Administrador de Layout de Flow encaja los tantos componentes como posible en la fila de la cima y derramamientos el resto en una segunda fila. Los componentes siempre aparecen, dejó corregir, en el orden en que ellos se agregaron a su recipiente.

FIGURE 9.8

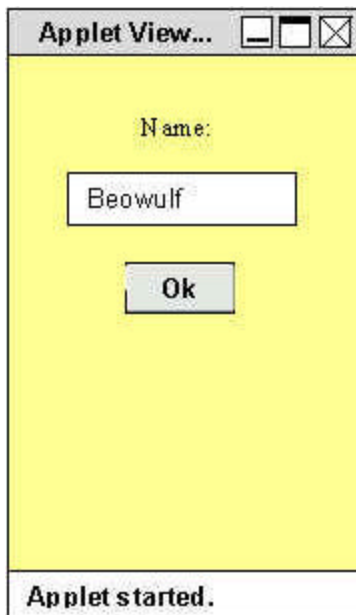
Un applet del angosto



Si el applet todavía está más delgado, como entonces en Figura 9.9, el Administrador de Layout de Flow crea todavía otra fila.

FIGURE 9.9

Un applet muy estrechos



Dentro de cada fila, los componentes se espacian uniformemente, y el racimo de componentes se centra. La justificación del arracimarse puede controlarse pasando un parámetro al constructor de `FlowLayout`. El posible arco de valores `FlowLayout.LEFT`, `FlowLayout.CENTER`, y `FlowLayout.RIGHT`. El applet listado explícitamente debajo construye a un Administrador de Layout de Flow para derecho-justificar cuatro botones:

```

1. import java.awt.*;
2. import Java.applet.Applet;
3. public class FlowRight extends Applet {
4. public void init() {
5.     setLayout( new FlowLayout( FlowLayout.RIGHT ) );
6.     for ( int i = 0; i < 5; i++ ) {
7.         add( new Button( "#" + i ) );
8.     }
9. }
10.    }

```

Figure 9.10 muestras el applet resultante con una ventana ancha.

FIGURE 9.10

Administrador de Layout de Flow derecho-justificativo

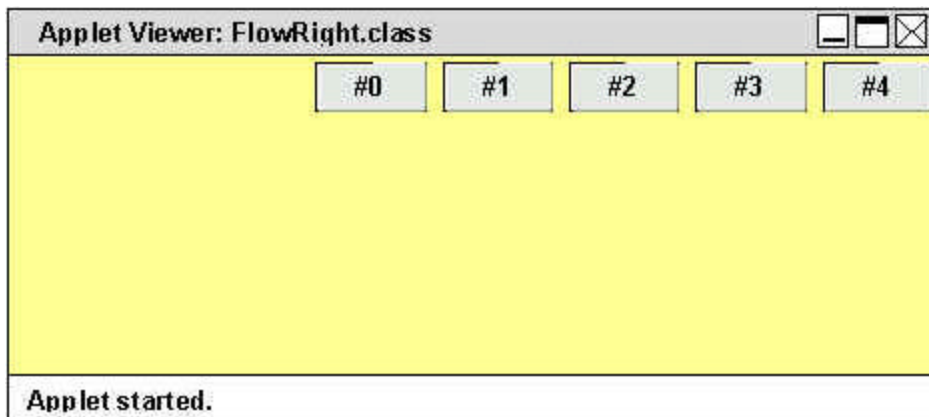
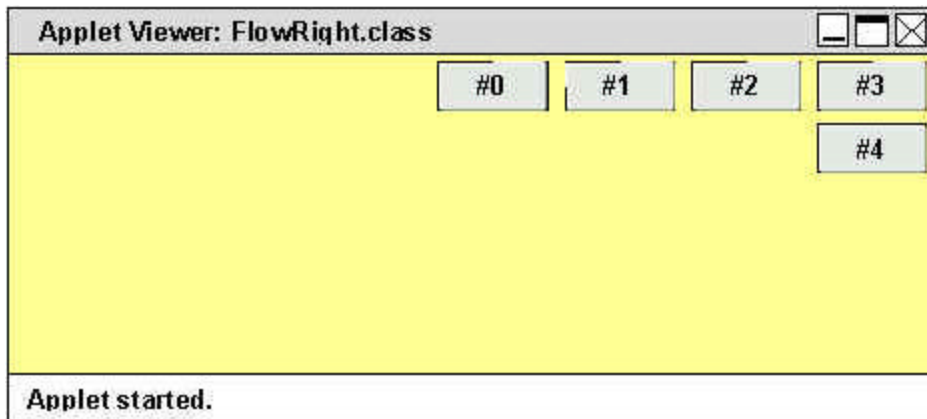


Figure 9.11 usos el mismo Administrador del Layout y componentes como Figura 9.10, pero el applet es el narrower.

FIGURE 9.11

Administrador de Layout de Flow derecho-justificativo estrecho



Por defecto, el Administrador de Layout de Flow deja un hueco de cinco pixel entre los componentes en ambas direcciones horizontales y verticales. Este valor por defecto puede cambiarse llamando una versión cargada excesivamente del constructor de FlowLayout, mientras pasando en los huecos horizontales y verticales deseados. Todos los Administradores de Layout tienen esta capacidad. No se cubren los huecos en el examen de la certificación, pero ellos son ciertamente bueno saber sobre ellos. Una modificación del hueco pequeña puede mejorar la apariencia de un GUI grandemente. En el programa de la muestra en la sección de este capítulo "los Recipientes y Componentes" titularon, se usaron los huecos en las líneas 12 y 16.

El Administrador de Layout de Grid

El Administrador de Layout de Flow siempre honra el tamaño preferido de un componente. El Administrador de Layout de Grid toma el extremo contrario: siempre ignora el tamaño preferido de un componente.

El Administrador de Layout de Grid subdivide su territorio en una matriz de filas y columnas. El número de filas y número de columnas se especifica como los parámetros al constructor del Administrador:

GridLayout (el nRows del int, nColumns del int)

El código listado debajo de los usos un Administrador de Layout de Grid para dividir un applet en 5 filas y 3 columnas, y entonces pone un botón en cada célula de la Grid:

1. **import** java.awt.*;
2. **import** java.applet.Applet;
3. **public class** Grid5x3 **extends** Applet {
4. **public void** init() {

```

5. setLayout( new GridLayout( 5, 3 ) );
6. for ( int row = 0; row < 5; row++ ) {
7. add( new Label( "Label " + row ) );
8. add( new Button( "Button " + row ) );
9. add( new TextField( "TextField " + row ) );
10. }
11. }
12. }

```

La nota que el constructor en línea 6 rinde cinco filas y tres columnas, no al revés. Después de los tantos años de programar con las coordenadas de Cartesiano, es probablemente segunda naturaleza para la mayoría de los programadores especificar clases horizontales de información antes de la coma, y clases verticales de información después de la coma. El constructor de GridLayout usa "fila-mayor" anotación que a veces está confundiendo los humanos.

Como usted puede ver en Figura 9.12, cada componente en el applet es exactamente el mismo tamaño. Los componentes aparecen en el orden en que ellos fueron agregados.

Si los mismos componentes serán puestos fuera en un más alto, applet de angosto, entonces cada componente es proporcionalmente más alto y angosto, como mostrado en Figura 9.13.

FIGURE 9.12

El Layout de la Grid

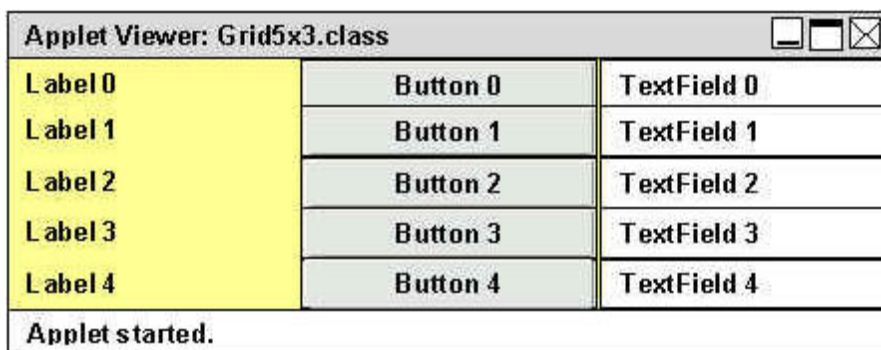
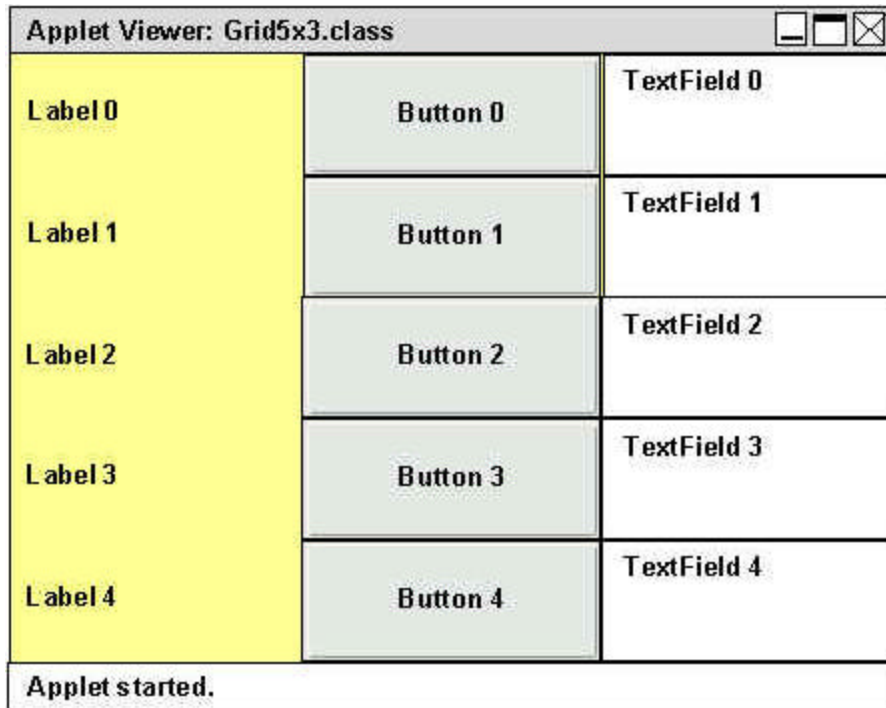


FIGURE 9.13

Un Layout de la Grid más alto



Administradores de Layout de Grid se comportan extrañamente cuando usted les hace manejar los muy pocos componentes (es decir, significativamente menos que el número de tiempos de las filas el número de columnas) o muchos componentes (es decir, más del número de tiempos de las filas el número de columnas).

El Administrador del Layout Border

El Administrador del Layout Border es el Administrador predefinido para los Frames. Da fuerza a un mucho de las políticas del Layout menos intuitivas que el Flow o Administradores de la Grid.

El Administrador de FlowLayout siempre honra el tamaño preferido de un componente; el Administrador de GridLayout nunca hace. El Administrador del Layout Border hace el alguna cosa entre ellos.

El Administrador del Layout Border divide su territorio en cinco regiones. Los nombres de estas regiones son Nortes, Sur, Orientales, Orientales, y Centro. Cada región puede contener un solo componente (pero ninguna región se exige contener un componente).

El componente al Norte se posiciona a la cima del recipiente, y el componente a Sur se posiciona al fondo. El Administrador del Layout honra la altura preferida

de los componentes Nortes y Sur, y les obliga a que sean exactamente tan ancho como el recipiente.

Las regiones Nortes y Sur son útiles para el toolbars, el estado línea, y algunos otros mandos que deben ser tan anchos como posible, pero ningún más alto que necesario. Figure 9.14 muestras un applet que usa a un Administrador de BorderLayout para posicionar un toolbar al Norte y una línea de estado a Sur. El conjunto de caracteres de la línea de estado es fijo grande ilustrar que la altura de cada uno de estas regiones se dicta por la altura preferida del componente en la región. (Para la simplicidad, el toolbar es simplemente un Panel que contiene unos botones).

FIGURE 9.14

El Layout Border para el toolbar y línea de estado

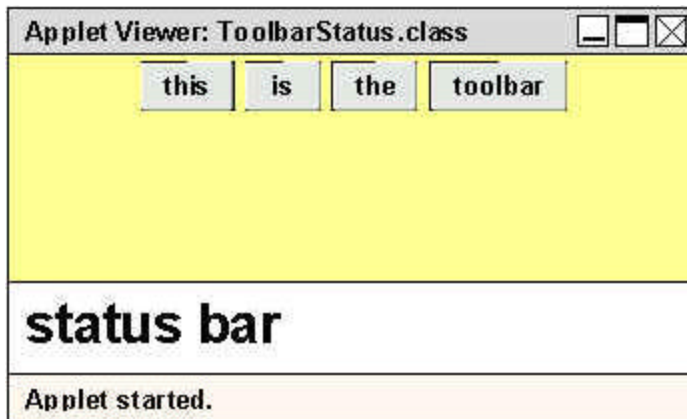
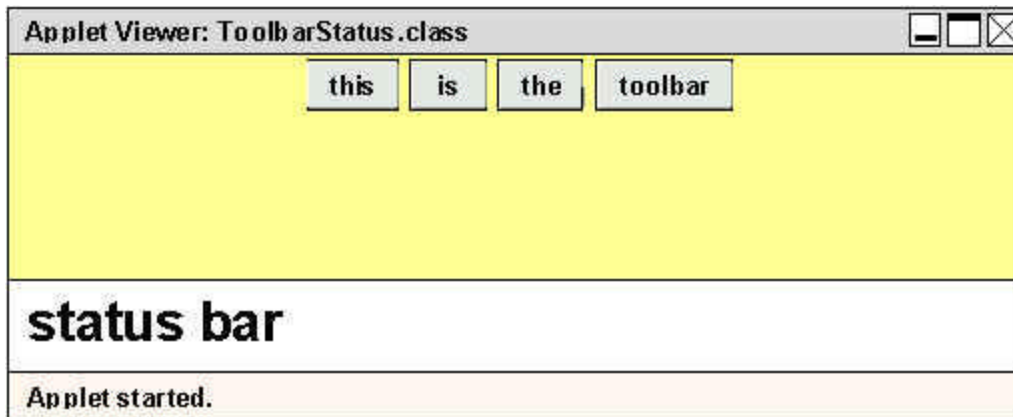


Figure 9.15 muestras lo que pasa si el mismo código es acostumbrado al Layout un applet más grande. El aviso que el toolbar todavía está en la cima, y los estados todavía linean al fondo. El toolbar y los estados linean es tan alto como ellos estaban en Figura 9.14, y ellos son automáticamente tan anchos como el propio applet.

FIGURE 9.15

El Layout Border más grande



El código que produjo estos tiros de la pantalla aparece debajo:

```
1. import java.awt.*;
2. import java.applet.Applet;
3. public class ToolbarStatus extends Applet {
4. public void init() {
5.     setLayout( new BorderLayout() );
6.
7.     // build, populate, and add toolbar
8.     Panel toolbar = new Panel();
9.     toolbar.add( new Button( "This" ) );
10.    toolbar.add( new Button( "Is" ) );
11.    toolbar.add( new Button( "The" ) );
12.    toolbar.add( new Button( "Toolbar" ) );
13.    add( toolbar, BorderLayout.NORTH );
14.    // add status line.
15.    TextField status = new TextField( "Status" );
16.    status.setFont( new Font( "Monospaced", Font.BOLD, 48 ) );
17.    add( status, BorderLayout.SOUTH );
18. }
```

19. }

Note en las líneas 14 y 19, una forma cargada excesivamente del `add ()` el método se usa. El `Layout Border` no es afectado por el orden en que usted agrega el componente. En cambio, usted debe especificar qué de las cinco regiones el componente que usted está agregando recibirá. La versión cargada excesivamente de `add ()` toma dos parámetros: primero el componente a agregándose, y segundo un Objeto. El uso apropiado del Administrador de `BorderLayout` requiere que el segundo parámetro es un Cordón que especifica el nombre de la región; los valores válidos para este Cordón son:

- **"North"**
- **"South"**
- **"East"**
- **"West"**
- **"Center"**

El cordón debe deletrearse exactamente como mostrado anteriormente. La clase de `BorderLayout` ha definido constantes que usted puede usar en lugar de los cordones (el hacer trampas-stants se define para ser los cordones ellos). es una idea buena para usar las constantes definidas en lugar de los cordones, porque si usted deletrea mal el nombre de una constante, el compilador le permitirá saber (por otro lado, si usted usa un Cordón deletreado mal literal, una excepción del runtime se tirará). Los cinco hacer trampas-stants es:

- **`BorderLayout.NORTH`**
- **`BorderLayout.SOUTH`**
- **`BorderLayout.EAST`**
- **`BorderLayout.WEST`**
- **`BorderLayout.CENTER`**

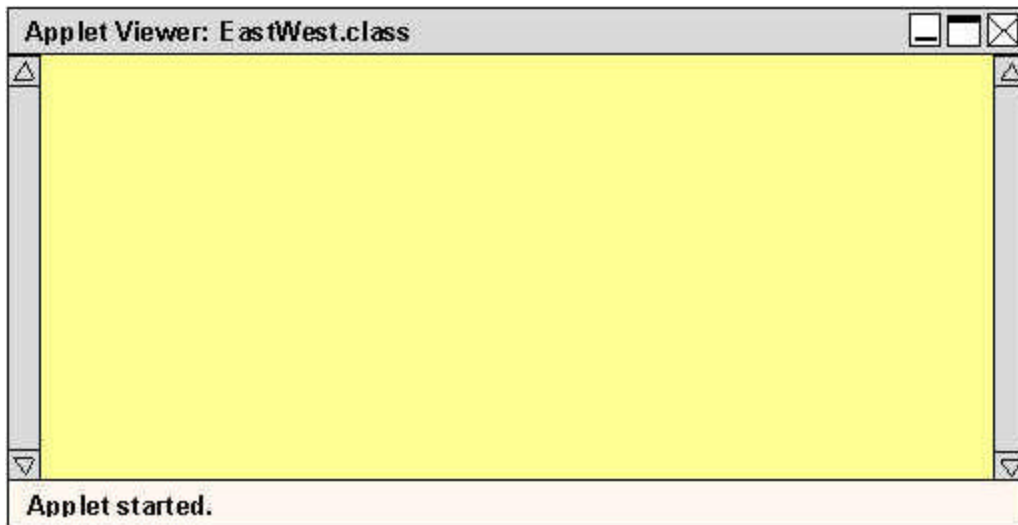
Las regiones Orientales y Occidentales son el contrario de Norte y Sur: en el Este y Hacia el oeste, un componente consigue ser su anchura preferida pero tiene su altura reprimida. Aquí un componente se extiende verticalmente a al fondo del componente Norte (si hay uno) o a la cima del recipiente (si no hay ningún componente Norte). UN componente se extiende abajo a la cima del

componente Sur (si hay uno) o al fondo del recipiente (si no hay ningún componente Sur).

Figuras 9.16 a través de 9.19 applets de muestra que acostumbran a un Administrador de BorderLayout al Layout dos barras del pergamino, uno al Este y uno a Hacia el oeste. En Figura 9.16, no hay ningún componente al Norte o Sur para contender con.

FIGURE 9.16

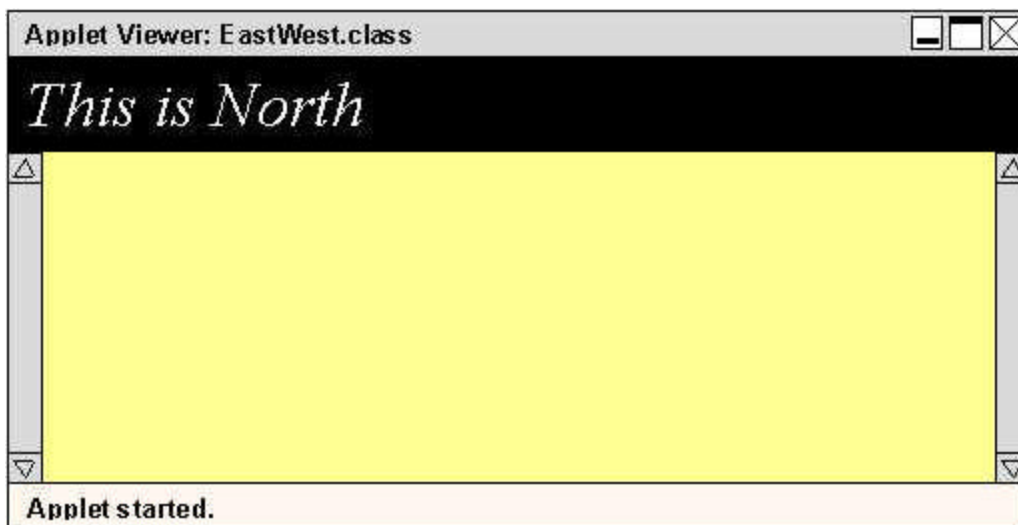
Del este y Hacia el oeste



En Figura 9.17, hay una etiqueta al Norte.

FIGURE 9.17

Del este y Norte de West,with

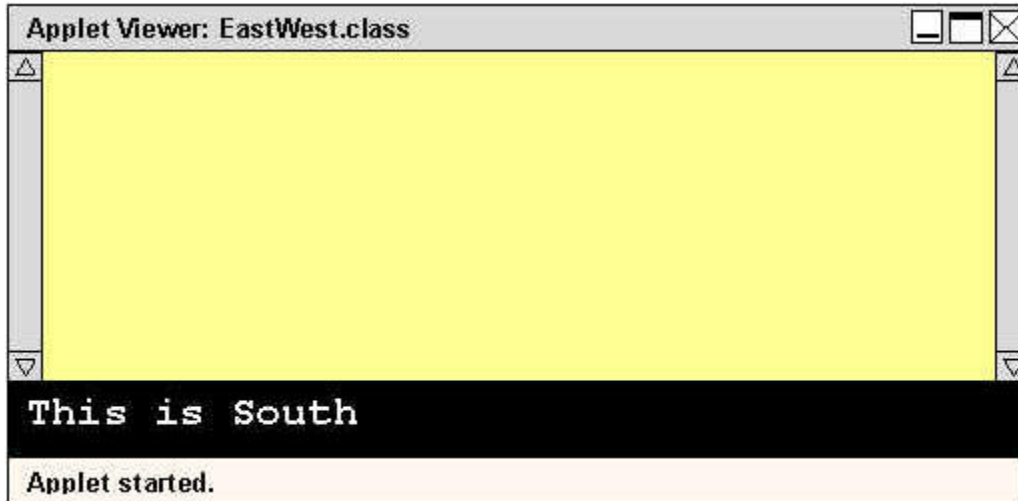


En Figura 9.18, hay una etiqueta a Sur. La etiqueta tiene el texto blanco en el

fondo negro para que usted pueda ver exactamente donde la región Sur es.

FIGURE 9.18

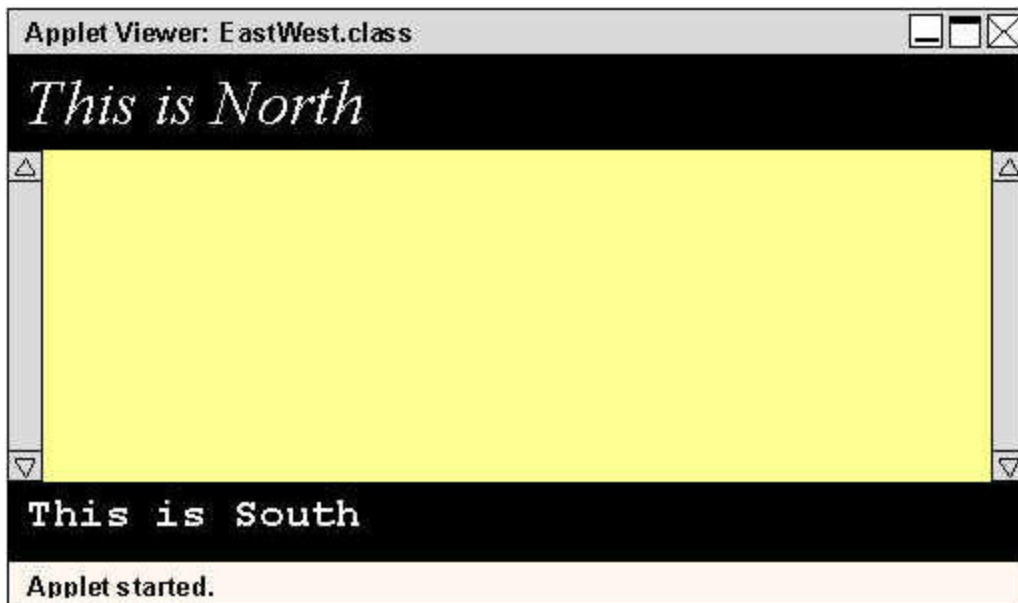
Del este y Hacia el oeste, con Sur



En Figura 9.19, hay etiquetas a Norte y " Sur. Las etiquetas tienen el texto blanco en el fondo negro para que usted pueda ver exactamente donde las regiones Nortes y Sur son.

FIGURE 9.19

Del este y Hacia el oeste, con Norte y " Sur



El código que generó estos cuatro applets se lista hay sólo un programa. El código, como mostrado, genera Figura 9.19 (Norte y " Sur); línea 19 y 24 se comentaron juiciosamente fuera generar las otras figuras:

```

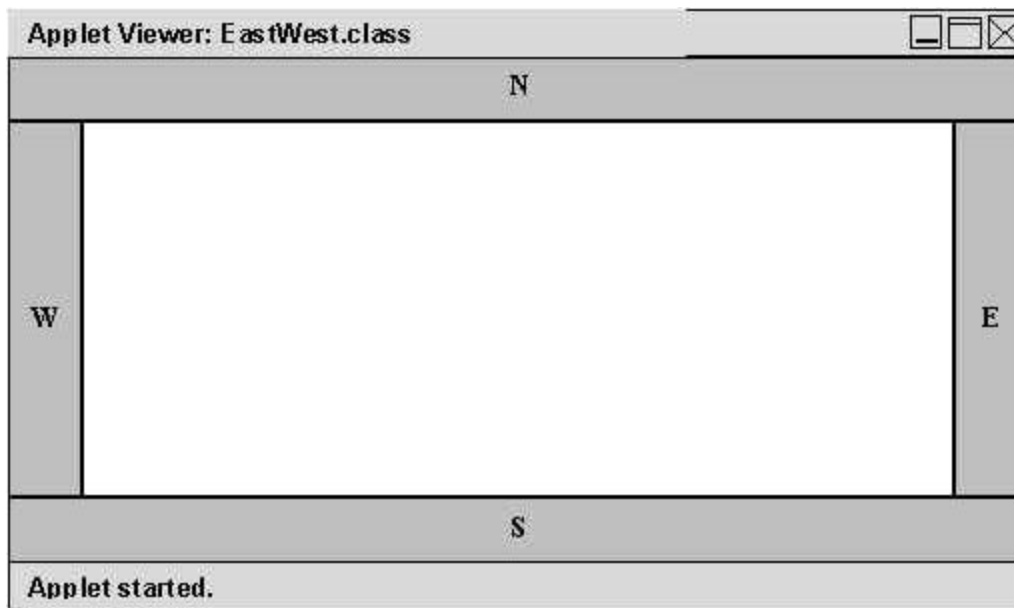
1. import java.awt.*;
2. import Java.applet.Applet;
3. public class EastWest extends Applet {
4. public void init() {
5. setLayout( new BorderLayout() );
6. // scroll bars at East and West
7. Scrollbar sbRight = new Scrollbar( Scrollbar.VERTICAL );
8. add( sbRight, BorderLayout.EAST );
9. Scrollbar sbLeft = new Scrollbar( Scrollbar.VERTICAL );
10.    add( sbLeft, BorderLayout.WEST );
11.    // labels at North and South
12.    Label labelTop = new Label( "This is North" );
13.    labelTop.setFont( new Font( "Serif", Font.ITALIC, 36 ) );
14.    labelTop.setForeground( Color.white );
15.    labelTop.setBackground( Color.black );
16.    add( labelTop, BorderLayout.NORTH );
17.    Label labelBottom = new Label( "This is South" );
18.    labelBottom.setFont( new Font( "Monospaced",Font.BOLD,18 ) );
19.    labelBottom.setForeground( Color.white );
20.    labelBottom.setBackground( Color.black );
21.    add( labelBottom, BorderLayout.SOUTH );
22.    }
23.    }

```

La quinta región que un mando de Administrador de Layout Borders se llaman el Centro. El centro simplemente es la parte de un recipiente que permanece después Norte, Sur, Oriental, y Hacia el oeste se ha asignado. Figure 9.20 muestras un applet con Norte, Sur, Oriental, y Hacia el oeste, y un Panel al Centro. El Panel es la región blanca.

FIGURE 9.20

El centro



El código que generó Figura 9.20 se lista debajo:

```
1. import java.awt.*;  
2. import Java.applet.Applet;  
3. public class Center extends Applet {  
4.   public void init() {  
5.     setLayout( new BorderLayout() );  
6.     add( new Button( "N" ), BorderLayout.NORTH );  
7.     add( new Button( "S" ), BorderLayout.SOUTH );  
8.     add( new Button( "E" ), BorderLayout.EAST );  
9.     add( new Button( "W" ), BorderLayout.WEST );  
10.    Panel p = new Panel();  
11.    p.setBackground( Color.white );  
12.    add( p, BorderLayout.CENTER );  
13.  }
```

14. }

En línea 13, el Panel blanco se agrega a la región del Centro. Al agregar un componente para Centrar, es legal omitir el segundo parámetro al `add()` la llamada; el Administrador de BorderLayout asumirá que usted quiso decir el Centro. Sin embargo, es más fácil para otras personas entender su código si usted especifica la región explícitamente, como en línea 13 anteriormente.

Otras Opciones del Layout

El examen de la certificación sólo le exige que sepa sobre el Flow, Grid, y los Layouts Borders. Sin embargo, es útil saber un poco el pedazo sobre las otras opciones. Si usted está en una situación donde el Flow, Grid, y el `Border` no crean el Layout usted necesita, sus opciones son:

- para usar a un GridBag Layout Administrador
- para usar a un Administrador de Layout de Tarjeta
- para usar a ningún Administrador del Layout
- para crear a su propio Administrador del Layout

GridBag es por lejos el Administrador del Layout más complicado. Divide su recipiente en una serie de células, pero (diferente las células de un Administrador de GridLayout) las filas de la célula diferentes pueden tener las alturas diferentes, y las columnas celulares diferentes pueden tener las anchuras diferentes.

Un componente puede ocupar una sola célula o puede medir por palmos varios células. Un GridBag Layout Administrador exige a mucha información saber dónde poner un componente. Una clase del auxiliar llamada `GridBagConstraints` es acostumbrado a sostener toda la información de posición de Layout. Cuando usted agrega un componente, usted usa el `add()` (el Componente, Objeto) la versión del `add()` el método, pasando un caso de `GridBagConstraints` como el parámetro del Objeto.

El Administrador de Layout de Tarjeta pone sus componentes a tiempo en lugar en el espacio. En cualquier momento, un recipiente que usa a un Administrador de Layout de Tarjeta está desplegando uno u otro de sus componentes; todos los otros componentes están inadvertidos. Una llamada del método al Administrador del Layout puede decirle que despliegue un componente diferente. Todos los componentes (qué normalmente es los Panels) es los `resized` para ocupar el recipiente entero. El resultado es similar a un Panel del `tabbed` sin las etiquetas.

Usted siempre tiene la opción de usar a ningún Administrador del Layout en

absoluto. Para hacer esto, sólo llamada `myContainer.setLayout (nulo);`

Si un recipiente no tiene ningún Administrador del Layout, honra el `x` de cada componente, `y`, anchura, y valores de altura. Así usted puede llamar el `setBounds ()` en un componente, `add ()` él a un recipiente que no tiene ningún Administrador del Layout, y tiene el extremo del componente a dónde usted lo espera para ser. Esto está tentando ciertamente, pero esperanzadamente la primera parte de este capítulo lo ha convencido que Administrador de Layout son simples y eficaces trabajar.

Es más, si su recipiente reside en un recipiente más grande (un Frame, por ejemplo) eso consigue el `resized`. Las personas que pusieron al Administrador del Layout de un recipiente a hallazgo nulo que ellos tienen que escribir el código para descubrir cuando el `resizes` del recipiente, y más codifica para hacer la cosa correcta cuando el `resizing` ocurre. Esto termina siendo más complicado que creando a su propio Administrador del Layout.

Está más allá del alcance de este libro mostrarle cómo preparar a su propio Administrador del Layout, pero para las políticas del Layout simples no es especialmente difícil hacer para que. La ventaja de crear a un Administrador de Layout de costumbre encima de poner al Administrador del Layout de un recipiente a nulo es que usted ya no tiene que escribir el código para descubrir `resizing` del recipiente; usted apenas tiene que escribir el código para llevar a cabo la política del Layout, y el sistema hará las llamadas correctas en el momento correcto.

Escribiendo su propia clase de Administrador de Layout involucra llevando a cabo el `LayoutManager` una (o posiblemente los `LayoutManager2` unen). Para una referencia buena con los ejemplos en cómo hacer esto, vea *Java 2 Developer's Mano-libro* (Sybex, 1999).

Administrador de Layout le exigen que use su habilidad de visualizar en dos dimensiones más de cualquier otro tema Java-relacionado. Cuando usted toma el examen del certificación, usted se dará la herramienta perfecta para apoyar el pensamiento bidimensional: una hoja pálida de papel improvisado. Ésta es la única cosa le permitirán traer en su cubículo de la prueba (y usted tendrá que devolverlo cuando usted sale).

Aparte de los problemas de Administrador de Layout, es difícil de imaginar lo para que el papel improvisado es bueno. Considere el dibujo un cuadro de cada problema de Administrador de Layout, si o no se siente como usted necesitara uno. Usted no correrá fuera de papel. Su cuadro no podría convencerle que escogiera una respuesta diferente que usted habría por otra, pero si este truco le ayuda a hacer la respuesta correcta a incluso un problema de Administrador de Layout extra, entonces ha hecho su trabajo.

El Resumen del capítulo

Administrador de Layout proporcionan una capa de apoyo geométrico que lo releva de tener que especificar el tamaño y posición de cada componente de GUI usted crea.

El tradeoff es que usted tiene que ser consciente de la implementación de política de Layout por cada uno de los varios Administrador de Layout. Le obligan a que piense por lo que se refiere a la política, en lugar de la anchura absoluta, altura, y posición.

Este capítulo ha discutido los tres más Administrador de Layout básicos: El Flow, Grid, y Frontera. El Administrador del Layout más avanzado, GridBag, se discute en el próximo curso, "Diseñando la Interface del Usuario."

10. EVENTOS

OBJETIVOS

Este capítulo cubre aspectos del siguiente objetivo del examen de certificación de Java:

- Escribir el código para implementar clases y métodos oyente, y en métodos oyente, extraer información del evento para determinar el componente afectado, posición del ratón, naturaleza y tiempo del evento. Declarar el classname del evento para cualquier interface específica de oyente en el paquete de java.awt.event.

El modelo de evento original de Java "outward rippling" ("ondeo externo") demostró tener algunos pequeños resultados . Un nuevo modelo "event delegation" ("comisión de evento") se introdujo, en consecuencia, en la versión 1.1 del JDK. Ambos modelos están basados en Java 2, pero eventualmente el modelo antiguo tenderá a desaparecer. Para ahora, todos los métodos que soportan al modelo de evento viejo no son tenidos en cuenta.

Los dos modelos son mutuamente incompatibles. Es probable que un programa de Java que usa a ambos modelos falle, con eventos que están perdidos o incorrectamente procesados.

Este capítulo repasa al nuevo modelo en detalle.

MOTIVACIÓN PARA EL MODELO EVENT DELEGATION (COMISIÓN DE EVENTO)

Ciertas fallas en el modelo de evento original aparecieron cuando Java ya llevaba mucho tiempo en el mundo desarrollando programas. El problema mayor era que un evento sólo pudiera ser manejado por el componente que originó el evento o por uno de los recipientes que contuvieron el componente originando. Esta restricción violó uno de los principios fundamentales de la programación orientada a objetos: la funcionalidad debe residir en la clase más apropiada. A menudo la clase más apropiada para manejar un evento no es un miembro de la jerarquía contenedora del componente generador.

Otro inconveniente del modelo original era que un número grande de ciclos de CPU se utilizaba en eventos poco interesantes.

Un evento en el cual un programa no tenía interés ondearía a través de toda la jerarquía que lo contiene antes de ser eventualmente descartado. El modelo original de evento no proporcionó ninguna manera de desactivar el procesamiento de eventos no pertinentes.

En el modelo de delegación de eventos, un componente puede decir qué objeto u objetos deben notificarse cuando el componente genera un tipo particular de evento. Si un componente no está interesado en un tipo de evento, entonces no se propagarán eventos de ese tipo.

El modelo de delegación es basado en cuatro conceptos:

- Eventos Clases
- Eventos Oyentes

- Activación explícita del evento
- Adaptadores

Este capítulo explica cada uno de estos conceptos a su vez.

LA JERARQUÍA DE EVENTOS CLASE

El modelo de delegación de eventos define un gran número de nuevos eventos clases. La jerarquía de eventos clases que usted necesita conocer para el examen se muestra en la Figura 10.1. La Mayoría de las clases evento reside en el paquete de **java.awt.event**.

La superclase de más alto nivel de todas los nuevos eventos clases es **java.util.EventObject**. Esta una clase muy general, con sólo un método de interés:

Object getSource (): Retorna el objeto que originó el evento

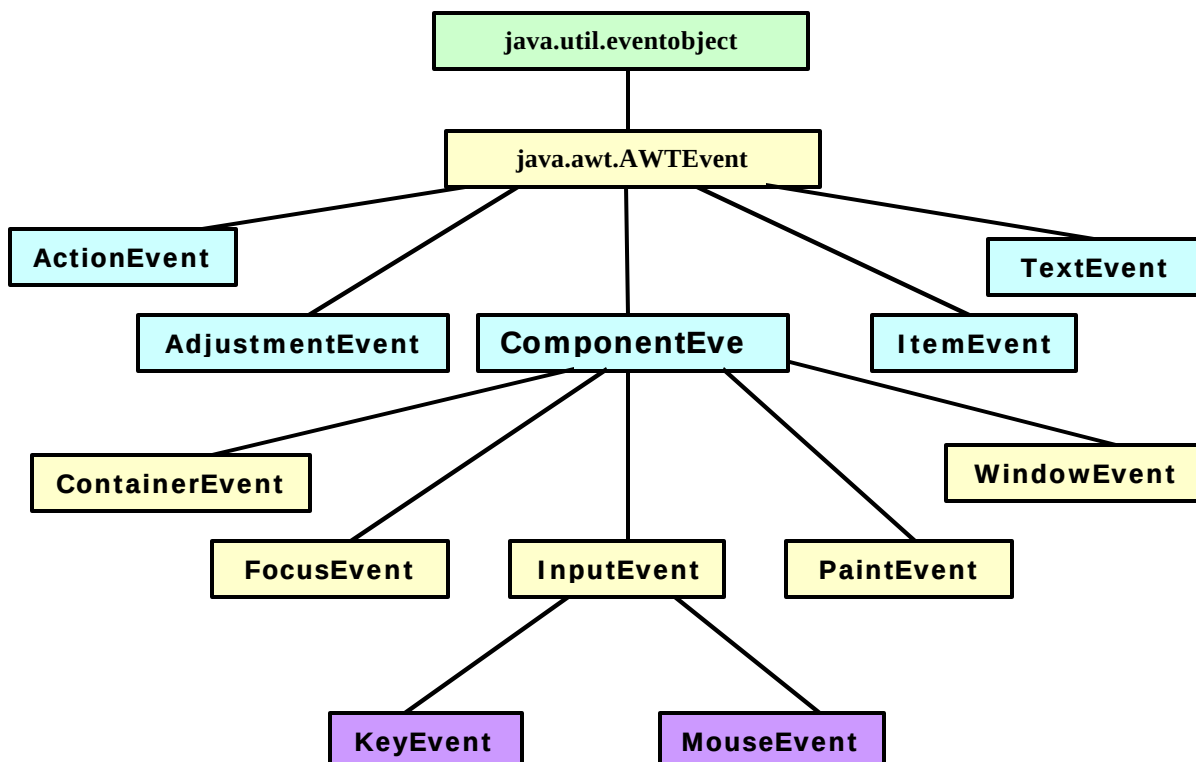
Una subclase de **EventObject** es **java.awt.AWTEvent** que es la superclase de todos los eventos clases del modelo de delegación. De nuevo, hay sólo un método de interés:

int getID (): Retorna el ID del evento

El ID de un evento es un entero que especifica la naturaleza exacta del evento. Por ejemplo, una instancia de la clase **MouseEvent** puede representar una de siete ocurrencias: un click, un arrastre, una entrada, una salida, un movimiento, un seleccionamiento, o un deseleccionamiento.

FIGURA 10.1

JERARQUÍA DE LOS EVENTOS CLASE



Todas las clases hacen parte del paquete **java.awt.event** a menos que se defina lo contrario

Cada uno de estas posibilidades es representado por un int:

MouseEvent.MOUSE_CLICKED, **MouseEvent.MOUSE_DRAGGED**, y así sucesivamente.

Las subclases de **java.awt.AWTEvent** representan la variedad de tipos de eventos que pueden ser generados por los diferentes componentes de **AWT**. Éstos tipos de evento son:

ActionEvent: generado por activación de componentes

AdjustmentEvent: generado por ajuste de componentes ajustables como barras de desplazamiento

ContainerEvent: generado cuando los componentes se agregan o se quitan de un recipiente

FocusEvent: generado cuando un componente entra o sale del foco

ItemEvent: generado cuando un artículo se selecciona de una lista, opción, o caja de chequeo

KeyEvent: generado por actividad del teclado

MouseEvent: generado por actividad del ratón

PaintEvent: generado cuando un componente se pinta

TextEvent: generado cuando un componente del texto se modifica

WindowEvent: generado por actividad de la ventana (como iconifying o de-iconifying)

La superclase **InputEvent** tiene un **getWhen()** método que devuelve el tiempo cuando el evento tuvo lugar; el tipo del retorno es largo. La clase de **MouseEvent** tiene **getX ()** y **getY ()** métodos que devuelven la posición del ratón dentro del componente originando en el momento el evento tuvieron lugar; los tipos del retorno son ambos int.

Hay dos maneras de manejar los eventos listadas previamente. La primera manera es delegar el manejo de evento a un objeto oyente. La segunda manera es explícitamente habilitar al componente origen a manejar sus propios eventos. Estas dos estrategias se discuten en las próximas dos secciones.

EVENTOS OYENTE

Un Evento Oyente es un objeto al que un componente ha delegado la tarea de manejar un tipo particular de evento. Cuando el componente experimenta entrada, un evento del tipo apropiado se construye; el evento se pasa entonces como parámetro a una llamada del método en el oyente. Un oyente debe llevar a cabo la interface que contiene el método de manejo del evento.

Por ejemplo, considere un botón en un applet. Cuando se pulsa el botón, un evento de acción será enviado a una instancia de la clase **MyActionListener**. El código para **MyActionListener** es como sigue:

```
1. class MyActionListener implements ActionListener {
2. public void actionPerformed (ActionEvent ae) {
3. System.out.println ("la Acción se realizó.");
4.}
5.}
```

La clase implementa la interface ActionListener, garantizando así la presencia de un método **actionPerformed()**. El código para el applet es algo como esto:

```
1. public class ListenerTest extends Applet {
2. public void init() {
3. Button btn = new Button("OK");
4. MyActionListener listener = new MyActionListener();
5. btn.addActionListener(listener);
6. add(btn);
7.}
8.}
```

En la línea 4, se crea una instancia de MyActionListener. En la línea 5, esta instancia es definida como uno de los oyentes de acción del botón. El código sigue una fórmula estándar para dar una acción de oyente al componente; la fórmula puede resumirse como sigue:

1. cree una clase oyente que implemente la interface ActionListener.
2. construya el componente.
3. construya una instancia de la clase oyente.
4. En el componente haga un llamado a addActionListener (), pasando el objeto oyente.

En total, hay 11 tipos de oyentes, cada uno representado por una interface. La Tabla 10.1 lista las interfaces oyentes, junto con los métodos de la interface y los métodos del addXXXListener().

Tabla 10.1

INTERFACES OYENTE

Interface	Metodos de la Interface	Adicionar Metodo
ActionListener	actionPerformed((ActionEvent))	addActionListener()
AdjustmentListener	adjustmentValueChanged(AdjustmentEvent)	addAdjustmentListener()
ComponentListener	componentHidden(ComponentEvent)	addComponentListener()
	componentMoved(ComponentEvent)	

	componentResized(ComponentEvent)	
	componentShown(ComponentEvent)	
ContainerListener	componentAdded(ContainerEvent)	addContainerListener()
	componentRemoved(ContainerEvent)	
FocusListener	focusGained(FocusEvent)	addFocusListener()
	focusLost(FocusEvent)	
ItemListener	itemStateChanged(ItemEvent)	addItemListener()
KeyListener	keyPressed(KeyEvent)	addKeyListener()
	keyReleased(KeyEvent)	
	keyTyped(KeyEvent)	
MouseListener	mouseClicked(MouseEvent)	addMouseListener()
	mouseEntered(MouseEvent)	
	mouseExited(MouseEvent)	
	mousePressed(MouseEvent)	
	mouseReleased(MouseEvent)	
MouseMotionListener	mouseMoved(MouseEvent)	addMouseMotionListener()
	mouseDragged(MouseEvent)	
TextListener	textValueChanged(TextEvent)	addTextListener()
WindowListener	windowActivated(WindowEvent)	addWindowListener()
	windowClosed(WindowEvent)	
	windowClosing(WindowEvent)	
	windowDeactivated(WindowEvent)	
	windowDeiconified(WindowEvent)	
	windowIconified(WindowEvent)	
	windowOpened(WindowEvent)	

Un componente puede tener múltiples oyentes para un solo tipo de evento. Nada garantiza que se notificarán los oyentes en el orden en el que ellos fueron agregados. Ni que toda la notificación del oyente ocurrirá en el mismo hilo; así los oyentes deben tomar precauciones para no adulterar datos compartidos.

Un evento oyente puede ser removido de la lista de oyentes de un componente con un llamado al método `removeXXXListener()`, pasando como parámetro el oyente a ser quitado. Por ejemplo, el código siguiente quita al oyente de acción **al** del **Button btn**:

btn.removeActionListener (al);

Las técnicas descritas en esta sección representan la manera normal de manejar eventos en el modelo de delegación. La delegación de eventos es suficiente en la mayoría de las situaciones; sin embargo, hay veces que para un componente es preferible manejar sus propios eventos, en lugar de delegarlos a los oyentes.

La próxima sección describe cómo hacer que un componente maneje sus propios eventos.

ACTIVACIÓN EXPLÍCITA DEL EVENTO

Hay una alternativa para delegar los eventos de un componente. Es posible dividir en subclases el componente, sobrecargar el método que recibe eventos y dárselos a los oyentes. Por ejemplo, componentes que originan eventos de acción tienen un método

llamado *processActionEvent* (**ActionEvent**) que despacha las activaciones de los eventos al oyente de cada acción.

El siguiente código implementa una subclase de Button que sobrecarga a **processActionEvent()**:

```
1. class MyBtn extiende Button {
2. MyBtn public (String label) {
3.     super(label);
4.     enableEvents (AWTEvent.ACTION_EVENT_MASK);
5. }
6.
7. public void processActionEvent (ActionEvent ae) {
8. System.out.println("Procesando un evento de acción.");
9. super.processActionEvent (ae);
10. }
11. }
```

En la línea 4, el constructor llama a **enableEvents()**, pasando una constante que habilita el procesamiento de eventos acción. La clase **AWTEvent** define 11 constantes que pueden usarse para habilitar el procesamiento de eventos; estas constantes se listan en la Tabla 10.2. (el procesamiento del evento se habilita automáticamente cuando se agregan oyentes de evento, así si usted se restringe al modelo del oyente, usted nunca tiene que llamar **enableEvents ()**).

La línea 7 es el principio de la versión de subclases del método **processActionEvent()**. Note la llamada en línea 9 a la versión del superclass'. Esta llamada es necesaria porque la versión del superclass' es responsable por llamar a **actionPerformed()** en los oyentes de acción del botón; sin la línea 9, se ignorarían los oyentes de acción.

Por supuesto, usted siempre puede hacer un subclass del componente que maneje sus propios eventos haciendo un evento oyente de sí mismo, como el siguiente código:

```
1. class MyBtn extends Button implements ActionListener {
2.     public MyBtn( String label ) {
3.         super( label );
4.         addActionListener( this );
5.     }
6.
7.     public void actionPerformed( ActionEvent ae ) {
8.         // handle the event here
9.     }
10. }
```

La única diferencia entre esta estrategia y la estrategia **enableEvents()** es el orden en el cual se invocan los eventos manejadores. Cuando usted llama **enableEvents()** explícitamente, el método **processActionEvent()** del componente será llamado antes que cualquier oyente de acción sea notificado. Cuando el subclass del componente es su propio evento oyente, no hay ninguna garantía acerca del orden de notificación.

Cada uno de los 11 tipos de oyente tiene definida una constante **XXX_EVENT_MASK** correspondiente en la clase **AWTEvent**, y el correspondiente método **processXXXEvent()**. La Tabla 10.2 lista las constantes de la máscara y los métodos del proceso.

Tabla 10.2

MÁSCARAS DE EVENTO

Mascara	Método
AWTEvent.ACTION_EVENT_MASK	processActionEvent()
AWTEvent.ADJUSTMENT_EVENT_MASK	processAdjustmentEvent()
AWTEvent.COMPONENT_EVENT_MASK	processComponentEvent()
AWTEvent.CONTAINER_EVENT_MASK	AprocessContainerEvent()
AWTEvent.FOCUS_EVENT_MASK	processFocusEvent()
AWTEvent.ITEM_EVENT_MASK	processItemEvent()
AWTEvent.KEY_EVENT_MASK	processKeyEvent()
AWTEvent.MOUSE_EVENT_MASK	processMouseEvent()
AWTEvent.MOUSE_MOTION_EVENT_MASK	processMouseMotionEvent()
AWTEvent.TEXT_EVENT_MASK AWT	processTextEvent()
Event.WINDOW_EVENT_MASK	processWindowEvent()

La estrategia de habilitar explícitamente eventos para un componente puede resumirse como sigue:

1. cree un subclass del componente.
- 2., Llame a **enableEvents(AWTEvent.XXXEVENT_MASK)**. en el constructor del subclass
3. proporcione la subclass con un método **processXXXEvent()**; este método debe llamar la versión del superclass' antes de retornar.

ADAPTADORES

Si usted mira la Tabla 10.1 que lista los métodos de las 11 interfaces oyentes de eventos, verá que algunas de las interfaces tienen un solo método, mientras otros tienen varios métodos. La interface más grande, **WindowListener**, tiene siete métodos.

Suponga que usted quiere coger eventos iconified en un **frame**. Usted podría intentar crear la siguiente clase:

1. **class** MylkeListener **extends** WindowAdapter {
2. **public void** windowIconified(**WindowEvent** we) {
3. // process the event
4. }
5. }

Desgraciadamente, esta clase no compilará. La interface **WindowListener** define siete métodos, y la clase `MylkeListener` necesita implementar los otros seis para satisfacer al compilador.

Podría pensarse en declarar los métodos restantes dándoles cuerpos vacíos, pero es tedioso. El paquete **java.awt.event** proporciona siete clases adaptadoras, una para cada interface oyente que define varios métodos.

Un adaptador simplemente es una clase que implementa una interface que proporciona métodos que no hacen nada. Por ejemplo, las clases **WindowAdapter** implementa la interface **WindowListener** con siete métodos que no hacen nada.

Nuestro ejemplo puede modificarse para sacar provecho de este adaptador:

1. la clase `MylkeListener` extiende `WindowAdapter` {
2. el `windowIconified` nulo público (`WindowEvent` nosotros) {
3. // el proceso el evento
- 4.}
- 5.}

La Tabla 10.3 lista todas las clases adaptador, junto con las interfaces del oyente que ellos implementan.

Tabla 10.3

ADAPTADORES

Adapter Class	Listener Interface
<code>ComponentAdapter</code>	<code>ComponentListener</code>
<code>ContainerAdapter</code>	<code>ContainerListener</code>
<code>FocusAdapter</code>	<code>FocusListener</code>
<code>KeyAdapter</code>	<code>KeyListener</code>
<code>MouseAdapter</code>	<code>MouseListener</code>
<code>MouseMotionAdapter</code>	<code>MouseMotionListener</code>
<code>WindowAdapter</code>	<code>WindowListener</code>

RESUMEN DEL CAPÍTULO

El modelo de delegación de eventos le permite designar cualquier objeto como un oyente para los eventos de un componente. Un componente puede tener múltiples oyentes para cualquier tipo de evento. Todos los oyentes deben implementar la interface apropiada. Si la interface define más de un método, el oyente puede extender la clase del adaptador apropiada.

Una subclase del componente puede manejar sus propios eventos llamando **enableEvents()**, pasando una máscara de evento. Con esta estrategia, se llama un método **processXXXEvent()** antes que algún oyente sea notificado.

Capítulo 11: Componentes

Objetivos

Los componentes son bloques de construcción para crear interfaces gráficas de usuario. Algunos tipos de componentes, como los botones y las barras de desplazamiento son usados para el control del IGU (GUI en inglés, Interfaz Gráfica de Usuario). Otras clases de componentes (aquellos que heredan desde la clase abstracta `Container`) proveen organización espacial. IGUs son una importante parte de cualquier programa. Las herramientas de ventanas de JAVA (AWT) proveen extensiva funcionalidad. En este capítulo se revisarán y se mencionarán las más importantes.

Introducción

Los componentes en JAVA son implementados por varias subclases de las super-clases `java.awt.Component` y `java.awt.MenuComponent`. Hay 19 clases de componentes y la manera de organizarlos es la siguiente:

- Componentes visuales: Hay 11
- Componentes de contenedores: Hay 4
- Componentes de menú: Hay 4

Hay varios métodos implementados por todos los componentes visuales y contenedores, por herencia de **`java.awt.Component`** (los componentes de menú extiende de **`java.awt.MenuComponent`** por lo que no hereda la misma funcionalidad de la superclase).

Estos métodos:

`getSize()`: Retorna el tamaño del componente. El retorno es de tipo `Dimension`, el cual tiene los miembros públicos `height` (alto), y `width` (ancho).

`setForeground()` y `setBackground()`: Pone el color principal y el color de fondo, respectivamente, de un componente. Cada método toma un solo argumento, el cual es una instancia de la clase **`java.awt.Color`**. Generalmente el color de foreground de una componente es usada para pintar texto, el color de background es usado para pintar el área no-textual de un componente. Por ejemplo, una etiqueta (label) con un color principal azul y un color de fondo negro, mostrara el texto azul sobre un fondo negro. Si no se especifica los colores principal y de fondo, el componente los toma de su contenedor inmediato.

`setFont()` este método determina la fuente que el componente usará para renderizar cualquier texto que necesita mostrar. El método toma un solo argumento, el cual es una instancia de la clase `java.awt.Font`. Si no se especifica la fuente del componente, este tomara la de su contenedor inmediato. Por ejemplo, si usted tiene un applet cuya fuente es "bold" "Serif" de 48 puntos, y usted agrega un checkbox al applet sin llamar a `setFont()` en el checkbox, este tomara como etiquetas con fuentes "bold" "Serif" de 48 puntos.

`setEnabled()` Este método toma un solo argumento de tipo booleano, si este es "true" (verdadero), entonces el componente tiene su apariencia normal. Si el argumento es "false" (falso) entonces el componente se opaca y no responde a las entradas del usuario. Este método reemplaza a `enable()` y `disable()` los cuales ya han sido obsoletos.

`setSize()` y `setBounds()` Estos métodos especifican la geometría del componente, o tratan de hacerlo. Reemplazan a los métodos 1.0 `resize()` y `reshape()`. El método `setSize()` toma dos argumentos: `width` y `height` (ancho y alto); una forma sobrecargada toma una sola dimensión. El método `setBounds()` toma cuatro argumentos: `x`, `y`, `width` y `height`; una forma sobrecargada toma un solo rectángulo. Si

usted ha tratado de llamar a estos métodos, sabe que usualmente son inútiles. En el capítulo 9 se explica que el tamaño y la posición que usted intenta darle a un componente esta dominada por el manejador de esquemas (layout manager). De hecho, estos componentes existen mas que todo para el uso de los manejadores de esquemas. La excepción mayor a esta regla es la clase Frame, la cual no esta dentro de ningun esquema y puede ser perfectamente especificada en su tamaño y posición. Esto es explicado luego en la sección Frame de este capítulo.

setVisible() Este método toma un argumento booleano que determina si este componente se verá en la pantalla. Este es otro método que solo trabaja con Frame, a menos que usted aprenda otras técnicas que van más allá de este curso de Certificación. De nuevo, este método es explicado luego en la sección Frame.

Componentes Visuales

Los componentes Visuales Los componentes visuales son los únicos con los que el usuario puede interactuar, Los once componentes visuales son:

- Button
- Canvas
- Checkbox
- Choice
- FileDialog
- Label
- List
- ScrollPane
- Scrollbar
- TextArea
- TextField

Para usar uno de estos componentes en una IGU, usted primero debe crear una instancia llamando al constructor apropiado, luego usted incluye el componente en el contenedor. Añadir un componente a un contenedor no es tan trivial, de hecho este tópico fue cubierto en el capítulo 9. Para efectos de este capítulo, usted deberá suponer que los componentes que usted vea en este capítulo ya están contenidos en algún contenedor de manera apropiada, o dentro de un applet. No todas las formas de constructor estan dadas, la intención aquí no es de proveerlo a usted de todas las formas dadas con una lista exhaustiva (para ello busque las paginas de la API de Java), sino que se expondrán las necesarias para el examen de certificación. Las componentes en los applets mostrados en esta página pueden variar de maquina a maquina.

Button

La clase Button, por supuesto, implementa un botón para empujar. El botón mostrado en el applet tiene el siguiente código de construcción.

```
new Button( "Este es un boton" );
```

El constructor toma como parámetro el texto que lleva como etiqueta. Cuando el botón es pulsado, este envía un evento Action. Eventos Action y el modelo total de delegación de eventos se explican en detalle en el capítulo 10.

Canvas

Un canvas (lienzo) es un componente que no tiene comportamiento ni apariencia por defecto. Usted puede hacer heredar Canvas para crear regiones de dibujo de usuario, áreas de trabajo, componentes, entre otras. Los lienzos reciben eventos de entrada

desde el mouse y el teclado. Esto hace que el programados transforme estas entradas en alguna acción y apariencia importante. El tamaño predeterminado (o más propiamente, el tamaño preferido, como lo vio en el capítulo 9) de un lienzo es inútilmente pequeño. Una manera de enfrentar este problema es usar un manejador de esquemas que acomodara al canvas en su tamaño. Otra manera es llamando la función `setSize()` en el lienzo mismo; los lienzos son un caso raro donde esto todavía funciona.

El código para crear un lienzo (canvas) como el del applet es el siguiente:

1. **`Canvas canv = new Canvas();`**
2. **`canv.setBackground( Color.yellow );`**
3. **`canv.setSize( 100, 50 );`**

Los lienzos envían eventos de mouse `Mouse`, `MouseMotion` y eventos de teclado, como se explicó en el capítulo 10.

Checkbox

Un check box (caja de verificación) es un botón de dos estados: Verdadero (chequeado) y falso (no chequeado). Las dos formas básicas del constructor `Checkbox` son las siguientes:

- `Checkbox( String label )`
- `Checkbox( String label, boolean initialState )`

Si usted no especifica un estado inicial, el que sale por defecto es `false`. Los siguientes dos métodos leen y especifican el estado de un checkbox:

- `boolean getState()`
- `void setState( boolean state )`

Los checkboxes pueden agruparse juntos en grupos con comportamiento radial: solo un miembro del grupo puede estar chequeado en el mismo instante, seleccionar un nuevo miembro del cambia el estado del miembro seleccionado previamente a falso. Algunos sistemas de ventanas (`Motif` y `NextStep`, por ejemplo) implementan grupos radio como componentes en su propio derecho. El siguiente applet muestra como `CheckboxGroup` los checkbox `Opcion 1`, `Opcion 2` y `Opcion 3`, y un `Checkbox` sencillo. En Java, la clase **`java.awt.CheckboxGroup`** no es una componente, es simplemente una clase no visible que organiza check boxes. Esto significa que Java no impone restricciones en las relaciones espaciales entre miembros de un grupo checkbox. Si usted prefiere, usted puede poner un miembro de un grupo en la esquina superior izquierda de un frame, otra en la esquina inferior derecha y otra en otro frame diferente todos juntos. Obviamente el resultado no seria muy útil. Para usar un grupo checkbox, usted debe primero crear una instancia de la clase `CheckboxGroup`, y luego pasarle la instancia como parametro al constructor checkbox. El código siguiente agrega los tres checkboxes del applet a mostrar

1. **`CheckboxGroup cbg = new CheckboxGroup();`**
2. **`add( new Checkbox( "Opcion 1", false, cbg ) );`**
3. **`add( new Checkbox( "Opcion 2", false, cbg ) );`**
4. **`add( new Checkbox( "Opcion 3", true, cbg ) );`**

Los dos métodos de la clase `CheckboxGroup` para leer y especificar el miembro seleccionado del grupo, son los siguientes:

- `Checkbox getSelectedCheckbox()`
- `void setSelectedCheckbox( Checkbox newSelection )`

Los Checkboxes envían eventos de `Item` cuando estos son seleccionados, como se explico en el capitulo 10.

Choice

Un Choice (Selección) es una lista de despliegue. Para crear un Choice, primero hay que llamar al constructor, y luego adicionar ítems a la instancia llamando a `addItem()`.

El siguiente código muestra como crear el que se muestra en el applet

1. **Choice ch1 = new Choice();**
2. **ch1.addItem("Opcion 1 del choice");**
3. **ch1.addItem("opcion 2 ");**
4. **ch1.addItem("Opcion 3");**
5. **ch1.addItem("Opcion 3");**

Los Choices como los checkboxes, envían eventos de Item cuando alguno de los ítems es seleccionado.

FileDialog

La clase `FileDialog` representa un cuadro de dialogo de abrir o salvar archivo. La apariencia de estos diálogos varia mucho de plataforma a plataforma. Un dialogo de archivo es modal, es decir que toda entrada del frame padre del dialogo es totalmente exclusiva del dialogo, tanto como éste dure mostrándose. El dialogo se remueve automáticamente cuando el usuario especifica un archivo o hace clic en el botón cancelar. El constructor más usado de `FileDialog` tiene la siguiente forma: `FileDialog(Frame parent, String title, int mode )` El padre del dialogo (parent) es el frame sobre el cual el dialogo aparecerá. El título title aparecerá en la barra de titulo del cuadro de dialogo (en algunas plataformas). El modo mode puede estar entre **`FileDialog.LOAD`** o **`FileDialog.SAVE`**. Después de que el usuario ha especificado un archivo, el nombre del archivo o su directorio pueden ser recuperados con los siguientes métodos:

- `String getFile()`
- `String getDirectory()`

En el fragmento de código siguiente, se construye un cuadro de dialogo, y se muestra sobre el frame f. Después de que el usuario ha especificado un archivo, el nombre de este archivo es recuperado y desplegado.

1. **`FileDialog fidi = new FileDialog( f, "Choose!", FileDialog.LOAD );`**
2. **`fidi.setVisible( true );`**
3. **`System.out.println( fidi.getFile() );`**

Label

El componente más simple de la AWT de Java es el Label (Etiqueta). Las etiquetas no responden a entradas de usuario y no envían ningún evento. Hay tres maneras de construir un label:

- `Label()`
- `Label( String text )`
- `Label( String text, int alignment )`

La alineación por defecto para las etiquetas es hacia la izquierda. Para determinar la alineación, se usa la tercera forma de constructor y se pasa como argumento `align` uno de los siguientes: **`Label.LEFT`** **`Label.CENTER`** **`Label.RIGHT`** Existen dos métodos para recuperar y determinar el texto de una etiqueta, respectivamente:

- `String getText()`
- `void setText( String newText )`

Si usted no usa argumentos en el constructor, usted debe especificar el texto de la etiqueta con **setText()** en algún momento. En el applet de etiqueta, se especifico la siguiente llamada:

```
Label lab = new Label("Ejemplo de label o etiqueta");
```

Lista

Una lista es una colección de Items de texto, dispuestos verticalmente. Si una lista contiene más elementos de los que puede mostrar, adquiere una barra de desplazamiento vertical. La lista tiene tres formas de constructor:

- List()
- List(int nVisibleRows)
- List(int nVisibleRows, boolean bMultiSelectOk)

El número de filas visibles nVisibleRows determina el alto de la lista. La primera versión no especifica un número de filas visibles, por lo que presumiblemente la altura de tal lista la determina su manejador de esquemas (layout manager). Si la versión del tercer constructor es usada y el argumento bMultiSelectOk es true (verdadero), entonces la lista soportará múltiple selección. Si la múltiple selección no está habilitada entonces seleccionar un nuevo ítem deselecta uno previamente seleccionado. El siguiente código muestra la construcción de la lista que se muestra en el applet.

1. **List lista = new List(5,true);**
2. **lista.addItem("Elemento 1 de la lista");**
3. **lista.addItem("elemento 2");**
4. **lista.addItem("el 3");**
5. **lista.addItem("otro que es el 4");**
6. **lista.addItem("y el ultimo");**

La lista tiene cinco ítems y los cinco son visibles, por lo que no necesita barra de desplazamiento a menos que el manejador de esquemas lo determine. La selección múltiple esta activada, por lo que permite seleccionar varios ítems al mismo tiempo. La clase List provee un gran número de métodos soporte. Una lista parcial de estos métodos aparece a continuación:

- void addItem(String text): Añade un ítem al final de la lista
- void addItem(String text, int index): Inserta un ítem en la posición especificada por index
- String getItem(int index): retorna el ítem con el index especificado
- int getItemCount(): retorna el número de ítems en la lista
- int getRows(): retorna el número de líneas visibles en la lista
- int getSelectedIndex(): retorna el índice del ítem seleccionado (la lista tiene que estar en modo de selección sencillo)
- int[] getSelectedIndexes(): retorna un arreglo con los índices de los ítems seleccionados (para modo de multiple selección)
- String getSelectedItem(): retorna una cadena que refleja el ítem selección(para modo de selección sencillo)
- String[] getSelectedItems(): retorna una arreglo de cadenas con los ítems seleccionados (para modo de múltiple selección)
- Seleccionar un ítem de una lista hace que esta envíe un evento de Item; hacer doble click en un ítem causa un evento de Action.

ScrollPane

El `ScrollPane` es una clase muy útil introducida en Java 1.1. Un scroll pane (Cuadro desplazable) puede contener un solo componente, el cual puede ser más grande y ancho que el cuadro desplazable: si esto pasa, el cuadro adquiere barras de desplazamiento verticales y/o horizontales cuando sea necesario. Hay dos constructores para esta clase:

- `ScrollPane()`: Construye un `ScrollPane` con comportamiento de barras de desplazamiento por defecto
- `ScrollPane( int scrollbarPolicy )`: Construye una `ScrollPane` con comportamiento de barras de desplazamiento especificado por `scrollbarPolicy`

Si se usa la segunda forma de constructor, entonces `scrollbarPolicy` puede usar una de las siguientes tipo de parámetros:

- `ScrollPane.SCROLLBARS_AS_NEEDED`
- `ScrollPane.SCROLLBARS_ALWAYS`
- `ScrollPane.SCROLLBARS_NEVER`

El código siguiente crea una `scrollpane` con comportamiento de barras de desplazamiento por defecto. El `scrollpane` contiene una etiqueta considerablemente grande, por lo que las barras de desplazamiento serán necesarias.

1. **`ScrollPane spane = new ScrollPane();`**
2. **`Label labelGrande = new Label("Una etiqueta grandisima dentro de Scrollpane!!");`**
3. **`labelGrande.setFont(new Font("Serif",Font.ITALIC,80));`**
4. **`spane.add(labelGrande);`**
5. **`add(spane);`**

Los `scrollpanes` envían mensajes de eventos de `Mouse` y `MouseMotion`

Scrollbar

El componente `scrollbar` (barra de desplazamiento), que ajusta listas y `scrollpanes`, está disponible como un componente mismo. Estos son sus tres constructores:

- `Scrollbar()`: Construye una barra de desplazamiento vertical
- `Scrollbar( int orientation )`: construye una barra de desplazamiento con la orientación especificada
- `Scrollbar( int orientation, int initialValue, int sliderSize, int minValue, int maxValue )`: construye una barra de desplazamiento con los parametros especificados

Para constructores que toman un parámetro de orientación `orientation`, el valor debe ser uno de los siguientes:

- `Scrollbar.HORIZONTAL`
- `Scrollbar.VERTICAL`

En la tercera forma de constructor, el parámetro `sliderSize` es un poco confuso. En la terminología de Java, el pedazo de `scrollbar` que se desliza es el deslizador o `slider`. El parámetro `sliderSize` controla el tamaño del deslizador pero no en unidades de pixel, sino definidas por el proporcionado entre el máximo y el mínimo valor de la `scrollbar`. Por ejemplo considere una barra de desplazamiento horizontal cuyo valor mínimo es 600 y máximo valor es 700. El espacio que cubre esta barra es la diferencia entre estos dos números, o 100. Un tamaño de 50 para el deslizador representa la mitad de esta distancia y sería la mitad del ancho para esta barra. Un tamaño de 10 representaría una décima parte de este tamaño y así un tamaño de un décimo del ancho de la barra.. El siguiente código crea una barra horizontal con un rango de 600

a 700. El valor inicial es de 625. El deslizador tiene un tamaño de 25, o sea un cuarto del ancho de la barra.

Scrollbar sbar = new Scrollbar(Scrollbar.HORIZONTAL, 625, 25, 600, 700);

La scrollbar genera eventos de ajuste Adjustment, explicados en el capítulo 10

TextField y TextArea

Las clases TextField y TextArea implementan componentes de una y dos dimensiones para la entrada, visualización y edición de texto. Ambas clases extienden a la superclase TextComponent como se muestra en la figura. Ambas clases tienen varios constructores, los cuales ofrecen la opción de especificar o no una cadena inicial o un tamaño. Los constructores que no especifican tamaño, son usados para trabajar con Layout Managers que le asignan un tamaño. Los constructores para TextField se listan abajo:

- **TextField():** Construye un campo de texto vacío
- **TextField(int nCols):** Construye un campo de texto vacío con el número de columnas especificado
- **TextField(String text):** Construye un campo de texto cuyo contenido inicial es text
- **TextField(String text, int nCols):** Construye un campo de texto cuyo contenido inicial es text con el número de columnas especificado

Los constructores para TextArea se listan abajo:

- **TextArea():** Construye un área de texto vacío
- **TextArea(int nRows, int nCols):** Construye un área de texto vacío con el número de filas y columnas especificado
- **TextArea(String text):** Construye un área de texto cuyo contenido inicial es text
- **TextArea(String text, int nRows, int nCols):** Construye un área de texto cuyo contenido inicial es text, con el número de filas y columnas especificadas
- **TextArea(String text, int nRows, int nCols, int scrollbarPolicy):** Lo mismo que el anterior, pero las políticas de las barras de desplazamiento son determinadas por el último parámetro el cual puede ser uno de los siguientes:
 - **TextArea.SCROLLBARS_BOTH**
 - **TextArea.SCROLLBARS_NONE**
 - **TextArea.SCROLLBARS_HORIZONTAL_ONLY**
 - **TextArea.SCROLLBARS_VERTICAL_ONLY**

Para ambas clases, hay algunos problemas sorprendentes al número de parámetro de las columnas. Primero el número de columnas es una medida de ancho en términos de columnas de texto, como dado en una fuente particular. Un área de texto de 25 columnas con una fuente diminuta será muy estrecha, mientras que un área de texto de 5 columnas con una fuente grande será sumamente ancha. Hay el problema de fuentes proporcionales luego. Para una fuente de anchura fija, es obvio cuál debe ser la anchura de la columna. Para una fuente proporcional, la anchura de la columna se toma para ser el promedio de las anchuras del carácter de la fuente. Un problema final es la pregunta de lo que pasa cuando un usuario teclea más allá del carácter más a la derecha de la columna en uno de estos componentes. En ambos casos, el texto visible desplaza a la izquierda. El punto de inserción permanece en el lugar, en la columna de más a la derecha. El componente contiene más texto ahora que puede desplegar, entonces el desplazamiento se requiere. Las áreas de texto soportan las barras de desplazamiento. Los campos de texto pueden ser desplazados usando las teclas <--y -

->. Ambas clases heredan alguna funcionalidad de su superclase `TextComponent`.

Estos métodos incluyen :

- **`String getSelectedText()`**: Devuelve el texto seleccionado actualmente
- **`String getText()`**: Devuelve el texto contenido en el componente
- **`void setEditable(boolean editable)`**: Si `editable` es verdadero, no permite al usuario editar el contenido del campo
- **`void setText(String text)`**: Coloca la cadena `text` en el componente.

Una experiencia comun para los programadores que comienzan AWT es por ejemplo que necesitan adquirir el contenido de un campo de texto. no hay ningun nombre prometedor entre los métodos listados en la página del API para `TextField`. No hay nada prometedor que se encuentre alli, y de repente los campos de texto parecen inútiles. El problema, claro, es la herencia: Los métodos deseados están disponibles, pero ellos son heredados de `TextComponent` y se documentaron en una página diferente. Si usted sabe que una clase debe llevar a cabo un cierto método (porque usted ha oído que lo hace, o porque usted se recuerda usando el método hace tiempo, o porque la clase sería por otra parte inútil si no lo hiciera) no se rinda si usted no encuentra lo que usted quiere en la lista de métodos de la clase. Mire en la sección titulado " Métodos heredados de la clase XXX" (o si usted está usando las paginas de API 1.1 , siga los eslabones de jerarquía de las superclases que se acercan a la cima de la página). El código debajo crea tres campos del texto. Cada uno es de cinco columnas de ancho, pero todos ellos usan fuentes diferentes. (Las fuentes son todas de 24 puntos, para que las diferencias seán sutiles).

```
1. TextField tf1 = new TextField(5);
2. tf1.setFont(new Font("Serif", Font.PLAIN, 24));
3. tf1.setText("12345");
4. TextField tf2 = new TextField(5);
5. tf2.setFont(new Font("SansSerif", Font.PLAIN, 24));
6. tf2.setText("12343");
7. TextField tf3 = new TextField(5);
8. tf3.setFont(new Font("Monospaced", Font.PLAIN, 24));
9. tf3.setText("12345");
```

Sorprendentemente, sólo cuatro caracteres aparecen en cada campo (aunque el punto cerca del derecho del primer campo se parece la cola truncada de los " 5" sospechosamente) Éste no es un bug. Los campos realmente son de cinco columnas de ancho, pero algunos espacios son tomados por los espacios en blanco entre caracteres. El código de abajo implementa tres áreas de texto, cada una con seis filas y cinco columnas. De nuevo, cada componente tiene una familia diferente de fuentes de 24 puntos. (El nombre de la fuente aparece en la primera fila de cada componente.) Las primeras dos fuentes son proporcionales, tanto que muchas más `i`s` que `w`s` pueden caber en una fila. De nuevo, los componentes realmente tienen cinco columnas, pero el espacio en blanco reduce el número de caracteres visibles.

```
1. TextArea ta1 = new TextArea(6, 5);
2. ta1.setFont(new Font("Serif" , Font.PLAIN, 24));
3. ta1.setText("Serif \n 12345\n abcde \n iiiiiiiii \n WWWWW");
4. TextArea ta2 = new TextArea(6, 5);
5. ta2.setFont(new Font("SansSerif" , Font.PLAIN, 24));
6. ta2.setText("Sans \n 12345 \n abcde \n iiiiiiiii \n WWWWW");
7. TextArea ta3 = new TextArea(6, 5);
8. ta3.setFont(new Font("Monospaced", Font.PLAIN, 24));
```

9. **ta3.setText("Mono \n 12345 \n abcde \n iiiiiii \n WWWWW");**

Los campos de texto y las áreas de texto generan TextEvent y KeyEvent.

Adicionalmente, los campos del texto generan los eventos de Acción en el recibo de un Entre en la pulsación.

Componentes Contenedores

Las cuatro no-superclases contenedoras componentes clases son:

- Applet
- Frame
- Panel
- Dialog

Técnicamente Scrollpane también es un recipiente, porque hereda de la superclase del Recipiente, pero no presenta las características que los otros tres tienen. La figura muestra la jerarquía de herencia de estas clases. Los recipientes son los componentes capaces de sostener otros componentes dentro de sus límites. Cada pantalla disparada hasta ahora en este capítulo ha mostrado una acción como un recipiente para el componente a ilustrarse. Los Componentes agregados a los Recipientes requieren actuar recíprocamente con administradores de diseño; este tema entero se cubrió a fondo en el Capítulo 9. Las próximas cuatro secciones son breves.

Applet

El único problema que necesita la atención aquí es el problema de redimensionar.

Applets, en virtud de heredar del Componente, tienen los métodos setSize () y setBounds (). Los Applets sólo existen en los navegadores. Cambiar el tamaño de un applet es permitido o prohibido por el navegador del applet, y durante el ciclo de desarrollo usted no puede saber qué marca de navegador estará ejecutando su applet. El navegador más fácil para el desarrollo es el visor de applet que permite redimensionar los applets. Es común para un applet tener un setSize() temporal llamado por su método init (), porque esto proporciona una manera fácil de jugar con los tamaños diferentes. Si usted usa esta técnica, recuerde anular el setSize() llamado antes de la última entrega y ponga el tamaño en su etiqueta del HTML.

Frame

Un Frame es una ventana independiente, decorada por el sistema de la ventana subyacente y capaz de ser movido alrededor en la pantalla independiente de otras ventanas del GUI. Cualquier aplicación que requiere un GUI debe usar uno o más Frames para contener los componentes deseados. Hay sólo dos formas del constructor del Frame:

- **Frame():** Construye un frame con una barra de título vacía
- **Frame(String title):** Construye un frame con el título especificado

Cuando un Frame se construye, no tiene ningún tamaño y no se despliega en la pantalla. Para dar un tamaño a un Frame, llame uno de los métodos setSize() o setBounds () heredados. (Si usted llama el setBounds (), los (x ,y) parámetros dicen el Frame dónde aparecerá en la pantalla.) Una vez un Frame se ha dado un tamaño, usted puede desplegarlo llamando el método setVisible(true). Para quitar un Frame no deseado de la pantalla, usted puede llamar el método setVisible(false). Esto no destruye el Frame o lo daña de forma alguna; usted siempre puede desplegarlo de nuevo llamando el setVisible(true). Cuando usted quiere acabar con un Frame, usted necesita reciclar sus recursos de memoria. (La memoria se segará por el basurero.) los recursos de memoria son dependientes del sistema; basta lo para decir que toma

mucho para conectar un GUI de Java a un sistema de la Ventana subyacente. Por ejemplo, en una plataforma de UNIX/Motif los recursos de memoria de un Frame incluirían descriptores del archivo y ventana de X por lo menos. Para soltar los recursos de memoria de un Frame, simplemente llame el metodo dispose(). El código abajo construye y muestra un Frame de 500 x 350 ; 30 segundos después el Frame está alejado de la pantalla y eliminado de memoria.

```
1. // Construct and display
2. Frame f=new Frame('This is a frame);
3. f.setBounds(10, 10, 500, 350);
4. f.setVisible(true);
5. // Delay
6. try { Thread.sleep(30*1000); }
7. catch (InterruptedException e) { }
8. // Remove and dispose
9. f.setVisible(false);
10.f.dispose();
```

CAUTION: Si un applet intenta desplegar un Frame, el navegador del applet confiere con el gerente de seguridad local. La mayoría de los navegadores tiene gerentes de seguridad que imponen una restricción en los Frames. El despliegue del Frame se permite, pero el Frame contiene una etiqueta que marca el Frame como "no confiable" inesperadamente. La razón es que el Frame se podría haber desplegado por un applet que estaba cargado de la Internet, para que cualquier información delicada entre en los componentes del Frame posiblemente podría transmitirse a las fiestas de fibra moral dudosa.

Panel

Los Applet y los Frames sirven como los componentes del GUI de alto nivel o extremos. El Panel mantiene un nivel intermedio de organización espacial en los GUIs. Usted es libre agregar todos los componentes de un GUI directamente en un applet o un Frame, pero usted puede proporcionar niveles adicionales de agrupamiento agregando los componentes a los Panels y agregando los Panels a un applet o Frame. Este proceso es recursivo: Los componentes que usted agrega a los Panels se encapsulan así sea en Panels, y así sucesivamente al nivel de profundidad que usted quiera. Para lograr que los componentes vayan exactamente donde usted los quiere dentro de un Panel, es el asunto del Capítulo 9 mencionado.

Dialog

Un diálogo es una ventana automática que acepta la entrada del usuario. Pueden hacerse los diálogos opcionalmente de forma manual. La clase dialog es la superclase de la clase de FileDialog. El gerente del diseño predefinido para esta clase es el diseño fronterizo.

Componentes de Menu

Java apoya dos tipos de menú: desplegable y automático. El examen de Certificación no cubre los menus automaticos . Se acceden a los menús desplegables vía una barra de menú que puede contener multiples menus. Las barras de menú sólo pueden aparecer en los Frame. (Por consiguiente los menús desplegables también pueden aparecer sólo en los Frames) Para crear un Frame con una barra de menú que contiene un menú desplegable. usted necesita pasar por lo siguiente pasos:

1. Cree una barra de menú y átelo al Frame.
2. Cree y pueble el menú.
3. Ate el menú a la barra de menú.

Para crear una barra de menu, simplemente cree una instancia de la clase `MenuBar`. Para atarlo a un Frame, páselo como parametro en el metodo `setMenuBar()` del Frame. Para crear un menú, simplemente construya una instancia de la clase `Menu`. El constructor más común toma una cadena que es la etiqueta del menú; esta etiqueta aparece en la barra de menú. Hay cuatro tipos de elemento que puede mezclarse y puede emparejarse para poblar un menú:

- **Menu items**
- **Check-box menu items**
- **Separators**
- **Menus**

MenuItem(String text)

donde el texto es la etiqueta del artículo del menú. Un elemento del menu es como un boton que pasa a vivir en un menu Como los botones un menu genera `ActionEvents`. Un `CheckboxMenuItem` se parece un elemneto del menú con una casilla de verificación a la izquierda de su etiqueta. Cuando un `CheckboxMenuItem` es seleccionado, la casilla de verificación cambia su estado. El constructor básico para la clase de `CheckboxMenuItem` es

CheckboxMenuItem(String text)

donde el texto es la etiqueta del artículo. Un `CheckboxMenuItem` es como una casilla de verificación que pasa para vivir en un menú; usted puede leer y puede poner el estado de un artículo llamando los metodos `getState()` y `setState()` así como usted haría con una casilla de verificación llana. Los `CheckboxMenuItem` generan `ItemEvent`.

Un separador es simplemente una marca horizontal usada para dividir un menú visualmente en secciones. Para agregar un separador a un menú, llame el metodo `addSeparator()` del menú.

Cuando usted agrega un menú a otro menú, la etiqueta del primer menú aparece en el segundo menú, con un icono del despliegue derecho. Desplegando el ratón a la derecha causa que el menu subalterno aparezca.

Después de que un menú se puebla totalmente, usted lo ata a una barra de menú llamando el metodo `add()` de la barra de menú. Si usted quiere que el menú aparezca en la posición de menú de Ayuda a la derecha de otros menús, llame el metodo `setHelpMenu()` en vez de `add()`.

El código debajo crea y despliega un Frame con una barra de menú y dos menús. El primer menú contiene uno de cada clase (`menuItem`, `checkboxmenuItem`, `separador`, y `submenú`). El segundo menú es un menú de Ayuda y simplemente contiene dos artículos de menú.

1. **Frame frame;**
2. **MenuBar bar;**
3. **Menu fileMenu, subMenu, helpMenu;**
4. **// Create frame and install menu bar.**
5. **frame = new Frame("Menu demo");**
6. **frame.setSize(400, 300);**
7. **bar = new MenuBar();**
8. **frame.setMenuBar(bar);**
9. **.**

```

10.// Create submenu.
11.subMenu = new Menu("Pull me");
12.subMenu.add(new MenuItem("Sub-This"));
13.subMenu.add(new MenuItem("Sub-That"));
14.// Create and add file menu.
15.fileMenu = new Menu("File");
16.fileMenu.add(new MenuItem("New"));
17.fileMenu.add(new MenuItem("Open"));
18.fileMenu.addSeparator();
19.fileMenu.add(new CheckboxMenuItem("Print Preview Mode"));
20.fileMenu.add(subMenu);
21.bar.add(fileMenu);
22.// Create help menu.
23.helpMenu = new Menu("Help");
24.helpMenu.add(new MenuItem("Contents . . ."));
25.helpMenu.add(new MenuItem("About this program .. "));
26.bar.setHelpMenu(helpMenu);
27.// Now that the frame is completely built, display it.
28.frame.setVisible(true);

```

La figura muestra el Frame con el menú del Archivo y el submenú visible.

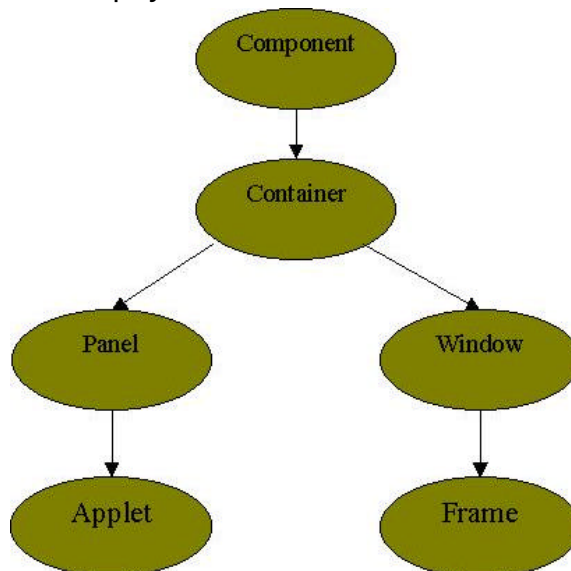
La figura muestra el Frame con el menú de ayuda visible

Resumen del Capitulo

Este capítulo ha introducido tres categorías de componentes:

- **Los componentes visuales**
- **Los componentes contenedores**
- **Los componentes del menú**

Los componentes visuales son los componentes con que el usuario actúa recíprocamente. Los componentes del recipiente contienen otros componentes. Los componentes del menú apoyan los menús en los Frames.



Test Yourself

1. Un TextField se construye y se da un color prioritario blanco y una fuente de 64-puntos Bold Serif. El texto se agrega entonces a un applet que tiene un color prioritario rojo, el color del fondo azul, y una fuente de 7-puntos sans-serif. ¿Cuál la declaración debajo de es verdad sobre el Texto Presente?

- A. El color prioritario es negro, el fondo es blanco, la fuente es de 64-puntos bold serif.
- B. El color prioritario es rojo, el fondo es azul, la fuente es de 64-puntos bold serif
- C. El color prioritario es rojo, el fondo es azul, la fuente es de 7-puntos bold serif.
- D. El color prioritario es blanco, el fondo es azul, la fuente es de 7-puntos bold serif.
- E. El color prioritario es blanco, el fondo es azul, la fuente es de 64-puntos bold serif.

2. Usted tiene una casilla de verificación en un Panel; el Panel está en un applet. El applet no contiene ningún otro componente. Usando setFont(), usted da una fuente de 100-puntos al applet, y usted da una fuente de 6-puntos al Panel. ¿Qué frases o declaraciones debajo son correcto?

- A. La casilla de verificación usa una fuente de 12-puntos.
- B. La casilla de verificación usa una fuente de 6-puntos.
- C. La casilla de verificación usa una fuente de 100-puntos.
- D. La casilla de verificación usa la fuente del applet, porque usted no puede poner una fuente en un Panel.
- E. La casilla de verificación usa la fuente del Panel, porque usted no puso una fuente explícitamente para la casilla de verificación.

3. Usted tiene una casilla de verificación en un Panel; el Panel está en un applet. El applet no contiene ningún otro componente. Usando setFont(), usted da una fuente de 100-puntos al applet. ¿Qué declaración o declaraciones debajo son correctas?

- A. La casilla de verificación usa una fuente del 12-punto.
- B. La casilla de verificación usa una fuente de 6-puntos.
- C. La casilla de verificación usa una fuente de 100-puntos.
- D. La casilla de verificación usa la fuente del applet.
- E. La casilla de verificación usa la fuente del Panel, porque usted no puso una fuente explícitamente para la casilla de verificación.

4. Usted quiere construir una área del texto que es 80 caracteres de ancho y 10 caracteres de alto. ¿Qué código usted usa? A. new TextArea(80, 10) B. new TextArea(10, 80) 5. Usted construye una lista llamando new List(10,

falso). ¿Qué frases o declaraciones debajo son correctas? (Asuma que los gerentes de diseño no modifican la lista de forma alguna.)

- A. La lista tiene 10 artículos.
- B. La lista soporta la selección múltiple.
- C. La lista tiene a 10 artículos visibles.
- D. La lista no soporta la selección múltiple.
- E. La lista adquirirá una barra de desplazamiento vertical si necesita.

6. Un campo del texto tiene una fuente de anchura variable. Se construye llamando new TextField ("iiii"). Qué pasa si usted cambia el contenido del texto por "WWWWW" (Tenga en mente que i es uno de los caracteres más delgados, y w es uno del más anchos.)

- A. El campo del texto se pone más ancho.
- B. El campo del texto se vuelve estrecho.
- C. El campo del texto se queda de la misma anchura; para ver los contenidos enteros usted tendrá que desplazar usando las teclas <--y--> .
- D. El campo del texto se queda la misma anchura; para ver los volúmenes enteros usted tendrá que desplazar usando la barra de desplazamiento horizontal del campo del texto.

7. ¿Cuales de los siguientes elementos puede contener un menú? (Escoge todos los que aplican)

- A. Un separador
- B. Un checkbox
- C. Un menu
- D. Un boton
- E. Un panel

8. ¿Cuales de los siguientes componentes pueden contener una barra de menú? (Escoge todos los que aplican.)

- A. Un panel
- B. Un frame
- C. Un applet
- D. Un menubar
- E. Un menu

9. Su aplicación construye un Frame llamando `Frame f = new Frame();` pero cuando usted ejecuta el código, el Frame no aparece en la pantalla. ¿Qué código hará el Frame aparecer? (Escoge uno.)

- A. `f.setSize(300, 200);`
- B. `f.setFontOnnew Font("SansSerif" , Font.BOLD, 24));`
- C. `f.setForeground(Color.white)`
- D. `f.setVisible(true);`
- E. `f.setSize(300, 200); f.setVisible(true);`

10. La clase `CheckboxGroup` es una subclase de la clase `Component`.

- A. True
- B. False

CAPITULO 12: DIBUJADO

Los mecanismos de pintado de Java proveen la vía para que usted presente los componentes.

El mecanismo es robusto y si usted lo usa correctamente puede crear código eficiente, escalable y reutilizable.

Los conceptos fundamentales de pintado son:

- ? El método paint () y el contexto gráfico
- ? El hilo GUI y el método repaint ()
- ? Pintado Espontaneo
- ? Pintando en las imágenes.

Este capítulo le llevara paso a paso de tal manera que usted entienda y aplique estos conceptos.

Método Paint y el Contexto Gráfico.

La mayoría de los programadores encuentran el método paint (), bastante pronto, en los libros

introdutorios de Java.

El applet presentado a continuación es un ejemplo sencillo de este método.

```
1. import java.applet.Applet;  
2. import java.awt.*;  
3. public class SimplePaint extends Applet {  
4.     public void paint( Graphics g ) {  
5.         g.setColor( Color.black );  
6.         g.fillRect( 0, 0, 300, 300 );  
7.         g.setColor( Color.white );  
8.         g.fillOval( 30, 30, 50, 50 );  
9.     }  
10. }
```

Un punto interesante acerca de este applet es que no hace ninguna llamada al método paint(). Este

método ya esta provisto.

El ambiente parece hacer un buen trabajo al llamar al método paint(), esto se conoce como pintado

espontaneo y es abordado más adelante en este capítulo.

El dibujado de un componente es realizado haciendo llamadas a un contexto gráfico, el cual es una

instancia de la clase Graphics.

Un contexto gráfico debe saber como presentar en otro destino.

Los tres medios que un contexto gráfico puede presentar son:

Componentes

Imágenes

Impresoras

Cualquier tipo de componente puede ser asociado con un contexto gráfico. Dicha asociación es

permanente; un contexto gráfico no puede ser reasignado a otro componente.

Aunque es posible usar un contexto gráfico para pintar cualquier componente; es inusual hacerlo con aquellos componentes que ya tienen una apariencia definida. Botones, listas, cajas de selección, etiquetas, barras de desplazamiento, cajas de texto y áreas de texto no requieren frecuentemente que el programador redefina su apariencia. Estos componentes usan la versión paint que heredan de la super clase componente.

Sin embargo hay cuatro clases de componentes que no tienen una apariencia por omisión y se ven como rectángulos vacíos a menos que una subclases de estos se les proporcione métodos paint().

Estos cuatro componentes son:

- ? Applet
- ? Canvas
- ? Frame
- ? Panel

Si usted mira la línea del código presentando anteriormente , se observa que un contexto gráfico es pasado como parámetro del método paint.

Las cuatro operaciones principales provistas por la clase Graphics son:

- Seleccionar un color
- Seleccionar una fuente
- Dibujar y rellenar.
- Recortar.

SELECCIONAR UN COLOR

Los colores son seleccionados invocando el método setColor(). El argumento es una instancia de la clase Color.

Existen 13 colores predefinidos, los cuales se pueden acceder mediante una variable estática final de la clase Color.

Los colores predefinidos son:

- ? Color.red
- ? Color.yellow
- ? Color.blue
- ? Color.green
- ? Color.orange
- ? Color.magenta
- ? Color.cyan
- ? Color.pink

? Color.lightGray
? Color.darkGray
? Color.gray
? Color.white
? Color.black

Si se desea usar un color que no esta en la lista, usted puede construir uno. Existen varias versiones del constructor de la clase Color, el más simple es:
Color (int redLevel, int greenLevel, int blueLevel)

Los tres parámetros corresponden a niveles de intensidad de los colores primarios, el rango de dicha intensidad esta entre 0 y 255. El fragmento de código presentado a continuación presenta la primera parte de un método paint() que establece el color del contexto gráfico en verde pálido.

1. public void paint(Graphics g) {
2. Color c = new Color(170, 255, 170);
3. g.setColor(c);
4. ...

Después de la línea tres todos los gráficos serán dibujados en verde pálido. hasta la siguiente llamada a g.setColor(). La instrucción g.setColor() no cambia el color de ningún objeto que ya ha sido dibujado, esto sólo afecta las operaciones siguientes.

SELECCIONAR FUENTE

Establecer la fuente para un contexto gráfico es similar a seleccionar un color para dicho contexto. Las siguientes instrucciones de dibujo de cadenas usaran la nueva fuente, mientras que las que fueron dibujadas previamente no se verán afectadas.

Antes que usted pueda fijar una fuente, se debe crear una. El constructor de la clase Font es el siguiente:

Font(String fontname, int style, int size)

El primer parámetro es el nombre de la fuente. La disponibilidad de las fuentes depende de la plataforma que se este utilizando. Usted puede pedir una lista de los nombres de las fuentes disponibles , llamando al método getFontList el cual retornara un arreglo de cadenas. A continuación se verá un ejemplo:

String fontnames[] = Toolkit.getDefaultToolkit().getFontList()

Tres nombres de fuentes están disponibles independientemente de la plataforma que se este utilizando:

? "Serif"

? "SansSerif"

? "Monospaced"

En las versiones anteriores de JDK estas eran llamadas respectivamente:

? "TimesRoman"

? "Helvetica"

? "Courier".

El parámetro style del constructor de la clase Font puede tomar uno de los siguientes tres enteros:

? Font.PLAIN

? Font.BOLD

? Font.ITALIC

El siguiente fragmento de código establece la fuente del contexto gráfico gc como sans serif con negrilla

y de un tamaño de 24 puntos.

1. Font f = new Font("SansSerif", Font.BOLD, 24);

2. gc.setFont(f);

Se puede especificar una fuente itálica y en negrilla pasando como parámetro style en el constructor

Font.BOLD + Font.ITALIC.

DIBUJADO Y RELLENADO

Todos los métodos de presentación de la clase Graphics especifican la posición en pixeles.

Cada componente tiene su propio espacio de coordenadas , donde el punto de origen del componente

es la esquina superior izquierda. X se incrementa hacia la derecha y Y se incrementa hacia abajo. Según

lo muestra la figura.

FIGURA 12.2

El sistema de coordenadas
para los componentes

Los contextos gráficos no tienen un extensivo repertorio de métodos de dibujado. (Presentaciones

sofisticadas se realizan con APIs extendidas tales como 2D, 3D y Animation.)

Los métodos que usted debe conocer son:

? drawLine()
? drawRect() and fillRect()
? drawOval() and fillOval()
? drawArc() and fillArc()
? drawPolygon() and fillPolygon()
? drawPolyline()
? drawString()

Estos métodos son explicados en detalle en las siguientes secciones de este capítulo.

drawLine()

Este método dibuja una línea desde un punto (x0, y0) a un punto (x1 , y1).
`public void drawLine( int x0, int y0, int x1, int y1 );`

La figura 12.3 muestra un sencillo applet que en su método paint() implementa la siguiente instrucción.
`g.drawLine( 20, 120, 100, 50 );`

Figura 12.3
drawLine()

drawRect() and fillRect()

Estos métodos se usan para dibujar y rellenar rectángulos respectivamente.
`public void drawRect( int x, int y, int width, int height );`
`public void fillRect( int x, int y, int width, int height );`

Los parámetros x y y son las coordenadas de la esquina superior izquierda de el rectángulo. Los parámetros width y height no corresponden a las coordenadas de la esquina opuesta, sino al ancho y alto que tendrá el rectángulo respectivo. Dichos parámetros deben ser números positivos o la figura no será dibujada. (Esto se aplica para todos los métodos del contexto gráfico que usan width y height).

La figura 12.4 muestra un applet que en su método paint implementa la siguiente instrucción:
`g.drawRect( 20, 20, 100, 80 );`

Figura 12.4
drawRect()

La figura 12.5 muestra un applet que en su método paint implementa la siguiente instrucción:

```
g.fillRect( 20, 20, 100, 80 );
```

Figura 12.5

fillRect()

drawOval() and fillOval()

Estos métodos se usan para dibujar óvalos simples y óvalos rellenos respectivamente. Un óvalo se especifica por medio de una caja rectangular. El óvalo se coloca dentro de una caja

limitante y es tangente a cada lado de la caja limitante, como lo muestra la figura 12.6

Para dibujar un

círculo se usa una caja limitante cuadrada.

Figura 12.6

Caja limitante

de un óvalo

Los dos métodos para dibujar oválos requieren que usted especifica una caja limite de la misma manera

como se especifica un rectángulo en los métodos drawRect() y fillRect().

```
public void drawRect( int x, int y, int width, int height );
```

```
public void fillRect( int x, int y, int width, int height );
```

En estos métodos x y y son las coordenadas de la esquina superior izquierda de la caja limite y los

parámetros height y width son el alto y el ancho de la caja.

La figura 12.7 muestra un applet, que implementa la siguiente instrucción en su método paint();

```
g.drawOval( 10, 10, 150, 100 );
```

FIGURE 12.7

drawOval()

La Figura 12.8 muestra un applet que en su método paint() implementa la siguiente instrucción

```
g.fillOval( 10, 10, 150, 100 );
```

FIGURE 12.8

drawArc() and fillArc()

Un arco es un segmento de un óvalo. Para definir un arco usted inicialmente debe especificar la caja

límite, exactamente como lo hace con los métodos drawOval() y fillOval(). También debe especificar los puntos inicial y final del arco, los cuales son un ángulo inicial y un ángulo final.

Los ángulos se miden en grados . Para el ángulo inicial, 0 grados es a la derecha, 90 grados es arriba y así se incrementan en forma contraria a las manecillas del reloj.

Un arco relleno es la región comprendida por el arco sí mismo y los dos de radios desde el centro del óvalo hasta el punto final del arco.

```
public void drawArc( int x, int y, int width, int height, int startDegrees, int arcDegrees );  
public void fillArc( int x, int y, int width, int height, int startDegrees, int arcDegrees );
```

La figura 12.9 muestra un applet que en su método paint () implementa la siguiente instrucción:

```
g.drawArc( 10, 10, 150, 100, 45, 180);
```

FIGURA 12.9
drawArc()

La figura 12.10 muestra un applet que en su método paint () implementa la siguiente instrucción:

```
g.fillArc( 10, 10, 150, 100, 45, 180);
```

FIGURA 12.10
fillArc()

drawPolygon() and fillPolygon()

Un polígono es una figura cerrada con un número arbitrario de vértices. Los vértices son pasados a los métodos drawPolygon() y fillPolygon() como dos arreglos de números enteros. El primer arreglo contiene las coordenadas en x de los vértices y el segundo arreglo contiene las coordenadas en y de los vértices. un tercer parámetro especifica el número de vértices del polígono.

```
public void drawPolygon( int[] xs, int[] ys, int numPoints );  
public void fill Polygon( int[] xs, int[] ys, int numPoints );
```

La figura 12.11 muestra un applet que implementa la siguiente instrucción en su método paint ().

```
1. int[] polyXs = { 20, 150, 150 };
```

```
2. int[] polyYs = { 20, 20, 120 };  
3. g.drawPolygon( polyXs, polyYs, 3 );
```

FIGURA 12.11 drawPolygon()

La figura 12.12 muestra un applet que implementa la siguiente instrucción en su método paint()

```
1. int[] polyXs = { 20, 150, 150 };  
2. int[] polyYs = { 20, 20, 120 };  
3. g.fillPolygon( polyXs, polyYs, 3 );
```

FIGURA 12.12 fillPolygon()

drawPolyline()

Una polilínea es similar a un polígono, pero es una figura abierta en vez de cerrada. No hay un

segmento de línea que conecta el último vértice con el primero.

Los parámetros de drawPolyline () son los mismos que los de drawPolygon(): Dos arreglos de números

enteros que representan los vértices y un número entero que especifica cuantos vértices tiene la

polilínea. No existe el método fillPolyline() ya que fillPolygon () daría el mismo resultado.

```
public void drawPolyline( int[] xs, int[] ys, int numPoints );
```

La figura 12.13 muestra un applet que en su método paint () implementa el siguiente código:

```
1. int[] polyXs = { 20, 150, 150 };  
2. int[] polyYs = { 20, 20, 120 };  
3. g.drawPolyline( polyXs, polyYs, 3 );
```

FIGURA 12.13 drawPolyline()

drawString()

El método drawString() pinta una cadena de texto.

```
public void drawString( String s, int x, int y );
```

Los parámetros x y y especifican el borde izquierdo de la línea base de la cadena. Los caracteres con trazos descendentes (g, j,p,q,y, etc) se extienden debajo de la línea base. Por omisión un contexto gráfico usa la fuente del componente al que esta asociado. Sin embargo usted puede fijar una fuente diferente para el contexto gráfico llamando al método `setFont()`.

Figura 12.14 muestra un applet que implementa en el método `paint()` el siguiente código:

```
1. Font font = new Font( "Serif", Font.PLAIN, 24 );
2. g.setFont( font );
3. g.drawString( "juggle quickly", 20, 50 );
4. g.setColor( Color.gray );
5. g.drawLine( 20, 50, 150, 50 );
```

La cadena de la línea 3 contiene cinco caracteres descendentes. Las líneas 4 y 5 dibujan la línea base del texto para que usted pueda apreciar como se posicionan los caracteres descendentes.

FIGURA 12.14
`drawString()`

`drawImage`

Una imagen es una representación en pantalla de matriz de pixeles. El soporte de imágenes en Java es complicado y una completa descripción de este tema, se sale del alcance de este material. En la última sección de este capítulo se comentan los tópicos que se necesitan conocer para crear y manipular imágenes.

Desde ahora asuma que de algún modo usted obtiene una imagen (la cual es una instancia de la clase `java.awt.Image`) la cual usted desea presentar en la pantalla usando un determinado contexto gráfico.

La forma de hacer esto es llamar en el contexto gráfico al método `drawImage()`.

```
void drawImage( Image im, int x, int y, ImageObserver observer );
```

Existen otras versiones de este método pero esta es la más comúnmente usada.

Obviamente `im` es la

imagen que va a ser presentada y `x` e `y` son las coordenadas de la esquina superior izquierda del

componente destino. `ImageObserver` debe ser un objeto que implementa la interfaz `ImageObserver`.

Recorte

La mayoría de los programadores realizan operaciones de selección de colores, dibujado y rellenado en los contextos gráficos. Una operación menos frecuente es el recortado. Recortar es simplemente restringir la región que un contexto gráfico puede modificar. Todas las instancias de la clase Graphics tienen una región de recortado. La cual puede ser todo un componente o una parte de este. Cuando usted llama uno de los métodos drawXXX() ó fillXXX() de la clase Graphics , sólo aquellos pixeles que están contenidos en la región de recortado del contexto gráfico son modificados. Por omisión la región de recortado para un contexto gráfico es todo el componente. Existen métodos que recuperan y modifican una región de recortado. En un momento usted vera un ejemplo de recortado, pero previamente se debe tener en cuenta el siguiente código:

```
1. public void paint( Graphics g ) {  
2.     for ( int i = 10; i < 500; i += 20 )  
3.         for ( int j = 10; j < 500; j += 20 )  
4.             g.fillOval( i, j, 15, 15 );  
5. }
```

FIGURA 12.15

Applet con región de recortado por omisión.

Este método dibuja una secuencia de puntos. Tenga en cuenta que este es el método paint() de un applet que tiene 300 pixeles de ancho por 300 pixeles de alto. Debido a que el contador del ciclo llega hasta 500, el método intenta dibujar fuera de los límites del applet. Esto no es problema, porque el contexto gráfico por omisión tiene una región de recortado que coincide con la del applet. Para fijar una región de recorte rectangular, usted puede llamar el método setClip(x , y, width, height) , pasando como parámetros cuatro números enteros que describen la posición y tamaño de la región rectangular deseada. Por ejemplo el código anteriormente presentado se puede modificar de la siguiente forma

```
1. public void paint( Graphics g ) {  
2.     g.setClip( 100, 100, 100, 100 );  
3.     for ( int i = 10; i < 500; i += 20 )
```

```

4.         for ( int j = 10; j < 500; j += 20 )
5.         g.fillOval( i, j, 15, 15 );
6.     }

```

Ahora el dibujo es recortado en un rectángulo de 100 x 100, el cual se ubica en le centro de un applet de 300 x 300 pixeles. Como se ve en la figura 12.16.

FIGURA 12.16

Applet con una región de recortado especifica

Pintando un Componente Contenedor

Si un applet o un frame contienen componentes que tiene sus propios métodos paint (), todos los métodos paint () serán llamados por el ambiente cuando sea necesario. Por ejemplo si un frame contiene un panel y un canvas , en un momento determinado el entorno llamara el método paint () del frame, el método paint () del panel y el método paint () del canvas.

El código presentado a continuación implementa un frame que contiene un panel y un canvas. El frame usa un administrador de disposición GridLayout con tres filas y una columna. El panel es el primero en ser adicionado al frame , por lo tanto aparece en la parte superior de este. El canvas es agregado de segundo, por lo tanto aparece en la fila de la mitad. como no hay un componente en la última celda, usted vera en la tercera fila la última porción del frame. El panel dibuja rectángulos concéntricos. El canvas dibuja ovalos concéntricos. El frame dibuja texto.

```

1. import java.awt.*;
2. public class ThingsInFrame extends Frame {
3.     public ThingsInFrame() {
4.         super( "Panel and Canvas in a Frame" );
5.         setSize( 350, 500 );
6.         setLayout( new GridLayout(3, 1 ) );
7.         add( new RectsPanel() );
8.         add( new OvalsCanvas() );
9.     }
10.    public static void main( String[] args ) {
11.        ThingsInFrame tif = new ThingsInFrame();
12.        tif.setVisible( true );
13.    }
14.    public void paint( Graphics g ) {

```

```

15.         Rectangle bounds = getBounds();
16.         int y = 12;
17.         while ( y < bounds.height ) {
18.             g.drawString( "frame frame frame frame frame frame" ,
19.                 60, y );
20.             y += 12;
21.         }
22.     }
23. class RectsPanel extends Panel {
24.     public RectsPanel() {
25.         setBackground( Color.lightGray );
26.     }
27.     public void paint( Graphics g ) {
28.         Rectangle bounds = getBounds();
29.         int x = 0;
30.         int y = 0;
31.         int w = bounds.width - 1;
32.         int h = bounds.height - 1;
33.         for ( int i = 0; i < 10; i++ ) {
34.             g.drawRect( x, y, w, h );
35.             x += 10;
36.             y += 10;
37.             w -= 20;
38.             h -= 20;
39.         }
40.     }
41. }
42. class OvalsCanvas extends Canvas {
43.     public OvalsCanvas() {
44.         setForeground( Color.white );
45.         setBackground( Color.darkGray );
46.     }
47.     public void paint( Graphics g ) {
48.         Rectangle bounds = getBounds();
49.         int x = 0;
50.         int y = 0;
51.         int w = bounds.width - 1;
52.         int h = bounds.height - 1;
53.         for ( int i = 0; i < 10; i++ ) {
54.             g.drawOval( x, y, w, h );
55.             x += 10;
56.             y += 10;
57.             w -= 20;
58.             h -= 20;
59.         }
60.     }
61. }

```

FIGURA 12.17

Un frame con componentes
contenedores

En la línea 31, el constructor para RectsPanel llama al método setBackground(). En las líneas 53 y 54 el constructor de OvalCanvas llama a los métodos setBackground() y setForeground().

El Hilo GUI y el método repaint()

En el capítulo 7 usted repaso las facilidades de Java para crear y controlar hilos. En tiempo de ejecución el entorno crea y controla sus propios hilos los cuales operan por detrás de la escena , uno de estos hilos es el responsable de la administración de la interfaz gráfica de usuario. El hilo GUI es la herramienta del entorno usada para aceptar eventos de entrada y llamar el método paint () de los componentes cuando estos necesitan ser pintados. Todas las llamadas al método paint () no son generadas por el ambiente. Los programas en Java pueden hacer llamados directos al método paint (lo cual no es recomendable) o llamados indirectos (usando el método repaint()). Las siguientes dos secciones cubren las dos formas en que las llamadas al método paint () pueden ser generadas.

Pintado Espontáneo

Pintado espontaneo no es un termino oficial de Java. Pero se usa para dar a entender dicha acción. Algunos dibujados ocurren por si mismos, no por ser llamados en el programa. Por ejemplo, como lo explican los libros introductorios a Java, cuando un navegador inicia la ejecución de un applet, poco antes que el método init() se complete, se hace un llamado al método paint (). También cuando una parte o todo el navegador es cubierto por otra ventana, cuando esta desaparece, se hace una llamada al método paint () .

El Hilo GUI es el que hace los llamados al método paint(). Todos los applets y las aplicaciones que tienen una interfaz gráfica de usuario tienen un hilo GUI. El hilo GUI hace llamados espontáneos al método paint () bajo cuatro circunstancias, dos de las cuales se aplican sólo para los applets. Después de la exposición de otra ventana.

Después de de iconificar

Poco antes que el método `init` termine (sólo para applets)

Cuando un navegador regresa a una página previamente mostrada.

Cuando el hilo GUI llama al método `paint()` , es necesario suministrar un contexto gráfico, el parámetro

del método `paint ()` es una instancia de la clase `Graphics`.

Método `repaint()`

Hay ocasiones cuando el programa, no el ambiente , debe iniciar el pintado. Esto usualmente sucede en

respuesta a eventos de entrada.

Suponga que usted que necesita dibujar un punto rojo en la ultima posición donde se hizo click con el

ratón. El applet debe ser amarillo.

Asuma que el applet maneja sus propios eventos de ratón. Su manejador de eventos se vería de la

siguiente manera:

```
1. public void mouseClicked( MouseEvent e ) {  
2.     Graphics g = getGraphics();  
3.     g.setColor( Color.yellow );  
4.     g.fillRect( 0, 0, getSize().width, getSize().height );  
5.     g.setColor( Color.red ); // red dot  
6.     g.fillOval( e.getX()-10, e.getY()-10, 20, 20 );  
7. }
```

Existen dos razones por las cuales esta aproximación esta lejos de ser optima.

Primero, siempre que el applet se cubre y se muestra, el hilo GUI llama al método `paint ()`.

Desafortunadamente `paint()` no sabe del círculo rojo que fue dibujado en el método `mouseClicked ()`,

por lo tanto el círculo rojo no será repintado.

Una buena práctica es realizar todas las operaciones de dibujado en el método `paint ()` o en métodos

llamados desde el método `paint()`, para que el hilo GUI pueda reparar las porciones que han sido

dañadas. El hilo GUI supone que el método `paint()` puede reconstruir la pantalla en un determinado momento.

La forma de dar al hilo GUI lo que espera, es remover todo el código de dibujado de los manejadores

de eventos. Manejadores de eventos similares a `mouseClicked()` pueden almacenar información de

estado en variables instanciadas. El método `paint ()` puede usar los valores de las variables instanciadas

como instrucciones a ser dibujadas. En nuestro ejemplo mouseClicked() fue modificado y se ve de la siguiente manera, asuma que la clase tiene unas variables instanciadas mouseX y mouseY.

```
1. public void mousedicked( MouseEvent e ) {  
2.     m_mouseX = e.getX();  
3.     m_mouseY = e.getY();  
4.     Graphics g = getGraphics();  
5.     paint( g );  
6. }
```

El método paint () es el siguiente:

```
1. public void paint( Graphics g ) {  
2.     g.setColor( Color.yellow );      // yellow background  
3.     g.fillRect( 0, 0, getSize().width, getSize().height );  
4.     g.setColor(Color.red);          // red dot  
5.     g.fillOval( m_mouseX-10, m_mouseY-10, 20, 20 );  
6. }
```

Mucho mejor. Ahora si un punto es cubierto y expuesto, el daño puede ser reparado automáticamente

El programa puede ser simplificado un poco Existe un método de la clase Component llamado update()

, el cual limpia el componente a su color de fondo y luego llama al método paint(). El parámetro de

entrada del método update() es un contexto gráfico. El método init del applet puede fijar el color de

fondo en amarillo:

```
setBackground( Color.yellow );
```

Ahora el manejador de eventos puede llamar al método update en vez del método paint() y el método

update () sólo necesita dibujar el punto rojo.

```
1. public void mouseClicked( MouseEvent e ) {  
2.     m_mouseX = e.getX();  
3.     m_mouseY = e.getY();  
4.     Graphics g = getGraphics();  
5.     update( g );  
6. }  
7. public void paint( Graphics g ) {  
8.     g.setColor( Color.red );  
9.     g.fillOval( m_mouseX-10, m_mouseY-10, 20, 20 );  
10. }
```

El código trabaja eficientemente. Este es un simple programa que sólo responde a eventos de click con

el ratón. En el mundo real los programadores necesitan rastrear diferentes eventos como lo son: eventos de acción, eventos de ajuste, eventos de teclado, eventos de enfoque, eventos de mouse entre otros.

Considere que pasaría si un gran número de eventos de diferente tipo fueran generadas en una rápida sucesión. (Esto no es inusual, moviendo o arrastrando el ratón se pueden crear un gran número de eventos en un corto lapso). De tiempo en tiempo un evento puede ser generado, los manejadores de eventos pueden modificar variables instanciadas y el método paint() puede modificar la pantalla por medio de un ciclo repetitivo. Sería ideal si el manejador de eventos pudiera simplemente, modificar las variables instanciadas y correr método paint () periódicamente.

El método repaint() calendariza una llamada al método update(). Esto significa que una bandera es activada por en el hilo GUI. Cada 100 milisegundos (en la mayoría de plataformas), el hilo GUI chequea la bandera. Si ha sido activada, el hilo hace un llamado al método update() y desactiva la bandera. Así se hagan muchas peticiones durante el período de 100 milisegundos, solo se hace una llamada al método update(). El código de ejemplo puede ser modificado por última vez y se vería de la siguiente manera:

```
1. public void mousedicked( MouseEvent e ) {  
2.     m_mouseX = e.getX();  
3.     m_mouseY = e.getY();  
4.     repaint();  
5. }  
6. public void paint( Graphics g ) {  
7.     g.setColor( Color.red );  
8.     g.fillOval( m_mouseX-10, m_mouseY-10, 20, 20 );  
9. }
```

La llamada al método repaint() en la línea 4, reemplaza la llamada al método getGraphics() y la llamada al método update().

El código anterior muestra el enfoque preferido para manejar eventos que hacen que la pantalla se cargue: los manejadores de eventos almacenan información en variables y llaman al método repaint() y el método paint() dibuja en la pantalla de acuerdo con la información de las variables. Los dos beneficios de este enfoque son:

La pantalla es correctamente reconstruida cuando el entorno espontáneamente llama al método `paint()`.

La máquina virtual nunca será sobrecargada por eventos.

Si usted desea acumular puntos, en vez de borrar cada punto cada vez que dibuja uno nuevo, usted

puede sobre escribir el método `update()` para que este no borre. El método `update()` necesita para este

caso hacer una llamada al método `paint()` como se muestra a continuación:

```
1. public void update( Graphics g ) {  
2.     paint( g );  
3. }
```

Esta es una técnica estándar.

IMÁGENES

Las imágenes son representaciones en pantalla de una matriz de píxeles.

Existen tres cosas que usted puede hacer con las imágenes:

Crearla

Modificarla

Dibujarlas en pantalla o en otras imágenes.

Hay dos formas de crear una imagen vacía ó crear una partir de una archivo `.gif` ó `.jpeg`. Para crear una imagen vacía llame al método `createImage()` de la clase `Component`, y pase como

parámetros el ancho y el alto deseado. Por ejemplo la siguiente línea crea una imagen llamada `im1` la

cual tiene 400 píxeles de ancho por 250 píxeles de alto:

```
Image im = createImage( 400, 250 );
```

Una imagen puede ser creada basándose en la información de un archivo `.gif` ó `.jpeg`.

Las clases `Applet`

y `Toolkit` tienen un método llamado `getImage()`, el cual tiene dos formas comunes:

```
? getImage( URL fileURL )
```

```
? getImage( URL dirURL, String path )
```

La primera forma toma una URL que hace referencia a la imagen deseada. La segunda forma toma una

URL que se refiere a un directorio y la ruta de la imagen deseada, relativa al directorio.

El siguiente

fragmento de código muestra el método `init()` de un applet , el cual carga una imagen de un archivo que

reside en el mismo directorio de la pagina que contiene al applet.

```
1. public void init() {  
2. Image im = getImage( getDocumentBase(), "thePicture.gif" );  
3. }
```

Si usted carga una imagen de un archivo, puede modificarla. Afortunadamente las imágenes tienen un contexto gráfico. Todo lo que usted necesita es obtener un contexto gráfico de una imagen y hacer uso de los métodos que fueron discutidos en la sección Dibujado y Rellenado, de este capítulo. Para obtener un contexto gráfico simplemente llame al método `getGraphics()`. El siguiente código implementa un applet que en su método `init()` crea una imagen y después de obtener su contexto gráfico dibuja un círculo azul sobre un fondo amarillo. el método `paint()` presenta la imagen en la pantalla usando el método `drawImage()`, el cual fue documentado anteriormente en este capítulo.

```
1. import java.applet.Applet;
2. import java.awt.*;
3. public class PaintImage extends Applet {
4.     Image im;
5.     public void init() {
6.         im = createImage( 300, 200 );
7.         Graphics imgc = im.getGraphics();
8.         imgc.setColor( Color.yellow );
9.         imgc.fillRect( 0, 0, 300, 200 );
10.        imgc.setColor( Color.blue );
11.        imgc.fillOval( 50, 50, 100, 100 );
12.    }
13.    public void paint( Graphics g ) {
14.        g.drawImage( im, 25, 80, this );
15.    }
16. }
```

Note que en las líneas 9-13 , `imgc` es un contexto gráfico que dibuja en la imagen `im`. En las líneas 16-

17, `g` es un contexto gráfico que dibuja el applet.

Resumen del Capítulo

El método `paint()`, provee un contexto gráfico para dibujar. La funcionalidad del contexto gráfico (clase `Graphics`) incluye:

- Seleccionar un color
- Seleccionar una fuente
- Dibujar y rellenar
- Recortar

Llamadas al método `paint()` puede ser generadas espontáneamente por el sistema bajo cuatro

circunstancias:

Después de la exposición de otra ventana.

Después de de iconificar

Poco antes que el método init termine (sólo para applets)

Cuando un navegador regresa a una página previamente mostrada. (Sólo para applets)

En todos los casos, la región de recorte de un contexto gráfico se puede fijar

apropiadamente. Los

manejadores de eventos que necesitan modificar la pantalla puede almacenar el estado de la

información en variables y posteriormente llamar al método repaint(). Este método calendariza una

llamada al método update(), el cual limpia el fondo de un componente y llama al método paint().

Las imágenes pueden ser creadas iniciando desde cero ó cargandolas de archivos externos. una imagen

puede ser modificada usando un contexto gráfico.

Ejemplos Adicionales

Ejemplo 1

```
import java.awt.*;
public class ajedrez extends java.applet.Applet
{
    public void init()
    {
        resize(300,300);
    }

    public void paint(Graphics g)
    {
        int x=30; // Ancho del recuadro
        int y=30; // Alto del recuadro
        int c1=0; // Coordenada x
        int c2=0; // Coordenada y

        for (int i=0; i<=7; i++)
        {
            for (int j=0;j<=7;j++)
            {
                if ( i % 2 ==0)
                {
                    if ( j %2 ==0 )
                    {
                        g.setColor(Color.yellow);
                    }
                    else
```

```

        {
            g.setColor(Color.black);
        }
    }
    else
    {
        if ( j %2 ==0 )
        {
            g.setColor(Color.black);
        }
        else
        {
            g.setColor(Color.yellow);
        }
    }
    c1+=x;
    g.fillRect(c1,c2,x,y);
}
c2+=y;
c1=0;
}
}
}

```

Código anterior muestra un tablero de ajedrez en la pantalla, se construye usando rectángulos con relleno. El color de relleno se intercala según sea la fila y columna del recuadro

Ejemplo 2

```

import java.awt.*;
import java.awt.image.*;
public class imagen1 extends Frame
{
    imagen1()
    {
        setSize(300,300);
    }

    public void paint (Graphics g)
    {
        Image imagen = getToolkit().getImage("escudoun.gif");
        g.drawImage(imagen,0,0,this);
    }

    public static void main(String args[])
    {
        imagen1 IM = new imagen1();
    }
}

```

```

        IM.show();
    }

}

```

Ejemplo 3

```

public class CentradoTexto extends java.applet.Applet
{
    public void init()
    {
        resize(300,300);
    }

    public void paint(Graphics g)
    {
        Font f= new Font ("Serif",Font.PLAIN,30);
        g.setFont(f);
        FontMetrics fme = g.getFontMetrics();
        String cadena = "Java 2";
        int xinicial = ( getSize().width - fme.stringWidth(cadena))/2;
        int yinicial = ( getSize().height - fme.getHeight())/2;
        g.drawString(cadena,xinicial,yinicial);

    }
}

```

El código anterior dibuja la cadena Java2 con el tipo de letra Serif, en negrilla y de un tamaño de 30 puntos centrada en el espacio asignado al applet.

Examínese usted mismo

1. Cómo se fija el color de un contexto gráfico llamado g en cian.

- A. g.setColor(Color.cyan);
- B. g.setCurrentColor(cyan);
- C. g.setColor("Color.cyan");
- D. g.setColor("cyan");
- E. g.setColor(new Color(cyan));

2. El siguiente código dibuja una línea. ¿ De qué color es la línea?

- 1. g.setColor(Color.red.green.yellow.red.cyan);

2. `g.drawLine( 0, 0, 100, 100 );`

- A. Rojo
- B. Verde
- C. Amarillo
- D. Cian
- E. Negro

3 ¿ Qué dibuja el siguiente código?

- 1. `g.setColor( Color.black );`
- 2. `g.drawLine( 10, 10, 10, 50 );`
- 3. `g.setColor( Color.red );`
- 4. `g.drawRect( 100, 100, 150, 150 );`

- A. Una línea vertical roja que tiene 40 píxeles de longitud y un cuadrado rojo que tiene 150 píxeles de lado.
- B. Una línea vertical negra que tiene 40 píxeles de longitud y un cuadrado rojo con lados de 150 píxeles.
- C. Una línea vertical negra que tiene 50 píxeles de longitud y un cuadrado rojo con lados de 150 píxeles.
- D. Una línea vertical roja que tiene 50 píxeles de longitud y un cuadrado con lados de 150 píxeles.
- E. Una línea vertical negra que tiene 40 píxeles de longitud y un cuadrado rojo con lados de 100 píxeles.

4. La figura 12.18 muestra dos formas. ¿ Cual de las dos es dibujada por el siguiente código ?

- 1. `g.fillArc( 10, 10, 100, 100, 0, 90 );`

FIGURA 12.18

Pregunta 4.

1. ¿Cuál de los siguientes enunciados es verdadero ?

- A. Una polinea siempre tiene relleno
- B. Una polinea no puede tener relleno.
- C. Un poligono siempre tiene relleno
- D. Un poligono siempre es cerrado.
- E. Un poligono puede tener relleno o no tenerlo.

1. Cuando el hilo GUI llama al método `paint()` el orden para reparar un daño por exposición es el siguiente , el método `paint()` debe determinar cual es el daño y establecer una región de recorte apropiada.

- A. Verdadero
- B. Falso

7. Su manejador de eventos `mouseDragged()` y su método `paint()` son los siguientes:

```
1. public void mouseDragged( MouseEvent e ) {  
2.     mouseX = e.getX();  
3.     mouseY = e.getY();  
4.     repaint();  
5. }  
6. public void paint( Graphics g ) {  
7.     g.setColor( Color.cyan );  
8.     g.drawLine( mouseX, mouseY, mouseX+10, mouseY+10 );  
9. }
```

Usted desea modificar su código para que las líneas de color cian se acumulen en la pantalla , en vez de que sean borradas continuamente cuando `repaint()` llama a `update()` . Cual es la forma más simple para hacerlo ?

- A. En la línea 4, reemplace `repaint()` con `paint()`.
- B. En la línea 4, reemplace `repaint()` con `update()`.
- C. Después de la línea 7, agregue esta línea: `super. update(g);`
- D. Adicione el siguiente método

```
1. public void update( Graphics g ) { paint( g ); }
```

8. ¿ Qué código usaría usted para construir una fuente serif, con negrilla y de 24 puntos?

- A. `new Font( Font.SERIF, 24, Font.BOLD );`
- B. `new Font( "Serif", 24, "Bold" );`
- C. `new Font( "Bold", 24, Font.SERIF );`
- D. `new Font( "Serif", Font.BOLD, 24 );`
- E. `new Font( Font.SERIF, "Bold", 24 );`

9. ¿ Qué dibuja el siguiente método `paint()` ?

```
1. public void paint( Graphics g ) {  
2.     g.drawString( "question #9", 10, 0 );  
3. }
```

- A. La cadena "question #9", con su esquina superior izquierda en 10, 0
- B. Una pequeña porción baja desde la parte superior del componente

10. ¿ Qué dibuja el siguiente método paint() ?

```
public void paint( Graphics g ) {  
1.    g.drawOval( 0, 100, 44 );  
2. }
```

- A. Un círculo en (100, 100) con radio 44
- B. Un círculo en (100, 44) con radio 100
- C. Un círculo en (100, 44) con radio 44
- D. El código no compila

11. ¿ Qué dibuja el siguiente método paint() ?

```
public void paint(Graphics g)  
{  
    g.setColor(Color.blue);  
    g.fillOval(100,100,100,50);  
    setForeground(Color.red);  
    setBackground(Color.yellow);  
    g.fillRect(50,50,20,30);  
    g.drawString("Applet",100,200);  
}
```

- A. Se dibuja un óvalo con relleno azul, un rectángulo de color rojo con 20 pixeles de ancho, 30 de alto y en las coordenada (100,200) aparece la cadena "Applet" de color rojo. Todo esto sobre un fondo amarillo.
- B. Se dibuja un óvalo con relleno negro, un rectángulo de color azul con 20 pixeles de ancho, 30 de alto y en las coordenada (100,200) aparece la cadena "Applet" de color rojo. Todo esto sobre un fondo amarillo.
- C. Se dibuja un óvalo con relleno azul, un rectángulo de color azul con 20 pixeles de ancho, 30 de alto y en las coordenada (100,200) aparece la cadena "Applet" de color azul. Todo esto sobre un fondo amarillo.
- D. Se dibuja un óvalo con relleno azul, un rectángulo de color rojo con 20 pixeles de ancho, 30 de alto y en las coordenada (100,200) aparece la cadena "Applet" de color azul. Todo esto sobre un fondo amarillo.
- E. El código no compila.

12. ¿ Qué dibuja el siguiente método paint() ?

```
public void paint(Graphics g)
```

```

{
int a[] = {30,60,60,30,50};
int b[] = {10,10,40,40,50};
g.fillPolygon(a,b,4);
}

```

- A. El código no compila.
- B. El código compila pero lanza una excepción en tiempos de ejecución.
- C. El código compila, no lanza ninguna excepción y pinta un cuadrilátero con relleno de color negro
- D. El código compila, no se lanza ninguna excepción, pero no se ve ningún dibujo.

13 ¿ Qué dibuja el siguiente método paint() ?

```

public void paint(Graphics g)
{
    setForeground(Color.red);
    setBackground(Color.yellow);
    g.setColor(Color.blue);
    g.drawRect(-10,0,30,30);
}

```

- A. Código no compila
- B. Código compila pero al ejecutarse no muestra ninguna figura.
- C. Pinta un cuadrado desde la coordenada (0,0) de color rojo
- D. Pinta un cuadrado desde la coordenada (0,0) de color azul
- E. Ninguna de las anteriores

14. Considere el siguiente código, seleccione una o más opciones:

```

import java.awt.*;

class ejemplo4 extends Frame
{
    Image im;
    public void paint(Graphics g)
    {

        im = createImage( 300, 200 );
        Graphics imgc = im.getGraphics();
        imgc.setColor( Color.yellow );
        imgc.fillRect( 0, 0, 300, 200 );
        g.drawImage( im, 25, 80, this );
    }

    public static void main(String args[])
    {
        ejemplo4 e = new ejemplo4();
    }
}

```

```
e.show();
```

```
}
```

```
}
```

- A. El código compila, pero lanza una excepción en tiempo de ejecución.
- B. El código no compila porque no se puede sobrescribir el método paint() de un frame
- C. El código compila y a la hora de ejecutarse se ve un frame que contiene un polígono relleno de color azul.
- D. El código no compila porque el método createImage() no esta implementado en la clase ejemplo4 ni en ninguna de sus super clases.

15. Una Polilinea puede ser una figura cerrada

- A. Verdadero
- B. Falso

Capítulo 13: ENTRADA Y SALIDA

OBJETIVOS

Java soporta entrada y salida con un conjunto flexible de Clases de Flujos. Los Archivos I/O requieren un poco de soporte adicional, el cual provee Java en sus Clases **File** y **RandomAccessFile**.

Todas las operaciones dentro y fuera de una Máquina Virtual Java son contingentes sobre la aprobación por el Manejador de Seguridad. La mayoría de los browsers prohíben todo el acceso a Archivos, así que las clases **File** y **RandomAccessFile** se usan generalmente en aplicaciones.

Archivos de Entrada y Salida

Las clases **File** y **RandomAccessFile** de Java proporcionan a las funciones para navegar el sistema de archivos local, describir archivos y directorios, y tener acceso a archivos en orden no secuencial (Tener acceso a archivos secuencialmente se hace con streams, readers, and writers, que se describen más adelante en este capítulo. Los archivos a menudo contienen texto. La representación del texto de Java va más allá del ASCII tradicional. Puesto que el examen de la certificación requiere familiarizarse con la manera cómo Java representa el texto, es útil revisar este tema antes los archivos de mirar los archivos de I/O.

Representación del Texto y Codificación de Caracter

Java utiliza dos formas de representación del texto:

- Unicode para la representación interna de caracteres y de cadenas
- UTF para la entrada y salida

Unicode utiliza 16 bits para representar cada caracter. Si los 9 bits más significativos son todos ceros, entonces la codificación es simplemente ASCII estándar, con el byte menos significativo conteniendo la representación del caracter. Si no, los bits representan un caracter que no es representado en 7-bit ASCII.

El tipo de datos char de Java utiliza la codificación de Unicode, y la clase **String** contiene una colección de caracteres de Java. Los 16 bits de Unicode son suficientes codificar la mayoría de los alfabetos, pero los lenguajes picto-gráficos asiáticos presentan un problema. Los comités de estándares han desarrollado compromisos para permitir subconjuntos limitados pero útiles de chino, japonés, y coreano que se representarán en Unicode, pero ha llegado a estar claro que un esquema ideal global de la representación del texto debe utilizar más de 16 bits por caracter.

La respuesta es **UTF**. La abreviatura significa " Formato de Transformación UCS " y el UCS significa " Conjunto Universal de Caracteres ". Mucha gente cree que UTF es corto para el "Formato de Texto Universal, " y mientras ellos están en desacuerdo, su versión es más descriptiva que la versión verdadera.

La codificación de UTF utiliza tantos bits como sea necesario para codificar un caracter, pocos bits para alfabetos más pequeños, más bits para los alfabetos asiáticos más grandes. Puesto que cada caracter puede ser representado, UTF es un esquema de codificación verdaderamente global.

Una *codificación del caracter* es un mapeo entre un conjunto de caracteres y un rango de números binarios. Cada plataforma de Java tiene una codificación de caracteres por defecto, que se utiliza para interpretar entre Unicode interno y los bytes externos. La codificación de caracteres por defecto refleja el lenguaje y la cultura locales. Cada

codificación tiene un nombre. Por ejemplo, " 8859_1 " es ASCII común, " 8859_8 " es ISO Latin/Hebrew, y " Cp1258 " es Vietnamita.

Cuando una operación I/O se realiza en Java, el sistema necesita saber qué codificación de caracteres va a utilizar. Las diferentes clases de I/O utilizan la codificación local por defecto, a menos que explícitamente se especifique una codificación diferente. Para la mayoría de las operaciones, la codificación local por defecto es perfectamente adecuada. Sin embargo, al comunicarse a través de una red con otro ordenador, ambas máquinas necesitan utilizar la misma codificación, incluso si residen en diversos países. En tales casos, es una buena idea solicitar explícitamente " 8859_1."

La Clase File

La clase de **java.io.File** representa el nombre de un archivo o de un directorio que pudieron existir en el sistema de archivos del computador. La forma más simple del constructor para esta clase es:

File(String pathname);

Es importante saber que la construcción de una instancia File no crea un archivo en sistema de Archivos local. Llamar al constructor crea simplemente una instancia que encapsula la cadena especificada como parametro. Por supuesto, si la instancia va a estar en cualquier uso, debería encapsularse muy probablemente una cadena que representa un archivo o un directorio existente, o uno que sea creado pronto. Sin embargo, en el tiempo de la construcción no se hace ningún chequeo.

Hay dos otras versiones del constructor File:

File(String dir, String subpath);

File(File dir, String subpath);

Ambas versiones requieren que usted proporcione un directorio y un path relativo (el argumento subpath) dentro de ese directorio.

En una versión usted utiliza una cadena que especifica el directorio; en la otra, usted utiliza una instancia de File (recuerde que la clase File puede representar un directorio así como un archivo). Usted puede, por ejemplo, ejecutar el siguiente código en una máquina UNIX:

1. File f = new File("/tmp", "xyz"); // asuma / tmp es un directorio

Usted puede ejecutar el siguiente código en una plataforma de Windows:

1. File f1 = new File("C:\\a"); // asuma que C:\\a es un archivo del directorio

2. File f2 = new File(f1, "xyz.java");

(En la línea 1, el primer backslash es un caracter de escape que se asegura de que el segundo backslash esté validado literalmente.

Por supuesto, no hay razón teórica por la que usted no podría ejecutar el ejemplo en una máquina de Windows y el segundo ejemplo en una plataforma de UNIX. Hasta este punto usted no está haciendo nada más que construir objetos que encapsulan cadenas. En la práctica, sin embargo, no se gana nada usando la semántica incorrecta del pathname.

Después de construir una instancia de File, usted puede hacer un número de llamadas a sus métodos. Algunas de estas llamadas hacen simplemente la manipulación de cadenas en el pathname del archivo, mientras que otras tienen acceso o modifican al sistema de archivos local.

Los métodos que soportan la navegación se enumeran abajo:

- `boolean exists()`: Este retorna **true** si el archivo o el directorio existen, si no retorna **false**.
- `String getAbsolutePath()`: Este retorna el path absoluto, es decir el path no relativo del archivo o del directorio.
- `String getCanonicalPath()`: Este retorna el path canónico del archivo o el directorio. Es similar a **getAbsolutePath()**, pero los simbolos `.` y `..` son resueltos.
- `String getName()`: Este retorna el nombre del archivo o directorio. El nombre es el último elemento del path.
- `String getParent()`: Este retorna el nombre del archivo o directorio que contiene el archivo.
- `boolean isDirectory()`: Este retorna **true** si el **Archivo** describe un directorio que existe en el sistema de archivos.
- `boolean isFile()`: Este retorna **true** si el **Archivo** describe un archivo que existe en el sistema de archivos.
- `String[] List()`: Este retorna una arreglo que contiene los nombres de los archivos y directorios dentro del **Archivo**. El **Archivo** debería describir un directorio, no un archivo

Los métodos que se listaron arriba no son la totalidad de los métodos de la clase.

Algunos métodos no-navigables son:

- `boolean canRead()`: Este retorna **true** si el archivo o el directorio pueden ser leídos.
- `boolean canWrite()`: Este retorna **true** si el archivo o el directorio pueden modificados.
- `boolean delete()`: Este intenta borrar el archivo o el directorio.
- `long length()`: Este retorna la longitud del archivo.
- `boolean mkdir()`: Este intenta crear un directorio cuyo path es descrito por el archivo.
- `boolean renameTo( File newname )`: Este renombra el archivo o directorio. Retorna **true** si el renombramiento ha sucedido, de lo contrario retorna **false**. El programa listado abajo usa algunos de los métodos de navegación para crear un listado recurrente de un directorio. La aplicación espera que el directorio sea especificado en la línea de comando. El listado aparece en un área de texto dentro de un Frame.

```

1. import java.awt.*;
2. import java.io.File;
3. public class Lister extends Frame {
4.     TextArea ta;
5.     public static void main( String[] args ) {
6.         // get path or dir to be listed. Default to cwd if
7.         // no command line arg
8.         String path = ".";
9.         if ( args.length >= 1 )
10.            path = args[ 0 ];
11.        // make sure path exists and is a directory
12.        File f = new File( path );
13.        if ( !f.isDirectory() ) {
14.            System.out.println( "Doesn't exist or not dir: " + path );
15.            System.exit( 0 );

```

```

16.}
17.// recursively list contents
18.Lister lister = new Lister( f );
19.lister.setVisible( true );
20.}
21.Lister( File f ) {
22.setsize( 300, 450 );
23.ta = new TextArea();
24.ta.setFont( new font( "Monospaced" , font.PLAIN, 14 ) );
25.add( ta, BorderLayout.CENTER );
26.recurse( f, 0 );
27.}
28.//
29.// recursively list the contents of dirfile,
30.// indent 5 spaces for each level of depth
31.//
32.void recurse( File dirfile, int depth ) {
33.String contents[] = dirfile.list();
34.for ( int i = 0; contents.length; i++ ) // for each child...
35.for ( int spaces = 0; spaces < depth; spaces++ ) // indent
36.ta.append( " " );
37.ta.append( contents[ i ] + "\n" ); // print name
38.File child = new File( dirfile, contents[ i ] );
39.if ( child.isDirectory() )
40.recurse( child, depth+1 ); // recurse if dir
41.}
42.}

```

} La Figura 13.1 muestra una muestra de la salida de este programa.

El programa primero chequea un argumento de la línea de comandos(líneas 10-12). Si se provee uno, este se asume para ser el nombre del directorio que se listará; si no hay argumento, el directorio de trabajo actual será listado.

Observe la llamada a `isDirectory()` en la línea 16. Esta llamada retorna `true` solamente si el path representa un directorio existente.

Después de establecer que el objeto que se listará realmente es un directorio, el código construye una instancia de **Lister**, que hace una llamada a **recurse()**, pasando en el archivo que se listará en el parámetro `dirfile`.

El método **recurse()** hace una llamada a **list()** (línea 39) para conseguir un listado del contenido del directorio. Cada archivo o subdirectorio se imprime (línea 43) después del sangrado de márgenes apropiado (5 espacios por nivel, líneas 41 y 42). Si el hijo es un directorio (probado en la línea 45), su contenido se listará recurrentemente.

El programa del **Lister** muestra una forma para utilizar los métodos de la clase **File** para navegar el sistema de archivos local. Estos métodos no modifican el contenido de los archivos de ninguna manera; para modificar un archivo usted debe utilizar la clase **RandomAccessFile** o los recursos `stream`, `reader`, y `writer` de Java

Todos estos tópicos se cubren en las secciones que siguen, pero primero aquí esta un resumen de las puntos dominantes referentes a la clase **File**:

- Una instancia de **File** describe un archivo o directorio.

- El archivo o directorio podría o no podría existir.
- La colección Constructing/garbage de una instancia File no tiene ningún efecto en el sistema de archivos local

Figura 13.1

Ejemplo de listado

image
ColorModel.java
DirectColorModel.java
ImageConsumer.java
ImageFilter.java
ImageObserver.java
IndexColorModel.java
ImageProducer.java
RGBImageFilter.java
MemoryImageSource.java
FilteredImageSource.java
CropImageFilter.java
PixelGrabber.java
ReplicateScaleFilter.java
AreaAveragingScaleFilter.java
datatransfer
Clipboard.java
ClipboardOwner.java
DataFlavor.java
StringSelection.java
Transferable.java
UnsupportedFlavorException.java

Event
KeyEvent.java
ActionEvent.java
ActionListener.java
FocusEvent.java
ComponentAdapter.java
ComponentEvent.java

La Clase RandomAccessFile

Una forma para leer o para modificar un archivo es utilizar la clase de **java.io.RandomAccessFile**. Esta clase presenta un modelo de archivos que es incompatible con el modelo stream/reader/writer descrito más adelante en este capítulo.

El modelo de stream/reader/writer fue desarrollado para I/O general, mientras que la clase RandomAccessFile se aprovecha de un comportamiento particular de los archivos que no se encuentra generalmente en dispositivos de I/O .

Con un archivo de acceso aleatorio, usted puede buscar a una posición deseado dentro un archivo y entonces leer o escribir una cantidad deseada de datos. La clase de RandomAccessFile proporciona los métodos que soportan búsqueda, lectura, y escritura.

Los constructores para la clase son:

- RandomAccessFile(String file, String mode);
- RandomAccessFile(File file, String mode);

La cadena **mode** debe ser " r " o " rw." Utilice " r " para abrir el archivo y leerlo solamente, y utilice " rw " para abrirlo para lectura y escritura.

La segunda forma del constructor es útil cuando usted desea utilizar algunos de los métodos de la clase File antes de abrir un archivo de acceso aleatorio, de modo que usted tenga ya una instancia de File actual cuando llegue la hora de llamar al constructor de RandomAccessFile. Por ejemplo, el fragmento del código abajo construye una instancia de File para verificar que el path de la cadena representa un archivo que exista y pueda ser escrito. Si éste es el caso, llaman el constructor de RandomAccessFile; si no una excepción se lanza.

1. **File** file = **new File**(path);
2. **if** (!file.isFile() || !file.canRead() || !file.canWrite()) {
3. throw **new IOException**();
4. }
5. **RandomAccessFile** raf = **new RandomAccessFile**(file, "rw");

Cuando no existe el archivo nombrado, construir un RandomAccessFile es diferente de construir un archivo ordinario. En esta situación, si el RandomAccessFile se construye en modo inalterable, se lanza un **FileNotFoundException**. Si el RandomAccessFile se construye en modo de lectura-escritura, después se crea un archivo de cero-longitud.

Después de que se construya un archivo de acceso aleatorio, usted puede buscar a cualquier posición del byte dentro del archivo y entonces leer o escribir. Los sistemas Pre-Java (la biblioteca estándar I/O de C , por ejemplo) han soportado búsquedas a una posición concerniente al principio del archivo, el final del archivo, o de la posición actual dentro del archivo. Los archivos de acceso aleatorio de Java utilizan solamente búsquedas relativas al principio del archivo, pero hay los métodos que señalan la posición actual y la longitud del archivo, así que usted puede realizar con eficacia las otras clases de búsqueda mientras usted está dispuesto a hacer la aritmética.

Los métodos que soportan búsqueda son:

- `long getFilePointer()` lanza `IOException`: Este retorna la posición actual dentro del archivo, en bytes. La lectura y escritura subsecuentes ocurrirán comenzando en esta posición.
- `long length()` lanza `IOException`: Este retorna la longitud del archivo, en bytes.
- `void seek(long position)` lanza `IOException`: Este fija la posición actual dentro del archivo, en bytes. La lectura y escritura subsecuentes ocurrirán comenzando en esta posición. Los archivos comienzan en la posición 0.

El código enumerado abajo es una subclase de `RandomAccessFile` que adiciona dos nuevos métodos para realizar búsqueda de la posición actual o del final del archivo. El código ilustra el uso de los métodos listados arriba.

```
1. class GeneralRAF extends RandomAccessFile {  
2. public GeneralRAF( File path, String mode ) throws IOException {  
3.     super( path, mode );  
4. }  
5. public GeneralRAF( String path, String mode ) throws IOException {  
6.     super( path, mode );  
7. }  
8. public void seekFromEnd( long offset ) throws IOException {  
9.     seek( length() – offset );  
10.}  
11.public void seekFromCurrent( long offset ) throws IOException {  
12.    seek( getFilePointer() + offset );  
13.}  
14.}
```

Todo el punto de búsqueda, por supuesto, es leer desde o escribir a una posición dentro de un archivo. Los archivos son colecciones pedidas de bytes, y la clase de `RandomAccessFile` tiene varios métodos que utilicen la lectura y escritura de bytes. Sin embargo, los bytes en un archivo a menudo se combinan para representar formatos de datos más ricos. Por ejemplo, dos bytes podrían representar un carácter Unicode; cuatro bytes podrían representar un **float** o un **int**. Todos los métodos de lectura y de escritura avanzan la posición actual del archivo.

Los métodos más comunes que utilizan la lectura y escritura de bytes son:

- `int read ()` lanza `IOException`: Este retorna el siguiente byte del archivo (almacenado en los 8 bits menos significativos de un `int`) ó -1 si esta en el final del archivo
- `int read(byte dest[])` lanza `IOException`: Este procura leer bastantes bytes para llenar el arreglo `dest[]`. Retorna el número de bytes leídos o -1 si el archivo esta en el final archivo.

- `int read (byte dest[], int offset, int len)` lanza `IOException`: Esto procura leer `len` bytes en el arreglo `dest[]`, comenzando en el desplazamiento. Retorna el número de bytes leídos o `-1` si el archivo esta en el final del archivo.
- `void write(int b)` lanza `IOException`: Este escribe el byte menos significativo de `b`.
- `void write(byte [])` lanza `IOException`: Escribe todo el arreglo `b[]` de bytes.
- `void write (byte [], int offset, int len)` lanza `IOException`: Este escribe `len` bytes del arreglo `b[]` de bytes, comenzando en el desplazamiento.

Los archivos de acceso aleatorio utilizan la lectura y escritura de todos los tipos de datos primitivos. Cada operación de lectura o escritura avanza de la la posición actual del archivo por el número de bytes leídos o escritos. La Tabla 13.1 presenta varios métodos primitivo-orientados, todos lanzan `IOException`.

Tabla 13.1 .

Métodos del archivo de acceso aleatorio la para los tipos de datos primitivos.

Data Type	Read Method	Write Method
boolean	boolean <code>readBoolean()</code>	void <code>writeBoolean(boolean b)</code>
Byte	byte <code>readByte()</code>	void <code>writeByte(int b)</code>
Short	short <code>readShort()</code>	void <code>writeShort(int s)</code>
Char	char <code>readChar()</code>	void <code>writeChar(int c)</code>
Int	int <code>readInt()</code>	void <code>writeInt(int i)</code>
Long	long <code>readLong()</code>	void <code>writeLong(long l)</code>
Float	float <code>readFloat()</code>	void <code>writeFloat(float f)</code>
double	double <code>readDouble()</code>	void <code>writeDouble(double d)</code>
Byte	int <code>readUnsignedByte()</code>	None
Short	int <code>readUnsignedShort()</code>	None
line of text	String <code>readLine()</code>	None
UTF string	String <code>readUTF()</code>	void <code>writeUTF(String s)</code>

Hay varios métodos de archivos de acceso aleatorio que soportan lectura y escritura de los tipos de datos no-absolutamente-primitivos. Estos métodos se ocupan de los unsigned bytes ,unsigned short, líneas de texto, y de las cadenas de UTF, según lo mostrado en la Tabla 13.1.

Cuando un archivo de acceso aleatorio no se necesita más debería ser cerrado:

- `void close()` lanza `IOException`.

El método del `close()` libera los recursos de sistema non-memory asociados al archivo.

Para resumir, los archivos de acceso aleatorio ofrecen las siguientes funciones :

- Búsqueda de cualquier posición dentro de un archivo.
- Lectura y escritura de un solo o múltiples bytes
- Lectura y escritura de grupos de bytes, tratados como tipos datos de alto nivel
- Cierre.

Streams, Readers, and Writers

La clases de Java stream, readers y writers ven la entrada y salida de datos como secuencias ordenadas de bytes. Por supuesto, el ocuparse terminantemente de bytes sería enormemente fastidioso, porque los datos aparecen a veces como bytes, a veces como enteros, a veces como Reales, etcétera. Usted ha visto ya cómo la clase de

RandomAccessFile permite que usted lea y que escriba todos los tipos de datos primitivos de Java. El método del readInt(), por ejemplo, lee cuatro bytes de un archivo, junta las piezas de ellos , y devuelve un entero. Las clases generales de I/O de Java proporcionan a un acercamiento estructurado similar:

- Un flujo de salida de bajo nivel recibe bytes y escribe bytes a un dispositivo de salida.
- Un Filtro de flujo de salida de alto nivel recibe datos de formato-general, tales como primitivos, y escribe bytes a un flujo de salida de bajo nivel o a otro Filtro de flujo de salida.
- Un writer es similar a un Filtro de flujo de salida pero esta especializado en escribir cadenas Java en Unidades de caracteres Unicode.
- Un flujo de entrada de bajo nivel lee bytes desde un dispositivo de entrada de bajo nivel y retorna bytes a su llamador.
- Un Filtro de flujo de entrada de alto nivel lee bytes desde un flujo de entrada de bajo nivel. o desde otro filtro de flujo de entrada, y retorna datos de formato-general a su llamador.
- Un reader es similar a un filtro de flujo de entrada pero esta especializado para leer cadenas UTF en unidades de caracteres Unicode.

Las clases streams, readers y writers no son muy complicados. La manera más fácil de repasarlos es comenzar con los flujos de bajo nivel.

Flujos de Bajo Nivel

Los flujos de entrada de bajo nivel tienen métodos que leen la entrada de información y retornan la entrada de información como bytes. Los flujos de salida de bajo nivel tienen métodos que reciben bytes, y escriben los bytes como salida. Las clases **FileInputStream** y **FileOutputStream** son ejemplos excelentes.

Los dos constructores del flujo de entradas de archivos más comunes son:

- **FileInputStream(String pathname)**
- **FileInputStream(File file)**

Después de que se haya construido un flujo de entradas de archivo, usted puede llamar métodos para leer un solo byte, un arreglo de bytes, o una porción de un arreglo de bytes. Las funciones son similares a los métodos de byte-entrada que usted ha visto ya en la clase RandomAccessFile:

- **int read()** lanza IOException: Este devuelve el byte siguiente del archivo (almacenado en los 8 bits de orden inferior de un entero) o -1 si esta en el final del archivo.
- **int read (byte[] dest)** lanza IOException: Este procura leer bastantes bytes para llenar el arreglo dest[]. Retorna el número de los bytes leídos o -1 si esta en el final del archivo.
- **int read (byte[] dest,int offset, int len)** lanza IOException: Este procura leer len bytes en el arreglo dest[], comenzando en el desplazamiento. Retorna el número de los bytes leídos o -1 si esta en el final del archivo.

El fragmento del código abajo ilustra el uso de estos métodos leyendo un solo byte en el byte b, después bastantes bytes para llenar de bytes el arreglo byte[], y finalmente 20 bytes en las primeras 20 localizaciones del arreglo de bytes morebytes[]:

1. **byte** b;
2. **byte** bytes[] = **new byte**[100];
3. **byte** morebytes[] = **new byte**[50];
4. try {

```

5. FileInputStream fis = new FileInputStream( "some_file_name" );
6. b = (byte)fis.read(); // single byte
7. fis.read( bytes ); // fill the array
8. fis.read( morebytes, 0, 20 ); // 1st 20 elements
9. fis.close();
10.} catch ( IOException e ) { }

```

La clase de `FileInputStream` tiene algunos métodos utilitarios muy útiles:

- `int available()` lanza `IOException`: Este retorna los bytes que pueden ser leídos sin el bloqueo.
- `void close()` lanza `IOException`: Este libera los recursos de sistema de los no miembro-asociados al archivo. Un flujo de entradas de archivo debe siempre cerrarse cuando no se necesita más.
- `long skip(long nbytes)` lanza `IOException`: Este procura leer y desechar bytes de los `nbytes`. Retorna el número AC de los bytes saltados.

No es de sorprender que los flujo de salida de archivo son casi idénticas a los flujos de entradas de archivo. Los constructores comúnmente usados son:

- **`FileOutputStream( String pathname )`**
- **`FileOutputStream( File file )`**

Hay métodos que soportan escribir un solo byte, un arreglo de bytes, o un subconjunto de un arreglo de bytes:

- `void write(int b)` lanza `IOException`: Este escribe el byte de orden inferior de `b`.
- `void write(byte bytes[])` lanza `IOException`: Este escribe a todos los miembros del arreglo de bytes `bytes[]`.
- `void write(byte bytes[], int offset, int len)` lanza `IOException`: Este escribe `len` bytes de bytes del arreglo `bytes[]`, comenzando en el desplazamiento.

La clase de `FileOutputStream` también tiene un método **`close()`**, que debe ser llamado siempre cuando un flujo de salida de archivo no se necesite más.

Además de las dos clases descritas arriba, el package `java.io` tiene un número de otras clases de flujos de entrada y salida de bajo nivel.

- `InputStream` y `OutputStream`: Estas son las superclases de las otras clases de flujos de bajo nivel. Pueden ser utilizados para la lectura y la escritura de sockets en redes .
- `ByteArrayInputStream` y `ByteArrayOutputStream`: Estas clases leen y escriben arreglos de bytes. Los arreglos de bytes no son ciertamente dispositivos de I/O de hardware, pero las clases son útiles cuando usted desea procesar o crear flujos de bytes.
- `PipedInputStream` y `PipedOutputStream`: Estas clases proporcionan un mecanismo para sincronizar la comunicación entre los hilos.

Filtros Para Flujos de Alto Nivel

El poder leer bytes de los dispositivos de entrada y escribir bytes en los dispositivos de salida, está muy bien si los bytes son la unidad de información que desea manejar. Sin embargo es más frecuente que se deseen leer tipos de información de alto nivel como enteros o cadenas.

Java brinda soporte para entrada/salida de alto nivel por medio de flujos de alto nivel. El más común de estos flujos (y el que se cubre en este capítulo) descende de las súper clases `FilterInputStream` y `FilterOutputStream`. Los flujos de entrada de alto nivel no leen desde dispositivos de entrada como archivos o sockets; en su lugar, lo

hacen desde otros flujos. Los flujos de salida de alto nivel no escriben en los dispositivos de salida, lo hacen en otros flujos.

Un buen ejemplo de flujo de entrada de alto nivel es el `DataInputStream`.

Existe un único constructor para esta clase:

`DataStream(InputStream instream)`

El constructor espera que usted le provea de un flujo de entrada. Esta instancia puede ser el flujo de entrada de un archivo (puesto que `FileInputStream` descende de `InputStream`), el flujo de entrada de un socket, o cualquier otro tipo de flujo de entrada. Cuando la instancia de `DataInputStream` es llamada por quien transporta los datos, realiza un número de llamadas `read()` en el flujo `instream`, procesa los bytes y retorna el valor apropiado. Los métodos más comúnmente usados de la clase `DataInputStream` son:

`boolean readBoolean() throws IOException`

- `byte readByte() throws IOException`
- `char readChar() throws IOException`
- `double readDouble() throws IOException`
- `float readFloat() throws IOException`
- `int readInt() throws IOException`
- `long readLong() throws IOException`
- `short readShort() throws IOException`
- `String readUTF() throws IOException`

Además existe, por supuesto, un método `close()`.

Nota:

Cuando se crean cadenas de flujos, es recomendable que usted cierre los flujos que no necesite más, asegurándose de hacerlo en el orden opuesto en que los flujos fueron contruidos.

El siguiente fragmento de código ilustra una pequeña cadena de entrada:

```
1. try{
2. //Construye la cadena
3. FileInputStream fis = new FileInputStream("el_archivo");
4. DataInputStream dis = new DataInputStream(fis);
5. //Lee la cadena
6. double d = dis.readDouble();
7. int i = dis.readInt();
8. String s = dis.readUTF();
9. //Cierra la cadena
10. dis.close(); //Cierra primero dis
11. fis.close(); //ya que se creo después de fis
12. }
13. catch (IOException e){ }
```

La figura 13.2 ilustra la jerarquía de la cadena de entrada.

El código espera que los primeros ocho bytes en el archivo representen un `double`, los siguientes cuatro bytes representen un `int`, y los siguientes (sin saber exactamente cuantos sean) bytes representen un UTF string. Esto significa que el código con el que originalmente se haya creado el archivo debe haber escrito un `double`, un `int` y un UTF String. No es necesario que el archivo haya sido creado por un programa de Java, pero en caso de haberlo sido, la mejor forma de hacerlo es empleando un `DataOutputStream`.

La clase `DataOutputStream` es la imagen espejo de la clase `DataInputStream`. El constructor es:

`DataOutputStream(OutputStream ostream)`

El constructor requiere que usted le pase un flujo de salida. Cuando usted escribe al flujo de salida, éste convierte los parámetros de los métodos `write()` a bytes y luego los escribe en `ostream`. Los métodos de salida de la clase `DataOutputStream` que se emplean más a menudo son:

- `void writeBoolean(boolean b) throws IOException`
- `void writeByte(int b) throws IOException`
- `void writeBytes(String s) throws IOException`
- `void writeChar(int c) throws IOException`
- `void writeDouble(double d) throws IOException`
- `void writeFloat(float b) throws IOException`
- `void writeInt(int i) throws IOException`
- `void writeLong (long l) throws IOException`
- `void writeShort (int s) throws IOException`
- `void writeUTF(String s) throws IOException`

Todos estos métodos convierten sus entradas a bytes de la forma obvia, con excepción de `writeBytes()`, el cual escribe solamente el byte de menor orden de cada carácter en su cadena. Como es usual también existe un método `close()`. También es necesario que las cadenas de flujos de salida sean cerradas en el orden inverso al de su creación.

Con los métodos listados anteriormente, usted está en capacidad de escribir un código que genere el archivo que debe ser leído por el código del ejemplo anterior. En este ejemplo, el archivo contendrá un `double`, un `int` y un `UTF String`. El archivo puede ser creado como se ilustra a continuación:

```
1. try{
2. //Crea la cadena
3. FileOutputStream fos = new FileOutputStream("el_archivo");
4. DataOutputStream dos = new DataOutputStream(fos);
5. //Escribe en la cadena
6. dos.writeDouble(123.456);
7. dos.writeInt(55);
8. dos.writeUTF("The moving finger writes");
9. //Cierra la cadena
10.dos.close();
11.fos.close();
12.}
13.catch (IOException e){ }
```

Adicionalmente a los `data input stream` y a los `data output stream`, el paquete `java.io` ofrece varias clases de flujos de alto nivel. Todos los constructores para los flujos de entrada de alto nivel requieren que usted les provea de un flujo de entrada, el cual debe ser el siguiente elemento de la cadena, ya que desde este flujo es de donde se obtendrán todos los datos para el nuevo objeto. De igual forma, los constructores para flujos de salida de alto nivel, requieren que usted les provea de un flujo de salida, en el cual serán escritos todos los datos generados por el nuevo objeto. Algunos de estos flujos de alto nivel se listan a continuación:

BufferedInputStream y BufferedOutputStream: Estas clases cuentan con buffers de almacenamiento interno, de tal forma que es posible leer o escribir grandes bloques de bytes, minimizando la sobrecarga de operaciones de entrada/salida.

PrintStream: Esta clase puede utilizarse para escribir texto o primitivas. Los objetos `System.out` y `System.err` son ejemplos de esta clase.

PushBackInputStream: Esta clase permite recuperar el último byte leído y llevarlo de vuelta al flujo, como si aún no hubiera sido leído. Esta clase es muy utilizada por ciertas clases de analizadores gramaticales.

Es posible crear cadenas de flujos de cualquier longitud. Por ejemplo, el fragmento de código siguiente implementa un flujo de datos de entrada (data input stream) que lee desde un buffered input stream, el cual a su turno lee desde un flujo de entrada de un archivo (file input stream):

1. `FileInputStream fis = new FileInputStream("lea_esto");`
2. `BufferedInputStream bis = new BufferedInputStream(fis);`
3. `DataInputStream dis = new DataInputStream(bis);`

Lectores y Escritores

Los lectores (readers) y escritores (writers) son como los flujos de entrada y salida:

Los de bajo nivel se comunican directamente con los dispositivos de entrada/salida, mientras que los de alto nivel se comunican con los de bajo nivel. Lo que hace que los lectores y escritores sean diferentes de los flujos es que son exclusivamente orientados a los caracteres UNICODE.

Un buen ejemplo de un lector de bajo nivel es la clase `FileReader`. Sus constructores más empleados son:

- `FileReader(String ruta)`
- `FileReader(File file)`

Por supuesto, cualquier archivo que sea pasado a estos constructores debe contener cadenas UTF.

El escritor correspondiente es la clase `FileWriter`:

- `FileWriter(String ruta)`
- `FileWriter(File file)`

Las otras clases de lectores y escritores de bajo nivel son:

- `CharArrayReader` y `CharArrayWriter`: estas clases leen y escriben arreglos de caracteres.
- `PipedReader` y `PipedWriter`: Estas clases proveen un mecanismo para la comunicación entre hilos.
- `StringReader` y `StringWriter`: Estas clases leen y escriben cadenas.

Los lectores de bajo nivel descenden de la súper clase abstracta `Reader`. Esta clase ofrece el ahora familiar trío de métodos `read()` para leer un sólo carácter, un arreglo de caracteres, o un subconjunto de un arreglo de caracteres. Note como ahora, la unidad de información es el char en lugar del byte. Los tres métodos son:

- `int read() throws IOException`: Este retorna el siguiente carácter (almacenado en los 16 bits de orden inferior del entero devuelto) o -1 si se trata del fin de la entrada.
- `int read(char dest[]) throws IOException`: Este método lee los caracteres del arreglo `dest[]`. Devuelve el número de caracteres leídos o -1 si se trata del fin de la entrada.
- `abstract int read(char dest[], int offset, int len) throws IOException`: Este método lee el número `len` de caracteres, comenzando en la posición `offset` desde

el arreglo `dest[]`. Devuelve el número de caracteres leídos o -1 si se trata del fin de la entrada.

Los escritores de bajo nivel descienden de la súper clase abstracta `Writer`. Esta clase provee un conjunto de métodos un poco diferente del trío estándar de métodos

`write()`:

- `void write(int ch) throws IOException`: escribe el carácter que aparece en los 16 bits de orden inferior de `ch`.
- `void write(String str) throws IOException`: escribe la cadena `str`.
- `void write(String str, int offset, int len) throws IOException`: escribe la subcadena de `str` que comienza en la posición `offset` y cuya longitud es igual a `len`.
- `void write(char chars[]) throws IOException`: escribe el arreglo de caracteres `chars`.
- `void write(char chars[], int offset, int len) throws IOException`: escribe el número `len` de caracteres del arreglo `chars`, comenzando en la posición `offset`.

Los lectores y escritores de alto nivel descienden de las súper clases `Reader` y `Writer`, por lo cual soportan los métodos listados anteriormente. Al igual que con los flujos de alto nivel, cuando usted construye un lector o un escritor de alto nivel debe pasarle como parámetro al constructor, el lector o escritor que se encuentra a continuación en la cadena que está siendo creada. Las clases de alto nivel son:

- `BufferedReader` y `BufferedWriter`: Estas clases cuentan con buffers internos que les permiten leer y escribir grandes bloques de datos, minimizando la sobrecarga de operaciones de entrada/salida. Son similares a las clases `BufferedInputStream` y `BufferedOutputStream`.
- `InputStreamReader` y `OutputStreamWriter`: Estas clases realizan la conversión entre flujos de bytes y secuencias de caracteres UNICODE. Por defecto, estas clases asumen que los flujos utilizan la codificación por defecto de la plataforma empleada; pero se proveen constructores alternativos para poder especificar alguna otra codificación.
- `LineNumberReader`: Esta clase considera su entrada como una secuencia de líneas de texto. Existe un método llamado `readLine()` que devuelve la línea siguiente, y la clase siempre lleva control sobre el número de línea en el que se encuentra.
- `PrintWriter`: Esta clase es similar a `PrintStream`, pero escribe caracteres en lugar de bytes.
- `PushBackReader`: Esta clase es similar a `PushBackInputStream`, pero lee caracteres en lugar de leer bytes.

El fragmento de código siguiente encadena un lector de número de línea

(`LineNumberReader`) con un lector de archivo. El código imprime cada línea del archivo precediéndola con el número de línea correspondiente:

```
1. 1. try{
2. 2. FileReader fr = new FileReader("datos");
3. 3. LineNumberReader lnr = new LineNumberReader(fr);
4. 4. String s;
5. 5. int lineNum;
6. 6. while((s = lnr.readLine())!=null){
7. 7. System.out.println(lnr.getLineNumber()+" "+s);
8. 8. }
9. 9. lnr.close();
10.10. fr.close();
```

```
11.11. }
```

```
12.12. catch(IOException x){ }
```

La figura 13.4 muestra la forma en que la cadena del lector es implementada por este código.

Resumen del Capítulo

Este capítulo ha cubierto la cuatro ideas principales relacionadas con el soporte de entrada/salida en Java:

- Dentro de la máquina virtual de Java, el texto es representado por caracteres **UNICODE** de 16 bits y cadenas. Para entrada/salida el texto puede estar representando también por cadenas **UTF**.
- La clase **File** es muy útil para navegar por el sistema de archivos local.
- La clase **RandomAccessFile** le permite leer y escribir en posiciones arbitrarias dentro de un archivo.
- Los flujos de entrada, los flujos de salida, los lectores y los escritores, brindan mecanismos para la creación de cadenas de entrada y de salida. Los flujos de entrada y salida operan sobre bytes, mientras que los lectores y los escritores lo hacen sobre caracteres.

Prueba Personal:

1. Cual de las siguientes afirmaciones es verdadera? (elija ninguna, algunas o todas.)
 - A. Todos los caracteres **UTF** son de 8 bits.
 - B. Todos los caracteres **UTF** son de 16 bits.
 - C. Todos los caracteres **UTF** son de 24 bits.
 - D. Todos los caracteres **UNICODE** son de 16 bits.
 - E. Todos los caracteres Bytecode son de 16 bits.
2. Cual de las siguientes afirmaciones es verdadera? (elija ninguna, algunas o todas.)
 - A. Cuando usted construye una instancia de **File**, si usted no utiliza la semántica empleada en los nombres de archivo de la máquina local, el constructor arrojará una **IOException**.
 - B. Cuando usted construye una instancia de **File**, si el correspondiente archivo no existe en el sistema de archivos local, entonces será creado.
 - C. Cuando una instancia de **File** es recolectada por el garbage collector, el archivo correspondiente en el sistema de archivos es eliminado.
3. La clase **File** contiene un método que se emplea para cambiarse al directorio actual de trabajo.
 - A. Verdadero
 - B. Falso
4. Es posible usar la clase **File** para listar el contenido del directorio de trabajo actual.
 - A. Verdadero
 - B. Falso

5. Cuantos bytes escribe el siguiente fragmento de código en el archivo destfile?

```
1. try {  
2. FileOutputStream fos = new FileOutputStream("destfile");  
3. DataOutputStream dos = new DataOutputStream(fos);  
4. dos.writeInt(3);  
5. dos.writeDouble(0.0001);  
6. dos.close();  
7. fos.close();  
8. }  
9. catch (IOException e){}
```

A. 2

B. 8

C. 12

D. 16

E. El número de bytes depende de la plataforma sobre la cual se esté trabajando.

6. Cual es el resultado de la impresión en la línea 9 del siguiente código?

```
1. FileOutputStream fos = new FileOutputStream("xx");  
2. for(byte b=10; b<50; b++)  
3. fos.write(b);  
4. fos.close();  
5. RandomAccessFile raf = new RandomAccessFile("xx", "r");  
6. raf.seek(10);  
7. int i = raf.read();  
8. raf.close();  
9. System.out.println("i = " + i);
```

A. La salida es **i** = 30.

B. La salida es **i** = 20.

C. La salida es **i** = 10.

D. No se produce ninguna salida puesto que en la línea 1 el código arroja una **IOException**.

E. No se produce ninguna salida puesto que en la línea 5 el código arroja una **IOException**.

7. Un archivo es creado empleando el siguiente código:

1. **FileOutputStream** fos = new **FileOutputStream**("datafile");
2. **DataOutputStream** dos = new **DataOutputStream**(fos);
3. for (int i=0; i<500; i++)
4. dos.**writeInt**(i);

Usted desea escribir un código para leer los datos desde este archivo. Cual de las siguientes soluciones funcionaría? (elija ninguna, algunas, o todas)

A. Construir un **FileInputStream**, pasándole el nombre del archivo. Encadenar al **FileInputStream** un **DataInputStream**, y llamar el método **readInt()** de este último.

B. Construir un **FileReader**, pasándole el nombre del archivo y llamando luego su método **readInt()**.

C. Construir un **PipedInputStream**, pasándole el nombre del archivo y llamando luego su método **readInt()**.

D. Construir un **RandomAccessFile**, pasándole el nombre del archivo y llamando luego su método **readInt()**.

E. Construir un **FileReader**, pasándole el nombre del archivo. Encadenar al **FileReader** un **DataInputStream**, y llamar el método **readInt()** de este último.

8. Los lectores tienen métodos que pueden leer y retornar floats y doubles.

A. Verdadero

B. Falso

9. Usted ejecuta el siguiente código en un directorio vacío, cuál es el resultado de esta operación?

1. **File** f1 = new **File**("dirname");
2. **File** f2 = new **File**(f1, "filename");

A. Se crea un nuevo directorio con el nombre dirname, dentro del directorio actual de trabajo.

B. Se crea un nuevo directorio con el nombre dirname, dentro del directorio actual de trabajo. Se crea un nuevo archivo con el nombre filename dentro del recién creado directorio dirname.

C. Se crean un nuevo directorio llamado `dirname`, y un nuevo archivo con el nombre `filename`, en el directorio actual de trabajo.

D. Se crea un nuevo archivo con el nombre `filename` en el directorio actual de trabajo.

E. No se crea ningún directorio, ni ningún archivo.

10.Cuál es el resultado de tratar de compilar y ejecutar el siguiente fragmento de código? Asuma que el código hace parte de un programa que cuenta con permisos de escritura sobre el directorio actual de trabajo. Asuma también que antes de la ejecución del código, en el directorio actual de trabajo no existe ningún archivo con el nombre `datafile`.

```
1. try {  
2. RandomAccessFile raf = new RandomAccessFile ("datafile", "rw");  
3. BufferedOutputStream bos = new BufferedOutputStream(raf);  
4. DataOutputStream dos = new DataOutputStream(bos);  
5. dos.writeDouble(Math.PI);  
6. dos.close();  
7. bos.close();  
8. raf.close();  
9. } catch (IOException e){ }
```

A. El código no compila.

B. El código compila, pero arroja una excepción en la línea 3.

C. El código compila y se ejecuta, pero no tiene ningún efecto sobre el sistema local de archivos.

D. El código compila y se ejecuta; después de esto, el directorio actual de trabajo contendrá un archivo llamado `datafile`.